

```
const 발전단계 = [  
  for (const 하나 of 발전단계) {  
    const 사람이적는것 = [  
      "오늘 3호 프레스를 점검했고 이상이 없었다",  
      "베어링에서 소음이 나서 교체했다",  
      "불량이 12개 나왔는데 원인은 금형 마모로 보인다",  
      "신입에게 절단기 조작법을 교육했다",
```

1 권 · 요청과 응답

백엔드

요청을 받아 응답을 돌려주는 것에서 시작해,
파일에 저장하고 화면과 잇는 데까지 갑니다.

7

단원

34

개념

161

직접 해보기

118

문제

차례

00	제조 현장 이해	9
01	스마트팩토리란 무엇인가	10
02	현장 문서 여섯 가지	17
03	문서마다 모양이 다르다	26
04	어떤 값이 들어 있나	27
05	왜 AI 를 붙이나	36
01	Node 시작하기	51
01	Node.js 란 무엇인가	52
02	파일 나눠 쓰기 (모듈)	59
03	npm 과 package.json	65
04	에러 메시지 읽는 법	72
02	파일 다루기	89
01	path, 경로 다루기	90
02	파일 읽고 쓰기 (fs)	98
03	동기와 비동기 (Sync 를 붙일 것인가)	106
04	폴더 다루기	114
05	JSON 파일로 데이터 저장하기	123
03	HTTP와 서버	141
01	클라이언트와 서버, 그리고 URL	142
02	HTTP 메서드와 상태코드	151
03	맨손으로 서버 만들기 (http 모듈)	154
04	요청 읽고 응답하기 (req 자세히 보기 + Postman)	161
05	JSON API 하나를 끝까지 만들어 보기	172
04	Express 시작하기	199
01	Express 시작하기	200

02	라우팅 (주소와 함수를 짝지어 등록하기)	210
03	요청에서 값 꺼내기 (params · query · body · headers)	220
04	응답 보내기와 에러 처리	229
05	nodemon 과 파일 나누기 (express.Router)	239
05	미들웨어	261
01	미들웨어란 무엇인가	262
02	쓸모 있는 미들웨어 직접 만들기	270
03	인증과 권한 (401 과 403)	280
04	검증을 미들웨어로 빼기	289
05	morgan 과 미들웨어 파일 나누기	290
06	REST API 설계	317
01	자원과 주소 설계	318
02	목록 다루기 (페이징 · 정렬 · 부분 응답)	327
03	검색과 필터	338
04	응답 형식 통일하기	348
05	API 문서와 버전 관리	359
부록 가	연습문제·종합연습 정답	385
부록 나	심화문제 정답	465

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

시작하기

```
cd 백엔드자료
npm install          # 한 번만
node 01_Node_시작하기/개념01_Node란_무엇인가.js
```

패키지는 **맨 위 폴더에 한 번만** 설치합니다. Node 가 `node_modules` 를 위쪽으로 거슬러 올라가며 찾기 때문에, 단위 폴더 안에서 실행해도 잘 찾습니다.

도구	왜
Node 18 이상	<code>fetch</code> 가 내장돼 있어야 합니다 (24 권장)
VS Code	아무 에디터나 됩니다
Postman	03단원부터 필수입니다. 브라우저로는 POST 를 못 보냅니다
크롬	08·10단원에서 F12 를 계속 씁니다

파일 이름 규칙

개념01_....js	읽고 실행하며 배우는 것
연습문제.js	서버 없이 푸는 문제 (TODO 를 채웁니다)
연습문제_정답.js	
연습문제_서버.js	서버를 직접 만드는 문제
연습문제_서버_정답.js	

정답을 먼저 보지 마세요. 설명은 정답 파일에 더 많지만, 막힌 뒤에 읽어야 남습니다.

이 자료의 약속

적힌 출력은 전부 진짜입니다

```
console.log(1 + 2);
// 출력: 3
```

이 주석은 손으로 적은 것이 아니라 **기계로 대조한 것**입니다. 서버 자료도 마찬가지입니다.

```
// 확인: GET /health
// 응답: 200 {"status":"ok"}
```

이건 진짜로 서버를 띄워 요청을 보내고 응답을 맞춰 본 것입니다. 화면도 진짜 크롬으로 버튼을 눌러 확인했습니다.

결과가 다르면 여러분이 틀린 것입니다. 자료를 의심하지 말고 코드를 보세요. 그래도 다르면 알려 주세요. 자료가 틀렸을 수 있습니다.

틀린 예를 일부러 넣어 뒀습니다

자주 하는 실수 절에 있는 것들은 대부분 **자료를 만들면서 실제로 겪은** 것입니다.  직접 해보기 에 "일부러 망가뜨려 보세요" 가 자주 나옵니다. 꼭 해 보세요. **확인한 뒤에는 반드시 되돌리세요.**

한글은 값에만 씁니다

이 자료가 계속 강조하는 규칙입니다.

곳	한글	안 쓰면 무슨 일이
라우트 주소	×	에러 없이 조용히 404
응답 헤더 값	×	500 으로 터짐
요청 헤더 값	×	요청이 아예 안 나감
업로드 필드 이름	×	Unexpected field
업로드 파일 이름	△	깨져서 도착 (되돌려야 함)
쿼리·경로·본문·응답	○	—

안 되는 것은 전부 **헤더에 실려 가는** 것입니다. HTTP 헤더는 아주 오래된 약속이라 영어와 숫자만 담깁니다.

단 원 00

제조 현장 이해

OPEN 00_제조_현장_이해

```
const 발전단계 = [  
  for (const 하나 of 발전단계) {  
    const 사람이적는것 = [  
      "오늘 3호 프레스를 점검했고 이상이 없었다",
```

01 스마트팩토리란 무엇인가

02 현장 문서 여섯 가지

03 문서마다 모양이 다르다

04 어떤 값이 들어 있나

05 왜 AI 를 붙이나

- 연습문제

스마트팩토리란 무엇인가

RUN node 개념01_스마트팩토리란_무엇인가.js

이 과정에서 우리가 만들 것은 이것입니다.

제조 현장의 문서를 AI가 읽고 답해 주는 웹서비스

코드를 짜기 전에 먼저 알아야 할 게 있습니다.

· 제조 현장이 어떤 곳인가 · 거기에 어떤 문서가 있나 · 왜 그 문서를 못 찾아서 고생하나

이걸 모르면 만들면서 계속 헛갈립니다. "이 기능이 왜 필요하지?" 를 스스로 답할 수 없게 됩니다.

이 단원에서 쓰는 것

순수 자바스크립트만 씁니다. Node 만의 기능은 아직 안 씁니다. PART 1 에서 배운 배열·객체·함수면 충분합니다. (require, fs 같은 것은 다음 단원에서 배웁니다)

공장이 어떻게 바뀌어 왔나

섹션 1

공장의 발전 단계를 흔히 이렇게 나눕니다.

```
const 발전단계 = [
  { 단계: 1, 이름: "기계화", 무엇이: "사람 손 → 기계", 기록: "종이 장부" },
  { 단계: 2, 이름: "자동화", 무엇이: "기계를 자동으로", 기록: "종이 + 엑셀" },
  { 단계: 3, 이름: "정보화", 무엇이: "컴퓨터로 관리", 기록: "엑셀 + 사내 시스템" },
  { 단계: 4, 이름: "지능화", 무엇이: "데이터로 판단", 기록: "★ 전부 데이터로" },
];

for (const 하나 of 발전단계) {
  console.log(`${하나.단계}단계 ${하나.이름.padEnd(4, " ")} | ${하나.
    무엇이.padEnd(14, " ")} | 기록: ${하나.기록}`);
}
```

출력	1단계 기계화	사람 손 → 기계	기록: 종이 장부
	2단계 자동화	기계를 자동으로	기록: 종이 + 엑셀
	3단계 정보화	컴퓨터로 관리	기록: 엑셀 + 사내 시스템
	4단계 지능화	데이터로 판단	기록: ★ 전부 데이터로

흔히 "스마트팩토리" 라고 하면 3~4단계를 가리킵니다.

그런데 여기서 중요한 것은 로봇이나 센서가 아닙니다.

★ 4단계로 가려면 기록이 데이터여야 합니다.

종이에 적힌 것은 컴퓨터가 못 읽습니다.

우리나라 공장 대부분은 2~3단계에 있습니다. 기계는 자동인데 기록은 아직 종이나 엑셀인 곳이 많습니다.

▢ 직접 해보기 1

발전단계 배열에서 기록에 "종이"가 들어간 단계만 골라서 이름을 출력해 보세요. (filter를 쓰세요)

자동화가 돼도 문서는 사람이 씁니다

섹션 2

여기가 이 과정의 출발점입니다.

설비는 자동으로 돌아도, 이런 것들은 여전히 사람이 적습니다.

```
const 사람이적는것 = [
  "오늘 3호 프레스를 점검했고 이상이 없었다",
  "베어링에서 소음이 나서 교체했다",
  "불량이 12개 나왔는데 원인은 금형 마모로 보인다",
  "신입에게 절단기 조작법을 교육했다",
];
```

```
console.log("");
```

출력

```
console.log("설비가 자동이어도 이걸 사람이 적습니다:");
```

출력 설비가 자동이어도 이걸 사람이 적습니다:

```
for (const 하나 of 사람이적는것) {
  console.log(` · ${하나}`);
}
```

출력 · 오늘 3호 프레스를 점검했고 이상이 없었다
· 베어링에서 소음이 나서 교체했다
· 불량이 12개 나왔는데 원인은 금형 마모로 보인다
· 신입에게 절단기 조작법을 교육했다

```
}
```

센서는 "온도 82도"는 알려 주지만 "왜 82도까지 올라갔는지"는 사람이 적습니다.

★★ 그 사람이 적은 글이 바로 우리가 다룰 문서입니다.

“그냥 안 적으면 안 되나요?” 라는 질문이 나옵니다. 안 됩니다.

```
const 적는이유 = [
  { 이유: "법으로 정해져 있다", 예: "산업안전보건법 - 안전점검 기록 보관" },
  { 이유: "인증을 받으려면 필요하다", 예: "ISO 9001 - 품질 기록을 남겨야 함" },
  { 이유: "고객사가 요구한다", 예: "납품처가 공정 기록 제출을 요구" },
  { 이유: "사고가 났을 때 근거가 된다", 예: "'절차대로 했다' 를 증명" },
  { 이유: "사람이 바뀌어도 이어져야 한다", 예: "담당자 퇴사 후에도 알아야 함" },
];
```

```
console.log("");
```

출력

```
for (const 하나 of 적는이유) {
  console.log(`${하나.이유.padEnd(24, " ")} ${하나.예}`);
}
```

출력	법으로 정해져 있다	산업안전보건법 - 안전점검 기록 보관
	인증을 받으려면 필요하다	ISO 9001 - 품질 기록을 남겨야 함
	고객사가 요구한다	납품처가 공정 기록 제출을 요구
	사고가 났을 때 근거가 된다	'절차대로 했다' 를 증명
	사람이 바뀌어도 이어져야 한다	담당자 퇴사 후에도 알아야 함

```
}
```

★★★ 다섯 번째가 이 과정과 제일 가깝습니다.

“20년 일한 반장님만 아는 것” 이 문서 어딘가에 있긴 합니다. 그런데 **못 찾습니다**. 그래서 결국 반장님한테 물어봅니다. 반장님이 퇴사하면 그 지식이 사라집니다.

⇒ 직접 해보기

2

적는이유 배열에서 “법” 이나 “인증” 이 들어간 것만 골라 개수를 세어 출력해 보세요.

문서가 얼마나 쌓이나

감이 잘 안 올 수 있어서 숫자로 세어 봅니다. 설비 30대짜리 중소 제조업체를 가정합니다.

```
const 설비수 = 30;
const 일하는날 = 250; // 1년
const 한해문서 = {
  점검일지: 설비수 * 일하는날, // 설비마다 매일
  불량보고서: 12 * 30, // 월 30건쯤
  작업표준서: 45, // 공정마다 하나, 잘 안 바뀜
}
```

```
설비이력카드: 설비수, // 설비마다 하나 (계속 덧붙임)
자재출납: 일하는날 * 8, // 하루 8건쯤
};
```

```
let 합계 = 0;
console.log("");
```

출력

```
for (const 이름 in 한해문서) {
  const 수 = 한해문서[이름];
  합계 += 수;
  console.log(`${이름.padEnd(12, " ")} ${String(수).padStart(6, " ")} 건/년`);
}
```

출력	점검일지	7500 건/년
	불량보고서	360 건/년
	작업표준서	45 건/년
	설비이력카드	30 건/년
	자재출납	2000 건/년

}

```
console.log(`${"합계".padEnd(12, " ")} ${String(합계).padStart(6, " ")} 건/년`);
```

출력	합계	9935 건/년
----	----	----------

```
console.log(`5년이면 ${(합계 * 5).toLocaleString("ko-KR")} 건입니다.`);
```

출력	5년이면 49,675 건입니다.
----	-------------------

★★ 설비 30대짜리 작은 공장인데도 5년이면 5만 건 가까이 됩니다.

이걸 엑셀 파일 수백 개와 종이 파일철에 나눠 두고 있습니다. “작년에 5호기 베어링 언제 갈았지?” 를 찾으려면 사람이 뒤져야 합니다.

그래서 이 과정이 만들 것

섹션 5

우리가 만들 서비스는 이 흐름입니다.

```
const 흐름 = [
  { 단계: "① 올린다", 무엇: "현장 문서를 웹에 업로드", 어디서: "PART 3" },
  { 단계: "② 저장한다", 무엇: "클라우드에 안전하게", 어디서: "PART 4" },
  { 단계: "③ 답한다", 무엇: "물어보면 문서를 찾아 답변", 어디서: "PART 5" },
  { 단계: "④ 합친다", 무엇: "팀으로 실제 서비스 완성", 어디서: "PART 6" },
];

console.log("");
```

출력

```
for (const 하나 of 흐름) {
  console.log(`${하나.단계.padEnd(8, " ")} ${하나.무엇.padEnd(24, " ")} (${하나.
  어디서})`);
}
```

출력	① 올린다	현장 문서를 웹에 업로드	(PART 3)
	② 저장한다	클라우드에 안전하게	(PART 4)
	③ 답한다	물어보면 문서를 찾아 답변	(PART 5)
	④ 합친다	팀으로 실제 서비스 완성	(PART 6)

}

▹ 직접 해보기 3

흐름 배열에서 "PART 5" 인 단계만 찾아 그 단계의 무엇 값을 출력해 보세요. (find 를 쓰세요)

자주 하는 실수

섹션 6

이런 실수를 합니다

"스마트팩토리 = 로봇" 이라고 생각하기

로봇은 2단계입니다. 이미 오래된 이야기입니다. 지금 어려운 것은 **흩어진 기록을 쓸 수 있게 만드는** 것입니다. 우리가 만들 것도 로봇이 아니라 그 기록을 다루는 웹서비스입니다.

이런 실수를 합니다

"AI 를 붙이면 문서 관리가 저절로 된다" 고 생각하기

반대입니다. **문서가 정리돼 있어야 AI 가 답합니다.** 개정판이 섞여 있으면 AI 는 옛 규정으로 답합니다. AI 는 정리를 대신 해 주지 않습니다. (PART 5 에서 직접 보게 됩니다)

이런 실수를 합니다**현장 용어를 몰라도 된다고 생각하기**

“가동률”, “치공구”, “부적합”, “로트” 같은 말이 계속 나옵니다. 모르면 요구사항을 잘못 읽습니다. ★ 모르는 말이 나오면 바로 물어보세요. 넘어가면 나중에 더 큼니다.

이런 실수를 합니다**문서를 다 똑같이 생겼다고 가정하기**

점검일지는 표이고, 작업표준서는 글이고, MSDS 는 규격 서식입니다. 전부 다르게 다뤄야 합니다. (개념03에서 봅니다)

이런 실수를 합니다**개인정보를 안 챙기기**

현장 문서에는 **작성자 이름·사번·연락처**가 거의 항상 들어 있습니다. 그걸 AI 에게 그대로 보내면 회사 밖으로 나갑니다. ★ PART 5 에서 크게 다룹니다. 지금부터 의식하고 보세요.

정리

- 1 스마트팩토리의 핵심은 로봇이 아니라 **기록이 데이터가 되는 것**이다.
- 2 설비가 자동이어도 “왜 그랬는지” 는 여전히 사람이 글로 적는다.
- 3 법·인증·고객사·사고 대비·인수인계 때문에 안 적을 수가 없다.
- 4 설비 30대짜리 공장도 5년이면 문서가 5만 건 가까이 쌓인다.
- 5 문제는 “없다” 가 아니라 “**있는데 못 찾는다**” 이다.
- 6 이 과정은 그 문서를 올리고(3) 저장하고(4) 답하게(5) 만들어 합친다(6).

직접 해보기 정답

```
1) const 종이단계 = 발전단계.filter((하나) => 하나.기록.includes("종이"));
   console.log(종이단계.map((하나) => 하나.이름));
   // 출력: [ '기계화', '자동화' ]
```

→ 1~2단계까지는 종이가 남아 있습니다.

2) const 걸린것 = 적는이유.filter(

```
(하나) => 하나.이유.includes("법") || 하나.이유.includes("인증")
```

```
);  
console.log(걸린것.length);  
// 출력: 2
```

→ “법으로 정해져 있다” 와 “인증을 받으려면 필요하다” 입니다.

★ 이 둘은 **안 지키면 벌금이나 인증 취소**로 이어집니다.
그래서 현장에서 문서를 없앨 수가 없습니다.

```
3) const 다섯 = 흐름.find((하나) => 하나.어디서 === "PART 5");  
console.log(다섯.무엇);  
// 출력: 물어보면 문서를 찾아 답변
```

→ find 는 조건에 맞는 **첫 번째 하나**를 돌려줍니다.

filter 는 배열을 돌려주니 여기서는 find 가 맞습니다.

현장 문서 여섯 가지

```
RUN node 개념02_현장_문서_여섯_가지.js
```

00

개념01에서 “문서가 5만 건 쌓인다”를 봤습니다. 그 5만 건이 다 같은 것은 아닙니다.

현장에서 자주 보는 여섯 가지를 봅니다. 앞으로 이 과정 전체에서 **이 여섯 가지를 소재**로 씁니다.

★ 이 단원의 [data/샘플문서/](#) 폴더에 실제 모양의 예시가 들어 있습니다. PART 3 업로드 실습, PART 5 문서 Q&A, PART 6 프로젝트에서 그대로 씁니다.

미리 밝혀 둘 것

여기 나오는 서식은 **일반적인 형태**입니다. 현장마다 회사마다 다릅니다. 항목 이름도 다릅니다. ★ 실제 프로젝트를 할 때는 **그 회사 문서를 먼저 받아 보세요.**

```

const 문서종류 = [
  {
    이름: "점검일지",
    다른이름: "일상점검표 · 체크시트",
    누가: "현장 작업자",
    언제: "매일 (교대마다)",
    무엇: "설비가 정상인지 항목별로 체크",
  },
  {
    이름: "작업표준서",
    다른이름: "SOP · 작업지도서",
    누가: "생산기술팀",
    언제: "공정이 바뀔 때",
    무엇: "이 작업을 어떤 순서로 하는가",
  },
  {
    이름: "설비이력카드",
    다른이름: "설비대장 · 이력관리표",
    누가: "설비보전팀",
    언제: "정비할 때마다 덧붙임",
    무엇: "이 설비에 언제 무엇을 했는가",
  },
  {
    이름: "불량보고서",
    다른이름: "부적합보고서 · NCR",
    누가: "품질관리팀",
    언제: "불량이 났을 때",
    무엇: "무슨 불량인 왜 났고 어떻게 조치했는가",
  },
  {
    이름: "자재출납대장",
    다른이름: "입출고대장 · 재고카드",
    누가: "자재팀",
    언제: "입고·출고할 때",
    무엇: "무엇이 언제 얼마나 들어오고 나갔는가",
  },
]

```

```

{
  이름: "MSDS",
  다른이름: "물질안전보건자료",
  누가: "제조사가 만들어 줌",
  언제: "화학물질을 들일 때",
  무엇: "이 물질이 위험한가, 사고 나면 어떻게 하는가",
},
];

for (const 하나 of 문서종류) {
  console.log(`【${하나.이름}】 ${하나.다른이름}`);
  console.log(`  누가: ${하나.누가} / 언제: ${하나.언제}`);
  console.log(`  무엇: ${하나.무엇}`);
}

```

출력

【점검일지】 일상점검표 · 체크시트
 누가: 현장 작업자 / 언제: 매일 (교대마다)
 무엇: 설비가 정상인지 항목별로 체크

【작업표준서】 SOP · 작업지도서
 누가: 생산기술팀 / 언제: 공정이 바뀔 때
 무엇: 이 작업을 어떤 순서로 하는가

【설비이력카드】 설비대장 · 이력관리표
 누가: 설비보전팀 / 언제: 정비할 때마다 덧붙임
 무엇: 이 설비에 언제 무엇을 했는가

【불량보고서】 부적합보고서 · NCR
 누가: 품질관리팀 / 언제: 불량이 났을 때
 무엇: 무슨 불량이 왜 났고 어떻게 조치했는가

【자재출납대장】 입출고대장 · 재고카드
 누가: 자재팀 / 언제: 입고·출고할 때
 무엇: 무엇이 언제 얼마나 들어오고 나갔는가

【MSDS】 물질안전보건자료
 누가: 제조사가 만들어 줌 / 언제: 화학물질을 들일 때
 무엇: 이 물질이 위험한가, 사고 나면 어떻게 하는가

★ “다른이름”을 넣어 둔 이유

같은 문서를 회사마다 다르게 부릅니다. “점검일지 주세요”라고 했는데 “그런 거 없는데요, 체크시트는 있어요”라는 대화가 실제로 자주 오갑니다. ★ 요구사항을 받을 때 **다른 이름도 같이 물어보세요.**

▫ 직접 해보기

1

문서종류에서 “언제”에 “매일”이 들어간 것만 골라 이름을 출력해 보세요.

얼마나 자주 생기나

섹션 2

문서마다 쌓이는 속도가 완전히 다릅니다. 이게 설계에 영향을 줍니다.

```
const 생기는속도 = [
  { 이름: "점검일지", 한해건수: 7500, 성격: "계속 쌓임" },
  { 이름: "자재출납대장", 한해건수: 2000, 성격: "계속 쌓임" },
  { 이름: "불량보고서", 한해건수: 360, 성격: "가끔 생김" },
  { 이름: "작업표준서", 한해건수: 45, 성격: "거의 안 바뀜" },
  { 이름: "설비이력카드", 한해건수: 30, 성격: "덧붙여 자람" },
  { 이름: "MSDS", 한해건수: 20, 성격: "거의 안 바뀜" },
];

console.log("");
```

출력

```
for (const 하나 of 생기는속도) {
  const 막대 = "■".repeat(Math.min(20, Math.ceil(하나.한해건수 / 400)));
  console.log(`${하나.이름.padEnd(14, " ")} ${String(하나.한해건수).padStart(5, " ")}
건 ${막대} ${하나.성격}`);
}
```

출력	점검일지	7500건	■ 계속 쌓임
자재출납대장	2000건	■ 계속 쌓임	
불량보고서	360건	■ 가끔 생김	
작업표준서	45건	■ 거의 안 바뀜	
설비이력카드	30건	■ 덧붙여 자람	
MSDS	20건	■ 거의 안 바뀜	

★★ 여기서 설계 판단이 갑니다.

“계속 쌓임” 인 것 → 검색해서 찾아야 합니다. 사람이 못 뒤집니다 “거의 안 바뀔” 인 것 → 개수가 적으니 통째로 다뤄도 됩니다

★ PART 5 에서 “문서가 작으면 통째로 넣으세요” 를 배웁니다.

작업표준서 45개, MSDS 20개는 그쪽에 가깝습니다.
점검일지 7500건은 반드시 찾기(검색)가 필요합니다.

무엇을 물어보게 되나

섹션 3

문서마다 사람들이 물어보는 질문이 다릅니다. 이게 우리가 만들 기능을 정합니다.

```
const 자주묻는것 = [
  { 문서: "점검일지", 질문: "5호기 지난달에 이상 있었나요?", 종류: "찾기" },
  { 문서: "작업표준서", 질문: "절단 작업 순서가 어떻게 되나요?", 종류: "설명" },
  { 문서: "설비이력카드", 질문: "3호기 베어링 마지막에 언제 갈았죠?", 종류: "찾기" },
  { 문서: "불량보고서", 질문: "이런 불량 전에도 있었나요?", 종류: "찾기" },
  { 문서: "자재출납대장", 질문: "이번 달 베어링 몇 개 나갔죠?", 종류: "★ 계산" },
  { 문서: "MSDS", 질문: "이거 피부에 묻으면 어떻게 하나요?", 종류: "설명" },
];

console.log("");
```

출력

```
for (const 하나 of 자주묻는것) {
  console.log(`[${하나.종류.padEnd(4, " ")}] ${하나.문서.padEnd(14, " ")} "${하나.
  질문}"`);
}
```

출력	[찾기]	점검일지	"5호기 지난달에 이상 있었나요?"
	[설명]	작업표준서	"절단 작업 순서가 어떻게 되나요?"
	[찾기]	설비이력카드	"3호기 베어링 마지막에 언제 갈았죠?"
	[찾기]	불량보고서	"이런 불량 전에도 있었나요?"
	[★ 계산]	자재출납대장	"이번 달 베어링 몇 개 나갔죠?"
	[설명]	MSDS	"이거 피부에 묻으면 어떻게 하나요?"

★★★ “계산” 을 따로 표시해 둔 이유

“몇 개 나갔죠?” 는 세어야 답이 나옵니다. ★ AI 는 계산을 잘 못합니다. 그럴듯한 숫자를 만들어 냅니다.

이런 건 DB 에 넣고 SQL 로 세야 합니다. (PART 4 에서 배웁니다) AI 에게 시키면 안 되는 것을 지금부터
갈라 두세요.

▹ 직접 해보기 2

자주묻는것 에서 종류가 “찾기” 인 것이 몇 개인지 세어 출력해 보세요.

어디에 보관돼 있나

섹션 4

문서가 있어도 못 찾는 진짜 이유입니다.

```
const 보관장소 = [
  { 어디: "종이 파일철", 비율: 30, 문제: "찾으려면 캐비닛을 뒤져야 함" },
  { 어디: "개인 PC 엑셀", 비율: 35, 문제: "★ 그 사람 없으면 못 봄" },
  { 어디: "공유 폴더", 비율: 25, 문제: "파일명이 제각각이라 못 찾음" },
  { 어디: "사내 시스템", 비율: 10, 문제: "그나마 낫지만 검색이 약함" },
];

console.log("");
```

출력

```
let 확인 = 0;
for (const 하나 of 보관장소) {
  확인 += 하나.비율;
  console.log(`${String(하나.비율).padStart(3, " ")}% ${하나.어디.padEnd(12, "
  ")} ${하나.문제}`);
}
```

출력	30%	종이 파일철	찾으려면 캐비닛을 뒤져야 함
	35%	개인 PC 엑셀	★ 그 사람 없으면 못 봄
	25%	공유 폴더	파일명이 제각각이라 못 찾음
	10%	사내 시스템	그나마 낫지만 검색이 약함

```
console.log(`합계 ${확인}%`);
```

출력 합계 100%

★ 위 비율은 **감을 잡으라고 넣은 예시 숫자**입니다. 회사마다 다릅니다. 다만 “개인 PC 에 흩어져 있다” 가 가장 큰 덩어리라는 건 어디나 비슷합니다.

★★ 그래서 우리가 만들 서비스의 첫 기능이 **업로드**입니다. 흩어진 것을 한 곳에 모으는 것부터가 시작입니다. (PART 3)

파일 형식은 무엇인가

섹션 5

업로드를 받으려면 어떤 파일이 올라올지 알아야 합니다.

```
const 파일형식 = [
  { 확장자: ".xlsx", 무엇: "엑셀", 혼한가: "매우 혼함", 글자뽑기: "어려움 - 표 구조" },
  { 확장자: ".pdf", 무엇: "PDF", 혼한가: "혼함", 글자뽑기: "보통 - 스캔본이면 어려움" },
  { 확장자: ".hwp", 무엇: "한글", 혼한가: "혼함", 글자뽑기: "★ 어려움 - 전용 형식" },
  { 확장자: ".docx", 무엇: "워드", 혼한가: "보통", 글자뽑기: "쉬움" },
  { 확장자: ".jpg", 무엇: "사진", 혼한가: "보통", 글자뽑기: "★★ 매우 어려움 - OCR 필요" },
  { 확장자: ".txt", 무엇: "글자", 혼한가: "드물", 글자뽑기: "매우 쉬움" },
];

console.log("");
```

출력

```
for (const 하나 of 파일형식) {
  console.log(`${하나.확장자.padEnd(7, " ")} ${하나.무엇.padEnd(5, " ")} ${하나.
  혼한가.padEnd(8, " ")} 글자 뽑기: ${하나.글자뽑기}`);
}
```

출력	.xlsx	엑셀	매우 혼함	글자 뽑기: 어려움 - 표 구조
	.pdf	PDF	혼함	글자 뽑기: 보통 - 스캔본이면 어려움
	.hwp	한글	혼함	글자 뽑기: ★ 어려움 - 전용 형식
	.docx	워드	보통	글자 뽑기: 쉬움
	.jpg	사진	보통	글자 뽑기: ★★ 매우 어려움 - OCR 필요
	.txt	글자	드물	글자 뽑기: 매우 쉬움

★★★ 여기서 프로젝트 범위가 정해집니다.

전부 다루려고 하면 못 끝납니다. ★ PART 6 프로젝트에서는 **글자를 뽑기 쉬운 형식부터** 하세요.

```
.txt · .docx · (텍스트가 든) .pdf
```

★★ 스캔한 사진은 OCR 이 필요합니다. 이 과정 범위 밖입니다.

"안 되는 것" 을 미리 정해 두는 것도 설계입니다.

◦ 직접 해보기

3

파일형식 에서 글자뽑기 에 "★" 가 들어간 것만 골라 확장자를 출력해 보세요.

이런 실수를 합니다

문서 이름을 하나로만 알고 가기

“점검일지” 만 알고 가면 “체크시트” 라고 부르는 회사에서 못 알아듣습니다. ★ 다른 이름을 같이 물어보세요.

이런 실수를 합니다

모든 문서를 같은 방식으로 다루려고 하기

점검일지 7500건과 MSDS 20건은 완전히 다릅니다. 앞의 것은 검색이 필요하고, 뒤의 것은 통째로 넣어도 됩니다.

이런 실수를 합니다

“계산” 질문을 AI 에게 시키기

“이번 달 몇 개 나갔죠?” 는 세어야 합니다. AI 는 세는 걸 못 믿습니다. ★ DB 와 SQL 로 하세요. (PART 4)

이런 실수를 합니다

.hwp 와 스캔 PDF 를 만만하게 보기

.hwp 는 전용 형식이라 라이브러리가 마땅치 않습니다. 스캔본은 사진이라 글자가 아예 없습니다. OCR 이 따로 필요합니다. ★ 프로젝트 범위를 정할 때 이걸 먼저 확인하세요.

이런 실수를 합니다

샘플 문서 없이 개발 시작하기

“대충 이렇게 생겼겠지” 로 만들면 실제 문서를 받았을 때 다 뜯어고칩니다. ★ 시작할 때 **실제 문서 한두 장**을 꼭 받아 보세요. 이 단원의 data/샘플문서/ 는 그 대신입니다. 진짜는 아닙니다.

정리

- 1 현장 문서는 크게 여섯 가지다. 점검일지 · 작업표준서 · 설비이력카드 · 불량보고서 · 자재출납대장 · MSDS
- 2 같은 문서를 회사마다 다르게 부른다. 다른 이름도 같이 물어봐야 한다.
- 3 쌓이는 속도가 다르다. 많이 쌓이는 것만 검색이 필요하다.
- 4 물어보는 질문이 “찾기 / 설명 / 계산” 으로 갈린다. ★ 계산은 AI 가 아니라 DB 가 해야 한다.
- 5 대부분 개인 PC 나 종이에 흩어져 있다. 그래서 모으는 것부터가 시작이다.
- 6 파일 형식이 제각각이다. .hwp 와 스캔 PDF 는 글자 뽑기가 어렵다. ★ 프로젝트 범위를 정할 때 형식부터 확인한다.

직접 해보기 정답

```
1) const 매일것 = 문서종류.filter((하나) => 하나.언제.includes("매일"));
console.log(매일것.map((하나) => 하나.이름));
// 출력: [ '점검일지' ]
```

→ 매일 생기는 건 점검일지뿐입니다. 그래서 제일 많이 쌓입니다.

```
2) const 찾기수 = 자주묻는것.filter((하나) => 하나.종류 === "찾기").length;
console.log(찾기수);
// 출력: 3
```

→ 여섯 중 셋이 "찾기" 입니다.

★ 그래서 이 과정의 핵심 기능이 **문서 검색**입니다.

```
3) const 어려운것 = 파일형식.filter((하나) => 하나.글자뽑기.includes("*"));
console.log(어려운것.map((하나) => 하나.확장자));
// 출력: [ '.hwp', '.jpg' ]
```

→ 이 둘은 프로젝트 범위에서 빼는 게 보통입니다.

★ "안 되는 것을 정하는 것" 도 설계입니다.
다 하려다 아무것도 못 끝내는 게 제일 흔한 실패입니다.

문서마다 모양이 다르다

RUN node 개념03_문서마다_모양이_다르다.js

개념02 에서 여섯 가지 문서를 봤습니다. 이번에는 그 문서들이 어떻게 생겼는지 를 봅니다.

★★★ 왜 이게 중요한가

나중에 이 문서를 컴퓨터가 읽어야 합니다. 그런데 모양이 다르면 읽는 방법이 다릅니다.

표는 칸을 나눠 읽어야 하고, 줄글은 문단으로 잘라야 합니다. 같은 방법으로 다루면 반드시 깨집니다.

이 파일에서 쓰는 것

data/샘플문서/ 의 여섯 개 중 일부를 글자로 옮겨 왔습니다. (파일을 읽는 방법은 02단원에서 배웁니다. 지금은 글자로 봅니다)

세 가지 모양

섹션 1

```
const 모양들 = [
  {
    모양: "표형",
    설명: "칸이 나뉘어 있고 값이 들어감",
    문서: "점검일지 · 자재출납대장",
    특징: "행마다 같은 항목이 반복됨",
  },
  {
    모양: "서식형",
    설명: "번호가 붙은 항목이 정해져 있음",
    문서: "작업표준서 · 불량보고서 · MSDS",
    특징: "1. 2. 3. 처럼 절이 나뉨",
  },
  {
    모양: "덧붙임형",
    설명: "날짜순으로 계속 이어 붙임",
    문서: "설비이력카드",
    특징: "끝에 계속 추가됨. 길어짐",
  },
];

for (const 하나 of 모양들) {
  console.log(`[${하나.모양}] ${하나.설명}`);
  console.log(`      문서: ${하나.문서}`);
}
```

어떤 값이 들어 있나

RUN node 개념04_어떤_값이_들어_있나.js

00

개념03 에서 문서의 **모양**을 봤습니다. 이번에는 그 안에 **어떤 값이 들어 있는지** 를 봅니다.

★★★ 이걸 왜 보나

① DB 표를 설계할 때 이 목록이 필요합니다 (PART 4) ② ★★ **밖으로 나가면 안 되는 값**을 미리 알아야 합니다 (PART 5)

②가 특히 중요합니다. 현장 문서에는 **사람 이름과 사번이 거의 항상** 들어 있습니다. 그걸 모르고 AI 에게 통째로 넘기면 회사 밖으로 나갑니다.

문서마다 어떤 칸이 있나

섹션 1

```
const 문서칸 = [
  {
    문서: "점검일지",
    칸: ["설비명", "설비번호", "점검일", "근무조", "점검자", "사번", "항목별 결과",
    "종합판정", "조치사항", "확인자"],
  },
  {
    문서: "작업표준서",
    칸: ["문서번호", "작업명", "개정판", "개정일", "작성자", "승인자", "적용범위",
    "준비물", "작업순서", "주의사항"],
  },
  {
    문서: "설비이력카드",
    칸: ["설비명", "설비번호", "도입일", "제조사", "담당팀", "이력(날짜+내용)",
    "비고"],
  },
  {
    문서: "불량보고서",
    칸: ["보고번호", "발생일", "공정", "설비", "로트번호", "작성자", "부적합내용",
    "수량", "원인", "조치", "확인자"],
  },
  {
    문서: "자재출납대장",
    칸: ["자재명", "품번", "단위", "안전재고", "날짜", "구분", "수량", "잔량",
    "사용처", "담당"],
  },
]
```

```

{
  문서: "작업표준서",
  칸: ["문서번호", "작업명", "개정판", "개정일", "작성자", "승인자", "적용범위",
"준비물", "작업순서", "주의사항"],
},
{
  문서: "설비이력카드",
  칸: ["설비명", "설비번호", "도입일", "제조사", "담당팀", "이력(날짜+내용)",
"비고"],
},
{
  문서: "불량보고서",
  칸: ["보고번호", "발생일", "공정", "설비", "로트번호", "작성자", "부적합내용",
"수량", "원인", "조치", "확인자"],
},
{
  문서: "자재출납대장",
  칸: ["자재명", "품번", "단위", "안전재고", "날짜", "구분", "수량", "잔량",
"사용처", "담당"],
},
{
  문서: "MSDS",
  칸: ["제품명", "제조사", "개정일", "관리번호", "유해성분류", "응급조치",
"취급보관", "보호구", "누출대처"],
},
];

for (const 하나 of 문서칸) {
  console.log(`${하나.문서.padEnd(14, " ")} 칸 ${String(하나.칸.length).padStart(2, "
")}${개}`);
}

```

출력	점검일지	칸 10개
	작업표준서	칸 10개
	설비이력카드	칸 7개
	불량보고서	칸 11개
	자재출납대장	칸 10개
	MSDS	칸 9개

▹ 직접 해보기 1

문서칸 에서 칸이 10개 이상인 문서만 골라 이름을 출력해 보세요.

공통으로 나오는 칸

섹션 2

여섯 문서에 몇 번씩 나오는지 세어 봅니다.

```
const 칸셈 = {};

for (const 하나 of 문서칸) {
  for (const 칸 of 하나.칸) {
    칸셈[칸] = (칸셈[칸] ?? 0) + 1;
  }
}

const 여러번 = Object.entries(칸셈)
  .filter(([ , 수]) => 수 >= 2)
  .sort((가, 나) => 나[1] - 가[1]);

console.log("");
```

출력

```
console.log("두 문서 이상에 나오는 칸:");
```

출력 두 문서 이상에 나오는 칸:

```
for (const [칸, 수] of 여러번) {
  console.log(` ${칸.padEnd(10, " ")} ${수}개 문서`);
}
```

출력	설비명	2개 문서
	설비번호	2개 문서
	확인자	2개 문서
	개정일	2개 문서
	작성자	2개 문서
	제조사	2개 문서
	수량	2개 문서

★★ “설비명:설비번호” 가 겹칩니다.

그래서 DB 를 만들 때 **설비 표를 따로** 두고 나머지가 그걸 가리키게 합니다. 같은 설비 이름을 여러 표에 적어 두면, 이름이 바뀔 때 전부 고쳐야 합니다. ★ PART 4 에서 이걸 배웁니다. (표 여러 개 나누기)

★★★ 밖으로 나가면 안 되는 값

섹션 3

여기가 이 파일에서 제일 중요한 부분입니다.

```
const 값등급 = [
  { 칸: "설비명", 등급: "공개", 이유: "설비 이름은 비밀이 아님" },
  { 칸: "점검일", 등급: "공개", 이유: "날짜는 그 자체로 문제없음" },
  { 칸: "항목별 결과", 등급: "공개", 이유: "AI 가 답하려면 필요함" },
```

```
{ 칸: "점검일", 등급: "공개", 이유: "날짜는 그 자체로 문제없음" },
{ 칸: "항목별 결과", 등급: "공개", 이유: "AI 가 답하려면 필요함" },
{ 칸: "점검자", 등급: "★ 개인정보", 이유: "사람 이름" },
{ 칸: "사번", 등급: "★ 개인정보", 이유: "개인을 특정함" },
{ 칸: "확인자", 등급: "★ 개인정보", 이유: "사람 이름" },
{ 칸: "담당 연락처", 등급: "★★ 개인정보", 이유: "전화번호는 더 민감" },
{ 칸: "로트번호", 등급: "△ 사내한정", 이유: "고객사가 추적할 수 있음" },
{ 칸: "단가·거래처", 등급: "△△ 영업비밀", 이유: "밖으로 나가면 큰일" },
];

console.log("");
```

출력

```
for (const 하나 of 값등급) {
  console.log(`${하나.등급.padEnd(12, " ")} ${하나.칸.padEnd(12, " ")} ${하나.이유}`);
}
```

출력	공개	설비명	설비 이름은 비밀이 아님
	공개	점검일	날짜는 그 자체로 문제없음
	공개	항목별 결과	AI 가 답하려면 필요함
	★ 개인정보	점검자	사람 이름
	★ 개인정보	사번	개인을 특정함
	★ 개인정보	확인자	사람 이름
	★★ 개인정보	담당 연락처	전화번호는 더 민감
	△ 사내한정	로트번호	고객사가 추적할 수 있음
	△△ 영업비밀	단가·거래처	밖으로 나가면 큰일

★★★ 등급을 나눠 두는 이유

AI 에게 문서를 통째로 넘기면 **저 아래 것들도 같이** 나갑니다. 그런데 대부분 AI 가 답하는 데 **필요하지도 않습니다.**

"3호 프레스 점검에서 뭐가 문제였나요?" 에 답하는 데 점검자 이름과 사번이 필요할까요? 필요 없습니다.

★ 그러니 **넘기기 전에 빼면** 됩니다.

PART 5 에서 이걸 "애초에 안 넣기" 로 배웁니다.

빼고 넘기기

섹션 4

점검일지 한 건을 예로 들어 봅니다.

```
const 점검일지 = {
  설비명: "3호 프레스",
  설비번호: "PRS-003",
  점검일: "2026-08-18",
  근무조: "주간",
  점검자: "김민준",
  사번: "E20250042",
  종합판정: "조건부 가동",
  조치사항: "하부 소음 확인 요망. 설비보전팀에 유선 통보함.",
  확인자: "박서준 반장",
};
```

★ 넘겨도 되는 칸만 고릅니다

```
const 넘길칸 = ["설비명", "설비번호", "점검일", "근무조", "종합판정", "조치사항"];

function 골라내기(원본, 허용칸) {
  const 결과 = {};
  for (const 칸 of 허용칸) {
    if (칸 in 원본) 결과[칸] = 원본[칸];
  }
  return 결과;
}

console.log("");
```

출력

```
console.log("[원본 - 그대로 넘기면]");
```

출력 [원본 - 그대로 넘기면]

```
console.log(JSON.stringify(점검일지));
```

출력 {"설비명":"3호 프레스","설비번호":"PRS-003","점검일":"2026-08-18","근무조":"주간","점검자":"김민준","사번":"E20250042","종합판정":"조건부 가동","조치사항":"하부 소음 확인 요망. 설비보전팀에 유선 통보함.","확인자":"박서준 반장"}

```
console.log("");
```

출력

```
console.log("[골라서 넘기면]");
```

출력 [골라서 넘기면]

```
console.log(JSON.stringify(골라내기(점검일지, 넘길칸)));
```

출력

```
{ "설비명": "3호 프레스", "설비번호": "PRS-003", "점검일": "2026-08-18", "근무조": "주간", "종합판정": "조건부 가동", "조치사항": "하부 소음 확인 요망. 설비보전팀에 유선 통보함." }
```

★★ 이름과 사번이 빠졌습니다. 그런데 답하는 데 아무 지장이 없습니다.

★ “빠는 목록” 이 아니라 “**넣을 목록**” 으로 만든 것이 요점입니다.

빠는 목록으로 만들면, 나중에 칸이 하나 늘었을 때 그게 그대로 나갑니다.
넣을 목록이면 새 칸은 기본으로 안 나갑니다.

▸ 직접 해보기

2

넘길칸 에 “점검자” 를 넣어 보고 결과가 어떻게 달라지는지 확인해 보세요. 그리고 다시 빼세요.

값이 비어 있는 경우

섹션 5

현장 문서는 칸이 자주 비어 있습니다. 이걸 미리 알아야 합니다.

```
const 실제기록들 = [
  { 설비명: "3호 프레스", 점검일: "2026-08-18", 종합판정: "조건부 가동", 조치사항: "하부 소음 확인 요망" },
  { 설비명: "5호 컨베이어", 점검일: "2026-08-18", 종합판정: "가동", 조치사항: "" },
  { 설비명: "7호 절단기", 점검일: "", 종합판정: "가동", 조치사항: "" },
  { 설비명: "", 점검일: "2026-08-17", 종합판정: "가동", 조치사항: "" },
];

function 빈칸찾기(기록) {
  const 빈것 = [];
  for (const 칸 in 기록) {
    if (기록[칸] === "" || 기록[칸] === null || 기록[칸] === undefined)
      빈것.push(칸);
  }
  return 빈것;
}

console.log("");
```

출력


```
for (const 하나 of 실제기록들) {
  const 빈것 = 빈칸찾기(하나);
  const 이름 = 하나.설비명 || "(설비명 없음)";
  console.log(`${이름.padEnd(14, " ")} 빈 칸: ${빈것.length ? 빈것.join(", ") :
  "없음"}`);
}
```

출력	3호 프레스	빈 칸: 없음
	5호 컨베이어	빈 칸: 조치사항
	7호 절단기	빈 칸: 점검일, 조치사항
	(설비명 없음)	빈 칸: 설비명, 조치사항

★★ 빈 칸에도 등급이 있습니다.

“조치사항” 이 비었다 → 정상입니다. 조치할 게 없었던 것입니다 “점검일” 이 비었다 → ★ 문제입니다.
언제 점검했는지 모릅니다 “설비명” 이 비었다 → ★★ 심각합니다. 어느 설비인지 모릅니다

★ 그래서 꼭 있어야 하는 칸을 정해 두고 검사해야 합니다.

PART 3 에서 “검증” 으로, PART 5 에서 “값 검사” 로 다시 만납니다.

▸ 직접 해보기 3

실제기록들 중에서 설비명이나 점검일이 빈 것만 골라 개수를 세어 출력해 보세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

문서를 통째로 AI 에게 넘기기

이름·사번·연락처가 같이 나옵니다. ★ 대부분 답하는 데 **필요 없는 값**입니다. 빼고 넘기세요.

이런 실수를 합니다

“빨 목록” 으로 만들기

나중에 칸이 하나 늘면 그게 그대로 새어 나옵니다. ★ **“넣을 목록”** 으로 만드세요. 새 칸은 기본으로 안 나옵니다.

이런 실수를 합니다

빈 칸이 없다고 가정하기

현장 문서는 칸이 자주 비어 있습니다. 손으로 쓰니까요. 기록.점검일.slice(0, 7) 같은 코드가 빈 값에서 터집니다.

이런 실수를 합니다

빈 칸을 다 똑같이 다루기

“조치사항 없음”은 정상이고 “설비명 없음”은 심각합니다. ★ 꼭 있어야 하는 칸을 정해 두세요.

이런 실수를 합니다

사람 이름이 개인정보라는 걸 잊기

“회사 안에서만 쓰는데요?”라고 해도, AI에게 보내면 **회사 밖**입니다. ★ PART 5 개념02에서 크게 다룹니다.

정리

- 1 문서마다 칸이 7~11개쯤 있다. DB 표를 설계할 때 이 목록이 출발점이다.
- 2 설비명·설비번호가 여러 문서에 겹친다. → 설비 표를 따로 둔다 (PART 4)
- 3 ★★ 칸마다 등급이 다르다. 공개 / 개인정보 / 사내한정 / 영업비밀.
- 4 AI에게 넘길 때는 “**넣을 목록**”으로 골라서 넘긴다. 이름·사번은 대부분 답하는 데 필요 없다.
- 5 현장 문서는 칸이 자주 비어 있다. 코드가 거기서 터진다.
- 6 빈 칸에도 등급이 있다. 꼭 있어야 하는 칸을 정해 두고 검사한다.

직접 해보기 정답

```
1) const 많은것 = 문서칸.filter((하나) => 하나.칸.length >= 10);
console.log(많은것.map((하나) => 하나.문서));
// 출력: [ '점검일지', '작업표준서', '불량보고서', '자재출납대장' ]
```

→ 넷입니다. 칸이 많을수록 입력 화면 만들기가 번거롭습니다.

★ PART 3에서 업로드 화면을 만들 때 이 개수가 그대로 일이 됩니다.

```
2) const 넣어보기 = 골라내기(점검일지, [...넘길칸, "점검자"]);
console.log(JSON.stringify(넣어보기));
// → "점검자":"김민준" 이 결과에 들어옵니다.
```

★ 한 줄 고쳤을 뿐인데 개인정보가 나가게 됩니다.

그래서 이 목록은 ****아무나 못 고치게**** 관리해야 합니다.

3) const 문제있는것 = 실제기록들.filter(

```
(하나) => !하나.설비명 || !하나.점검일
```

```
);  
console.log(문제있는것.length);  
// 출력: 2
```

→ 7호 절단기(점검일 없음)와 설비명 없는 것, 둘입니다.

★ `!하나.설비명`` 은 빈 문자열도 걸러 냅니다.
`""` 는 거짓으로 취급되기 때문입니다.

왜 AI를 붙이나

```
RUN node 개념05_왜_AI를_붙이나.js
```

여기까지 봤습니다.

개념01 문서가 왜 쌓이나
개념02 어떤 문서가 있나
개념03 어떻게 생겼나
개념04 안에 무슨 값이 있나

마지막으로 왜 AI를 붙이는지 를 봅니다.

★★★ 이걸 안 정하고 만들면

“AI 니까 뭔가 대단한 걸 하겠지” 로 시작해서 “그래서 이게 뭐가 좋은 거죠?” 로 끝납니다.

★ 실제 불편에서 시작해야 합니다.

```

const 불편들 = [
  {
    번호: 1,
    불편: "찾는 데 오래 걸린다",
    상황: "작년 5호기 베어링 교체일을 찾으려고 파일철을 뒤짐",
    걸리는시간: 20,
  },
  {
    번호: 2,
    불편: "신입은 어디 있는지도 모른다",
    상황: "절단 작업 순서를 몰라 선배를 찾아다님",
    걸리는시간: 15,
  },
  {
    번호: 3,
    불편: "옛 개정판을 보고 작업한다",
    상황: "공유 폴더에 v2 와 v3 가 같이 있어 v2 를 열어 봄",
    걸리는시간: 0,
  },
  {
    번호: 4,
    불편: "결국 반장님께 물어본다",
    상황: "반장님이 하던 일을 멈추고 답해 줌",
    걸리는시간: 10,
  },
  {
    번호: 5,
    불편: "야간에는 물어볼 사람이 없다",
    상황: "야간조에 문제가 생겼는데 담당자가 퇴근함",
    걸리는시간: 60,
  },
];

for (const 하나 of 불편들) {
  console.log(`${하나.번호}. ${하나.불편}`);
  console.log(`    "${하나.상황}"`);
}

```

출력

1. 찾는 데 오래 걸린다
"작년 5호기 베어링 교체일을 찾으려고 파일철을 뒤집"
2. 신입은 어디 있는지도 모른다
"절단 작업 순서를 몰라 선배를 찾아다님"
3. 옛 개정판을 보고 작업한다
"공유 폴더에 v2 와 v3 가 같이 있어 v2 를 열어 봄"
4. 결국 반장님께 물어본다
"반장님이 하던 일을 멈추고 답해 줘"
5. 야간에는 물어볼 사람이 없다
"야간조에 문제가 생겼는데 담당자가 퇴근함"

▹ 직접 해보기

1

불편들 에서 걸리는시간 이 가장 큰 것을 찾아 그 불편 값을 출력해 보세요.

시간으로 세어 봅니다

섹션 2

“불편하다” 는 말로는 설득이 안 됩니다. 숫자로 세어 봅니다.

```
const 하루찾는횟수 = 6; // 한 사람이 하루에 문서를 찾는 횟수
const 사람수 = 20;
const 일하는날 = 250;
```

평균 몇 분 걸리나

```
const 시간합 = 불편들.reduce((합, 하나) => 합 + 하나.걸리는시간, 0);
const 평균분 = 시간합 / 불편들.length;

const 한해분 = 평균분 * 하루찾는횟수 * 사람수 * 일하는날;
const 한해시간 = Math.round(한해분 / 60);

console.log("");
```

출력

```
console.log(`평균 ${평균분}분 × 하루 ${하루찾는횟수}회 × ${사람수}명 × ${일하는날}일`);
```

출력 평균 21분 × 하루 6회 × 20명 × 250일

```
console.log(`= 한 해 ${한해시간.toLocaleString("ko-KR")}시간`);
```

출력 = 한 해 10,500시간

★ 위 숫자는 감을 잡으라고 만든 예시입니다. 회사마다 다릅니다. 다만 “문서 찾는 데 쓰는 시간이 생각보다 크다” 는 건 어디나 비슷합니다.

★★ 실제 프로젝트에서는 이걸 **직접 재야** 합니다. “몇 분 걸리세요?” 를 열 명에게 물어보면 됩니다. 그 숫자가 있어야 “만들 만한가” 를 판단할 수 있습니다.

AI 가 되는 것과 안 되는 것

섹션 3

00

```
const 판정 = [
  { 불편: "찾는 데 오래 걸린다", AI: "○ 잘 맞음", 이유: "문서에서 찾아 주는 게 딱 이것" },
  { 불편: "신입이 어디 있는지 모른다", AI: "○ 잘 맞음", 이유: "물어보면 답해 줌" },
  { 불편: "옛 개정판을 본다", AI: "△ 반쯤", 이유: "★ 문서 정리가 먼저. AI 는 정리를 못 함" },
  { 불편: "반장님께 물어본다", AI: "○ 잘 맞음", 이유: "간단한 건 AI 가 대신 답함" },
  { 불편: "야간에 물어볼 사람 없음", AI: "○ 잘 맞음", 이유: "24시간 답할 수 있음" },
];

console.log("");
```

출력

```
for (const 하나 of 판정) {
  console.log(`${하나.AI.padEnd(8, " ")} ${하나.불편.padEnd(24, " ")} ${하나.이유}`);
}
```

출력	○ 잘 맞음	찾는 데 오래 걸린다	문서에서 찾아 주는 게 딱 이것
	○ 잘 맞음	신입이 어디 있는지 모른다	물어보면 답해 줌
	△ 반쯤	옛 개정판을 본다	★ 문서 정리가 먼저. AI 는 정리를 못 함
	○ 잘 맞음	반장님께 물어본다	간단한 건 AI 가 대신 답함
	○ 잘 맞음	야간에 물어볼 사람 없음	24시간 답할 수 있음

★★★ 세 번째를 보세요.

개정판이 섞여 있으면 AI 는 **옛 규정으로 답합니다**. AI 가 “이건 옛날 거네” 를 알아서 판단해 주지 않습니다.

★ 폐지된 문서를 **치우는 것**이 먼저입니다. 그건 사람이 해야 합니다.

PART 5 에서 이걸 실제로 재 보고 확인합니다.

```
const 시키면안되는것 = [
  { 무엇: "숫자를 세는 일", 예: "이번 달 베어링 몇 개 나갔죠?", 대신: "DB 와 SQL (PART 4)" },
  { 무엇: "정확해야 하는 판정", 예: "이 제품 합격입니까?", 대신: "규칙을 코드로" },
  { 무엇: "되돌릴 수 없는 실행", 예: "이 기록 지워 줘", 대신: "사람이 버튼을 누름" },
  { 무엇: "법적 판단", 예: "이거 법 위반인가요?", 대신: "담당 부서에 연결" },
];

console.log("");
```

출력

```
for (const 하나 of 시키면안되는것) {
  console.log(`x ${하나.무엇}`);
  console.log(`   예: "${하나.예}"`);
  console.log(`   대신: ${하나.대신}`);
}
```

출력

- x 숫자를 세는 일
 - 예: "이번 달 베어링 몇 개 나갔죠?"
 - 대신: DB 와 SQL (PART 4)
- x 정확해야 하는 판정
 - 예: "이 제품 합격입니까?"
 - 대신: 규칙을 코드로
- x 되돌릴 수 없는 실행
 - 예: "이 기록 지워 줘"
 - 대신: 사람이 버튼을 누름
- x 법적 판단
 - 예: "이거 법 위반인가요?"
 - 대신: 담당 부서에 연결

★★ 이 목록이 프로젝트 범위를 지켜 줍니다.

“이것도 되나요?” 라는 요청이 계속 들어옵니다. 미리 정해 두면 “그건 AI 가 아니라 SQL 로 합니다” 라고 답할 수 있습니다.

◀ 직접 해보기 2

시키면안되는것 에서 대신 에 “PART” 가 들어간 것만 골라 무엇 값을 출력해 보세요.

이 과정이 만들 것

앞의 판정을 종합하면 만들 기능이 정해집니다.


```
const 만들기능 = [
  { 기능: "문서 올리기", 왜: "흩어진 것을 한 곳에", 어디: "PART 3" },
  { 기능: "목록·조회", 왜: "무엇이 있는지 보기", 어디: "PART 3" },
  { 기능: "클라우드 저장", 왜: "내 PC 말고 어디서나", 어디: "PART 4" },
  { 기능: "질문하면 답하기", 왜: "찾는 시간을 줄이기", 어디: "PART 5" },
  { 기능: "★ 출처 보여주기", 왜: "믿을 수 있게", 어디: "PART 5" },
  { 기능: "관리자 화면", 왜: "폐지 문서 치우기", 어디: "PART 6" },
];

console.log("");
```

출력

```
for (const 하나 of 만들기능) {
  console.log(`${하나.어디} ${하나.기능.padEnd(16, " ")} ${하나.왜}`);
}
```

출력	PART 3	문서 올리기	흩어진 것을 한 곳에
	PART 3	목록·조회	무엇이 있는지 보기
	PART 4	클라우드 저장	내 PC 말고 어디서나
	PART 5	질문하면 답하기	찾는 시간을 줄이기
	PART 5	★ 출처 보여주기	믿을 수 있게
	PART 6	관리자 화면	폐지 문서 치우기

★★★ “출처 보여주기” 가 왜 별표인가

현장에서 AI 답을 그냥 믿지 않습니다. 믿으면 더 위험합니다. “이 답의 근거가 어느 문서 몇 조인가” 를 보여줘야 씁니다.

★ 답만 보여 주면 아무도 안 씁니다. 틀렸을 때 알아챌 방법이 없으니까요.

이건 PART 5 에서 계속 나오는 이야기입니다.

▫ 직접 해보기 3

만들기능 에서 PART 5 인 것이 몇 개인지 세어 출력해 보세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

“AI 로 뭘 할 수 있을까” 부터 생각하기

순서가 거꾸로입니다. **현장 불편**에서 시작해야 합니다. ★ 불편이 없으면 아무리 좋은 기술도 안 씁니다.

이런 실수를 합니다

불편을 말로만 적기

“찾기 불편해요” 로는 만들 만한지 판단할 수 없습니다. ★ “평균 20분, 하루 6번, 20명” 처럼 **재세요**.

이런 실수를 합니다

AI 가 문서 정리까지 해 줄 거라 기대하기

개정판이 섞여 있으면 AI 는 옛것으로 답합니다. ★ 치우는 건 사람이 합니다. AI 는 정리된 것에서만 잘합니다.

이런 실수를 합니다

안 되는 것을 미리 안 정하기

“이것도 되나요?” 가 계속 들어옵니다. ★ 계산·판정·실행·법적 판단은 AI 밖이라고 못 박아 두세요.

이런 실수를 합니다

출처를 안 보여 주기

답만 보여 주면 아무도 안 씁니다. 틀렸는지 확인할 수 없으니까요. ★ 근거 문서를 같이 보여 주는 게 이 서비스의 핵심입니다.

정리

- 1 AI 가 아니라 **현장 불편**에서 시작한다.
- 2 불편은 말이 아니라 **숫자로** 잴다. 그래야 만들 만한지 판단한다.
- 3 AI 가 잘하는 것 — 찾기, 설명하기, 24시간 답하기.
- 4 ★ AI 가 못 하는 것 — 세기, 정확한 판정, 되돌릴 수 없는 실행, 법적 판단.
- 5 ★★ 문서 정리는 사람이 먼저 해야 한다. AI 가 대신 안 해 준다.
- 6 ★★★ 출처를 보여 줘야 쓴다. 답만 보여 주면 아무도 안 믿는다.

직접 해보기 정답

```
1) let 제일큰것 = 불편들[0];  
for (const 하나 of 불편들) {
```

```
if (하나.걸리는시간 > 제일큰것.걸리는시간) 제일큰것 = 하나;
```

```
}
console.log(제일큰것.불편);
// 출력: 야간에는 물어볼 사람이 없다
```

→ 60분입니다.

★ 이게 **AI 가 제일 크게 도움이 되는 자리**입니다.
사람은 퇴근하지만 서비스는 24시간 돕니다.

```
2) const 걸린것 = 시키면안되는것.filter((하나) => 하나.대신.includes("PART"));
console.log(걸린것.map((하나) => 하나.무엇));
// 출력: [ '숫자를 세는 일' ]
```

→ 하나뿐입니다. 이걸 PART 4 에서 SQL 로 배웁니다.

나머지 셋은 기술이 아니라 **설계로** 막는 것입니다.

```
3) const 다섯개수 = 만들기능.filter((하나) => 하나.어디 === "PART 5").length;
console.log(다섯개수);
// 출력: 2
```

→ “질문하면 답하기” 와 “출처 보여주기” 입니다.

★ 둘이 짝입니다. 답만 있고 출처가 없으면 반쪽입니다.

제조 현장 이해

RUN node 연습문제.js

- 1 TODO 자리에 코드를 씁니다.
- 2 저장하고 node 연습문제.js 로 실행합니다.
- 3 '기대 출력'과 똑같이 나오면 정답입니다.
- 4 10분 고민해도 안 되면 연습문제_정답.js 를 보세요.

1~9는 코드로 푸는 문제, 10~13은 [손으로] 답을 적는 문제입니다.

★ 이 단원은 순수 자바스크립트만 씁니다. require, fs 는 안 씁니다.

문제 1 (개념01)

아래 배열에서 기록이 "데이터" 로 끝나는 단계의 이름을 출력하세요.

기대 출력 지능화

```
const 발전단계 = [
  { 단계: 1, 이름: "기계화", 기록: "종이 장부" },
  { 단계: 2, 이름: "자동화", 기록: "종이 + 엑셀" },
  { 단계: 3, 이름: "정보화", 기록: "엑셀 + 사내 시스템" },
  { 단계: 4, 이름: "지능화", 기록: "전부 데이터" },
];
```

여기에 코드를 쓰세요

문제 2 (개념01)

설비 50대, 연 250일 근무일 때 점검일지가 한 해에 몇 건 생기는지 계산해 출력하세요. (설비마다 하루 한 건)

기대 출력 12500

여기에 코드를 쓰세요

00

문제 3 (개념02)

아래 문서 목록에서 “계속 쌓임” 인 것의 이름만 배열로 출력하세요.

기대 출력

— ‘점검일지’, ‘자재출납대장’

```
const 문서들 = [
  { 이름: "점검일지", 성격: "계속 쌓임" },
  { 이름: "작업표준서", 성격: "거의 안 바뀜" },
  { 이름: "자재출납대장", 성격: "계속 쌓임" },
  { 이름: "MSDS", 성격: "거의 안 바뀜" },
];
```

여기에 코드를 쓰세요

문제 4 (개념02)

질문 목록에서 종류별 개수를 세어 출력하세요.

기대 출력 { '찾기': 3, '설명': 2, '계산': 1 }

★ 키에 따옴표가 붙습니다. `console.log` 는 “따옴표 없이 써도 되는 이름인가” 를 ASCII 로만 판단하기 때문입니다. 한글 키는 문법상 멀쩡한데도 붙습니다. 보기에만 그렇습니다. `JSON.stringify` 로 내보내면 차이가 없습니다.

```
const 질문들 = [
  { 질문: "5호기 지난달 이상 있었나요?", 종류: "찾기" },
  { 질문: "절단 작업 순서가 어떻게 되나요?", 종류: "설명" },
  { 질문: "3호기 베어링 마지막 교체일은?", 종류: "찾기" },
  { 질문: "이런 불량 전에도 있었나요?", 종류: "찾기" },
  { 질문: "이번 달 베어링 몇 개 나갔죠?", 종류: "계산" },
  { 질문: "이거 피부에 묻으면 어떡하죠?", 종류: "설명" },
];
```

여기에 코드를 쓰세요

문제 5 (개념03)

글을 보고 “표형” 인지 아닌지 판별하는 함수를 만드세요. 표 그리는 문자(| 나 —)가 5개 이상이면 표형입니다.

기대 출력 true
 false

```
const 글가 = "| 4 | 이상 소음·진동 | 주의 | 하부 미세 소음 |";
const 글나 = "오늘 점검했고 이상 없었습니다.";
```

```
function 표형인가(글) {
```

여기에 코드를 쓰세요

```
}
```

```
console.log(표형인가(글가));
console.log(표형인가(글나));
```

문제 6 (개념03)

점검 행 하나를 검색하기 좋은 문장으로 바꾸세요. 비교가 비어 있으면 비교 부분을 통째로 빼야 합니다.

기대 출력 2026-08-18 5호 컨베이어 점검 2번 "벨트 장력" 결과는 주의. 비교: 늘어짐.
 2026-08-18 5호 컨베이어 점검 3번 "안전커버" 결과는 양호.

```
const 머리 = { 설비: "5호 컨베이어", 점검일: "2026-08-18" };
const 행1 = { 번호: 2, 항목: "벨트 장력", 결과: "주의", 비교: "늘어짐" };
const 행2 = { 번호: 3, 항목: "안전커버", 결과: "양호", 비교: "" };

function 문장으로(머리, 행) {
```

여기에 코드를 쓰세요

```
}
```

```
console.log(문장으로(머리, 행1));
console.log(문장으로(머리, 행2));
```

문제 7 (개념04)

★ 넘겨도 되는 칸만 골라 새 객체를 만드세요. 이름·사번·연락처는 빼야 합니다.

기대 출력 {"설비명": "7호 절단기", "점검일": "2026-08-18", "종합판정": "가동"}

```
const 기록 = {
  설비명: "7호 절단기",
  점검일: "2026-08-18",
  점검자: "한지민",
  사번: "E20240115",
  연락처: "010-1234-5678",
  종합판정: "가동",
};

const 넘길칸 = ["설비명", "점검일", "종합판정"];

function 골라내기(원본, 허용칸) {
```

여기에 코드를 쓰세요

```
}
```

```
console.log(JSON.stringify(골라내기(기록, 넘길칸)));
```

문제 8 (개념04)

꼭 있어야 하는 칸이 비어 있는 기록만 골라 개수를 출력하세요. 꼭 있어야 하는 칸: 설비명, 점검일

기대 출력 2

```
const 기록들 = [
  { 설비명: "3호 프레스", 점검일: "2026-08-18", 조치사항: "" },
  { 설비명: "5호 컨베이어", 점검일: "", 조치사항: "" },
  { 설비명: "", 점검일: "2026-08-17", 조치사항: "확인 요망" },
  { 설비명: "7호 절단기", 점검일: "2026-08-18", 조치사항: "" },
];

const 꼭필요한칸 = ["설비명", "점검일"];
```


여기에 코드를 쓰세요

00

문제 9 (개념05)

불편 목록에서 AI로 해결되는 것("O"로 시작)만 골라 그 걸리는시간의 합을 출력하세요.

기대 출력 105

```
const 불편목록 = [
  { 불편: "찾는 데 오래 걸린다", 걸리는시간: 20, 판정: "O 잘 맞음" },
  { 불편: "신입이 모른다", 걸리는시간: 15, 판정: "O 잘 맞음" },
  { 불편: "옛 개정판을 본다", 걸리는시간: 30, 판정: "Δ 반쯤" },
  { 불편: "반장님께 물어본다", 걸리는시간: 10, 판정: "O 잘 맞음" },
  { 불편: "야간에 물어볼 사람 없음", 걸리는시간: 60, 판정: "O 잘 맞음" },
];
```

여기에 코드를 쓰세요

손으로 · 푸는 문제 — 답을 주석으로 적으세요

문제 10 (개념02)

현장 담당자에게 이렇게 물었습니다.

“점검일지 좀 주시겠어요?”

담당자가 “그런 거 없는데요” 라고 답했습니다. 그런데 실제로는 매일 쓰고 있었습니다.

(1) 왜 이런 일이 생깁니까? (2) 어떻게 물었어야 합니까?

답:

문제 11 (개념02·03)

팀에서 이렇게 정했습니다.

"업로드는 .txt, .docx, .pdf 만 받자. .hwp 와 스캔 사진은 빼자."

(1) 왜 .hwp 와 스캔 사진을 뺐습니까? (2) 현장에서 "우리는 .hwp 만 쓰는데요" 라고 하면 어떻게 하겠습니까?

답:

문제 12 (개념04)

점검일지를 AI 에게 넘기는 코드를 이렇게 짰습니다.

```
const 빨칸 = ["점검자", "사번"];
const 넘길것 = { ...기록 };
for (const 칸 of 빨칸) delete 넘길것[칸];
```

지금은 잘 됩니다. 그런데 왜 위험합니까? 어떻게 바꿔야 합니까?

답:

문제 13 (개념05)

현장 반장님이 이렇게 말합니다.

"AI 붙이면 문서 정리도 알아서 해 주는 거죠? 지금 공유 폴더에 옛날 버전이랑 새 버전이 막 섞여 있는데."

(1) 뭐라고 답하겠습니까? (2) 그대로 두고 AI 를 붙이면 무슨 일이 생깁니까? (3) 먼저 해야 할 일은 무엇입니까?

답:

다 풀었으면 연습문제_정답.js 와 맞춰 보세요.

단 원 01

Node 시작하기

OPEN 01_Node_시작하기

```
console.log(typeof window);  
console.log(typeof document);  
console.log(typeof alert);  
console.log(prices);
```

01 Node.js 란 무엇인가

02 파일 나눠 쓰기 (모듈)

03 npm 과 package.json

04 에러 메시지 읽는 법

- 연습문제

Node.js 란 무엇인가

RUN node 개념01_Node란_무엇인가.js

지금까지 자바스크립트를 두 곳에서 실행해 봤습니다.

① 터미널에서 node 파일.js ② 브라우저에서 .html 파일을 열어서

①이 바로 Node.js 입니다. 사실 이미 쓰고 계셨습니다. 이번 단원에서는 그게 정확히 무엇인지, 왜 백엔드에 쓰는지를 봅니다.

한 줄로 말하면

Node.js = 브라우저 밖에서 자바스크립트를 실행해 주는 프로그램

원래 자바스크립트는 브라우저 안에서만 돌아왔습니다. 웹페이지를 움직이라고 만든 언어였으니까요. 그런데 2009년에 “브라우저 없이도 돌게 하자” 며 만든 것이 Node.js 입니다.

그때부터 자바스크립트로 서버를 만들 수 있게 되었습니다. 프론트와 백엔드를 같은 언어로 쓸 수 있다는 것이 가장 큰 장점입니다.

브라우저에는 있고 Node 에는 없는 것

섹션 1

브라우저에서 늘 쓰던 것들을 여기서 찾아봅시다.

```
console.log(typeof window);
```

출력 undefined

window 가 없습니다. 브라우저 창을 가리키던 그 window 입니다.

```
console.log(typeof document);
```

출력 undefined

document 도 없습니다. 화면이 없으니 당연합니다.

```
console.log(typeof alert);
```

출력 undefined

alert 도 없습니다. 띄울 창이 없으니까요.

```
console.log(typeof localStorage);
```

출력 undefined

localStorage도 없습니다. 브라우저가 주던 저장소였습니다.

Node에는 화면이 없습니다. 그래서 화면과 관련된 것이 전부 없습니다.

화면을 그리는 일 → 브라우저(프론트엔드)가 합니다. 데이터를 다루는 일 → Node(백엔드)가 합니다.

이 구분이 이 과정 전체의 뼈대입니다.

▸ 직접 해보기 1

`typeof fetch`를 찍어 보세요. (fetch는 브라우저 것일까요, Node 것일까요?)

그런데 이건 그대로 있습니다

섹션 2

자바스크립트 '언어' 자체는 똑같습니다. 지금까지 배운 문법이 전부 그대로 동작합니다.

```
const menus = ["아메리카노", "라떼", "케이크"];
const prices = menus.map((menu, index) => `${menu} ${(index + 1) * 1000}원`);

console.log(prices);
```

출력 ['아메리카노 1000원', '라떼 2000원', '케이크 3000원']

```
const total = [1000, 2000, 3000].reduce((sum, n) => sum + n, 0);
console.log(total);
```

출력 6000

map, reduce, 화살표 함수, 템플릿 리터럴 — 전부 그대로입니다. 새 언어를 배우는 게 아닙니다. 같은 언어를 다른 곳에서 쓰는 것뿐입니다.

```
console.log(typeof console.log);
```

출력 function

console은 양쪽에 다 있습니다. 다만 브라우저는 F12 창에, Node는 터미널에 찍는다는 점만 다릅니다.

▸ 직접 해보기 2

배열 [5, 3, 9]를 filter로 4보다 큰 것만 골라 출력해 보세요.

대신 브라우저에 없는 것이 생겼습니다. 그중 첫 번째가 process 입니다. “지금 돌고 있는 이 프로그램” 자신을 가리키는 객체입니다.

```
console.log(typeof process);
```

출력 object

버전 확인

```
console.log(process.version.startsWith("v"));
```

출력 true

process.version 은 “v24.18.0” 처럼 생겼습니다. 설치한 Node 버전에 따라 다르므로 값 자체는 적어 두지 않았습니다.

직접 찍어 보세요: `console.log(process.version);`

어떤 운영체제에서 도는지

```
console.log(typeof process.platform);
```

출력 string

윈도우는 “win32”, 맥은 “darwin”, 리눅스는 “linux” 가 나옵니다. 나중에 EC2(리눅스)에 올리면 “linux” 가 됩니다.

지금 이 파일이 어느 폴더에서 실행됐는지

```
console.log(typeof process.cwd());
```

출력 string

cwd = current working directory. 터미널이 지금 있는 폴더입니다.

▸ 직접 해보기 3

process.version, process.platform, process.cwd() 를 각각 찍어서 내 컴퓨터의 값을 확인해 보세요.

실행할 때 값 넘겨 주기

터미널에서 파일 이름 뒤에 값을 붙이면 프로그램에 전달됩니다.

```
node 개념01_Node란_무엇인가.js 안녕 반가워
```

그 값들은 process.argv 라는 배열에 담깁니다.

코드	출력
<code>console.log(Array.isArray(process.argv));</code>	▶ true
<code>console.log(process.argv.length >= 2);</code>	▶ true

최소 2개는 항상 들어 있습니다. [0] node 프로그램의 위치 [1] 지금 실행 중인 파일의 위치 [2] 부터가 우리가 붙인 값입니다

우리가 붙인 값만 꺼내려면 앞의 두 개를 잘라냅니다.

```
const args = process.argv.slice(2);
console.log(args);
```

출력 []

아무것도 안 붙이고 실행했으니 빈 배열입니다.

위 명령처럼 값을 붙여 실행하면 ['안녕', '반가워'] 가 나옵니다. 직접 해 보세요. 서버를 만들 때 포트 번호를 이렇게 넘기기도 합니다.

▫ 직접 해보기

4

터미널에서 아래처럼 실행하고 결과를 확인해 보세요. node 개념01_Node란_무엇인가.js 커피 3

왜 백엔드에 Node 를 쓰나

섹션 5

서버를 만드는 언어는 여러 가지가 있습니다. (파이썬, 자바, PHP, Go ...) 그중 Node 를 쓰는 이유는 이렇습니다.

1) 프론트와 같은 언어입니다

화면도 자바스크립트, 서버도 자바스크립트.
언어를 하나만 알아도 양쪽을 다 만들 수 있습니다.

2) 기다리는 일에 강합니다

서버가 하는 일의 대부분은 '기다리기' 입니다.
데이터베이스 응답을 기다리고, 파일을 읽길 기다리고, AI 응답을 기다립니다.
Node 는 기다리는 동안 다른 요청을 처리합니다. (JS자료 12단원의 비동기입니다)

3) 패키지가 많습니다

npm 에 남이 만들어 둔 것이 300만 개가 넘습니다.
파일 업로드, 로그인, 이미지 처리 - 대부분 갖다 쓰면 됩니다.

이 과정에서 만들 AI Assistant 도 "요청을 받아 → AI 응답을 기다렸다가 → 돌려주는" 일이 대부분입니다.
Node 가 잘하는 일입니다.

자주 하는 실수

섹션 6

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다.

이런 실수를 합니다

브라우저 코드를 그대로 가져옴 → ReferenceError

```
document.querySelector("#app");
```

```
ReferenceError: document is not defined Node
```

에는 화면이 없습니다. 인터넷 예제를 복사할 때 자주 납니다. "이 코드가 브라우저 것인지 서버 것인지" 를 항상 확인하세요.

이런 실수를 합니다

alert 로 확인하려 함 → ReferenceError

```
alert("여기까지 왔습니다");
```

서버에서 값을 확인할 때는 `console.log` 만 씁니다.

이런 실수를 합니다

파일 이름을 안 적고 실행

터미널에 그냥 node 라고만 치면 REPL 이라는 대화 모드로 들어갑니다. > 표시가 뜨고 한 줄씩 실행됩니다. 나가려면 .exit 을 치거나 Ctrl+C 를 두 번 누릅니다. 당황하지 마세요. 파일을 실행하려면 node 파일이름.js 입니다.

이런 실수를 합니다

확장자를 빼먹음

node 개념01_Node란_무엇인가 ← .js 없이도 되긴 합니다 node 개념01_Node란_무엇인가.js ← 이렇게 쓰는 습관을 들이세요 폴더에 같은 이름의 폴더가 있으면 엉뚱한 것이 실행될 수 있습니다.

이런 실수를 합니다**서버가 안 꺼져서 포트가 막힘**

아직 서버를 안 만들었지만 미리 알아 두세요. 서버를 실행하면 프로그램이 안 끝나고 계속 돕니다. 정상입니다. 끄려면 그 터미널에서 Ctrl + C 입니다. 안 끄고 또 실행하면 "포트가 이미 사용 중" 에러가 납니다. (04단원에서 만납니다)

정리

- 1 Node.js 는 브라우저 밖에서 자바스크립트를 실행해 주는 프로그램이다.
- 2 window, document, alert, localStorage 가 없다. 화면이 없기 때문이다.
- 3 언어 문법은 완전히 같다. 새로 배울 문법이 아니다.
- 4 process 로 실행 환경(버전·OS·폴더)과 실행 인자를 알 수 있다.
- 5 node 파일이름.js 로 실행한다. node 만 치면 REPL 로 들어간다(.exit 로 나감).
- 6 화면은 브라우저가, 데이터는 Node 가 담당한다.

직접 해보기 정답

```
1) console.log(typeof fetch);
// 출력: function
```

→ 예전에는 브라우저에만 있었지만, Node 18 부터는 Node 에도 들어왔습니다.

그래서 서버에서도 다른 서버에 요청을 보낼 수 있습니다.
(PART 5 에서 AI 서버를 부를 때 이걸 씁니다)

```
2 console.log([5, 3, 9].filter((n) => n > 4));
// 출력: [ 5, 9 ]

3 console.log(process.version); // 예: v24.18.0
  console.log(process.platform); // 윈도우면 win32
  console.log(process.cwd());    // 지금 터미널이 있는 폴더 경로
```

→ 값은 컴퓨터마다 다릅니다. 옆 사람과 달라도 정상입니다.

4) node 개념01_Node란_무엇인가.js 커피 3

```
// 섹션 4의 args 출력이 [ '커피', '3' ] 로 바뀝니다.
```

→ 숫자 3을 넘겼는데 '3' 이라는 문자열로 들어옵니다.

터미널에서 넘어오는 값은 언제나 문자열입니다. 계산하려면 `Number` 로 바꿔야 합니다.

파일 나눠 쓰기 (모듈)

RUN node 개념02_모듈_불러오기.js

01

서버 코드는 금방 커집니다. 한 파일에 다 넣으면 아무도 못 읽습니다. 그래서 기능별로 파일을 나누고, 필요한 것만 서로 꺼내 씁니다.

그런데 방식이 두 가지입니다

CommonJS	require / module.exports	← Node 가 처음부터 쓰던 방식
ESM	import / export	← 나중에 표준이 된 방식

프론트(React·Vite)에서는 ESM 을 씁니다. 백엔드에서는 아직도 CommonJS 가 훨씬 많이 보입니다.

이 자료는 **CommonJS** 를 기본으로 씁니다. 이유는 두 가지입니다. ① Express 문서와 인터넷 예제 대부분이 require 로 되어 있습니다 ② 검색해서 나오는 코드를 그대로 쓸 수 있어야 합니다

둘 다 알아야 합니다. 이 파일에서 차이를 정리합니다.

require 로 가져오기

섹션 1

같은 폴더의 계산기.js 를 통째로 가져옵니다.

```
const calc = require("./계산기");
```

가져온 것은 객체입니다. 점으로 꺼내 씁니다.

코드	출력
console.log(typeof calc);	object
console.log(calc.TAX_RATE);	0.1
console.log(calc.add(3, 4));	7
console.log(calc.addTax(10000));	11000

읽는 법

```
const 답을이름 = require("./파일경로");
```

./ 로 시작하면 "내가 만든 파일"

★ .js 를 안 붙여도 됩니다. require("./계산기") 로 충분합니다. (프론트의 import 는 .js 를 꼭 붙여야 했죠. 여기가 다릅니다)

▹ 직접 해보기

1

calc.keyLength() 를 찍어 보세요. 몇이 나올까요?

필요한 것만 꺼내 오기

섹션 2

매번 calc. 을 붙이기 번거로우면, 가져오면서 바로 꺼낼 수 있습니다. JS자료 09단원에서 배운 객체 구조분해입니다.

```
const { add, addTax } = require("./계산기");
```

```
console.log(add(10, 20));
```

출력 30

```
console.log(addTax(5000));
```

출력 5500

실무 코드에서 이 형태를 가장 많이 봅니다.

```
const express = require("express");
```

통째로

```
const { Router } = require("express");
```

필요한 것만

내보내지 않은 것은 가져올 수 없습니다.

```
const { SECRET_KEY } = require("./계산기");
```

```
console.log(SECRET_KEY);
```

출력 undefined

에러가 아니라 undefined 입니다. 조용해서 더 헛갈립니다. "왜 undefined 지?" 싶으면 그 파일의 module.exports 를 먼저 확인하세요.

▹ 직접 해보기

2

계산기.js 를 열어서 module.exports 에 무엇이 담겼는지 확인하세요.

한 번만 실행된다

섹션 3

같은 파일을 두 번 require 하면 어떻게 될까요?

```
const calc1 = require("./계산기");
const calc2 = require("./계산기");

console.log(calc1 === calc2);
```

출력 true

같은 것입니다. Node 는 한 번 읽은 모듈을 기억해 두고 다시 씁니다.

그래서 모듈 안에 값을 담아 두면 온 프로그램이 그 값을 공유합니다. 나중에 데이터베이스 연결을 한 곳에 두고 여기저기서 가져다 쓰는 것이 이 성질 덕분입니다.

⇒ 직접 해보기 3

계산기.js 맨 아래에 `console.log("계산기 불러옴")` 을 추가하고 이 파일을 실행해 보세요. 몇 번 찍힐까요?

ESM 과 뭐가 다른가

섹션 4

프론트에서 배운 것과 나란히 놓고 보세요.

— 내보내기

```
CommonJS module.exports = { add };
ESM export { add };                      / export default ...
```

— 가져오기

```
CommonJS const { add } = require("./계산기");
ESM import { add } from "./계산기.js";
```

— 확장자

CommonJS	생략 가능	<code>require("./계산기")</code>
ESM	꼭 써야 함	<code>import ... from "./계산기.js"</code>

— 쓸 수 있는 위치

CommonJS	어디서든 가능. 함수 안, <code>if</code> 안에서도 됩니다
ESM	파일 맨 위에서만

한 파일에서 둘을 섞어 쓸 수는 없습니다. `package.json` 에 `"type": "module"` 이 있으면 ESM, 없으면 CommonJS 입니다. 이 자료의 폴더에는 그게 없으니 CommonJS 로 동작합니다.

CommonJS 는 함수 안에서도 부를 수 있습니다. 이건 ESM 이 못 하는 일입니다.

```
function loadWhenNeeded() {
  const later = require("./계산기");
  return later.add(1, 1);
}

console.log(loadWhenNeeded());
```

출력 2

▫ 직접 해보기 4

위 표를 보고, 아래 코드가 왜 에러가 나는지 말해 보세요. `import { add } from "./계산기.js";`

Node 가 이미 갖고 있는 모듈

섹션 5

`./` 없이 이름만 쓰면 "내가 만든 파일" 이 아니라 "Node 에 원래 있는 것" 이나 "npm 으로 설치한 것" 을 찾습니다.

```
const path = require("path");
const os = require("os");

console.log(typeof path.join);
```

출력 function

```
console.log(path.join("uploads", "docs", "a.pdf"));
```

출력 uploads\docs\a.pdf

윈도우는 `\` 로, 맥·리눅스는 `/` 로 이어 줍니다. 운영체제마다 다른 것을 알아서 맞춰 주는 것이 `path` 의 역할입니다. (나중에 EC2 리눅스에 올릴 때 이것 덕분에 안 깨집니다)

```
console.log(typeof os.platform());
```

출력 string

설치가 필요 없습니다. Node 에 이미 들어 있습니다. 02단원에서 자세히 봅니다.

▹ 직접 해보기 5

`path.join("a", "b", "c.txt")` 를 찍어 보세요.

01

자주 하는 실수

섹션 6

★ 아래에서 `SyntaxError` 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. `SyntaxError` 는 파일을 아예 못 읽게 만들어 출력이 한 줄도 안 나옵니다.

이런 실수를 합니다

`./` 를 빼먹음 → 에러

```
const calc = require("계산기");
```

Error: Cannot find module '계산기' `./` 가 없으면 npm 패키지를 찾습니다. 내 파일은 반드시 `./` 로 시작합니다.

이런 실수를 합니다

`module.exports` 를 안 씀

파일에 함수를 만들어 놓고 `module.exports` 를 안 하면 `require` 로 가져와 봐야 빈 객체 `{}` 입니다. 에러도 안 납니다. "분명 만들었는데 `undefined` 가 나온다" 면 이걸 의심하세요.

이런 실수를 합니다

`exports` 와 `module.exports` 를 헷갈림

```
exports.add = add; ← 이것도 됩니다  
exports = { add }; ← 이걸 안 됩니다 (조용히 실패)
```

헷갈리니 이 자료에서는 `module.exports` 만 씁니다.

이런 실수를 합니다

CommonJS 파일에서 `import` 를 씀 → `SyntaxError`

```
import { add } from "./계산기.js";
```

```
SyntaxError: Cannot use import statement outside a module
```

이 풀더는 CommonJS 입니다. `require` 를 쓰세요.

이런 실수를 합니다

순환 참조

A.js 가 B.js 를 부르고, B.js 가 다시 A.js 를 부르는 경우입니다. 에러 없이 한쪽이 `undefined` 가 되어 아주 찾기 어렵습니다. 파일을 나눌 때 “한 방향으로만 흐르게” 설계하세요.

정리

- 1 이 자료는 CommonJS 를 쓴다. `require / module.exports`.
- 2 `const calc = require("./파일");` — 내 파일은 `./` 로 시작, `.js` 는 생략 가능.
- 3 `module.exports` 에 담은 것만 밖에서 쓸 수 있다. 안 담으면 `undefined`.
- 4 구조분해로 필요한 것만 꺼내는 형태를 가장 많이 쓴다.
- 5 같은 모듈을 여러 번 불러도 한 번만 실행되고 같은 것을 돌려준다.
- 6 `./` 없이 이름만 쓰면 Node 내장 모듈이나 npm 패키지를 찾는다.

직접 해보기 정답

```
1) console.log(calc.keyLength());
// 출력: 12
```

→ `SECRET_KEY` 는 못 가져오지만, 그 값을 쓰는 함수는 가져올 수 있습니다.

“밖으로 안 나갑니다” 가 12글자입니다.
값은 숨기고 기능만 열어 주는 것 - 모듈을 나누는 이유 중 하나입니다.

```
2) module.exports = { TAX_RATE, add, addTax, keyLength };
```

→ `SECRET_KEY` 는 없습니다. 그래서 `undefined` 가 나온 것입니다.

- 3 한 번만 찍힙니다. 이 파일에서 `require("./계산기")` 를 네 번이나 했는데도 한 번입니다. Node 가 처음 한 번만 실행하고 그 결과를 기억해 두기 때문입니다. (섹션 3) 확인했으면 `계산기.js` 에서 그 줄을 다시 지우세요.
- 4 이 폴더에는 `package.json` 의 “type”: “module” 이 없어서 CommonJS 로 동작합니다. CommonJS 파일에서는 `import` 를 쓸 수 없습니다. → `SyntaxError: Cannot use import statement outside a module`
- 5 `console.log(path.join("a", "b", "c.txt"));` // 출력: `a\b\c.txt` (윈도우 기준. 맥·리눅스는 `a/b/c.txt`)

npm 과 package.json

RUN node 개념03_npm과_package.json.js

01

서버를 만들 때 모든 것을 직접 짜지 않습니다. 남이 잘 만들어 둔 것을 가져다 씁니다. 그걸 '패키지' 라고 합니다.

파일 업로드 처리 → multer 서버 만들기 → express 날짜 다루기 → dayjs AWS 연동 → aws-sdk

이 과정에서 쓸 것이 전부 패키지입니다. 직접 만들지 않습니다.

npm 이 무엇인가

npm 은 두 가지를 뜻합니다.

① 패키지가 모여 있는 창고 (인터넷에 있습니다. 300만 개가 넘습니다) ② 그 창고에서 가져오는 도구 (내 컴퓨터에 있습니다. Node 설치할 때 같이 깔립니다)

터미널에 npm -v 를 쳐 보세요. 버전이 나오면 이미 있는 것입니다.

package.json — 프로젝트의 명세서

섹션 1

프로젝트 폴더마다 package.json 이라는 파일을 하나 둡니다. "이 프로젝트가 무엇이고, 무엇을 가져다 쓰는가" 를 적어 둔 파일입니다.

이 폴더에도 하나 있습니다. 열어 보세요.

CommonJS 에서는 JSON 파일도 require 로 읽을 수 있습니다. 읽으면 그냥 객체가 됩니다.

```
const pkg = require("./package.json");
```

```
console.log(typeof pkg);
```

출력 object

```
console.log(pkg.name);
```

출력 01-node-start

프로젝트 이름입니다. 소문자와 하이픈만 쓸 수 있습니다. 한글·대문자·공백은 안 됩니다.

```
console.log(pkg.version);
```

출력 1.0.0

버전입니다. 큰변화.기능추가.버그수정 순서로 올립니다.

```
console.log(pkg.description);
```

출력 01단원 실습용 프로젝트

```
console.log(pkg.main);
```

출력 개념01_Node란_무엇인가.js

“이 프로젝트의 시작 파일” 입니다. 나중에 서버 파일 이름을 여기 적습니다.

▸ 직접 해보기

1

pkg.license 와 pkg.author 를 찍어 보세요.

새 프로젝트 시작하기

섹션 2

새 폴더에서 아래 명령을 치면 package.json 이 만들어집니다.

```
npm init            질문에 하나씩 답하기 (이름? 버전? 설명? ...)
npm init -y        전부 기본값으로 한 번에 (-y = yes)
```

실무에서는 npm init -y 로 만들고 필요한 곳만 고칩니다.

중요 · 반드시 ‘프로젝트 폴더 안에서’ 실행하세요.

영동한 곳에서 치면 그 폴더에 package.json 이 생겨 버립니다.
터미널이 지금 어디에 있는지는 이렇게 확인합니다.

```
console.log(typeof process.cwd());
```

출력 string

직접 찍어 보면 지금 터미널이 있는 폴더 경로가 나옵니다. 이 파일을 실행한 곳이 01_Node_시작하기 폴더가 맞는지 확인해 보세요.

▸ 직접 해보기

2

바탕화면에 연습용 폴더를 하나 만들고, 터미널로 그 폴더에 들어가 npm init -y 를 실행해 보세요.
package.json 이 생겼는지 확인하세요.

패키지 설치하기

섹션 3

참고에서 가져오는 명령은 하나입니다.

```
npm install 패키지이름
npm i 패키지이름          (i 는 install 의 줄임)
```

이 폴더에는 dayjs 라는 날짜 패키지를 이미 설치해 두었습니다. 설치하면 package.json 의 dependencies 에 자동으로 적힙니다.

코드	출력
<code>console.log(Object.keys(pkg.dependencies));</code>	<code>['dayjs']</code>
<code>console.log(pkg.dependencies.dayjs.startsWith("^"));</code>	<code>true</code>

버전 앞에 ^ 가 붙어 있습니다. “이 버전 이상, 큰 변화 전까지는 괜찮다” 는 뜻입니다. ^1.11.21 → 1.11.21 이상 2.0.0 미만이면 OK 버그가 고쳐진 새 버전을 자동으로 받으라는 의미입니다.

설치할 때 붙이는 옵션이 하나 더 있습니다.

```
npm install nodemon --save-dev      또는  npm i -D nodemon
```

--save-dev 를 붙이면 dependencies 가 아니라 devDependencies 에 들어갑니다.

dependencies	실제로 서버가 돌아갈 때 필요한 것	(express, multer ...)
devDependencies	개발할 때만 필요한 것	(nodemon, 테스트 도구 ...)

나중에 서버에 올릴 때 devDependencies 는 빼고 설치할 수 있어 가볍습니다.

▫ 직접 해보기 3

위에서 만든 연습 폴더에서 npm i dayjs 를 실행하고, package.json 이 어떻게 바뀌었는지 열어서 확인하세요.

생기는 것들 — node_modules 와 package-lock.json

섹션 4

설치하면 폴더에 두 가지가 새로 생깁니다.

```
node_modules/      실제 코드가 들어 있는 폴더. 아주 큼니다.
package-lock.json  "정확히 이 버전을 설치했다" 는 기록
```

node_modules 안에 정말 들어 있는지 확인해 봅시다.

```
const dayjs = require("dayjs");

console.log(typeof dayjs);
```

출력 function

require("dayjs") 에 ./ 가 없습니다. 내 파일이 아니라 설치한 패키지니까요. (개념02)

```
console.log(dayjs("2026-08-13").format("YYYY년 M월 D일"));
```

```
출력      2026년 8월 13일
```

아주 중요 · node_modules 는 Git 에 올리지 않습니다.

왜냐하면

- 파일이 수만 개라 엄청나게 무겁습니다
- package.json 만 있으면 npm install 로 언제든지 다시 만들 수 있습니다

그래서 .gitignore 에 node_modules 를 적어 둡니다. (Git 단원에서 배웁니다) 대신 package.json 과 package-lock.json 은 반드시 올립니다.

팀원이 프로젝트를 받으면 이렇게 시작합니다.

```
git clone ...      코드 받기 (node_modules 는 없음)
npm install        package.json 을 보고 알아서 설치
```

package-lock.json 이 왜 필요한가 ^1.11.21 은 "1.11.21 이상" 이라 사람마다 다른 버전이 깔릴 수 있습니다. lock 파일은 "정확히 1.11.21" 을 못 박아 둡니다. 그래야 "내 컴퓨터에선 되는데요" 가 안 생깁니다.

▸ 직접 해보기 4

이 폴더의 node_modules 폴더를 열어 보세요. dayjs 폴더가 있는지 확인하세요.

npm scripts — 긴 명령을 짧게

섹션 5

package.json 의 scripts 에 이름을 붙여 두면 짧게 실행할 수 있습니다.

```
console.log(Object.keys(pkg.scripts));
```

```
출력      [ 'start', 'hello' ]
```

```
console.log(pkg.scripts.start);
```

```
출력      node 개념01_Node란_무엇인가.js
```

이렇게 적어 두면 터미널에서 이렇게 실행합니다.

```
npm start      → node 개념01_Node란_무엇인가.js 가 실행됩니다
npm run hello  → hello 에 적힌 명령이 실행됩니다
```

start 만 특별해서 run 을 생략할 수 있습니다. 나머지는 npm run 이름 입니다.

왜 쓰나 서버 실행 명령이 길어지기 때문입니다. 나중에 이렇게 됩니다.

```
"dev": "nodemon --watch src server.js"
```

이걸 매번 치는 대신 npm run dev 로 끝냅니다.

그리고 팀원이 "어떻게 실행해요?" 라고 물어볼 필요가 없어집니다. package.json 의 scripts 만 보면 됩니다.

▸ 직접 해보기 5

터미널에서 npm run hello 를 실행해 보세요.

자주 하는 실수

섹션 6

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. SyntaxError 는 파일을 아예 못 읽게 만들어 출력이 한 줄도 안 나옵니다.

이런 실수를 합니다

엉뚱한 폴더에서 npm install

바탕화면에서 npm i express 를 치면 바탕화면에 node_modules 가 생깁니다. 반드시 프로젝트 폴더로 cd 한 다음에 설치하세요. VS Code 에서 폴더를 우클릭 → "통합 터미널에서 열기" 가 가장 안전합니다.

이런 실수를 합니다

package.json 없이 설치

package.json 이 없는 폴더에서 설치하면 설치는 되지만 기록이 안 남습니다. 나중에 팀원이 npm install 을 해도 그 패키지가 안 깔립니다. 먼저 npm init -y 로 만들고 설치하세요.

이런 실수를 합니다

node_modules 를 Git 에 올림

파일이 수만 개라 push 가 몇 분씩 걸리고, 팀원과 충돌이 납니다. .gitignore 에 node_modules 를 꼭 넣으세요.

이런 실수를 합니다

package.json 을 손으로 고치고 설치를 안 함

dependencies 에 직접 이름을 적어 넣어도 설치는 안 됩니다. 적는 것과 받는 것은 별개입니다. npm install 을 실행해야 실제로 받아집니다.

이런 실수를 합니다

JSON 파일에 주석을 씌우면 → 에러

package.json 은 JSON 이라 // 주석을 쓸 수 없습니다. 주석을 넣으면 npm 이 파일을 못 읽어 모든 명령이 실패합니다.

이런 실수를 합니다

이름에 한글이나 대문자를 씌우면

npm init 에서 이름을 "내프로젝트" 나 "MyApp" 으로 하면 거부당합니다. 소문자, 숫자, 하이픈(-)만 쓰세요. my-app 같은 형태입니다.

정리

- 1 npm = 패키지 참고 + 가져오는 도구. Node 설치할 때 같이 깔린다.
- 2 package.json 은 프로젝트 명세서. npm init -y 로 만든다.
- 3 npm i 이름 으로 설치하면 dependencies 에 자동으로 적힌다.
- 4 개발할 때만 쓰는 것은 npm i -D 이름 (devDependencies).
- 5 node_modules 는 Git 에 올리지 않는다. package.json 만 있으면 다시 만들 수 있다.
- 6 package-lock.json 은 정확한 버전을 못 박아 "내 컴퓨터에선 되는데" 를 막는다.
- 7 scripts 에 이름을 붙여 두면 npm run 이름 으로 짧게 실행한다.

직접 해보기 정답

```
1) console.log(pkg.license); // 출력: MIT
   console.log(pkg.author); // 출력: 우리학원
```

→ license 는 "이 코드를 남이 어떻게 써도 되는가" 입니다.

MIT 는 "마음대로 쓰세요" 에 가깝습니다. 수업용이라 신경 안 써도 됩니다.

- 2 mkdir npm연습 (또는 탐색기에서 폴더 만들기) cd npm연습 npm init -y → package.json 이 생기고, name 은 폴더 이름이 됩니다.
- 3 npm i dayjs → package.json 에 dependencies 항목이 새로 생기고 그 안에 dayjs 가 들어갑니다.

node_modules 폴더와 package-lock.json 도 함께 생깁니다.

- 4 node_modules 안에 dayjs 폴더가 있고, 그 안에 실제 코드 파일들이 있습니다. 폴더가 몇 개인지도 세어 보세요. dayjs 는 혼자라 몇 개 없지만, express 를 설치하면 수십 개가 달려 옵니다. 패키지가 또 다른 패키지를 쓰기 때문입니다.

- 5 npm run hello → 스크립트로 실행됐습니다

package.json 의 scripts.hello 에 적힌 명령이 실행된 것입니다.

에러 메시지 읽는 법

RUN node 개념04_에러_읽는_법.js

백엔드를 시작하면 에러를 훨씬 자주 만납니다. 브라우저에서는 화면이 안 바뀌는 정도였지만, 서버에서는 프로그램이 통째로 죽습니다.

그런데 좋은 소식이 있습니다. 에러 메시지에는 **무엇이 잘못됐고 어디서 잘못됐는지**가 전부 적혀 있습니다.

초보자와 익숙한 사람의 차이는 “에러를 안 내는 것”이 아닙니다. 에러를 읽고 5초 만에 원인을 찾느냐, 30분을 헤매느냐의 차이입니다.

이 파일은 그 읽는 법만 다룹니다. 이 단원에서 가장 실용적인 파일입니다.

에러는 세 부분으로 되어 있다

섹션 1

에러를 일부러 하나 내 보고, 그 안을 뜯어 봅시다. (try/catch 로 잡았기 때문에 프로그램이 안 죽습니다. JS자료 12단원에서 배웠죠)

```
try {
  const user = null;
  console.log(user.length);
} catch (error) {
```

① 에러의 '종류'

```
console.log(error.name);
```

출력 TypeError

② 사람이 읽는 '설명'

```
console.log(error.message);
```

출력 Cannot read properties of null (reading 'length')

```
}
```

터미널에 빨강게 뜨는 것은 이 둘을 합친 모양입니다.

```
TypeError: Cannot read properties of null (reading 'length')
```


종류

설명

③ 세 번째는 '어디서' 입니다. `error.stack` 에 들어 있습니다. 아래 섹션 2에서 봅니다.

⇒ 직접 해보기

1

`try` 안의 `user` 를 `undefined` 로 바꿔 보세요. `error.message` 가 어떻게 달라지나요?

스택 — 어디서 났는지

섹션 2

```
try {
  const user = null;
  console.log(user.length);
} catch (error) {
  const firstLine = error.stack.split("\n")[1];

  console.log(firstLine.includes("개념04_에러_읽는_법.js"));
}
```

출력 true

스택의 둘째 줄에 '파일 이름' 이 들어 있습니다.

★ 맥에서는 `false` 가 나올 수 있습니다. 맥이 한글 파일 이름을 자소를 나눠(NFD) 저장해서, 눈에는 같아 보여도 컴퓨터에게는 다른 문자열이기 때문입니다. 윈도우에서는 `true` 입니다.

```
}
```

실제 스택은 이렇게 생겼습니다. 여러 줄입니다.

```
TypeError: Cannot read properties of null (reading 'length')
    at Object.<anonymous> (C:\...\개념04_에러_읽는_법.js:55:22)
    at Module._compile (node:internal/modules/cjs/loader:1234:14)
    at Module._load (node:internal/modules/cjs/loader:1012:12)
    ... 계속
```

★ 읽는 요령 딱 하나

"내 파일 이름이 나오는 첫 줄" 만 보세요.

위 예에서는 `개념04_에러_읽는_법.js:55:22` 입니다. 55번째 줄, 22번째 글자에서 났다는 뜻입니다. 거기로 가면 됩니다.

node:internal/... 로 시작하는 줄들은 Node 내부입니다. 볼 필요 없습니다. node_modules 가 들어간 줄도 대개 볼 필요 없습니다.

처음에는 빨간 글씨가 20줄쯤 쏟아져서 겁이 납니다. 그중 볼 것은 맨 윗줄 하나와 내 파일이 나오는 줄 하나, 딱 둘입니다.

▫ 직접 해보기 2

위 try 안에서 error.stack 을 통째로 찍어 보세요. 몇 줄이 나오고, 그중 내 파일은 몇 번째 줄인가요?

자주 만나는 에러 다섯 가지

섹션 3

백엔드에서 만나는 에러의 90%는 이 다섯입니다. 하나씩 일부러 내 보면서 메시지를 눈에 익히세요.

[1] ReferenceError — 없는 이름을 불렀다

```
try {
  console.log(존재하지않는변수);
} catch (error) {
  console.log(`${error.name}: ${error.message}`);
}
```

출력 ReferenceError: 존재하지않는변수 is not defined

```
}
```

원인: 오타이거나, 만들기 전에 썼거나, require 를 빼먹었습니다. 찾는 법: 그 이름을 파일에서 검색(Ctrl+F) 해 보세요. 한 번만 나오면 오타입니다.

[2] TypeError — 없는 것에서 뭔가를 꺼내려 했다

```
try {
  const data = null;
  console.log(data.name);
} catch (error) {
  console.log(`${error.name}: ${error.message}`);
}
```

출력 TypeError: Cannot read properties of null (reading 'name')

```
}
```

원인: 값이 null 이나 undefined 인데 점을 찍었습니다. ★ 백엔드에서 가장 많이 보는 에러입니다.

데이터베이스에서 못 찾았을 때, 요청에 값이 안 왔을 때 계속 납니다. 찾는 법: 점 왼쪽의 값을 console.log 로 찍어 보세요. 왜 비었는지가 진짜 원인입니다.

[3] TypeError — 함수가 아닌 것을 함수처럼 불렀다

```
try {
  const list = [1, 2, 3];
  list.push2(4);
} catch (error) {
  console.log(`${error.name}: ${error.message}`);
}
```

출력 `TypeError: list.push2 is not a function`

```
}
```

원인: 메소드 이름 오타이거나, 그 자료형에 없는 메소드입니다. 찾는 법: 메시지에 이름이 그대로 나옵니다.
push2 → push 로 고치면 됩니다.

[4] Cannot find module — 파일이나 패키지를 못 찾았다

```
try {
  require("./없는파일");
} catch (error) {
  console.log(error.message.split("\n")[0]);
}
```

출력 `Cannot find module './없는파일'`

```
}
```

원인 세 가지 중 하나입니다. · 경로가 틀렸다 → ./를 빼먹었거나 폴더가 다릅니다 · npm install 을 안 했다
→ node_modules 에 없습니다 · 파일 이름 오타 찾는 법: ./가 있으면 파일을, 없으면 package.json 을
확인하세요.

[5] SyntaxError — 문법이 깨졌다

```
try {
  JSON.parse("{잘못된}");
} catch (error) {
  console.log(error.name);
}
```

출력 `SyntaxError`

```
}
```

원인: 괄호·따옴표 짝이 안 맞거나, JSON 형식이 틀렸습니다. ★ 코드 자체의 SyntaxError 는 특별합니다.
파일을 아예 못 읽어서 **한 줄도 실행되지 않습니다**. console.log 를 넣어도 안 찍힙니다. 그럴 땐 문법부터
의심하세요.

[6] 예고 — EADDRINUSE 04단원에서 서버를 만들면 이걸 만납니다.

```
Error: listen EADDRINUSE: address already in use :::3000
```

“3000번 포트를 이미 누가 쓰고 있다”는 뜻입니다. 앞서 켜 둔 서버를 안 끄고 또 켜올 때 납니다. 그 터미널에서 Ctrl+C 로 끄면 됩니다.

▹ 직접 해보기 3

위 다섯 가지를 try/catch 없이 그냥 써 보고, 터미널에 실제로 어떻게 빨강게 나오는지 눈으로 보세요. (확인했으면 지우세요. 안 지우면 뒤 코드가 실행 안 됩니다)

에러가 나도 서버는 안 죽게

섹션 4

서버는 하루 종일 켜져 있어야 합니다. 요청 하나가 잘못됐다고 서버 전체가 죽으면 안 됩니다.

```
function 안전하게계산(value) {
  try {
    return value.toFixed(2);
  } catch (error) {
    console.log("계산 실패:", error.message);
    return "계산할 수 없음";
  }
}

console.log(안전하게계산(3.14159));
```

출력 3.14

```
console.log(안전하게계산(null));
```

출력 계산 실패: Cannot read properties of null (reading 'toFixed')

계산할 수 없음

두 번째 호출에서 에러가 났는데도 프로그램이 안 죽고 계속 돌립니다. 이게 try/catch 를 쓰는 이유입니다.

주의 · 그렇다고 모든 곳을 try/catch 로 감싸지는 마세요.

에러를 잡아 놓고 아무 말도 안 하면, 뭐가 잘못됐는지 영영 모르게 됩니다. 반드시 `console.log` 로 남기거나, 사용자에게 알려 주세요. 04단원에서 Express 의 에러 처리 방법을 따로 배웁니다.

▹ 직접 해보기 4

안전하게계산("문자열") 을 넣으면 어떻게 될까요? 먼저 예상하고 실행해 보세요.

console.log 로 원인 좁히기

섹션 5

에러 메시지만으로 안 풀릴 때가 있습니다. 그때 쓰는 방법입니다.

요령 1 · 이름표를 붙여 찍으세요

```
console.log(value);           ← 여러 개 찍으면 뭐가 뭔지 모릅니다
console.log("계산 전 value:", value); ← 이렇게
```

```
const order = { id: 1, items: ["라떼"] };

console.log("주문 확인:", order);
```

```
출력      주문 확인: { id: 1, items: [ '라떼' ] }
```

요령 2 · 점 왼쪽을 먼저 찍으세요

order.customer.name 에서 에러가 났다면 order 를 찍고, order.customer 를 찍어 보세요. 어디서 비는지 바로 보입니다.

```
console.log(order.customer);
```

```
출력      undefined
```

여기가 비어 있으니 order.customer.name 은 TypeError 가 납니다.

요령 3 · 자료형을 의심하세요

```
console.log(typeof order.id, typeof order.items);
```

```
출력      number object
```

숫자인 줄 알았는데 문자열인 경우가 정말 많습니다. 특히 터미널 인자, 요청으로 들어온 값은 전부 문자열입니다.

요령 4 · 여기까지 왔는지 확인하세요

```
console.log("여기1"); ... console.log("여기2");
```

어디까지 찍히는지 보면 어느 줄에서 멈췄는지 알 수 있습니다. 원시적이지만 가장 빠른 때가 많습니다.

▸ 직접 해보기 5

order 에 customer 를 추가한 뒤 order.customer.name 을 안전하게 꺼내 보세요. (JS자료 07단원의 ?.)

★ 아래에서 `SyntaxError` 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요.

이런 실수를 합니다

에러를 안 읽고 코드부터 고침

가장 많이 하는 실수입니다. 메시지에 답이 적혀 있는데 안 봅니다. 빨간 글씨가 뜨면 **맨 윗줄부터 소리 내어 읽으세요.**

이런 실수를 합니다

스택 전체를 붙여 놓고 “에러가 나요” 라고 물어봄

맨 윗줄과 내 파일이 나오는 줄만 보면 됩니다. 질문할 때도 그 두 줄과 해당 코드를 보여 주는 게 훨씬 빠릅니다.

이런 실수를 합니다

catch 에서 아무것도 안 함

```
try { ... } catch (e) {}
```

에러가 조용히 사라집니다. 나중에 원인을 절대 못 찾습니다. 최소한 `console.log(e.message)` 는 남기세요.

이런 실수를 합니다

SyntaxError 인데 `console.log` 를 넣어 찾으려 함

문법 에러는 파일을 아예 못 읽습니다. `console.log` 가 안 찍힙니다. “아무것도 안 찍힌다” 면 문법부터 보세요. VS Code 가 빨간 밑줄로 알려 줍니다.

이런 실수를 합니다

터미널을 안 보고 브라우저만 봄

백엔드 에러는 **터미널에 나옵니다.** 브라우저 화면은 그냥 멈춰 있습니다. 서버를 만들면 터미널을 항상 한쪽에 띄워 두세요.

정리

- 1 에러는 종류(name) + 설명(message) + 위치(stack) 세 부분이다.
- 2 스택은 '내 파일 이름이 나오는 첫 줄' 만 보면 된다. 줄 번호가 적혀 있다.
- 3 ReferenceError = 없는 이름 / TypeError = `null·undefined` 에 점 찍음
- 4 Cannot find module = 경로 오타이거나 npm install 을 안 했다.
- 5 SyntaxError 는 파일 전체가 안 돌아간다. 아무것도 안 찍히면 문법을 의심한다.
- 6 try/catch 로 잡되, catch 를 비워 두지 않는다.
- 7 막히면 점 왼쪽을 찍어 보고, 자료형을 의심하고, 이름표를 붙여 찍는다.
- 8 백엔드 에러는 브라우저가 아니라 터미널에 나온다.

직접 해보기 정답

```
1) const user = undefined;
```

→ TypeError: Cannot read properties of `undefined` (reading 'length')

`null` 이 `undefined` 로 바뀌었을 뿐 나머지는 같습니다.
둘 다 "값이 없는데 점을 찍었다" 는 같은 원인입니다.

```
2) console.log(error.stack);
```

→ 보통 8~12줄쯤 나옵니다.

```
1번째 줄: 에러 종류와 설명
2번째 줄: 내 파일과 줄 번호 ← 여기만 보면 됩니다
3번째 줄부터: node:internal/... (Node 내부. 볼 필요 없음)
```

3) 예를 들어 이렇게 써 보세요.

```
console.log(없는변수);
```

→ 터미널에 빨간 글씨가 쏟아지고 프로그램이 거기서 멈춥니다.

그 아래에 있던 코드는 실행되지 않습니다.
`try/catch` 가 있고 없고의 차이를 눈으로 확인하는 것이 목적입니다.

```
4) console.log(안전하게계산("문자열"));
// 출력: 계산 실패: value.toFixed is not a function
// 출력: 계산할 수 없음
```

→ 문자열에는 toFixed 가 없어서 TypeError 가 나고, catch 가 잡았습니다.

에러 종류는 달라도 처리 방식은 같습니다.

```
5) const order2 = { id: 1, items: ["라떼"], customer: { name: "김민준" } };
console.log(order2.customer?.name);      // 출력: 김민준
console.log(order.customer?.name);      // 출력: undefined
```

→ ?. 를 쓰면 customer 가 없어도 에러 없이 undefined 가 나옵니다.

서버에서 들어온 데이터를 다룰 때 이 습관이 에러를 크게 줄여 줍니다.

Node.js 시작하기

RUN node 연습문제.js

01

- 1 TODO 자리에 코드를 씁니다.
- 2 저장하고 node 연습문제.js 로 실행합니다.
- 3 '기대 출력'과 똑같이 나오면 정답입니다.
- 4 10분 고민해도 안 되면 연습문제_정답.js 를 보세요.

1~12는 기본, 13은 응용, 14는 [도전], 15는 에러 확인입니다.

문제 1 (개념01)

브라우저에는 있지만 Node 에는 없는 것 세 가지의 자료형을 한 줄에 나란히 출력하세요. (window, document, alert 순서)

기대 출력 undefined undefined undefined

여기에 코드를 쓰세요

문제 2 (개념01)

자바스크립트 문법은 그대로라는 것을 확인해 봅시다. 아래 배열에서 3000원 이상인 것만 골라 출력하세요.

```
const prices = [1500, 4000, 2500, 6000];
```

기대 출력

— 4000, 6000

여기에 코드를 쓰세요

문제 3 (개념01)

process.platform 이 문자열인지 확인해서 true/false 로 출력하세요. (값 자체는 컴퓨터마다 달라서 확인만 합니다)

기대 출력 true

여기에 코드를 쓰세요

문제 4 (개념01)

터미널에서 넘어온 값만 골라 담아 출력하세요. 아무것도 안 붙이고 실행하면 빈 배열이 나옵니다.

힌트: process.argv 의 앞 두 개는 Node 위치와 파일 위치입니다. 잘라내세요.

기대 출력

[]

(확인이 끝나면 node 연습문제.js 커피 3 으로도 실행해 보세요)

여기에 코드를 쓰세요

문제 5 (개념02)

같은 폴더의 계산기.js 를 통째로 가져와 calc 에 담고, TAX_RATE 를 출력하세요.

힌트: 내가 만든 파일은 ./ 로 시작합니다.

기대 출력 0.1

여기에 코드를 쓰세요

문제 6 (개념02)

이번엔 구조분해로 add 와 addTax 만 꺼내 와서 add(100, 200) 과 addTax(20000) 을 차례로 출력하세요.

기대 출력 300
22000

여기에 코드를 쓰세요

문제 7 (개념02)

계산기.js 에서 내보내지 않은 SECRET_KEY 를 꺼내 출력하세요. 에러가 날까요, 다른 게 나올까요? 먼저 예상해 보세요.

내 예상

기대 출력 undefined

여기에 코드를 쓰세요

문제 8 (개념02)

Node 내장 모듈 `path` 를 가져와, "uploads" "2026" "report.pdf" 세 조각을 이어 붙인 경로를 출력하세요.

힌트: 내장 모듈은 `.` 없이 이름만 씁니다.

기대 출력 `uploads\2026\report.pdf`

(윈도우 기준입니다. 맥·리눅스는 `uploads/2026/report.pdf` 로 나옵니다)

여기에 코드를 쓰세요

문제 9 (개념03)

이 폴더의 `package.json` 을 `require` 로 읽어서 `name` 과 `version` 을 한 줄에 나란히 출력하세요.

기대 출력 `01-node-start 1.0.0`

여기에 코드를 쓰세요

문제 10 (개념03)

`package.json` 의 `scripts` 에 어떤 이름들이 있는지 배열로 출력하세요.

힌트: 객체의 이름(키)만 모으는 것은 JS자료 07단원의 `Object.keys` 입니다.

기대 출력

— 'start', 'hello'

여기에 코드를 쓰세요

01

문제 11 (개념03)

설치되어 있는 dayjs 패키지를 가져와, "2026-03-05" 를 "2026년 3월 5일" 형태로 바꿔 출력하세요.

힌트: `dayjs("날짜").format("YYYY년 M월 D일")`

기대 출력 2026년 3월 5일

여기에 코드를 쓰세요

문제 12 (개념04)

없는 변수를 부르는 코드를 try 로 감싸고, catch 에서 에러의 '종류' 만 출력하세요.

힌트: `error.name` 입니다.

기대 출력 ReferenceError

여기에 코드를 쓰세요

문제 13 [응용]

아래 주문 정보에서 고객 도시를 꺼내 출력하는 코드를 쓰세요. 그런데 order2 에는 customer 가 아예 없습니다. 어느 쪽이든 에러 없이 동작해야 하고, 없으면 "정보 없음" 이 나와야 합니다.

힌트: JS자료 07단원의 ?. 와 03단원의 || 를 함께 씁니다. 한 줄로 됩니다.

```
const order1 = { id: 1, customer: { city: "부산" } };
const order2 = { id: 2 };
```

기대 출력 부산
 정보 없음

여기에 코드를 쓰세요

문제 14 [도전]

어떤 값이든 받아 소수 둘째 자리까지 문자열로 돌려주는 safeFixed 함수를 만드세요.

숫자면 → "3.14" 처럼 돌려준다 숫자가 아니면 → 에러를 잡아서 "계산 불가" 를 돌려준다 그리고 어느 쪽이든 프로그램이 죽으면 안 된다

힌트: toFixed 는 숫자에만 있습니다. try/catch 로 감싸세요.

catch 에서 아무 말도 안 하면 안 됩니다. 콘솔에 이유를 남기세요.

기대 출력 3.14
 실패 이유: value.toFixed is not a function
 계산 불가
 실패 이유: Cannot read properties of null (reading 'toFixed')
 계산 불가

여기에 코드를 쓰세요

(safeFixed(3.14159), safeFixed("문자열"), safeFixed(null) 을 차례로 출력)

문제 15 (에러 확인 - 맨 마지막)

아래 줄의 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.
(에러가 나면 프로그램이 거기서 멈춥니다. 그래서 맨 뒤에 있습니다)

```
const wrong = require("계산기");
```

무슨 에러가 났나요? 무엇을 빠뜨린 걸까요?

■ 답

계산기 — 개념02에서 '가져다 쓰는' 파일

이 파일은 직접 실행하지 않습니다.

개념02_모듈_불러오기.js 가 여기서 필요한 것을 꺼내 씁니다.

이렇게 '기능을 따로 담아 두고 필요할 때 꺼내 쓰는 파일'을 모듈(module)이라고 부릅니다.

```
const TAX_RATE = 0.1;

function add(a, b) {
  return a + b;
}

function addTax(price) {
  return Math.round(price * (1 + TAX_RATE));
}
```

이 파일 안에서만 쓰는 값입니다. 밖으로 안 내보냅니다.

```
const SECRET_KEY = "밖으로 안 나갑니다";

function keyLength() {
  return SECRET_KEY.length;
}
```

내보내기

module.exports 에 담은 것만 밖에서 쓸 수 있습니다. 담지 않은 것(SECRET_KEY)은 이 파일 안의 비밀입니다.

```
module.exports = {
  TAX_RATE,
  add,
  addTax,
  keyLength,
};
```

{ TAX_RATE, add, ... } 는 { TAX_RATE: TAX_RATE, add: add, ... } 의 줄임입니다. 이름과 값이 같으면 한 번만 써도 됩니다. (JS자료 09단원 객체 속성 축약)

파일 다루기

OPEN 02_파일_다루기

```
const path = require("path");
console.log(ext === ".pdf");
console.log(newName);
console.log(typeof __dirname);
```

01 path, 경로 다루기

02 파일 읽고 쓰기 (fs)

03 동기과 비동기 (Sync 를 붙일 것인가)

04 폴더 다루기

05 JSON 파일로 데이터 저장하기

- 연습문제

path, 경로 다루기

RUN node 개념01_path_경로_다루기.js

백엔드가 하는 일의 절반은 '파일 다루기' 입니다. 사용자가 올린 문서를 저장하고 저장한 문서를 찾아서 읽고 필요 없어지면 지웁니다

그런데 파일을 다루려면 먼저 '어디에 있는지' 를 정확히 말할 수 있어야 합니다. 그게 경로(path)입니다. 그래서 이 단원은 경로부터 시작합니다.

왜 직접 문자열로 안 쓰나

이렇게 쓰면 안 될까요?

```
const 파일경로 = "uploads/2026/보고서.pdf";
```

문제는 운영체제마다 구분자가 다르다는 것입니다.

윈도우	uploads\2026\보고서.pdf	(역슬래시)
맥·리눅스	uploads/2026/보고서.pdf	(슬래시)

내 컴퓨터(윈도우)에서 잘 되던 코드가 EC2(리눅스)에 올리는 순간 파일을 못 찾습니다.

그래서 Node 는 path 라는 내장 모듈을 줍니다. 설치할 필요 없습니다. Node 에 이미 들어 있습니다.

```
const path = require("path");
```

join — 경로 조각 이어 붙이기

섹션 1

```
console.log(path.join("uploads", "2026", "보고서.pdf"));
```

```
출력 uploads\2026\보고서.pdf
```

윈도우라서 \ 로 이어졌습니다. 리눅스에서 실행하면 / 로 이어집니다. 우리는 "이어 붙여라" 라고만 말했고, 구분자는 path 가 알아서 골랐습니다.

슬래시를 몇 개를 쓰든 알아서 정리해 줍니다.

```
console.log(path.join("uploads/", "/2026/", "a.pdf"));
```

```
출력 uploads\2026\a.pdf
```

직접 문자열을 더했다면 uploads//2026//a.pdf 가 됐을 겁니다.

.. 는 “한 단계 위로” 라는 뜻입니다. 이것도 정리해 줍니다.

```
console.log(path.join("uploads", "..", "docs", "a.txt"));
```

출력 docs\a.txt

uploads 로 들어갔다가 다시 나와서 docs 로 들어간 것이라 uploads 가 사라졌습니다.

⇒ 직접 해보기

1

path.join("public", "images", "logo.png") 를 출력해 보세요.

경로에서 정보 꺼내기

섹션 2

```
const filePath = "/data/uploads/보고서.pdf";
```

— 파일 이름만

```
console.log(path.basename(filePath));
```

출력 보고서.pdf

— 확장자를 뺀 이름만

```
console.log(path.basename(filePath, ".pdf"));
```

출력 보고서

두 번째 인자로 “뺄 확장자” 를 넘기면 그것만 떼어 줍니다.

— 확장자만

```
console.log(path.extname(filePath));
```

출력 .pdf

점이 함께 나옵니다. “.pdf” 이지 “pdf” 가 아닙니다.

확장자가 없으면 빈 문자열입니다.

```
console.log(path.extname("README") === "");
```

출력 true

— 폴더 부분만

```
console.log(path.dirname("uploads/2026/a.pdf"));
```

출력 uploads/2026

한 번에 다 꺼내는 방법도 있습니다.

```
const parsed = path.parse(filePath);
console.log(parsed.base, parsed.name, parsed.ext);
```

출력 보고서.pdf 보고서 .pdf

▸ 직접 해보기

2

"회의록_2026.docx" 에서 확장자와 확장자 뺀 이름을 각각 출력해 보세요.

왜 이게 중요한가 — 파일 업로드

섹션 3

나중에 사용자가 파일을 올리면 이런 일을 해야 합니다.

① 올린 파일의 확장자를 확인한다 → .pdf 만 허용 ② 이름이 겹치지 않게 새 이름을 만든다 → 덮어쓰기 방지 ③ 저장할 경로를 만든다

지금 배운 것만으로 다 됩니다.

```
const uploadedName = "작업표준서.pdf";
```

① 확장자 확인

```
const ext = path.extname(uploadedName);
console.log(ext === ".pdf");
```

출력 true

② 이름이 겹치지 않게 (시간을 붙이는 방법이 흔합니다)

```
const baseName = path.basename(uploadedName, ext);
const newName = `${baseName}_사본${ext}`;
console.log(newName);
```

출력 작업표준서_사본.pdf

③ 저장 경로 만들기

```
console.log(path.join("uploads", "2026", newName));
```

출력 uploads\2026\작업표준서_사본.pdf

09단원에서 multer 로 진짜 파일 업로드를 만들 때 이 세 줄을 그대로 씁니다.

▸ 직접 해보기

3

“검사성적서.xlsx” 를 받아 “uploads/검사성적서_백업.xlsx” 경로를 만들어 출력해 보세요.

__dirname — 이 파일이 있는 폴더

섹션 4

아주 중요한 함정이 하나 있습니다.

경로를 “uploads/a.txt” 라고 쓰면 ‘터미널이 지금 있는 폴더’ 를 기준으로 찾습니다. (process.cwd())

그런데 서버를 실행하는 위치는 매번 다를 수 있습니다. 그러면 같은 코드가 어떤 날은 되고 어떤 날은 안 됩니다.

```
console.log(typeof process.cwd());
```

출력 string

터미널이 있는 폴더. 어디서 실행하느냐에 따라 바뀝니다.

```
console.log(typeof __dirname);
```

출력 string

★ 이 ‘파일’ 이 있는 폴더. 어디서 실행하든 항상 같습니다.

```
console.log(typeof __filename);
```

출력 string

이 파일의 전체 경로(파일 이름까지)

그래서 실무에서는 항상 __dirname 을 기준으로 씁니다.

```
const 저장폴더 = path.join(__dirname, "uploads");
```

이렇게 쓰면 어디서 node 를 실행하든 같은 폴더를 가리킵니다.

```
console.log(path.basename(__dirname));
```

출력 02_파일_다루기

지금 이 파일이 들어 있는 폴더 이름입니다.

```
console.log(path.basename(__filename));
```

직접 해보기

4

`__dirname` 과 `process.cwd()` 를 각각 통째로 찍어 보세요. 그다음 한 단계 위 폴더로 나가서(`cd ..`) 다시 실행해 보세요. 무엇이 바뀌고 무엇이 안 바뀌나요?

절대 경로와 상대 경로

섹션 5

절대 경로 : 맨 처음부터 다 적은 것. `C:\Users\...` 또는 `/home/...` 상대 경로 : 지금 위치를 기준으로 한 것. `uploads/a.txt`

코드	출력
<code>console.log(path.isAbsolute("uploads/a.txt"));</code>	▶ false
<code>console.log(path.isAbsolute("C:/data"));</code>	▶ true

★ 위 `true` 는 윈도우 기준입니다. `C:` 는 윈도우의 드라이브 표기라서 맥·리눅스에서 실행하면 `false` 가 나옵니다. (거기서는 `/` 로 시작해야 절대 경로)

`resolve` 는 상대 경로를 절대 경로로 바꿔 줍니다.

<code>console.log(path.isAbsolute(path.resolve("uploads")));</code>
출력 true

`join` 과 `resolve` 의 차이 `join` 조각을 이어 붙이기만 합니다. 상대 경로면 상대 경로 그대로. `resolve` 항상 절대 경로를 만들어 줍니다. (기준은 터미널 위치)

파일을 실제로 읽고 쓸 때는 절대 경로가 안전합니다. 그래서 `path.join(__dirname, ...)` 형태를 씁니다. `__dirname` 이 절대 경로라서요.

<code>console.log(path.isAbsolute(path.join(__dirname, "uploads")));</code>
출력 true

직접 해보기

5

`path.join(__dirname, "data", "설정.json")` 을 출력해 보세요.

자주 하는 실수

섹션 6

★ 아래에서 `SyntaxError` 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요.

이런 실수를 합니다

경로를 직접 이어 붙임

```
const p = "uploads" + "/" + name;
```

윈도우에서는 되는데 리눅스 서버에 올리면 깨질 수 있습니다. 슬래시가 두 번 들어가거나 빠지는 일도 흔합니다. 무조건 path.join 을 쓰세요.

이런 실수를 합니다

확장자 비교에서 점을 빼먹음

```
console.log(path.extname("a.pdf") === "pdf");
```

출력 false

extname 은 ".pdf" 를 돌려줍니다. 점을 포함해서 비교해야 합니다.

이런 실수를 합니다

대소문자를 그대로 비교

```
console.log(path.extname("보고서.PDF") === ".pdf");
```

출력 false

사용자가 대문자로 올릴 수 있습니다. toLowerCase() 로 낮춰서 비교하세요.

```
console.log(path.extname("보고서.PDF").toLowerCase() === ".pdf");
```

출력 true

이런 실수를 합니다

__dirname 대신 상대 경로를 씀

```
fs.readFileSync("data.json")
```

터미널 위치가 바뀌면 파일을 못 찾습니다. "내 컴퓨터에선 되는데 서버에선 안 돼요" 의 단골 원인입니다. path.join(__dirname, "data.json") 으로 쓰세요.

이런 실수를 합니다

사용자가 준 파일명을 그대로 경로에 붙임

path.join("uploads", 사용자입력) 사용자가 ".././비밀.txt" 같은 걸 보내면 엉뚱한 폴더의 파일을 건드릴 수 있습니다. (경로 탈출 공격) 09단원에서 안전하게 처리하는 법을 배웁니다. 지금은 "사용자 입력을 그대로 경로에 쓰면 위험하다" 만 기억하세요.

정리

- 1 경로는 직접 이어 붙이지 말고 `path.join` 을 쓴다. 운영체제 차이를 없애 준다.
- 2 `basename(파일명)` / `extname(확장자, 점 포함)` / `dirname(폴더)` 으로 조각을 꺼낸다.
- 3 `extname` 은 `".pdf"` 다. 점을 빼먹지 말고, 대소문자는 낮춰서 비교한다.
- 4 `__dirname` 은 '이 파일이 있는 폴더'. 어디서 실행하든 안 바뀐다.
- 5 `process.cwd()` 는 '터미널이 있는 폴더'. 실행 위치에 따라 바뀐다.
- 6 파일을 실제로 다룰 때는 `path.join(__dirname, ...)` 형태를 쓴다.
- 7 사용자가 준 파일명을 검사 없이 경로에 붙이면 위험하다.

직접 해보기 정답

```
1 console.log(path.join("public", "images", "logo.png"));
  // 출력: public\images\logo.png

2 console.log(path.extname("회의록_2026.docx"));
  // 출력: .docx
  console.log(path.basename("회의록_2026.docx", ".docx"));
  // 출력: 회의록_2026
```

→ 밑줄과 숫자는 확장자와 상관없습니다. 마지막 점 뒤만 확장자입니다.

```
3 const name = "검사성적서.xlsx";
  const e = path.extname(name);
  const b = path.basename(name, e);
  console.log(path.join("uploads", `${b}_백업${e}`));
  // 출력: uploads\검사성적서_백업.xlsx

4 console.log(__dirname);
  console.log(process.cwd());
```

→ 같은 폴더에서 실행하면 둘이 같습니다.

한 단계 위로 나가서 `node 02_파일_다루기/개념01_....js` 로 실행하면
`__dirname` 은 그대로 `02_파일_다루기` 를 가리키고,
`process.cwd()` 는 위 폴더로 바뀝니다.
이 차이 때문에 상대 경로를 쓰면 파일을 못 찾게 됩니다.

```
5) console.log(path.join(__dirname, "data", "설정.json"));
  // 출력: (컴퓨터마다 다름) C:\Users\...\02_파일_다루기\data\설정.json
```


→ 앞부분은 컴퓨터마다 다르지만 뒤쪽 \data\설정.json 은 같습니다.

파일 읽고 쓰기 (fs)

RUN node 개념02_파일_읽고_쓰기.js

브라우저는 컴퓨터의 파일을 마음대로 못 읽습니다. 위험하니까요. 하지만 Node 는 읽고 쓰고 지울 수 있습니다. 서버니까요.

그 일을 하는 것이 fs 모듈입니다. (fs = file system, 파일 시스템) 설치할 필요 없습니다. Node 에 이미 들어 있습니다.

이 폴더에 샘플_문서.txt 를 미리 만들어 두었습니다. 열어 보고 오세요.

```
const fs = require("fs");
const path = require("path");
```

앞 단원에서 배운 대로 __dirname 을 기준으로 경로를 만듭니다. 이렇게 해야 어디서 실행하든 같은 파일을 가리킵니다.

```
const 샘플경로 = path.join(__dirname, "샘플_문서.txt");
```

파일 읽기

섹션 1

```
const text = fs.readFileSync(샘플경로, "utf-8");

console.log(typeof text);
```

출력 string

```
console.log(text.split("\n")[0]);
```

출력 작업표준서

첫 줄만 꺼냈습니다. 파일 전체가 하나의 긴 문자열로 들어옵니다.

빈 줄을 빼고 몇 줄인지 세어 봅시다.

```
const lines = text.split("\n").filter((line) => line !== "");
console.log(lines.length);
```

출력 4

```
console.log(lines[2]);
```

출력 담당: 김민준

읽어 온 것은 그냥 문자열입니다. 지금까지 배운 `split`, `filter`, `includes` 를 전부 쓸 수 있습니다.

```
console.log(text.includes("김민준"));
```

출력 true

▹ 직접 해보기 1

마지막 줄(작성일)만 꺼내어 출력해 보세요.

"utf-8" 을 빼먹으면

섹션 2

두 번째 인자를 안 주면 글자가 아니라 '바이트 덩어리' 가 나옵니다.

```
const raw = fs.readFileSync(샘플경로);

console.log(raw.constructor.name);
```

출력 Buffer

Buffer 는 "컴퓨터가 저장한 그대로의 0과 1" 입니다. 사람이 못 읽습니다.

찍어 보면 <Buffer ec9e91 ...> 처럼 알 수 없는 숫자가 나옵니다. 글자로 보려면 인코딩을 알려 줘야 합니다.

```
console.log(raw.toString("utf-8").split("\n")[0]);
```

출력 작업표준서

★ 규칙: 글자 파일을 읽을 때는 항상 "utf-8" 을 두 번째 인자로 주세요.

`fs.readFileSync(경로, "utf-8")` ← 글자로 읽기 (거의 항상 이것) `fs.readFileSync(경로)` ← 바이트 그대로 (이미지·PDF 등)

나중에 파일 업로드에서 PDF 나 이미지를 다룰 때는 utf-8 없이 읽습니다. 글자가 아니라 그냥 데이터 덩어리니까요.

▹ 직접 해보기 2

utf-8 없이 읽은 것을 그대로 `console.log` 해 보세요. 어떻게 생겼나요?

파일 쓰기

섹션 3

이 예제가 만드는 파일들은 매번 결과를 같게 하려고 먼저 지우고 시작합니다.

```
const 출력경로 = path.join(__dirname, "결과_보고서.txt");
const 로그경로 = path.join(__dirname, "결과_기록.txt");

[출력경로, 로그경로].forEach((p) => {
  if (fs.existsSync(p)) {
    fs.unlinkSync(p);
  }
});
```

writeFileSync 로 새 파일을 만듭니다.

```
fs.writeFileSync(출력경로, "첫 번째 줄입니다\n", "utf-8");

console.log(fs.existsSync(출력경로));
```

출력 true

파일이 진짜 생겼습니다. 탐색기에서 이 폴더를 열어 확인해 보세요.

```
console.log(fs.readFileSync(출력경로, "utf-8"));
```

출력 첫 번째 줄입니다

(파일 끝에 \n 이 있어서 빈 줄이 하나 더 찍힙니다)

★ 아주 중요 — writeFileSync 는 ‘덮어씹니다’

```
fs.writeFileSync(출력경로, "두 번째로 쓴 내용\n", "utf-8");

console.log(fs.readFileSync(출력경로, "utf-8").trim());
```

출력 두 번째로 쓴 내용

첫 번째 줄이 사라졌습니다. 파일을 통째로 새로 쓴 것입니다.

실수로 중요한 파일에 쓰면 원래 내용이 날아갑니다. 되돌릴 수 없습니다.

폴더가 없으면 에러가 납니다. 파일은 만들어 주지만 폴더는 안 만들어 줍니다. (폴더 만들기는 개념04에서 배웁니다)

▹ 직접 해보기

3

“메모.txt” 라는 파일에 자기 이름을 써 보세요. 그다음 탐색기에서 열어 확인하세요.

이어 쓰기

섹션 4

덮어쓰지 않고 뒤에 붙이려면 appendFileSync 입니다.

```
fs.appendFileSync(로그경로, "1번 접속\n", "utf-8");
fs.appendFileSync(로그경로, "2번 접속\n", "utf-8");
fs.appendFileSync(로그경로, "3번 접속\n", "utf-8");
```

```
const 기록 = fs.readFileSync(로그경로, "utf-8");
console.log(기록.trim().split("\n").length);
```

출력 3

```
console.log(기록.trim().split("\n")[2]);
```

출력 3번 접속

파일이 없으면 새로 만들어 주고, 있으면 뒤에 붙입니다.

```
writeFileSync    덮어쓰기    (설정 파일 저장 등)
appendFileSync   이어 쓰기    (접속 기록, 로그 등)
```

서버가 무슨 일을 했는지 기록을 남길 때 append 를 씁니다.

▹ 직접 해보기 4

로그경로에 "4번 접속" 을 한 줄 더 붙이고, 줄 수가 4가 되는지 확인해 보세요.

없는 파일을 읽으면

섹션 5

파일이 없으면 에러가 납니다. `undefined` 가 아닙니다.

```
console.log(fs.existsSync(path.join(__dirname, "없는파일.txt")));
```

출력 false

```
try {
  fs.readFileSync(path.join(__dirname, "없는파일.txt"), "utf-8");
} catch (error) {
  console.log(error.code);
}
```

출력 ENOENT

ENOENT = Error NO ENTry. "그런 파일이나 폴더가 없다" 는 뜻입니다. 파일을 다룰 때 가장 많이 보는 에러 코드입니다. 외워 두세요.

```
}
```

그래서 읽기 전에 두 가지 중 하나를 합니다.

① existsSync 로 먼저 확인한다 ② try/catch 로 감싼다

서버에서는 ②를 더 많이 씁니다. 확인하고 읽는 사이에 파일이 지워질 수도 있으니까요.

```
function 안전하게읽기(파일경로) {
  try {
    return fs.readFileSync(파일경로, "utf-8");
  } catch (error) {
    if (error.code === "ENOENT") {
      return null; // 파일이 없는 것은 에러라기보다 '없는 상태' 입니다
    }
    throw error; // 그 밖의 에러(권한 등)는 그대로 위로 넘깁니다
  }
}

console.log(안전하게읽기(path.join(__dirname, "없는파일.txt")));
```

출력 null

```
console.log(안전하게읽기(샘플경로).split("\n")[0]);
```

출력 작업표준서

▹ 직접 해보기

5

안전하게읽기 에 결과_기록.txt 경로를 넣어 보세요. null 이 나올까요, 내용이 나올까요?

파일 지우기

섹션 6

```
const 임시경로 = path.join(__dirname, "임시파일.txt");

fs.writeFileSync(임시경로, "곧 지워질 파일", "utf-8");
console.log(fs.existsSync(임시경로));
```

출력 true

```
fs.unlinkSync(임시경로);
console.log(fs.existsSync(임시경로));
```

출력 false

unlink 라는 이름이 낯설죠. delete 가 아닙니다. 리눅스에서 파일을 지우는 명령 이름이 unlink 라서 그대로 따랐습니다.

★ 휴지통으로 안 갑니다. 바로 사라집니다. 되돌릴 수 없습니다. 경로를 잘못 쓰면 엉뚱한 파일이 사라집니다. 항상 확인하고 지우세요.

▹ 직접 해보기 6

파일을 만들고, 있는지 확인하고, 지우고, 다시 확인하는 네 줄을 직접 써 보세요.

자주 하는 실수

섹션 7

★ 아래에서 `SyntaxError` 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요.

이런 실수를 합니다

"utf-8" 을 빼먹음

```
const t = fs.readFileSync(경로);
```

에러는 안 나지만 Buffer 가 나옵니다. `t.split(...)` 을 하려다 `TypeError` 가 납니다. "왜 글자가 이상하지?" 싶으면 이걸 의심하세요.

이런 실수를 합니다

상대 경로를 씀

```
fs.readFileSync("샘플_문서.txt", "utf-8");
```

터미널 위치에 따라 `ENOENT` 가 납니다. `path.join(__dirname, ...)` 으로 쓰세요. (개념01)

이런 실수를 합니다

writeFileSync 로 실수로 덮어씀

기존 파일에 쓰면 원래 내용이 통째로 사라집니다. 이어 쓰려던 거라면 `appendFileSync` 입니다.

이런 실수를 합니다

없는 폴더에 쓰려 함

```
fs.writeFileSync(path.join(__dirname, "없는폴더", "a.txt"), "내용");
```

`ENOENT` 가 납니다. 파일은 만들어 주지만 폴더는 안 만들어 줍니다. 폴더부터 만들어야 합니다. (개념04)

이런 실수를 합니다

Sync 를 안 붙임

```
const t = fs.readFile(경로, "utf-8");
```

`readFile(Sync 없음)` 은 비동기라서 결과를 바로 안 줍니다. `t` 에는 `undefined` 가 들어갑니다. 개념03에서 자세히 봅니다.

이런 실수를 합니다

지우기 전에 확인을 안 함

```
fs.unlinkSync(경로);
```

파일이 없으면 ENOENT 로 프로그램이 죽습니다. existsSync 로 확인하거나 try/catch 로 감싸세요.

정리

- 1 fs 는 Node 내장 모듈. 파일을 읽고 쓰고 지운다.
- 2 readFileSync(경로, "utf-8") — utf-8 을 빼면 Buffer 가 나온다.
- 3 writeFileSync 는 덮어쓰기, appendFileSync 는 이어 쓰기.
- 4 existsSync 로 있는지 확인한다. 없는 파일을 읽으면 ENOENT 에러.
- 5 ENOENT = 그런 파일이나 폴더가 없다. 가장 많이 보는 코드.
- 6 unlinkSync 로 지운다. 휴지통을 거치지 않는다.
- 7 경로는 항상 path.join(__dirname, ...) 으로 만든다.
- 8 폴더는 자동으로 안 만들어진다. 없으면 ENOENT.

직접 해보기 정답

```
1) const lines2 = text.split("\n").filter((line) => line !== "");
console.log(lines2[lines2.length - 1]);
// 출력: 작성일: 2026-03-05
```

→ at(-1) 을 써도 됩니다. lines2.at(-1)

```
2) console.log(fs.readFileSync(샘플경로));
// 출력: <Buffer ec 9e 91 ec 97 85 ...>  처럼  나옵니다
```

→ 16진수 숫자입니다. 한글 한 글자가 3바이트를 차지하는 것도 보입니다.

사람이 읽으라고 만든 게 아니라 컴퓨터가 저장한 그대로입니다.

```
3) fs.writeFileSync(path.join(__dirname, "메모.txt"), "김민준", "utf-8");
```


→ 폴더에 메모.txt 가 생깁니다. 확인했으면 지워도 됩니다.

4 `fs.appendFileSync(로그경로, "4번 접속\n", "utf-8");` `const 기록2 = fs.readFileSync(로그경로, "utf-8"); console.log(기록2.trim().split("\n").length);` // 출력: 4

5 내용이 나옵니다. 섹션 4에서 이미 만들었기 때문입니다.

```
console.log(안전하게읽기(로그경로).trim().split("\n")[0]);
// 출력: 1번 접속
```

```
6) const p = path.join(__dirname, "연습.txt");
fs.writeFileSync(p, "테스트", "utf-8");
console.log(fs.existsSync(p)); // 출력: true
fs.unlinkSync(p);
console.log(fs.existsSync(p)); // 출력: false
```

동기와 비동기 (Sync 를 붙일 것인가)

RUN node 개념03_동기와_비동기.js

앞 파일에서 쓴 것들을 다시 보세요.

```
fs.readFileSync
fs.writeFileSync
fs.existsSync
```

전부 Sync 로 끝납니다

Sync 가 없는 것도 있습니다. fs.readFile, fs.writeFile ... 이름만 다른 게 아니라 동작이 완전히 다릅니다.

Sync 붙음 → 다 끝날 때까지 '기다린다' Sync 없음 → 시켜 놓고 '먼저 다음 줄로 간다'

이 차이가 서버에서는 아주 중요합니다. 이 파일에서 그 이유를 봅니다.

★ 결론부터 — 서버 코드에서는 Sync 를 쓰지 않습니다. 왜 그런지 눈으로 확인하는 것이 이 파일의 목적입니다.

```
const fs = require("fs");
const path = require("path");

const 샘플경로 = path.join(__dirname, "샘플_문서.txt");
```

Sync — 다 읽을 때까지 멈춘다

섹션 1

```
console.log("A. 읽기 전");

const 내용 = fs.readFileSync(샘플경로, "utf-8");

console.log("B. 읽기 끝. 여기 오려면 파일을 다 읽어야 했습니다");
console.log(내용.split("\n")[0]);
```

출력 A. 읽기 전
 B. 읽기 끝. 여기 오려면 파일을 다 읽어야 했습니다
 작업표준서

순서가 A → B 입니다. 당연해 보이죠? 그런데 이 '당연함' 이 서버에서는 문제가 됩니다. 섹션 3에서 봅니다.

▸ 직접 해보기

1

A 와 B 사이에 `console.log("중간")` 을 넣어 보세요. 어디에 찍히나요?

비동기 — 시켜 놓고 먼저 간다

섹션 2

아래 코드는 전부 async 함수 안에 있습니다. CommonJS 에서는 파일 맨 바깥에서 await 를 쓸 수 없기 때문입니다. (JS자료 12단원에서 배운 규칙 — await 는 async 함수 안에서만)

```
async function main() {
  console.log("C. 비동기 읽기 시작");
```

await 를 안 붙이면 '시키기만' 합니다. 결과를 기다리지 않습니다.

```
const 약속 = fs.promises.readFile(샘플경로, "utf-8");

console.log("D. 아직 안 기다렸습니다. 파일은 지금 읽히는 중입니다");

console.log(약속.constructor.name);
```

출력 C. 비동기 읽기 시작
 D. 아직 안 기다렸습니다. 파일은 지금 읽히는 중입니다
 Promise

JS자료 12단원의 그 Promise 입니다. "나중에 값을 주겠다" 는 약속이죠. 진동벨을 받은 상태입니다. 커피는 아직 안 나왔습니다.

★★ 여기가 이 파일에서 가장 중요한 지점입니다 ★★

바로 아래 줄에서 await 를 만나면 main 이 '잠시 비켜섭니다'. 그 틈에 파일 맨 아래에 있는 `console.log("G. ...")` 가 먼저 실행됩니다. 그래서 실제 출력 순서에서 G 가 바로 여기에 끼어듭니다.

파일에 쓰인 순서와 실행되는 순서가 다릅니다. 처음 보면 반드시 헷갈립니다. 실행해서 눈으로 확인하세요.

출력 G. main 을 시켜 놓고 바로 여기로 왔습니다

```
const 값 = await 약속;
console.log("E. 이제 받았습시다");
console.log(값.split("\n")[0]);
```

출력 E. 이제 받았습시다
 작업표준서

보통은 한 줄로 씁니다. 약속을 따로 담아 둘 일이 별로 없어서요.

```
const 값2 = await fs.promises.readFile(샘플경로, "utf-8");
console.log(값2.split("\n")[1]);
```

출력 공정명: 조립 1라인

그래서 서버에서 왜 중요한가

섹션 3

서버는 여러 사람의 요청을 동시에 받습니다. 그런데 Node 는 요청을 처리하는 일꾼이 '한 명' 입니다.

— Sync 로 파일을 읽으면

김민준의 요청 → 파일 읽는 3초 동안 일꾼이 붙잡힘
 그동안 이서연·박지훈의 요청은 줄 서서 대기
 → 30명이 동시에 오면 마지막 사람은 90초를 기다립니다

— 비동기로 읽으면

김민준의 요청 → "파일 읽어 줘" 시켜 놓고 일꾼은 바로 다음 일로
 이서연·박지훈 요청도 즉시 접수
 파일이 다 읽히면 그때 이어서 처리
 → 30명이 와도 거의 동시에 처리됩니다

식당으로 비유하면 이렇습니다. Sync 주문받은 요리가 나올 때까지 종업원이 주방 앞에 서 있는 것 비동기 주문만 넣고 다음 손님 주문을 받으러 가는 것

종업원이 한 명이라면 어느 쪽이 나올지는 분명합니다.

Node 가 서버에 잘 맞는다는 말이 이 뜻입니다. 일꾼은 한 명이지만, 기다리는 일을 남에게 맡기고 계속 움직입니다.

여러 개를 한꺼번에

섹션 4

비동기의 진짜 장점은 여러 일을 겹쳐서 할 수 있다는 것입니다.

```
const 파일들 = [샘플경로, 샘플경로, 샘플경로];
```

하나씩 기다리기 · — 세 번을 차례로 기다립니다

```
const 순서대로 = [];
for (const p of 파일들) {
  const t = await fs.promises.readFile(p, "utf-8");
  순서대로.push(t.length);
}
console.log(순서대로.length);
```

출력 3

한꺼번에 · — 셋을 동시에 시켜 놓고 다 올 때까지만 기다립니다

```
const 동시에 = await Promise.all(파일들.map((p) => fs.promises.readFile(p, "utf-8")));
console.log(동시에.length);
```

출력 3

```
console.log(순서대로[0] === 동시에[0].length);
```

출력 true

결과는 같습니다. 걸린 시간만 다릅니다.

파일이 작아서 지금은 차이가 안 느껴집니다. 하지만 파일이 100개거나, 데이터베이스·AI 서버를 부르는 것이라면 순서대로는 100배, 한꺼번에는 1배입니다. (JS자료 12단원 심화에서 재 봤죠)

★ 서로 상관없는 일이면 Promise.all 을 쓰세요. 앞의 결과가 있어야 다음을 할 수 있으면 하나씩 await 해야 합니다.

쓰기도 비동기로

섹션 5

```
const 출력경로 = path.join(__dirname, "결과_비동기.txt");

await fs.promises.writeFile(출력경로, "비동기로 썼습니다\n", "utf-8");
const 확인 = await fs.promises.readFile(출력경로, "utf-8");
console.log(확인.trim());
```

출력 비동기로 썼습니다

```
await fs.promises.unlink(출력경로);
console.log(fs.existsSync(출력경로));
```

출력 false

fs.promises 아래에 있는 것들은 이름이 같습니다. Sync 만 떼 모양입니다.

```
fs.readFileSync → fs.promises.readFile
fs.writeFileSync → fs.promises.writeFile
fs.unlinkSync → fs.promises.unlink
fs.mkdirSync → fs.promises.mkdir
```

맨 위에서 이렇게 꺼내 쓰면 더 짧습니다.

```
const fsp = require("fs").promises;
await fsp.readFile(...)
```

에러 처리

섹션 6

비동기에서도 try/catch 가 그대로 동작합니다. await 를 감싸면 됩니다.

```
try {
  await fs.promises.readFile(path.join(__dirname, "없는파일.txt"), "utf-8");
} catch (error) {
  console.log(error.code);
}
```

출력 ENOENT

```
}
```

★ await 를 빼먹으면 try/catch 가 못 잡습니다. 에러가 나중에 터지는데 try 블록은 이미 끝나 있기 때문입니다. 비동기 함수를 부를 때는 await 를 붙였는지 항상 확인하세요.

```
console.log("F. 전부 끝");
```

출력 F. 전부 끝

```
}
```

```
main();
```

★ main() 을 부르면 어떻게 될까요? main 은 async 함수라서, 안에서 첫 await 를 만날 때까지만 쪽 실행되고 거기서 잠시 비켜섭니다. 그 순간 아래 줄이 실행됩니다. main 이 다 끝나기를 기다리지 않습니다.

```
console.log("G. main 을 시켜 놓고 바로 여기로 왔습니다");
```

(이 줄의 출력은 위 main 안, await 직전에 미리 적어 두었습니다. 실제로 찍히는 자리가 거기이기 때문입니다)

실제 출력 순서

A. 읽기 전 ← 동기 B. 읽기 끝 ← 동기 작업표준서 ← 동기 C. 비동기 읽기 시작 ← main 시작 D. 아직 안 기다렸습니다 ← 아직 await 전 Promise ← 아직 await 전 G. main 을 시켜 놓고... ← ★ 첫 await 를 만나 비켜선 순간 E. 이제 받았습니다 ← 파일이 다 읽혀 돌아옴 ... 나머지 F. 전부 끝

“쓰인 순서” 와 “실행 순서” 가 다릅니다. 이걸 이해하면 비동기의 절반은 끝난 것입니다.

자주 하는 실수

섹션 7

02

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요.

이런 실수를 합니다

await 를 빼먹음

```
const t = fs.promises.readFile(경로, "utf-8");  
console.log(t.length);
```

t 는 문자열이 아니라 Promise 입니다. length 는 undefined 입니다. “왜 undefined 지?” 싶으면 await 를 빼먹었는지 보세요.

이런 실수를 합니다

async 없이 await 를 씀 → SyntaxError

```
function load(){  
  const t = await fs.promises.readFile(경로);  
}
```

await 는 async 함수 안에서만 쓸 수 있습니다.

이런 실수를 합니다

파일 맨 바깥에서 await 를 씀 → SyntaxError

```
const t = await fs.promises.readFile(경로);
```

CommonJS 에서는 안 됩니다. async function main() 으로 감싸세요.

이런 실수를 합니다

Sync 와 비동기를 섞어 씀

```
const a = fs.readFileSync(경로1, "utf-8");  
const b = await fs.promises.readFile(경로2, "utf-8");
```

에러는 안 나지만 첫 줄에서 서버가 멈춥니다. 한 파일 안에서는 한쪽으로 통일하세요.

이런 실수를 합니다

forEach 안에서 await

```
파일들.forEach(async (p) => { await fs.promises.readFile(p); });
```

forEach 는 기다려 주지 않습니다. 전부 끝나기 전에 다음 줄로 갑니다. for...of 를 쓰거나 Promise.all 을 쓰세요. (JS자료 12단원에서 배운 그 실수입니다)

정리

- 1 Sync 가 붙으면 다 끝날 때까지 기다린다. 안 붙으면 시켜 놓고 먼저 간다.
- 2 서버에서는 Sync 를 쓰지 않는다. 일꾼이 한 명인데 붙잡히기 때문이다.
- 3 fs.promises.readFile 처럼 쓰고 await 를 붙인다.
- 4 await 는 async 함수 안에서만. CommonJS 는 맨 바깥에서 못 쓴다.
- 5 서로 상관없는 일은 Promise.all 로 한꺼번에.
- 6 try/catch 로 감싸되, await 를 빼먹으면 못 잡는다.
- 7 예외 — 서버가 처음 켜질 때 설정 파일을 읽는 것은 Sync 를 써도 된다. 아직 요청을 받기 전이라 붙잡힐 사람이 없기 때문이다.

직접 해보기 정답

- 1 `console.log("중간")` 을 A 와 B 사이에 넣으면 A → 중간 → B 순서로 찍힙니다. `readFileSync` 가 그 자리에서 멈춰 있다가 다 읽고 나서야 다음 줄로 갑니다. 이게 '동기' 입니다.
- 2 (섹션 4 참고) 파일 세 개를 각각 다른 파일로 바꿔서 Promise.all 과 for...of 의 결과가 같은지 확인해 보세요. 결과는 같고 걸리는 시간만 다릅니다.

더 해보기 · 아래를 파일 맨 아래에 넣고 실행해 보세요.

```
async function 순서확인() {
```

```
  console.log("1");
  await fs.promises.readFile(샘플경로, "utf-8");
  console.log("3");
```

```
}
순서확인();
console.log("2");
```


출력 순서: 1 → 2 → 3 await 를 만나는 순간 함수가 잠시 비켜서고, 바깥의 "2" 가 먼저 실행됩니다. 파일을 다 읽으면 멈췄던 자리로 돌아와 "3" 을 찍습니다. JS자료 12단원에서 배운 "await 는 이 함수 하나만 비켜서게 한다" 가 이것입니다.

폴더 다루기

RUN node 개념04_폴더_다루기.js

앞에서 파일을 읽고 썼습니다. 그런데 실제 서비스에서는 파일을 아무 데나 쌓아 두지 않습니다. 폴더로 정리합니다.

```
uploads/
  2026/
    03/
      작업표준서.pdf
      검사성적서.xlsx
```

이번 파일에서는 폴더를 만들고, 안을 들여다보고, 지우는 법을 배웁니다. 파일 업로드 서버를 만들 때 그대로 씁니다.

```
const fs = require("fs");
const path = require("path");
```

이 예제가 쓸 연습용 폴더입니다.

```
const 연습폴더 = path.join(__dirname, "연습_업로드");
```

매번 결과가 같도록 먼저 통째로 지우고 시작합니다. `force: true` 는 “없어도 에러 내지 말라” 는 뜻입니다.

```
fs.rmSync(연습폴더, { recursive: true, force: true });
```

폴더 만들기

섹션 1

```
fs.mkdirSync(연습폴더);

console.log(fs.existsSync(연습폴더));
```

```
출력      true
```

탐색기에서 이 폴더를 열어 보세요. 연습_업로드 폴더가 생겼습니다.

이미 있는 폴더를 또 만들면 에러가 납니다.

```
try {
  fs.mkdirSync(연습폴더);
} catch (error) {
  console.log(error.code);
}
```

출력 EEXIST

EEXIST = already EXISTS. "이미 있다" 는 뜻입니다.

```
}
```

그래서 보통 확인하고 만듭니다.

```
if (!fs.existsSync(연습폴더)) {
  fs.mkdirSync(연습폴더);
}
```

▹ 직접 해보기 1

"임시폴더" 를 만들고 existsSync 로 확인해 보세요.

여러 단계를 한 번에

섹션 2

uploads/2026/03 처럼 여러 단계를 만들려면 어떻게 할까요? 하나씩 만들 필요 없습니다. recursive 옵션을 주면 됩니다.

```
const 깊은경로 = path.join(연습폴더, "2026", "03");

fs.mkdirSync(깊은경로, { recursive: true });

console.log(fs.existsSync(깊은경로));
```

출력 true

2026 폴더도 03 폴더도 한 번에 만들어졌습니다.

recursive: true 는 보너스가 하나 더 있습니다. 이미 있어도 에러를 안 냅니다. EEXIST 가 안 납니다.

```
fs.mkdirSync(깊은경로, { recursive: true });
console.log("두 번 만들어도 에러가 안 났습니다");
```

출력 두 번 만들어도 에러가 안 났습니다

★ 그래서 실무에서는 거의 항상 이 형태를 씁니다.

```
fs.mkdirSync(저장폴더, { recursive: true });
```

확인할 필요도 없고, 몇 단계든 한 번에 만들어집니다.

▹ 직접 해보기 2

연습폴더 안에 "docs/pdf" 를 한 번에 만들어 보세요.

폴더 안 목록 보기

섹션 3

파일 몇 개를 넣어 봅시다.

```
fs.writeFileSync(path.join(연습폴더, "작업표준서.pdf"), "내용1", "utf-8");
fs.writeFileSync(path.join(연습폴더, "검사성적서.xlsx"), "내용22", "utf-8");
fs.writeFileSync(path.join(깊은경로, "3월보고서.pdf"), "내용333", "utf-8");

const 목록 = fs.readdirSync(연습폴더);

console.log(목록.sort());
```

출력 ['2026', '검사성적서.xlsx', '작업표준서.pdf']

세 가지를 눈여겨보세요. ① 폴더(2026)와 파일이 섞여 나옵니다 ② 이름만 나옵니다. 전체 경로가 아닙니다 ③ 안쪽 폴더(2026/03)의 파일은 안 나옵니다. 한 겹만 봅니다

sort() 를 붙인 이유 운영체제마다 순서가 다를 수 있습니다. 순서가 중요하면 직접 정렬하세요. (JS자료 08단원의 sort)

확장자로 걸러내는 것은 지금까지 배운 것으로 됩니다.

```
const pdf목록 = fs.readdirSync(연습폴더).filter((name) => path.extname(name) === ".pdf");
console.log(pdf목록);
```

출력 ['작업표준서.pdf']

▹ 직접 해보기 3

깊은경로 폴더의 목록을 출력해 보세요.

파일인가 폴더인가

섹션 4

목록에는 파일과 폴더가 섞여 나옵니다. 구분하려면 statSync 를 씁니다.

```
const 파일정보 = fs.statSync(path.join(연습폴더, "작업표준서.pdf"));

console.log(파일정보.isFile(), 파일정보.isDirectory());
```

출력 true false

```
const 폴더정보 = fs.statSync(path.join(연습폴더, "2026"));

console.log(폴더정보.isFile(), 폴더정보.isDirectory());
```

출력 false true

크기도 알 수 있습니다. '글자 수' 가 아니라 '바이트 수' 입니다.

```
console.log(파일정보.size);
```

출력 7

이 파일에 쓴 내용은 "내용1" — 세 글자입니다. 그런데 7바이트입니다.

```
내  → 3바이트
용  → 3바이트
1   → 1바이트
```

7바이트

utf-8 에서 한글은 한 글자에 3바이트, 영어와 숫자는 1바이트를 씁니다. 그래서 "글자 수 × 1 = 크기" 가 아닙니다.

나중에 "파일 크기 5MB 까지만 허용" 같은 제한을 걸 때 세는 단위가 바이트라는 것을 기억하세요. (09단원 multer)

수정 시각도 있습니다.

```
console.log(파일정보.mtime.constructor.name);
```

출력 Date

JS자료 14단원에서 배운 그 Date 객체입니다. `getFullYear()` 같은 걸 그대로 쓸 수 있습니다.

파일과 폴더를 한 번에 구분하는 더 편한 방법도 있습니다.

```
const 항목들 = fs.readdirSync(연습폴더, { withFileTypes: true });
const 구분 = 항목들.map((e) => `${e.name}:${e.isDirectory() ? "폴더" : "파일"}`).sort();

console.log(구분);
```

출력 ['2026:폴더', '검사성적서.xlsx:파일', '작업표준서.pdf:파일']

withFileTypes: true 를 주면 이름 대신 '정보가 담긴 객체' 가 나옵니다. statSync 를 하나씩 부르는 것보다 훨씬 빠릅니다.

▸ 직접 해보기 4

연습폴더에서 '폴더만' 골라 이름을 출력해 보세요.

지우기

섹션 5

파일 하나 · — 앞에서 배운 unlinkSync

```
fs.unlinkSync(path.join(연습폴더, "검사성적서.xlsx"));
console.log(fs.existsSync(path.join(연습폴더, "검사성적서.xlsx")));
```

출력 false

빈 폴더 · — rmdirSync

```
const 빈폴더 = path.join(연습폴더, "빈폴더");
fs.mkdirSync(빈폴더);
fs.rmdirSync(빈폴더);
console.log(fs.existsSync(빈폴더));
```

출력 false

안에 뭐가 든 폴더 · — rmSync 에 recursive

rmDirSync 로는 안 됩니다. 비어 있지 않다고 에러가 납니다.

```
console.log(fs.existsSync(깊은경로));
```

출력 true

```
fs.rmSync(path.join(연습폴더, "2026"), { recursive: true });
console.log(fs.existsSync(깊은경로));
```

출력 false

안에 있던 3월보고서.pdf 까지 통째로 사라졌습니다.

★★ 아주 위험합니다 ★★ rmSync 에 recursive: true 를 주면 그 아래를 전부 지웁니다. 경로를 잘못 쓰면 되돌릴 수 없습니다. 휴지통에도 안 갑니다.

실무에서는 지우기 전에 반드시 확인합니다.

- 경로를 `console.log` 로 찍어 보고
- 그 경로가 내가 만든 폴더 안인지 확인하고
- 그다음에 지웁니다

사용자가 보낸 값을 그대로 `rmSync` 에 넣는 일은 절대 하지 마세요.

▫ 직접 해보기 5

폴더를 만들고, 안에 파일을 하나 넣고, `rmdirSync` 로 지워 보세요. 무슨 에러가 나나요?

실전 — 업로드 폴더 준비하기

섹션 6

파일 업로드 서버가 시작할 때 하는 일을 그대로 만들어 봅시다. “오늘 날짜 폴더에 저장한다” 는 흔한 방식입니다.

```
function 저장폴더만들기(기준폴더, 날짜) {
  const 년 = 날짜.getFullYear();
  const 월 = String(날짜.getMonth() + 1).padStart(2, "0");

  const 대상 = path.join(기준폴더, String(년), 월);
```

없으면 만들고, 있으면 그냥 넘어갑니다

```
  fs.mkdirSync(대상, { recursive: true });

  return 대상;
}

const 오늘폴더 = 저장폴더만들기(연습폴더, new Date(2026, 2, 5));

console.log(path.basename(오늘폴더));
```

출력 03

```
console.log(path.basename(path.dirname(오늘폴더)));
```

출력 2026

2026/03 폴더가 만들어졌습니다. `getMonth()` 는 0부터라 +1 했습니다. (JS자료 14단원)

이제 여기에 파일을 저장하면 됩니다.

```
const 저장경로 = path.join(오늘폴더, "새문서.pdf");
fs.writeFileSync(저장경로, "업로드된 내용", "utf-8");

console.log(fs.existsSync(저장경로));
```

출력 true

09단원에서 multer 를 배우면 이 함수를 그대로 씁니다. 달라지는 것은 "파일 내용이 어디서 오느냐" 뿐입니다.

뒷정리 — 연습용이라 통째로 지웁니다.

```
fs.rmSync(연습폴더, { recursive: true, force: true });
console.log(fs.existsSync(연습폴더));
```

출력 false

▹ 직접 해보기 6

저장폴더만들기 에 new Date(2026, 11, 25) 를 넣으면 어떤 폴더가 만들어질까요? 먼저 예상하고 확인하세요.

자주 하는 실수

섹션 7

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요.

이런 실수를 합니다

폴더 없이 파일부터 쓰려 함

```
fs.writeFileSync(path.join(__dirname, "없는폴더", "a.txt"), "내용");
```

ENOENT 가 납니다. 파일은 만들어 주지만 폴더는 안 만들어 줍니다. 쓰기 전에 mkdirSync(폴더, { recursive: true }) 를 먼저 하세요.

이런 실수를 합니다

recursive 를 안 씀

```
fs.mkdirSync(path.join(a, "b", "c"));
```

b 가 없으면 ENOENT 입니다. 한 단계씩만 만들어 줍니다. { recursive: true } 를 붙이면 다 만들어 주고 이미 있어도 안전합니다.

이런 실수를 합니다

readdirSync 결과를 그대로 경로로 씀

```
const files = fs.readdirSync(폴더);
fs.readFileSync(files[0]);
```

readdirSync 는 '이름' 만 줍니다. 전체 경로가 아닙니다. path.join(폴더, files[0]) 로 합쳐야 합니다.

이런 실수를 합니다

안쪽 폴더까지 다 보일 거라 생각함

readdirSync 는 한 겹만 봅니다. 안쪽 폴더의 파일은 안 나옵니다. 전부 훑으려면 폴더를 만날 때마다 다시 들어가야 합니다.

이런 실수를 합니다

rmSync 를 함부로 씀

```
fs.rmSync(경로, { recursive: true, force: true });
```

경로가 틀리면 엉뚱한 폴더가 통째로 사라집니다. 특히 사용자 입력을 경로에 넣으면 절대 안 됩니다.

이런 실수를 합니다

폴더를 unlinkSync 로 지우려 함

```
fs.unlinkSync(폴더경로);
```

EPERM 이나 EISDIR 에러가 납니다. 파일은 unlink, 폴더는 rmdir 또는 rm 입니다.

정리

- 1 **mkdirSync**(경로, { recursive: true }) — 여러 단계를 한 번에, 있어도 안전.
- 2 **readdirSync**(폴더) — 한 겹의 이름 목록. 전체 경로가 아니다.
- 3 **withFileTypes: true** 를 주면 파일인지 폴더인지 바로 알 수 있다.
- 4 **statSync** 로 크기(size)와 수정 시각(mtime)을 알 수 있다.
- 5 파일은 **unlinkSync**, 빈 폴더는 **rmdirSync**, 내용 있는 폴더는 **rmSync + recursive**.
- 6 지우는 것은 되돌릴 수 없다. 경로를 반드시 확인하고 지운다.
- 7 파일을 쓰기 전에 폴더를 먼저 만든다. 폴더는 자동으로 안 생긴다.

직접 해보기 정답

```
1 const p = path.join(__dirname, "임시폴더");
  fs.mkdirSync(p, { recursive: true });
  console.log(fs.existsSync(p)); // 출력: true
  fs.rmSync(p, { recursive: true, force: true }); // 확인 후 정리

2 fs.mkdirSync(path.join(연습폴더, "docs", "pdf"), { recursive: true });
```

→ docs 와 pdf 가 한 번에 만들어집니다.

```
3) console.log(fs.readdirSync(깊은경로));
// 출력: [ '3월보고서.pdf' ]
```

→ 섹션 5에서 지우기 전에 실행해야 나옵니다.

```
4) const 폴더만 = fs.readdirSync(연습폴더, { withFileTypes: true })
```

```
.filter((e) => e.isDirectory())
.map((e) => e.name);
```

```
console.log(폴더만);
// 출력: [ '2026' ]
```

→ filter 와 map 은 JS자료 08단원에서 배운 그대로입니다.

```
5) const p = path.join(__dirname, "안빈폴더");
fs.mkdirSync(p, { recursive: true });
fs.writeFileSync(path.join(p, "a.txt"), "내용", "utf-8");
try {
```

```
fs.rmdirSync(p);
```

```
} catch (e) {
```

```
console.log(e.code); // 출력: ENOTEMPTY
```

```
}
fs.rmSync(p, { recursive: true, force: true });
```

→ ENOTEMPTY = NOT EMPTY. “비어 있지 않다” 는 뜻입니다.

안에 뭐가 있으면 rmdirSync 로는 못 지웁니다.

6) 2026/12 폴더가 만들어집니다. new Date(2026, 11, 25) 의 11 은 12월입니다. getMonth() 가 0부터라 +1 을 하면 12 가 되고, padStart 로 “12” 가 됩니다. (한 자리 달이면 “03” 처럼 앞에 0이 붙습니다)

JSON 파일로 데이터 저장하기

RUN node 개념05_JSON으로_데이터_저장.js

서버가 다루는 것은 파일만이 아닙니다. '데이터' 도 다룹니다.

누가 어떤 문서를 언제 올렸는가
그 문서의 제목과 담당자는 무엇인가

이런 것은 변수에 담아 두면 될까요? 안 됩니다. 서버를 껐다 켜면 변수는 전부 사라집니다.

그래서 어딘가에 '저장' 해야 합니다. 진짜 서비스에서는 데이터베이스를 씁니다. (PART 4 에서 배웁니다)
그 전에 JSON 파일로 먼저 해 봅시다. 원리가 똑같기 때문입니다.

JSON 을 다시 짚고 갑시다

JSON.stringify(값) 객체·배열 → 글자
JSON.parse(글자) 글자 → 객체·배열

JS자료 12단원에서 서버가 준 글자를 객체로 되돌릴 때 썼습니다. 여기서는 반대로, 내 객체를 글자로 바꿔 파일에 씁니다.

```
const fs = require("fs");
const path = require("path");

const 데이터파일 = path.join(__dirname, "문서목록.json");
```

매번 결과가 같도록 먼저 지우고 시작합니다.

```
if (fs.existsSync(데이터파일)) {
  fs.unlinkSync(데이터파일);
}
```

저장하기

섹션 1

```
const 문서들 = [
  { id: 1, title: "작업표준서", writer: "김민준", size: 240 },
  { id: 2, title: "검사성적서", writer: "이서연", size: 180 },
];
```

객체 배열을 그대로 파일에 쓰면 어떻게 될까요? 파일에는 '글자'만 쓸 수 있습니다. 그래서 먼저 글자로 바꿔야 합니다.

```
const 글자 = JSON.stringify(문서들);
console.log(typeof 글자);
```

출력 string

```
console.log(글자.slice(0, 30));
```

출력 [{"id":1,"title":"작업표준서","writ

한 줄로 쭉 이어진 글자입니다. 30글자만 잘라 봤는데도 벌써 답답합니다. 문서가 100개면 한 줄이 수천 글자가 됩니다. 사람이 열어 볼 수가 없습니다.

사람도 읽을 수 있게 줄바꿈과 들여쓰기를 넣을 수 있습니다.

```
const 예쁜글자 = JSON.stringify(문서들, null, 2);
console.log(예쁜글자.split("\n").length);
```

출력 14

```
JSON.stringify(값, null, 2)
      ──── ─
      (안 씀) 들여쓰기 칸 수
```

두 번째 자리는 지금 안 쓰니 `null`로 둡니다. 세 번째가 들여쓰기입니다. 보통 2를 씁니다.

```
fs.writeFileSync(데이터파일, 예쁜글자, "utf-8");
console.log(fs.existsSync(데이터파일));
```

출력 true

탐색기에서 문서목록.json을 열어 보세요. 사람이 읽을 수 있는 모양입니다.

▫ 직접 해보기 1

`JSON.stringify(문서들)`과 `JSON.stringify(문서들, null, 2)`를 각각 파일로 저장해 열어 보고 차이를 확인하세요.

읽기

섹션 2

```
const 읽은글자 = fs.readFileSync(데이터파일, "utf-8");
console.log(typeof 읽은글자);
```

출력 string

파일에서 읽으면 언제나 글자입니다. 아직 배열이 아닙니다.

```
const 읽은배열 = JSON.parse(읽은글자);
console.log(Array.isArray(읽은배열));
```

출력 true

이제 진짜 배열입니다. map, filter 를 그대로 쓸 수 있습니다.

코드	출력
<code>console.log(읽은배열.length);</code>	2
<code>console.log(읽은배열[0].title);</code>	작업표준서

★ 저장할 때 stringify, 읽을 때 parse. 이 한 쌍은 항상 짝입니다. JS자료 14단원 localStorage 에서 배운 것과 똑같습니다.

parse 를 빼먹으면 어떻게 되나

```
console.log(읽은글자.length);
```

출력 164

배열 길이 2 가 아니라 '글자 수' 가 나옵니다. 에러도 안 납니다. "왜 length 가 이상하지?" 싶으면 parse 를 빼먹었는지 보세요.

▹ 직접 해보기

2

읽은배열 에서 writer 만 뽑아 배열로 출력해 보세요.

파일이 없을 때

섹션 3

서버를 처음 켜를 때는 저장된 파일이 없습니다. 그냥 읽으면 ENOENT 로 서버가 죽습니다.

```
function 불러오기() {
  if (!fs.existsSync(데이터파일)) {
    return []; // 없으면 빈 목록으로 시작
  }
  return JSON.parse(fs.readFileSync(데이터파일, "utf-8"));
}

function 저장하기(목록) {
  fs.writeFileSync(데이터파일, JSON.stringify(목록, null, 2), "utf-8");
}

console.log(불러오기().length);
```

출력 2

이 두 함수가 이 파일의 핵심입니다. 앞으로 데이터를 다룰 때는 항상 이렇게 합니다.

- ① 불러오기()로 전부 읽는다
- ② 배열을 고친다 (지금까지 배운 map, filter, push)
- ③ 저장하기()로 전부 다시 쓴다

PART 4 에서 데이터베이스를 배우면 ①과 ③이 SQL 로 바뀔 뿐, ②의 생각은 똑같습니다.

▣ 직접 해보기 3

문서목록.json 을 손으로 지우고 불러오기() 를 실행해 보세요. 에러가 날까요, 빈 배열이 나올까요?

추가·수정·삭제

섹션 4

추가 · — id 를 어떻게 정할까요?

```
function 추가하기(title, writer) {
  const 목록 = 불러오기();
```

지금까지 쓴 id 중 가장 큰 것 + 1

```
const 다음id = 목록.length === 0 ? 1 : Math.max(...목록.map((d) => d.id)) + 1;

const 새문서 = { id: 다음id, title, writer, size: 0 };

저장하기([...목록, 새문서]); // 원본을 바꾸지 않고 새 배열로 (JS자료 09단원) return
새문서;
}

const 추가된것 = 추가하기("MSDS", "박지훈");
console.log(추가된것.id);
```

출력 3

```
console.log(불러오기().length);
```

출력 3

`Math.max(...배열)` 은 JS자료 09단원의 스프레드입니다. `Math.max` 는 배열을 통째로 못 받아서 펼쳐 넣습니다.

목록이 비었을 때를 따로 처리한 이유 `Math.max()` 를 빈 배열로 부르면 `-Infinity` 가 나옵니다. 그러면 `id` 가 `-Infinity + 1` 이 되어 버립니다.

수정 · — 하나만 바꾼 새 배열

```
function 수정하기(id, 바꿀내용) {
  const 목록 = 불러오기();
  const 새목록 = 목록.map((d) => (d.id === id ? { ...d, ...바꿀내용 } : d));
  저장하기(새목록);
}

수정하기(2, { writer: "최유진", size: 500 });

const 확인 = 불러오기().find((d) => d.id === 2);
console.log(확인.writer, 확인.size);
```

출력 최유진 500

```
console.log(확인.title);
```

출력 검사성적서

`title` 은 안 바뀌었습니다. `{ ...d, ...바꿀내용 }` 이 겹치는 것만 덮어썼습니다.

— 삭제

```
function 삭제하기(id) {  
  const 목록 = 불러오기();  
  저장하기(목록.filter((d) => d.id !== id));  
}  
  
삭제하기(1);  
console.log(불러오기().map((d) => d.id));
```

출력 [2, 3]

1번이 사라졌습니다. id 는 다시 매기지 않습니다. 그대로 둡니다.

— 찾기

```
const 찾은것 = 불러오기().find((d) => d.title === "MSDS");  
console.log(찾은것 ? 찾은것.writer : "없음");
```

출력 박지훈

```
const 못찾은것 = 불러오기().find((d) => d.title === "없는문서");  
console.log(못찾은것 ? 못찾은것.writer : "없음");
```

출력 없음

find 는 못 찾으면 undefined 입니다. 바로 점을 찍으면 TypeError 가 납니다. 반드시 확인하고 쓰세요.

⇒ 직접 해보기 4

문서를 하나 더 추가하고, 그 id 가 몇이 되는지 확인하세요.

이 방식의 한계 — 왜 데이터베이스가 필요한가

섹션 5

JSON 파일로도 저장은 됩니다. 그런데 실제 서비스에는 못 씁니다. 이유가 네 가지입니다. PART 4 에서 데이터베이스를 배우는 이유가 이것입니다.

① 동시에 두 사람이 오면 하나가 사라집니다

김민준이 읽고 → 이서연이 읽고 → 김민준이 쓰고 → 이서연이 씁니다
이서연은 김민준이 추가한 것을 모르는 상태로 덮어씁니다.
김민준의 문서가 흔적도 없이 사라집니다.

② 데이터가 커지면 느려집니다

문서 한 개를 고치려고 10만 개를 전부 읽고 전부 다시 씁니다.

③ 검색이 느립니다

"담당자가 김민준인 문서" 를 찾으려면 전부 훑어야 합니다.
데이터베이스는 색인(index)이 있어서 훨씬 빠릅니다.

④ 서버가 여러 대면 공유가 안 됩니다

1번 서버의 파일과 2번 서버의 파일이 서로 다른 내용이 됩니다.

그래도 이 단원을 하는 이유 데이터베이스도 결국 "읽고 → 고치고 → 저장" 입니다. 여기서 그 흐름을 손에 익히면 SQL 을 배울 때 훨씬 쉽습니다. 그리고 설정 파일이나 작은 목록에는 지금도 JSON 파일을 씁니다.

뒷정리

```
fs.unlinkSync(데이터파일);
console.log(fs.existsSync(데이터파일));
```

출력 false

➤ 직접 해보기 5

위 ①번 상황을 코드로 흉내 내 보세요. (힌트: 불러오기() 를 두 번 하고, 각각에 추가한 뒤 저장하기)

자주 하는 실수

섹션 6

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요.

이런 실수를 합니다

stringify 를 빼먹음

```
fs.writeFileSync(파일, 문서들, "utf-8");
```

배열이 "[object Object],[object Object]" 로 저장됩니다. 내용이 통째로 사라지고 되돌릴 수 없습니다.
JS자료 14단원 localStorage 에서 본 그 함정입니다.

이런 실수를 합니다

parse 를 빼먹음

```
const 목록 = fs.readFileSync(파일, "utf-8");
```

목록.filter(...) 문자열에도 filter 가 없어서 TypeError 가 납니다.

이런 실수를 합니다

파일이 없을 때를 생각 안 함

```
JSON.parse(fs.readFileSync(파일, "utf-8"))
```

처음 켄 때 ENOENT 로 서버가 죽습니다. existsSync 로 먼저 확인하세요.

이런 실수를 합니다

빈 파일을 parse 함

```
JSON.parse("")
```

```
SyntaxError: Unexpected end of JSON input
```

파일은 있는데 안이 비었을 때 납니다. 저장시 중간에 끊기면 이렇게 됩니다. try/catch 로 감싸는 게 안전합니다.

이런 실수를 합니다

JSON 파일에 주석을 씌

JSON 은 주석을 허용하지 않습니다. // 를 넣으면 parse 가 실패합니다. package.json 에 주석을 못 쓰는 이유와 같습니다. (01단원)

이런 실수를 합니다

원본 배열을 직접 고침

```
목록.push(새문서); 저장하기(목록);
```

지금은 동작하지만, 나중에 이 배열을 다른 곳에서도 쓰면 꼬입니다.

...목록, 새문서 · 처럼 새 배열을 만드는 습관을 들이세요. (JS자료 09단원)

이런 실수를 합니다

find 결과를 확인 없이 씌

불러오기().find((d) => d.id === 999).title 못 찾으면 undefined 라서 TypeError 가 납니다. ?. 를 쓰거나 if 로 확인하세요.

정리

- 1 변수는 서버를 끄면 사라진다. 남기려면 파일이나 데이터베이스에 저장해야 한다.
- 2 저장할 때 `JSON.stringify`, 읽을 때 `JSON.parse`. 항상 짝이다.
- 3 `JSON.stringify(값, null, 2)` 로 하면 사람이 읽을 수 있게 저장된다.
- 4 파일이 없을 때는 빈 배열로 시작한다. 그냥 읽으면 `ENOENT`.
- 5 흐름은 언제나 "전부 읽기 → 배열 고치기 → 전부 쓰기" 다.
- 6 고칠 때는 `map`, 지울 때는 `filter`, 찾을 때는 `find`. JS자료 08단원 그대로다.
- 7 이 방식은 동시 접근·성능·검색·서버 여러 대에서 한계가 있다. 그래서 PART 4 에서 데이터베이스를 배운다.

직접 해보기 정답

```
1) fs.writeFileSync(path.join(__dirname, "한줄.json"), JSON.stringify(문서들), "utf-8");
   fs.writeFileSync(path.join(__dirname, "여러줄.json"), JSON.stringify(문서들, null, 2), "utf-8");
```

→ 한줄.json 은 한 줄로 쭉 이어져 있고, 여러줄.json 은 들여쓰기가 되어 있습니다.

내용은 완전히 같습니다. 컴퓨터는 둘 다 똑같이 읽습니다.
사람이 열어 볼 파일이면 , `null, 2` 를 주는 게 좋습니다.
(확인했으면 두 파일은 지우세요)

- 2 `console.log(읽은배열.map((d) => d.writer));` // 출력: ['김민준', '이서연']
- 3 빈 배열이 나옵니다. 에러가 나지 않습니다. `불러오기()` 안에서 `existsSync` 로 먼저 확인하고 [] 를 돌려주기 때문입니다. 이 한 줄이 "서버를 처음 켰을 때 죽는 문제" 를 막아 줍니다.
- 4 추가하기(`"설비매뉴얼", "정하늘"`); `console.log(불러오기().map((d) => d.id));`

→ 섹션 4 를 다 실행한 뒤라면 [2, 3, 4] 가 됩니다.

1번을 지웠지만 `id` 는 다시 안 씁니다. 가장 큰 값(`3`) + `1` = `4` 입니다.
지운 번호를 재활용하면 옛날 데이터와 헷갈리기 때문에 그냥 건너뛸 겁니다.

```
5) const 김민준이읽은것 = 불러오기();
   const 이서연이읽은것 = 불러오기();      ← 둘 다 같은 시점의 목록
```

```
저장하기([...김민준이읽은것, { id: 100, title: "김민준 문서" }]);  
저장하기([...이서연이읽은것, { id: 200, title: "이서연 문서" }]);
```

```
console.log(불러오기().map((d) => d.title));
```

→ “김민준 문서” 가 없습니다. 이서연이 덮어썼기 때문입니다.

두 사람이 거의 동시에 요청하면 실제 서버에서 이 일이 일어납니다.
데이터베이스는 이런 일이 안 생기게 막아 줍니다.

파일 다루기

RUN node 연습문제.js

TODO 자리에 코드를 쓰고, '기대 출력'과 같은지 확인하세요. 1~12는 기본, 13은 응용, 14는 [도전], 15는 에러 확인입니다.

준비 · 아래 두 줄은 그대로 두세요. 모든 문제에서 씁니다.

```
const fs = require("fs");
const path = require("path");
```

이 연습문제가 쓸 폴더입니다. 매번 결과가 같도록 먼저 지우고 시작합니다.

```
const 작업폴더 = path.join(__dirname, "연습_작업");
fs.rmSync(작업폴더, { recursive: true, force: true });
fs.mkdirSync(작업폴더, { recursive: true });
```

문제 1 (개념01)

"uploads", "2026", "03", "보고서.pdf" 네 조각을 이어 붙인 경로를 출력하세요.

기대 출력 uploads\2026\03\보고서.pdf

(윈도우 기준입니다. 맥·리눅스는 uploads/2026/03/보고서.pdf 로 나옵니다)

여기에 코드를 쓰세요

문제 2 (개념01)

아래 파일 이름이 PDF 인지 판정해 true/false 로 출력하세요. 대문자로 올려도 true 가 나와야 합니다.

힌트: extname 은 점을 포함해 ".PDF" 를 줍니다. 소문자로 낮춰서 비교하세요.

```
const 파일명1 = "작업표준서.PDF";
```

기대 출력 true

여기에 코드를 쓰세요

문제 3 (개념01)

위 파일명에서 확장자를 떼 이름만 출력하세요. (대문자 확장자여도 제대로 떼어져야 합니다)

힌트: basename 의 두 번째 인자에 '떨 확장자' 를 넘깁니다.

그 확장자는 extname 으로 구할 수 있습니다.

기대 출력 작업표준서

여기에 코드를 쓰세요

문제 4 (개념02)

같은 폴더의 샘플_문서.txt 를 읽어서 첫 줄만 출력하세요.

힌트: 경로는 path.join(__dirname, ...) 으로 만드세요.

"utf-8" 을 빼먹으면 안 됩니다.

기대 출력 작업표준서

여기에 코드를 쓰세요

문제 5 (개념02)

같은 파일에서 빈 줄을 뺀 줄 수를 출력하세요.

기대 출력 4

여기에 코드를 쓰세요

문제 6 (개념02)

작업폴더 안에 "메모.txt" 를 만들고 "첫 줄\n" 을 쓰세요. 그다음 그 파일이 있는지 확인해 출력하세요.

기대 출력 true

여기에 코드를 쓰세요

문제 7 (개념02)

방금 만든 메모.txt 에 "둘째 줄\n" 과 "셋째 줄\n" 을 이어 붙이고, 전체 줄 수를 출력하세요. (뺀다면 안 됩니다)

힌트: 이어 쓰는 것은 appendFileSync 입니다.

기대 출력 3

여기에 코드를 쓰세요

문제 8 (개념02)

없는 파일을 읽어 보고, 에러 코드만 출력하세요. 프로그램이 죽으면 안 됩니다.

기대 출력 ENOENT

여기에 코드를 쓰세요

문제 9 (개념04)

작업폴더 안에 "docs/2026/03" 을 한 번에 만들고, 있는지 확인해 출력하세요.

힌트: 여러 단계를 한 번에 만드는 옵션이 있습니다.

기대 출력 true

여기에 코드를 쓰세요

문제 10 (개념04)

아래 세 파일을 작업폴더에 만든 뒤, 작업폴더에서 .txt 파일만 골라 이름을 정렬해 출력하세요.

(이 세 줄은 그대로 두세요)

```
fs.writeFileSync(path.join(작업폴더, "a.txt"), "A", "utf-8");
fs.writeFileSync(path.join(작업폴더, "b.pdf"), "B", "utf-8");
fs.writeFileSync(path.join(작업폴더, "c.txt"), "C", "utf-8");
```

기대 출력

— 'a.txt', 'c.txt', '메모.txt'

여기에 코드를 쓰세요

문제 11 (개념04)

작업폴더 안에서 '폴더' 인 것만 골라 이름을 출력하세요.

힌트: readdirSync 에 { withFileTypes: true } 를 주면

각 항목에 isDirectory() 를 쓸 수 있습니다.

기대 출력

— 'docs'

여기에 코드를 쓰세요

문제 12 (개념05)

아래 배열을 작업폴더의 "설비.json" 에 사람이 읽기 좋게 저장한 뒤, 다시 읽어서 두 번째 설비의 이름을 출력하세요.

```
const 설비들 = [
  { id: 1, name: "컨베이어", line: "A" },
  { id: 2, name: "프레스", line: "B" },
];
```

힌트: 저장할 때 stringify(값, null, 2), 읽을 때 parse 입니다.

기대 출력 프레스

여기에 코드를 쓰세요

문제 13 [응용]

업로드된 파일을 저장할 경로를 만드는 makeSavePath 함수를 만드세요.

```
makeSavePath("검사성적서.XLSX", new Date(2026, 2, 5))
```

```
→ "연습_작업\2026\03\검사성적서_20260305.xlsx"
```

규칙: - 확장자는 소문자로 통일합니다 - 폴더는 작업폴더/년/월 이고, 월은 두 자리입니다 (03) - 파일명 뒤에 _년월일 을 붙입니다 (20260305) - 폴더가 없으면 만들어야 합니다 - 만들어진 경로를 돌려줍니다

힌트: `getMonth()` 는 0부터입니다. `padStart(2, "0")` 으로 두 자리를 맞추세요.

```
path.join 이 만든 경로에서 작업폴더 부분을 빼고 보려면
path.relative(__dirname, 경로) 를 쓰면 됩니다.
```

```
기대 출력      연습_작업\2026\03\검사성적서_20260305.xlsx
               true
```

(경로는 윈도우 기준입니다. 맥·리눅스는 연습_작업/2026/03/... 로 나옵니다)

여기에 코드를 쓰세요

(두 번째 줄은 그 폴더가 실제로 만들어졌는지 확인해 출력하세요)

문제 14 [도전]

설비.json 을 비동기로 다루는 함수 세 개를 만들고 차례로 실행하세요.

불러오기() 파일이 없으면 [], 있으면 배열 추가하기(name, line) id 는 가장 큰 값 + 1 로 붙여서 추가
삭제하기(id) 그 id 를 뺀 목록으로 저장

전부 fs.promises 와 await 를 쓰세요. Sync 를 쓰면 안 됩니다.

힌트: await 는 async 함수 안에서만 쓸 수 있습니다.

```
async function main() { ... } 으로 감싸고 맨 아래에서 main() 을 부르세요.
```

★ 이 문제의 출력은 다른 문제들보다 '나중에' 찍힙니다. 정상입니다.

기대 출력 3

— '컨베이어', '프레스', '용접로봇'

— 1, 3

여기에 코드를 쓰세요

(추가하기("용접로봇", "C") 후 id / 이름 목록 / 삭제하기(2) 후 id 목록 순서로 출력)

문제 15 (에러 확인 - 맨 마지막)

아래 두 줄의 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.

```
const 없는폴더경로 = path.join(작업폴더, "없는폴더", "파일.txt");
fs.writeFileSync(없는폴더경로, "내용", "utf-8");
```

무슨 에러 코드가 났나요? 무엇을 먼저 했어야 할까요?

답

HTTP와 서버

OPEN 03_HTTP와_서버

```
console.log(주소.protocol);  
console.log(주소.hostname);  
console.log(주소.port);  
console.log(주소.pathname);
```

01 클라이언트와 서버, 그리고 URL

02 HTTP 메서드와 상태코드

03 맨손으로 서버 만들기 (http 모듈)

04 요청 읽고 응답하기 (req 자세히 보기 + Postman)

05 JSON API 하나를 끝까지 만들어 보기

- 연습문제

- 서버 연습문제

클라이언트와 서버, 그리고 URL

RUN node 개념01_클라이언트와_서버.js

지금까지 만든 것은 '내 컴퓨터에서 혼자 도는 프로그램'이었습니다. 이제부터는 '남이 접속해서 쓰는 프로그램'을 만듭니다. 그게 서버입니다.

클라이언트와 서버

클라이언트(client)	요청하는 쪽. 브라우저, 앱
서버(server)	응답하는 쪽. 우리가 만들 것

손님(클라이언트)이 주문하고, 주방(서버)이 만들어 내보냅니다.

지금까지 배운 것으로 나누면 이렇습니다.

HTML·CSS·React	→ 클라이언트. 브라우저에서 돌립니다
Node·Express	→ 서버. 우리 컴퓨터(나중엔 EC2)에서 돌립니다

한 번의 왕복

① 브라우저가 요청을 보낸다 "문서 목록 좀 주세요" ② 서버가 받아서 처리한다 파일을 읽거나 DB를 뒤집는다 ③ 서버가 응답을 보낸다 "여기 있습니다" + 데이터 ④ 브라우저가 받아서 화면에 그린다

12단원(JS자료)에서 fetch로 했던 것이 ①과 ④입니다. 이제 ②와 ③, 즉 '받아 주는 쪽'을 만듭니다.

★ 중요 — 서버는 먼저 말을 걸 수 없습니다. 항상 클라이언트가 물어봐야 대답합니다. 서버가 알아서 브라우저에 뭔가 보내는 일은 없습니다.

URL을 뜯어봅시다

섹션 1

요청을 보내려면 '어디로'를 알아야 합니다. 그게 URL입니다. Node에는 URL을 분석해 주는 도구가 이미 들어 있습니다.

```
const 주소 = new URL("https://api.example.com:8080/documents/12?type=pdf&page=2#top");
```

https://api.example.com:8080/documents/12?type=pdf&page=2#top

프로토콜 호스트 포트 경로 쿼리 해시

```
console.log(주소.protocol);
```

출력 https:

콜론이 함께 나옵니다. "https" 가 아니라 "https:" 입니다.

```
console.log(주소.hostname);
```

출력 api.example.com

어느 컴퓨터인지를 가리킵니다.

```
console.log(주소.port);
```

출력 8080

그 컴퓨터의 몇 번 문으로 들어갈지입니다. 섹션 3에서 자세히 봅니다.

```
console.log(주소.pathname);
```

출력 /documents/12

★ 서버에서 가장 중요한 부분입니다. "무엇을 달라는가" 가 여기 담깁니다.

```
console.log(주소.hash);
```

출력 #top

페이지 안의 위치입니다. ★ 서버로 전송되지 않습니다. 브라우저만 씁니다.

▫ 직접 해보기 1

"http://localhost:3000/api/users" 를 URL 로 만들어 hostname, port, pathname 을 각각 출력해 보세요.

쿼리 스트링 — 조건을 함께 보내기

섹션 2

경로 뒤 ? 부터가 쿼리입니다. "어떤 조건으로" 를 담습니다.

```
/documents?type=pdf&page=2
```

? 로 시작, & 로 여러 개, 각각은 이름=값

```
console.log(주소.search);
```

출력 ?type=pdf&page=2

통째로 문자열입니다. 직접 자르지 마세요. 아래 방법이 있습니다.

```
console.log(주소.searchParams.get("type"), 주소.searchParams.get("page"));
```

출력 pdf 2

```
console.log([...주소.searchParams.keys()]);
```

출력 ['type', 'page']

★ 아주 중요 — 꺼낸 값은 언제나 문자열입니다.

```
console.log(typeof 주소.searchParams.get("page"));
```

출력 string

page=2 라고 숫자를 보냈지만 "2" 로 들어옵니다. 계산하려면 Number 로 바꿔야 합니다. 01단원의 그 함정과 같습니다.

없는 것을 꺼내면 null 입니다.

```
console.log(주소.searchParams.get("없는것"));
```

출력 null

undefined 가 아니라 null 입니다. 확인하고 쓰세요.

쿼리가 아예 없으면 search 는 빈 문자열입니다.

```
const 주소2 = new URL("http://localhost:3000/api/docs");  
console.log(JSON.stringify(주소2.search));
```

출력 ""

▹ 직접 해보기 2

"/search?q=작업표준서&limit=10" 에서 q 와 limit 을 꺼내고, limit 의 자료형을 확인해 보세요. (앞에 http://localhost:3000 을 붙여야 URL 이 만들어집니다)

localhost 와 포트

섹션 3

서버를 만들면 이런 주소로 접속하게 됩니다.

```
http://localhost:3000
```

— localhost

“이 컴퓨터” 를 뜻하는 이름입니다. 127.0.0.1 과 같습니다. 내 컴퓨터에서 돌고 있는 서버에 내가 접속하는 것입니다. 옆자리 사람은 이 주소로 못 들어옵니다. 각자의 컴퓨터를 가리키니까요.

— 포트(port)

컴퓨터 하나에 프로그램이 여러 개 돌 수 있습니다. “그중 몇 번째 프로그램에게 갈 것인가” 를 정하는 번호입니다. 건물이 컴퓨터라면 포트는 호실 번호입니다.

3000, 5000, 8080 을 개발할 때 많이 씁니다. 아무 번호나 써도 됩니다. 다만 0~1023 은 이미 정해진 용도가 있어 피합니다.

```
80   → http (그래서 웹 주소에 포트를 안 씁니다)
443  → https
```

http://example.com 은 사실 http://example.com:80 입니다. 80 은 기본값이라 안 적는 것뿐입니다.

```
const 로컬 = new URL("http://localhost:3000/api/documents");

console.log(로컬.hostname, 로컬.port, 로컬.pathname);
```

```
출력      localhost 3000 /api/documents
```

★ 한 포트에는 프로그램 하나만 붙을 수 있습니다. 이미 3000 번을 쓰고 있는데 또 서버를 켜면 이 에러가 납니다.

```
Error: listen EADDRINUSE: address already in use :::3000
```

01단원에서 예고했던 그 에러입니다. 앞서 켜 둔 서버를 그 터미널에서 Ctrl + C 로 끄면 됩니다.

◦ 직접 해보기

3

포트를 4000 으로 바꾼 주소를 만들어 port 를 출력해 보세요.

서버는 경로를 보고 무엇을 할지 정합니다. 그래서 경로를 어떻게 지을지가 중요합니다.

```
/documents      문서 전체 목록
/documents/12   12번 문서 하나
/documents/12/파일 12번 문서의 파일
```

규칙 하나만 기억하세요.

★ 경로에는 '이름(명사)' 을 쓰고, '동작(동사)' 은 쓰지 않습니다.

```
좋은  /documents      나쁨  /getDocuments
좋은  /documents/12   나쁨  /deleteDocument?id=12
```

그럼 “가져오기” 와 “지우기” 는 어떻게 구분할까요? 경로가 아니라 ‘메서드’ 로 구분합니다. 다음 파일 (개념02)에서 배웁니다.

경로를 조각으로 나누는 것은 지금까지 배운 것으로 됩니다.

```
const 조각 = 로컬.pathname.split("/").filter((s) => s !== "");

console.log(조각);
```

```
출력      [ 'api', 'documents' ]
```

맨 앞의 / 때문에 빈 문자열이 하나 생겨서 filter 로 걸렸습니다.

```
const 상세 = new URL("http://localhost:3000/api/documents/12");
const 상세조각 = 상세.pathname.split("/").filter((s) => s !== "");

console.log(상세조각[2]);
```

```
출력      12
```

```
console.log(typeof 상세조각[2]);
```

```
출력      string
```

경로에서 꺼낸 것도 문자열입니다. 숫자로 쓰려면 Number 로 바꿔야 합니다.

▹ 직접 해보기

4

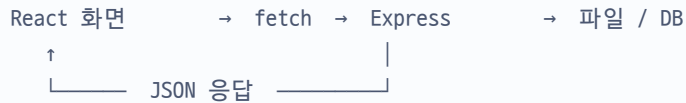
“/api/lines/A/machines/3” 을 조각으로 나눠 출력해 보세요.

우리가 만들 구조

섹션 5

이 과정에서 최종적으로 만들 모양입니다.

브라우저 · [서버] [저장소]



· 화면은 React 가 그립니다 (PART 2 에서 배웠습니다) · 데이터는 서버가 줍니다 (지금 배우는 것) · 둘은 fetch 로 대화합니다 (JS자료 12단원에서 배웠습니다) · 주고받는 것은 JSON 입니다

그래서 서버가 하는 일은 딱 이것뿐입니다.

"요청을 받아서 → 판단하고 → JSON 을 돌려준다"

화면을 그리는 일은 안 합니다. 그건 브라우저 몫입니다.

자주 하는 실수

섹션 6

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요.

이런 실수를 합니다

http:// 를 빼먹음 → 에러가 아니라 '조용히 이상한 결과'

```
const 잘못된주소 = new URL("localhost:3000/api");
console.log(잘못된주소.protocol, JSON.stringify(잘못된주소.hostname));
```

출력 localhost: ""

에러가 안 납니다. 그래서 더 위험합니다. "localhost:" 를 프로토콜로 읽어 버렸습니다. https: 자리에 들어간 것입니다. hostname 은 빈 문자열이 되어, 나중에 요청을 보내면 엉뚱한 곳으로 갑니다. http:// 를 반드시 붙이세요.

반대로 경로만 주면 그때는 에러가 납니다.

```
try {
  new URL("/api/documents");
} catch (error) {
  console.log(error.code);
}
```

출력 ERR_INVALID_URL

어디 컴퓨터인지가 없어서 URL 로 인정받지 못합니다.

```
}
```

이런 실수를 합니다

쿼리 값을 숫자로 착각

```
const page = 주소.searchParams.get("page");  
const 다음 = page + 1; → "21" (아어불아가)
```

Number(page) + 1 로 써야 3 이 됩니다.

이런 실수를 합니다

search 를 직접 자름

주소.search.split("=")[1] 값이 여러 개면 깨집니다. searchParams 를 쓰세요.

이런 실수를 합니다

경로에 동사를 씀

/getDocuments /createDocument /deleteDocument 문법 에러는 아니지만 실무에서 쓰지 않는 방식입니다. 경로는 명사, 동작은 메서드로 구분합니다. (개념02)

이런 실수를 합니다

해시(#)를 서버가 볼 수 있다고 생각

#top 은 브라우저에만 남고 서버로 전송되지 않습니다. 서버에서 읽으려 해도 절대 못 읽습니다.

이런 실수를 합니다

localhost 를 남에게 알려 줌

"http://localhost:3000 으로 들어와 보세요" 라고 해도 상대는 못 들어옵니다. 그 사람 컴퓨터의 3000 번을 보게 되니까요. 남이 접속하게 하려면 인터넷에 올려야 합니다. (PART 4 의 EC2)

정리

- 1 클라이언트가 요청하고 서버가 응답한다. 서버가 먼저 말을 걸 수는 없다.
- 2 URL 은 프로토콜 / 호스트 / 포트 / 경로 / 쿼리 / 해시로 되어 있다.
- 3 pathname 이 “무엇을”, search 가 “어떤 조건으로” 를 담는다.
- 4 searchParams.get 으로 쿼리를 꺼낸다. 값은 언제나 문자열, 없으면 null.
- 5 해시(#)는 서버로 전송되지 않는다.
- 6 localhost 는 내 컴퓨터, 포트는 그중 몇 번 프로그램인지.
- 7 한 포트에는 프로그램 하나. 중복되면 EADDRINUSE.
- 8 경로에는 명사를 쓴다. 동작은 메서드로 구분한다.

직접 해보기 정답

```

1 const u = new URL("http://localhost:3000/api/users");
  console.log(u.hostname); // 출력: localhost
  console.log(u.port);     // 출력: 3000
  console.log(u.pathname); // 출력: /api/users

2 const u = new URL("http://localhost:3000/search?q=작업표준서&limit=10");
  console.log(u.searchParams.get("q")); // 출력: 작업표준서
  console.log(u.searchParams.get("limit")); // 출력: 10
  console.log(typeof u.searchParams.get("limit")); // 출력: string

```

→ 10 이 아니라 “10” 입니다. 목록을 자를 때 slice(0, “10”) 처럼 쓰면

의도대로 동작하지 않습니다. Number 로 바꾸세요.

```

3) const u = new URL("http://localhost:4000/api");
  console.log(u.port); // 출력: 4000

```

→ 포트가 다르면 완전히 다른 프로그램입니다.

3000 번 서버를 켜 둔 채로 4000 번 서버를 또 켤 수 있습니다.

```

4) const u = new URL("http://localhost:3000/api/lines/A/machines/3");
  console.log(u.pathname.split("/").filter((s) => s !== ""));
  // 출력: [ 'api', 'lines', 'A', 'machines', '3' ]

```

→ “A 라인의 3번 설비” 라는 뜻입니다.

경로만 봐도 무엇을 달라는지 읽히는 것이 좋은 설계입니다.

HTTP 메서드와 상태코드

RUN node 개념02_메서드와_상태코드.js

앞 파일에서 “경로에는 명사를 쓰고 동작은 메서드로 구분한다” 고 했습니다. 이번 파일에서 그 메서드와, 서버가 결과를 알려 주는 상태코드를 배웁니다.

요청 하나는 이렇게 생겼습니다.

```
POST /api/documents      ← 메서드 + 경로
Content-Type: application/json ← 헤더
                               ← 빈 줄
{"title": "작업표준서"}    ← 본문(body)
```

응답은 이렇게 생겼습니다.

```
201 Created              ← 상태코드
Content-Type: application/json ← 헤더

{"id": 3, "title": "작업표준서"} ← 본문
```

메서드 — 무엇을 할 것인가

섹션 1

같은 /documents 경로라도 메서드에 따라 하는 일이 다릅니다.

```
GET    /documents      목록을 달라      (읽기)
POST   /documents      새로 만들어 달라   (생성)
GET    /documents/12   12번을 달라      (읽기)
PUT    /documents/12   12번을 통째로 바꿔 (수정)
PATCH /documents/12   12번의 일부만 바꿔 (부분 수정)
DELETE /documents/12   12번을 지워 달라   (삭제)
```

이 다섯 개면 거의 다 됩니다. 실무에서도 이것만 씁니다.

우리가 만들 서버가 하는 일을 함수로 표현하면 이렇게합니다.

```
function 무슨일인가(method, path) {
  const 조각 = path.split("/").filter((s) => s !== "");
  const 개별 = 조각.length > 1; // /documents/12 처럼 뒤에 번호가 있는가 if (method
=== "GET") {
    return 개별 ? "하나 조회" : "목록 조회";
```

```

}
if (method === "POST") {
  return "새로 만들기";
}
if (method === "PUT" || method === "PATCH") {
  return "수정";
}
if (method === "DELETE") {
  return "삭제";
}
return "모르는 메서드";
}

console.log(무슨일인가("GET", "/documents"));

```

출력 목록 조회

코드	▶ 출력
console.log(무슨일인가("GET", "/documents/12"));	▶ 하나 조회
console.log(무슨일인가("POST", "/documents"));	▶ 새로 만들기
console.log(무슨일인가("DELETE", "/documents/12"));	▶ 삭제

04단원부터 Express 를 쓰면 이 if 들을 직접 안 써도 됩니다. Express 가 메서드와 경로를 보고 알아서 갈라 줍니다. 다만 '무슨 일이 일어나는지' 는 알고 있어야 합니다.

▫ 직접 해보기

1

무슨일인가("PATCH", "/documents/5") 의 결과를 예상하고 확인하세요.

GET 과 POST 의 진짜 차이

섹션 2

가장 헷갈리는 부분입니다. 핵심은 '데이터를 어디에 심는가' 입니다.

GET	주소(쿼리)에 심는다	/search?q=작업표준서&page=2
POST	본문(body)에 심는다	보이지 않는 곳에 따로 담아 보냅니다

그래서 이런 차이가 생깁니다.

GET POST

데이터 위치 주소(쿼리) 본문 주소창에 보임 보인다 안 보인다 즐겨찾기 가능 불가능 길이 제한 있다(대략 2000자) 사실상 없다 파일 전송 못 한다 할 수 있다 새로고침 그냥 다시 됨 "다시 보내시겠습니까?" 가 뜸

★ 규칙 읽기만 하면 → GET 서버의 무언가가 바뀌면 → POST / PUT / PATCH / DELETE

비밀번호를 GET 으로 보내면 주소창과 서버 기록에 그대로 남습니다. 로그인이 POST 인 이유가 이것입니다.

GET 요청의 데이터를 꺼내는 법 (개념01 복습)

```
const 검색요청 = new URL("http://localhost:3000/search?q=작업표준서&page=2");

console.log(검색요청.searchParams.get("q"));
```

출력 작업표준서

```
console.log(Number(검색요청.searchParams.get("page")) + 1);
```

출력 3

Number 를 안 하면 "21" 이 됩니다. 쿼리는 언제나 문자열입니다.

POST 요청의 본문은 보통 JSON 입니다.

```
const 본문글자 = '{"title":"작업표준서","writer":"김민준"}';
const 본문 = JSON.parse(본문글자);

console.log(본문.title, 본문.writer);
```

출력 작업표준서 김민준

서버는 본문을 '글자' 로 받습니다. `JSON.parse` 로 객체를 만들어야 씁니다. 04단원에서 Express 가 이걸 자동으로 해 주는 법을 배웁니다.

▹ 직접 해보기

2

'{"id":5,"count":"3"}' 을 parse 해서 count 를 숫자로 바꿔 1 을 더해 출력해 보세요.

상태코드 — 결과를 숫자로

섹션 3

서버는 응답할 때 세 자리 숫자를 함께 보냅니다. 첫 자리가 성격을 정합니다.

```
2xx    성공했다
3xx    다른 데로 가라 (리다이렉트)
4xx    ★ 요청한 쪽 잘못이다
5xx    ★ 서버 잘못이다
```

4 와 5 를 구분하는 것이 중요합니다. 4xx 가 나오면 요청을 고쳐야 하고, 5xx 가 나오면 서버 코드를 고쳐야 합니다.

```
function 상태설명(code) {
  if (code >= 500) {
```

맨손으로 서버 만들기 (http 모듈)

```
RUN node 개념03_맨손으로_서버_만들기.js
```

그다음 브라우저에서 `http://localhost:3000` 을 여세요.

그러면 이 터미널에서 `Ctrl + C` 입니다.

04단원부터는 Express 를 씁니다. 훨씬 편하니까요. 그런데 그 전에 한 번은 맨손으로 만들어 봐야 합니다.

왜냐하면 Express 가 해 주는 일이 무엇인지 알아야 문제가 생겼을 때 어디를 봐야 할지 알 수 있기 때문입니다.

★ 이 파일은 지금까지와 다릅니다. 실행하면 프로그램이 안 끝납니다. 계속 켜져 있습니다. 그게 정상입니다. 서버는 요청을 기다려야 하니까요. 그러면 `Ctrl + C` 를 누르세요.

```
const http = require("http");
```

포트는 환경변수로 받고, 없으면 3000 을 씁니다. (나중에 서버에 올리면 포트를 밖에서 정해 주는 일이 많습니다. PART 4)

```
const PORT = process.env.PORT || 3000;
```

가장 작은 서버

섹션 1

`createServer` 에 함수를 하나 넘깁니다. 그 함수는 '요청이 올 때마다' 실행됩니다. 08단원(JS자료)의 콜백입니다.

```
(req, res) => { ... }
```

요청 응답

req 누가 무엇을 달라고 했는지가 담겨 있습니다 (읽기만)
res 내가 무엇을 돌려줄지를 담는 곳입니다 (쓰기만)

```
const server = http.createServer((req, res) => {
```

요청이 올 때마다 터미널에 기록을 남깁니다. 서버를 만들 때 이 한 줄이 아주 큰 도움이 됩니다.

```
console.log(`${req.method} ${req.url}`);
```

경로에 따라 다르게 응답하기

섹션 2

req.url 에는 경로와 쿼리가 함께 들어옵니다. (/documents?page=2) 경로만 떼어 내려면 URL 로 분석합니다.

```
const 주소 = new URL(req.url, `http://${req.headers.host}`);
const 경로 = 주소.pathname;
```

req.url 은 항상 상대 경로라 앞에 기준 주소를 붙여야 URL 이 만들어집니다. (개념01 실수 1에서 본 그 이유입니다)

```
if (경로 === "/") {
```

writeHead(상태코드, 헤더객체)

```
res.writeHead(200, { "Content-Type": "text/plain; charset=utf-8" });
res.end("안녕하세요. 문서 서버입니다.");
return; // * 응답한 뒤에는 반드시 빠져나옵니다
}
```

```
GET /
```

```
200 안녕하세요. 문서 서버입니다.
```

```
if (경로 === "/health") {
```

서버가 살아 있는지 확인하는 주소입니다. 실무에서 거의 항상 만듭니다. PART 4 에서 배포할 때 씁니다.

```
res.writeHead(200, { "Content-Type": "application/json; charset=utf-8" });
res.end(JSON.stringify({ status: "ok" }));
return;
}
```

```
GET /health
```

```
200 {"status":"ok"}
```

```

if (경로 === "/documents") {
  if (req.method === "GET") {
    const 목록 = [
      { id: 1, title: "작업표준서" },
      { id: 2, title: "검사성적서" },
    ];
    res.writeHead(200, { "Content-Type": "application/json; charset=utf-8" });
    res.end(JSON.stringify(목록));
    return;
  }
}

```

GET /documents

200 [{"id":1,"title":"작업표준서"},{"id":2,"title":"검사성적서"}]

```

if (req.method === "POST") {

```

새로 만들었으니 201 입니다. (개념02)

```

  res.writeHead(201, { "Content-Type": "application/json; charset=utf-8" });
  res.end(JSON.stringify({ id: 3, message: "만들었습니다" }));
  return;
}

```

POST /documents

201 {"id":3,"message":"만들었습니다"}

경로는 맞는데 메서드가 다른 경우입니다.

```

res.writeHead(405, { "Content-Type": "text/plain; charset=utf-8" });
res.end("이 주소에서는 쓸 수 없는 방법입니다");
return;

```

DELETE /documents

405 이 주소에서는 쓸 수 없는 방법입니다

```

}

```

쿼리 읽기

섹션 4

```
if (경로 === "/search") {
  const 검색어 = 주소.searchParams.get("q");

  if (!검색어) {
```

필수 값이 없으니 400 입니다. 500 이 아닙니다. (개념02)

```
res.writeHead(400, { "Content-Type": "application/json; charset=utf-8" });
res.end(JSON.stringify({ error: "q 를 넣어 주세요" }));
return;
}
```

GET /search

400 {"error": "q 를 넣어 주세요"}

```
res.writeHead(200, { "Content-Type": "application/json; charset=utf-8" });
res.end(JSON.stringify({ 검색어, 결과수: 0 }));
return;
```

GET /search?q=작업표준서

200 {"검색어": "작업표준서", "결과수": 0}

```
}
```

어디에도 안 걸리면 404

섹션 5

위에서 아무것도 return 하지 않았다면 모르는 주소입니다.

```
res.writeHead(404, { "Content-Type": "application/json; charset=utf-8" });
res.end(JSON.stringify({ error: "그런 주소가 없습니다" }));
```

GET /없는주소

404 {"error": "그런 주소가 없습니다"}

```
});
```

```
server.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
  console.log("고려면 Ctrl + C 를 누르세요.");
});
```

listen 을 부르는 순간부터 프로그램이 안 끝납니다. "요청 올 때까지 계속 기다리는 상태" 가 됩니다.

이 콜백은 서버가 다 켜졌을 때 딱 한 번 실행됩니다. 요청이 올 때마다 실행되는 것이 아닙니다. 위쪽 createServer 의 콜백과 다릅니다.

직접 해 볼 것

서버를 켜고 브라우저에서 아래 주소들을 차례로 열어 보세요. 그때마다 터미널에 GET / 처럼 기록이 남는 것도 함께 보세요.

http://localhost:3000/	→ 인사말
http://localhost:3000/health	→ {"status":"ok"}
http://localhost:3000/documents	→ 문서 목록
http://localhost:3000/search	→ 400 에러
http://localhost:3000/search?q=표준서	→ 검색어가 되돌아옴
http://localhost:3000/아무거나	→ 404

▹ 직접 해보기 1

/about 경로를 추가해 자기소개를 돌려주게 해 보세요. (고친 뒤에는 Ctrl+C 로 끄고 다시 켜야 반영됩니다)

▹ 직접 해보기 2

/documents 의 목록에 문서를 하나 더 추가해 보세요.

▹ 직접 해보기 3

브라우저 주소창으로는 POST 를 보낼 수 없습니다. 왜 그럴까요? (힌트: 주소창에 주소를 치면 항상 GET 입니다) POST 를 보내려면 Postman 이 필요합니다. 다음 파일에서 씁니다.

서버 코드를 고쳤는데 반영이 안 될 때

서버는 켜 때 파일을 한 번 읽고 그대로 둡니다. 파일을 고쳐도 이미 도는 서버는 옛날 코드입니다. Ctrl + C 로 끄고 다시 켜야 합니다.

매번 끄고 켜는 게 번거롭습니다. 그래서 04단원에서 nodemon 을 씁니다. 파일이 바뀌면 알아서 다시 켜 주는 도구입니다.

이 코드의 문제점

잘 동작하지만 벌써 불편한 것이 보입니다.

① if 가 계속 늘어납니다. 주소가 30개면 if 가 30개입니다. ② 경로마다 writeHead 와 `JSON.stringify` 를 반복해서 씁니다. ③ `/documents/12` 처럼 번호가 낀 주소는 직접 잘라 내야 합니다. ④ POST 로 온 본문 (body)을 읽으려면 코드가 훨씬 복잡해집니다.

이 넷을 전부 해결해 주는 것이 Express 입니다. 04단원에서 만납니다. 지금 불편함을 느껴 봐야 Express 가 왜 고마운지 알 수 있습니다.

자주 하는 실수

이런 실수를 합니다

res.end 를 안 부름

응답을 안 끝내면 브라우저가 계속 빙빙 돌립니다. 영원히 기다립니다. 어느 갈래로 가든 반드시 res.end 로 끝내세요.

이런 실수를 합니다

응답한 뒤에 return 을 안 함

아래 코드가 계속 실행되어 응답을 두 번 보내려 합니다. → Error: Cannot set headers after they are sent to the client 서버를 만들 때 가장 많이 보는 에러입니다. return 을 꼭 붙이세요.

이런 실수를 합니다

객체를 그대로 res.end 에 넣음

```
res.end({ id: 1 })
```

→ TypeError. 보낼 수 있는 것은 글자나 Buffer 뿐입니다. `JSON.stringify` 로 글자로 바꿔서 보내세요.

이런 실수를 합니다

charset=utf-8 을 빼먹음

한글이 깨져서 보입니다. Content-Type 에 꼭 붙이세요.

이런 실수를 합니다

서버를 안 끄고 또 켜

→ Error: listen EADDRINUSE: address already in use :::3000 앞 터미널에서 Ctrl + C 로 끄거나, 다른 포트를 쓰세요.

이런 실수를 합니다

파일을 고치고 새로고침만 함

서버는 다시 켜야 반영됩니다. 브라우저 새로고침으로는 안 됩니다.

정리

- 1 http.createServer 로 서버를 만들고 listen 으로 켜다. 실행하면 프로그램이 안 끝나는 것이 정상이다. 요청을 기다리는 중이다.
- 2 요청 하나가 오면 콜백이 한 번 돈다. req 는 들어온 것, res 는 내보낼 것이다.
- 3 갈래는 손으로 나눈다. req.url 과 req.method 를 if 로 하나씩 본다. 이 손으로 나누는 일을 04단원부터 Express 가 대신해 준다.
- 4 응답은 반드시 res.end 로 끝낸다. 안 부르면 브라우저가 계속 기다린다.
- 5 응답한 뒤에는 return 을 붙인다. 안 붙이면 아래로 흘러가 두 번 보낸다.
- 6 객체는 그대로 못 보낸다. JSON.stringify 로 글자로 바꿔서 보낸다.
- 7 한글이 깨지면 charset=utf-8 을 빼먹은 것이다.
- 8 고쳤으면 Ctrl + C 로 꺾다가 다시 켜야 반영된다. 새로고침만으로는 안 된다.

요청 읽고 응답하기 (req 자세히 보기 + Postman)

RUN node 개념04_요청_읽고_응답하기.js

그러면 Ctrl + C

개념03에서는 '경로' 만 보고 갈래를 나눴습니다. 이번에는 요청 안에 들어 있는 나머지를 전부 꺼내 봅시다.

① 메서드 req.method ② 주소 req.url → 경로 + 쿼리 ③ 헤더 req.headers ④ 본문 req.on("data") → ★ 이게 오늘의 핵심입니다

①②③ 은 요청이 도착하는 순간 이미 다 들어 있습니다. 그냥 꺼내 쓰면 됩니다. ④ 만 다릅니다. 본문은 '나중에, 조각조각' 도착합니다. 기다려서 모아야 합니다.

```
const http = require("http");

const PORT = process.env.PORT || 3000;
```

반복부터 없애고 갑시다

섹션 0

개념03 끝에서 불편한 점 네 가지를 봤습니다. 그중 ② "경로마다 writeHead 와 JSON.stringify 를 반복한다" 는 Express 없이도 우리가 직접 고칠 수 있습니다. 함수로 묶으면 됩니다.

```
function JSON응답(res, 상태코드, 데이터) {
  res.writeHead(상태코드, { "Content-Type": "application/json; charset=utf-8" });
  res.end(JSON.stringify(데이터));
}
```

이제 아래처럼 한 줄로 씁니다.

```
JSON응답(res, 200, { status: "ok" });
```

세 줄이 한 줄이 되었고, charset=utf-8 을 빠뜨릴 일도 없어졌습니다. "같은 코드를 세 번 이상 쓰고 있다" 싶으면 함수로 묶으세요. 서버 코드도 똑같습니다.

나머지 ①③④ 는 함수로 못 고칩니다. 그래서 Express 가 필요합니다.

본문은 왜 한 번에 안 오나

섹션 1

본문(body)은 요청에 딸려 오는 '내용물' 입니다. 회원가입 폼, 글쓰기 내용, 업로드한 파일이 전부 여기 들어옵니다.

그런데 Node 는 본문을 통째로 주지 않습니다. 조각(chunk)으로 나눠서, 도착하는 대로 하나씩 건네줍니다.

왜 그럴까요? 본문이 항상 작다는 보장이 없기 때문입니다. 500MB 짜리 동영상을 올리면, 통째로 주려다가 메모리가 터집니다. 조각으로 주면 받는 쪽이 "조금 받고 저장하고, 조금 받고 저장하고" 를 할 수 있습니다.

이런 방식을 스트림(stream)이라고 부릅니다. 물을 양동이째 건네는 게 아니라, 호스로 흘려보내는 것입니다.

그래서 코드가 이렇게 생겼습니다.

```
req.on("data", (조각) => { ... }) ← 조각이 올 때마다 실행
req.on("end", () => { ... }) ← 다 왔을 때 딱 한 번 실행
```

11단원(JS자료)에서 배운 addEventListener 와 똑같은 모양입니다. "무슨 일이 생기면 이 함수를 실행해 줘" 라는 뜻입니다. 버튼 클릭 대신 '데이터 도착' 이 그 일입니다.

본문 모으는 함수

섹션 2

조각을 모아서 하나의 글자로 만들어 돌려주는 함수입니다. 이 15줄은 서버를 만들 때 늘 나오는 정형구입니다. 통째로 외워도 됩니다.

```
function 본문읽기(req) {
  return new Promise((resolve, reject) => {
    let 쌓은글자 = "";
```

★ 조각을 글자로 알아서 바꿔 달라고 미리 부탁드립니다. 이 줄이 없으면 조각이 Buffer(숫자 덩어리)로 오고, 한글이 조각 경계에서 잘려 깨질 수 있습니다.

```
    req.setEncoding("utf-8");

    req.on("data", (조각) => {
      쌓은글자 += 조각;
    });

    req.on("end", () => {
      resolve(쌓은글자); // 다 모았으니 내놓습니다
    });

    req.on("error", (에러) => {
      reject(에러); // 중간에 연결이 끊긴 경우
    });
  });
}
```

왜 Promise 로 감쌌나 본문은 '기다려야' 얻을 수 있습니다. `return` 으로는 못 돌려줍니다. `end` 가 언제 올지 모르니까요. (02단원 개념03의 그 이야기입니다)

Promise 로 감싸 두면 쓰는 쪽에서 이렇게 한 줄이 됩니다.

```
const 글자 = await 본문읽기(req);
```

콜백 안의 콜백을 안 봐도 되니 훨씬 읽기 편합니다.

`resolve` 와 `reject` 는 무엇인가 `resolve(값) → await` 가 그 값을 받습니다. 성공입니다. `reject(에러) → await` 자리에서 에러가 납니다. `try/catch` 로 잡힙니다. 하나만 부르면 됩니다. 부르는 순간 그 Promise 는 끝납니다.

```
const server = http.createServer(async (req, res) => {
```

★ 콜백 앞에 `async` 가 붙었습니다. 안에서 `await` 를 쓰려면 반드시 있어야 합니다.

```
console.log(`${req.method} ${req.url}`);

const 주소 = new URL(req.url, `http://${req.headers.host}`);
const 경로 = 주소.pathname;
```

메서드와 쿼리 꺼내 보기

섹션 3

```
if (경로 === "/echo") {
```

`searchParams` 를 그대로 객체로 바꿔 봅니다. `Object.fromEntries` 는 [키,값] 짝들을 객체로 만들어 줍니다.

```
const 쿼리 = Object.fromEntries(주소.searchParams);

JSON응답(res, 200, {
  method: req.method,
  경로: 경로,
  쿼리: 쿼리,
});
return;
}
```

```
GET /echo?이름=김민준&나이=20
```

```
200 {"method":"GET","경로":"/echo","쿼리":{"이름":"김민준","나이":"20"}}
```

나이가 문자열 "20" 인 것을 보세요. 쿼리는 무엇을 넣든 항상 글자입니다. 숫자로 쓰려면 `Number()` 로 바꿔야 합니다. (개념01 실수 3)

```
if (경로 === "/headers") {
```

★ Node 에서 헤더 이름은 전부 소문자입니다. 브라우저가 User-Agent 라고 보내도 Node 는 user-agent 로 바꿔 답습니다.

```
JSON응답(res, 200, {
  "소문자로 찾으면": req.headers["user-agent"] !== undefined,
  "대문자로 찾으면": req.headers["User-Agent"] !== undefined,
});
return;
}
```

```
GET /headers
```

```
200 {"소문자로 찾으면":true,"대문자로 찾으면":false}
```

대문자로 찾으면 false 입니다. 없는 게 아니라 이름이 다른 것입니다. 헤더가 undefined 로 나오면 집중팔구 대소문자 문제입니다.

헤더에는 이런 것들이 들어 있습니다.

host	어느 주소로 들어왔나 (localhost:3000)
user-agent	어떤 프로그램이 보냈나 (크롬인지 Postman 인지)
content-type	본문이 무슨 형식인가 (application/json)
authorization	로그인 증표 - PART 4 에서 씁니다

본문 읽기

```
if (경로 === "/memos" && req.method === "POST") {
  const 글자 = await 본문읽기(req); // ← 여기서 기다립니다
```

본문이 아예 안 왔을 수 있습니다. 빈 글자입니다.

```
if (!글자) {
  JSON응답(res, 400, { error: "본문이 비어 있습니다" });
  return;
}
```

```
POST /memos
```

```
400 {"error":"본문이 비어 있습니다"}
```

JSON.parse 는 실패할 수 있습니다

섹션 6

받은 글자가 진짜 JSON 이라는 보장이 없습니다. 남이 보낸 것은 무엇이든 의심해야 합니다.

```
let 데이터;
try {
  데이터 = JSON.parse(글자);
} catch (에러) {
```

★ try/catch 를 안 하면 여기서 서버가 통째로 죽습니다.

```
JSON응답(res, 400, { error: "JSON 형식이 아닙니다" });
return;
}
```

```
POST /memos
{제목:"따옴표없음"}

400 {"error":"JSON 형식이 아닙니다"}
```

형식은 맞는데 필요한 값이 없을 수도 있습니다. 이것도 확인합니다.

```
if (!데이터.title) {
  JSON응답(res, 400, { error: "title 을 넣어 주세요" });
  return;
}
```

```
POST /memos
{"내용":"제목이없음"}

400 {"error":"title 을 넣어 주세요"}
```

여기까지 왔으면 쓸 수 있는 데이터입니다.

```
JSON응답(res, 201, {
  받은제목: 데이터.title,
  글자수: 데이터.title.length,
});
return;
}
```

POST /memos

```
{ "title": "작업표준서" }
```

```
201 { "받은제목": "작업표준서", "글자수": 5 }
```

PUT 도 본문 읽는 법은 똑같습니다

섹션 7

```
if (경로 === "/memos" && req.method === "PUT") {  
  const 글자 = await 본문읽기(req);  
  const 데이터 = JSON.parse(글자 || "{}"); // 빈 본문이면 빈 객체로 JSON응답(res,  
200, {  
    방법: req.method,  
    받은제목: 데이터.title || "(없음)",  
  });  
  return;  
}
```

PUT /memos

```
{ "title": "수정본" }
```

```
200 { "방법": "PUT", "받은제목": "수정본" }
```

POST 든 PUT 이든 PATCH 든 본문 읽는 코드는 완전히 같습니다. 메서드는 '의도'를 나타낼 뿐, 데이터가 오는 방식은 하나입니다.

GET 에는 본문이 없습니다

섹션 8

```
if (경로 === "/memos" && req.method === "GET") {  
  const 글자 = await 본문읽기(req);  
  
  JSON응답(res, 200, {  
    본문길이: 글자.length,  
    설명: "GET 에는 본문을 담지 않습니다",  
  });  
  return;  
}
```

GET /memos

```
200 { "본문길이": 0, "설명": "GET 에는 본문을 담지 않습니다" }
```

★ 중요 — GET 에도 본문읽기 를 부를 수는 있습니다. 멈추지 않습니다. 본문이 없으면 end 가 바로 오고, 빈 글자가 나옵니다. "await 를 걸면 GET 이 멈추는 것 아닌가?" 하는 걱정은 안 해도 됩니다.

```
JSON응답(res, 404, { error: "그런 주소가 없습니다" });
```

GET /없는주소

```
404 {"error":"그런 주소가 없습니다"}
```

```
});
```

```
server.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
});
```

Postman — POST 를 보내는 도구

브라우저 주소창에 주소를 치면 언제나 GET 입니다. 주소창에는 “본문을 여기 담아라” 를 적을 칸이 없기 때문입니다.

그래서 POST 를 시험하려면 도구가 필요합니다. Postman 을 씁니다.

설치

- 1 <https://www.postman.com/downloads> 에서 Windows 64-bit 를 받습니다.
- 2 설치한 뒤 실행합니다.
- 3 로그인하라고 나옵니다. 오른쪽 아래 작은 글씨의

"Skip and go to the app" 을 누르면 로그인 없이 쓸 수 있습니다.
(팀으로 공유하려면 가입하면 됩니다. 지금은 필요 없습니다)

GET 보내 보기

- 1 왼쪽 위 + 버튼 → 새 탭이 생깁니다.
- 2 주소칸에 `http://localhost:3000/echo?이름=김민준&나이=20`
- 3 파란 Send 버튼을 누릅니다.
- 4 아래쪽에 응답이 나옵니다.

Body 탭 → 돌려받은 내용
 Headers 탭 → 돌려받은 헤더
 오른쪽 위 → 200 OK 와 걸린 시간

POST 보내 보기 (제일 중요합니다)

- 1 주소칸 왼쪽의 GET 을 눌러 POST 로 바꿉니다.
- 2 주소칸에 `http://localhost:3000/memos`
- 3 주소칸 아래 줄에서 Body 탭을 누릅니다.
- 4 raw 를 고릅니다.
- 5 그 오른쪽 드롭다운을 Text → JSON 으로 바꿉니다. ★ 자주 빠뜨립니다
- 6 아래 큰 칸에 이렇게 적습니다.

```
{ "title": "작업표준서" }
```

7. Send. → 201 과 함께 {"받은제목":"작업표준서","글자수":5} 가 나옵니다.

5번을 안 하면 Content-Type 이 text/plain 으로 나갑니다. 지금 만든 서버는 그래도 동작하지만, 04단원 Express 에서는 본문이 통째로 빈 값이 되어 "왜 undefined 지?" 하고 헤매게 됩니다. 지금부터 습관을 들이세요.

일부러 틀려 보기

Body 를 아래처럼 바꿔 가며 Send 를 눌러 보세요.

(아무것도 안 씀)	→ 400 본문이 비어 있습니다
{ title: "따옴표없음" }	→ 400 JSON 형식이 아닙니다
{ "내용": "제목이없음" }	→ 400 title 을 넣어 주세요
{ "title": "작업표준서" }	→ 201 정상

두 번째가 왜 틀렸는지 보세요. JSON 은 속성 이름에도 큰따옴표가 필요합니다. 자바스크립트 객체와 생김새 비슷해서 정말 자주 틀립니다.

다른 도구도 있습니다

Thunder Client VS Code 확장. 에디터를 안 벗어나도 됩니다. 가볍습니다.

curl 터미널 명령. 서버에 올린 뒤에는 이것밖에 못 쓸 때가 많습니다.

```
curl -X POST http://localhost:3000/memos ^
-H "Content-Type: application/json" ^
-d "{\"title\": \"작업표준서\"}"
```

(윈도우 PowerShell에서는 줄 끝의 ^ 대신 `를 씁니다.
한 줄로 붙여 쓰면 그런 것 신경 안 써도 됩니다)

직접 해 볼 것

▫ 직접 해보기 1

/echo에 쿼리를 아무거나 붙여 보세요. `http://localhost:3000/echo?a=1&b=2&b=3`가 두 개일 때 어떻게 나오나요? 왜 그럴까요? (힌트: `fromEntries`는 같은 키가 오면 뒤엎것이 이깁니다. 둘 다 필요하다면 주소 `searchParams.getAll("b")`를 씁니다)

▫ 직접 해보기 2

/memos에 title을 200글자쯤 넣어 보내 보세요. 그리고 "50글자를 넘으면 400"이 되도록 검사를 추가해 보세요.

▫ 직접 해보기 3

본문읽기 안의 `console.log`를 켜서 조각이 몇 번 오는지 보세요.

```
req.on("data", (조각) => {
  console.log("조각 도착:", 조각.length);
  쌓은글자 += 조각;
});
```

짧은 본문은 조각이 한 번에 다 옵니다.
조각이 여러 번 오는 것을 보려면 아주 긴 글을 보내야 합니다.

▫ 직접 해보기 4

/memos에 DELETE로 요청을 보내면 지금 무엇이 나오나요? 왜 그렇게 나오는지 코드에서 짚어 보세요.

자주 하는 실수

이런 실수를 합니다

await 를 쓰면서 async 를 안 붙임

```
http.createServer((req, res) => { await 본문읽기(req) ... })
```

→ SyntaxError. 파일이 통째로 안 돌아갑니다. createServer 에 넘기는 콜백 앞에 async 를 붙이세요.

이런 실수를 합니다

본문을 안 기다리고 바로 씀

```
const 글자 = 본문읽기(req);  
JSON.parse(글자)
```

→ SyntaxError: Unexpected token 'o' ... "[object Promise]" 에러 메시지에 Promise 가 보이면 await 를 빠뜨린 것입니다.

이런 실수를 합니다

JSON.parse 를 try/catch 없이 씀

누군가 이상한 글자를 보내면 서버 전체가 죽습니다. 나 혼자 쓸 때는 안 죽지만, 남에게 열어 주는 순간 반드시 일어납니다. 남이 보낸 것을 parse 할 때는 예외 없이 try/catch 입니다.

이런 실수를 합니다

값이 없는 것을 500 으로 돌려줌

title 을 안 보낸 것은 '보낸 쪽' 잘못입니다. 400 입니다. 500 은 '내 코드' 가 터졌을 때만 씁니다. (개념02)

이런 실수를 합니다

Postman 에서 Body → raw 만 고르고 JSON 으로 안 바꿈

Content-Type 이 text/plain 으로 나갑니다. 04단원 Express 에서는 이러면 본문이 통째로 안 들어옵니다.

이런 실수를 합니다

GET 요청에 본문을 담으려 함

Postman 에서는 담아지지만, 브라우저 fetch 는 거부합니다. → TypeError: Request with GET/HEAD method cannot have body GET 으로 값을 보낼 때는 쿼리(?q=...)를 씁니다.

정리

- 1 메서드·주소·헤더는 요청이 도착하는 순간 이미 다 들어 있다. `req.method` · `req.url` · `req.headers` 로 그냥 꺼내 쓰면 된다.
- 2 본문만 다르다. 조각조각 나중에 도착한다. `req.on("data")` 로 모으고 `req.on("end")` 에서 끝낸다.
- 3 그래서 본문은 기다려야 한다. 안 기다리고 쓰면 빈 값이 나온다.
- 4 `setEncoding("utf8")` 을 미리 불러 두면 조각을 글자로 알아서 바꿔 준다.
- 5 헤더 이름은 Node 에서 전부 소문자다. `req.headers["content-type"]` 처럼 쓴다.
- 6 `JSON.parse` 는 실패할 수 있다. `try/catch` 없이 쓰면 서버가 통째로 죽는다. 형식이 틀린 것은 400 으로 돌려준다.
- 7 값이 빠진 것도 400 이다. 500 은 서버 잘못일 때만 쓴다.
- 8 GET 에는 본문이 없다. 부를 수는 있지만 빈 글자가 온다. GET 은 쿼리로 받는다.

JSON API 하나를 끝까지 만들어 보기

RUN node 개념05_JSON_API_만들어보기.js

그러면 Ctrl + C

지금까지 배운 것을 전부 합칩니다. 문서를 등록하고, 보고, 고치고, 지우는 API 를 만듭니다.

GET	/documents	목록 보기
GET	/documents?type=	종류로 걸러 보기
GET	/documents/1	하나만 보기
POST	/documents	새로 만들기
PUT	/documents/1	고치기

DELETE /documents/1 지우기

이 여섯 개를 CRUD 라고 부릅니다. Create(만들기) Read(읽기) Update(고치기) Delete(지우기)

세상의 거의 모든 서버가 이 네 가지를 합니다. 쇼핑몰이든 SNS 든 사내 문서 시스템이든 똑같습니다. 이 파일 하나를 이해하면 뼈대는 다 이해한 것입니다.

★ 04단원에서 똑같은 것을 Express 로 다시 만듭니다. 그때 이 파일을 옆에 놓고 비교해 보세요. 절반으로 줄어듭니다.

```
const http = require("http");

const PORT = process.env.PORT || 3000;
```

데이터는 어디에 두나

섹션 1

지금은 그냥 변수에 담아 둡니다. 이걸 '메모리에 둔다' 고 합니다.

let 인 이유: 추가·삭제하면서 배열 자체를 바꿔 끼울 것이기 때문입니다.

```
let 문서들 = [
  { id: 1, title: "작업표준서", type: "표준" },
  { id: 2, title: "검사성적서", type: "성적" },
];
```

새 문서에 붙일 번호입니다. 하나 만들 때마다 1씩 올립니다.

```
let 다음번호 = 3;
```

★ 여기에는 큰 문제가 있습니다. 미리 알고 가세요.

서버를 켜다 켜면 문서들이 처음 두 개로 돌아갑니다. 변수는 프로그램이 살아 있는 동안만 존재하기 때문입니다.

해결 순서는 이렇습니다.

- ① 파일에 저장한다 → 02단원에서 배웠습니다. 아래 ✎ 3번
- ② 데이터베이스에 저장한다 → PART 4 에서 배웁니다. 진짜 해결책입니다.

지금 단계에서는 '서버가 어떻게 생겼는지'에만 집중합니다. 저장 방법은 나중에 갈아 끼우면 됩니다. 그래서 이 순서로 배웁니다.

응답 도우미

섹션 2

```
function JSON응답(res, 상태코드, 데이터) {
  res.writeHead(상태코드, { "Content-Type": "application/json; charset=utf-8" });
  res.end(JSON.stringify(데이터));
}
```

204 는 '성공했지만 돌려줄 내용이 없다' 입니다. 본문을 아예 안 보냅니다. 그래서 Content-Type 도 붙이지 않습니다. 내용이 없으니 형식도 없습니다.

```
function 내용없음(res) {
  res.writeHead(204);
  res.end();
}

function 본문읽기(req) {
  return new Promise((resolve, reject) => {
    let 쌓은글자 = "";
    req.setEncoding("utf-8");
    req.on("data", (조각) => (쌓은글자 += 조각));
    req.on("end", () => resolve(쌓은글자));
    req.on("error", reject);
  });
}

const server = http.createServer(async (req, res) => {
  console.log(`${req.method} ${req.url}`);

  const 주소 = new URL(req.url, `http://${req.headers.host}`);
  const 경로 = 주소.pathname;
```

개념03 끝에서 말한 세 번째 불편함입니다. /documents/1 에서 1 을 직접 잘라 내야 합니다.

"/documents/1".split("/") → ['', 'documents', '1']

↑ 맨 앞이 빈 글자입니다

맨 앞 빈 글자가 거슬리니 filter 로 걸러 냅니다. filter(Boolean) 은 "빈 글자·0·undefined 를 버려라" 라는 뜻입니다. (JS자료 06단원)

```
const 조각들 = 경로.split("/").filter(Boolean);
```

/documents	→ ['documents']	번호글자 = undefined
/documents/1	→ ['documents', '1']	번호글자 = '1'

```
const 첫조각 = 조각들[0];
const 번호글자 = 조각들[1];

if (첫조각 !== "documents") {
  JSON응답(res, 404, { error: "그런 주소가 없습니다" });
  return;
}
```

GET /없는주소

404 {"error": "그런 주소가 없습니다"}

목록 보기 (Read)

```
if (번호글자 === undefined && req.method === "GET") {
  const 종류 = 주소.searchParams.get("type");
```

쿼리가 없으면 전부, 있으면 걸러서 돌려줍니다. 쿼리는 '있어도 되고 없어도 되는 것' 에 씁니다. (개념01)

```
const 결과 = 종류 ? 문서들.filter((문서) => 문서.type === 종류) : 문서들;

JSON응답(res, 200, 결과);
return;
}
```

GET /documents

```
200 [{"id":1,"title":"작업표준서","type":"표준"}, {"id":2,"title":"검사성적서","type":"성적"}]
```

GET /documents?type=표준

```
200 [{"id":1,"title":"작업표준서","type":"표준"}]
```

★ 결과가 없어도 404 가 아닙니다. ?type=없는종류 로 요청하면 빈 배열 [] 과 200 이 나옵니다. “목록을 달라” 는 요청은 성공했고, 그 목록이 비어 있을 뿐입니다. 빈 목록은 정상이지 에러가 아닙니다. 이걸 헛갈리는 사람이 아주 많습니다.

하나만 보기

섹션 5

```
if (번호글자 !== undefined && req.method === "GET") {
```

주소에서 꺼낸 것은 언제나 글자입니다. 숫자로 바꿔야 합니다.

```
const 번호 = Number(번호글자);
```

숫자로 바꿀 수 없는 것이 올 수 있습니다. /documents/abc 처럼요.

```
if (!Number.isInteger(번호)) {
  JSON응답(res, 400, { error: "번호는 숫자여야 합니다" });
  return;
}
```

GET /documents/abc

```
400 {"error":"번호는 숫자여야 합니다"}
```

```
const 문서 = 문서들.find((문서) => 문서.id === 번호);
```

find 는 못 찾으면 undefined 를 줍니다. 반드시 확인해야 합니다.

```
if (!문서) {
  JSON응답(res, 404, { error: `${번호}번 문서가 없습니다` });
  return;
}
```

GET /documents/99

```
404 {"error":"99번 문서가 없습니다"}
```

```
JSON응답(res, 200, 문서);
    return;
}
```

GET /documents/1

200 {"id":1,"title":"작업표준서","type":"표준"}

★ 여기가 목록과 결정적으로 다릅니다. "1번 문서를 달라" 했는데 없으면 그건 실패입니다. 404 가 맞습니다.

정리하면

목록 요청 + 결과 0건 → 200 []	(성공, 비어 있음)
개별 요청 + 못 찾음 → 404	(실패)

새로 만들기 (Create)

섹션 6

```
if (번호글자 === undefined && req.method === "POST") {
    const 글자 = await 본문읽기(req);

    let 데이터;
    try {
        데이터 = JSON.parse(글자 || "{}");
    } catch {
        JSON응답(res, 400, { error: "JSON 형식이 아닙니다" });
        return;
    }
}
```

catch 뒤의 (에러) 를 생략했습니다. 안 쓸 거라면 빼도 됩니다.

```
if (!데이터.title) {
    JSON응답(res, 400, { error: "title 을 넣어 주세요" });
    return;
}
```

POST /documents

{"type":"점검"}

400 {"error":"title 을 넣어 주세요"}


```
const 새문서 = {
  id: 다음번호,
  title: 데이터.title,
  type: 데이터.type || "기타", // 안 주면 기본값
};

다음번호 += 1;
문서들.push(새문서);
```

201 은 '만들었다' 입니다. 200 이 아닙니다. (개념02) Location 헤더로 "만든 것은 이 주소에 있다" 를 알려 주는 게 예의입니다.

```
res.writeHead(201, {
  "Content-Type": "application/json; charset=utf-8",
  Location: `/documents/${새문서.id}`,
});
res.end(JSON.stringify(새문서));
return;
}
```

POST /documents

```
{ "title": "설비점검표", "type": "점검" }
```

201 { "id": 3, "title": "설비점검표", "type": "점검" }

★ 만든 것을 그대로 돌려주는 이유 보낸 쪽은 id 를 모릅니다. 서버가 정해 주니까요. 돌려주지 않으면 화면을 새로 그리려고 목록을 또 불러와야 합니다. 만든 결과를 돌려주면 요청 한 번을 아깁니다. 습관처럼 돌려주세요.

고치기 (Update)

섹션 7

```
if (번호글자 !== undefined && req.method === "PUT") {
  const 번호 = Number(번호글자);
  const 문서 = 문서들.find((문서) => 문서.id === 번호);

  if (!문서) {
    JSON응답(res, 404, { error: `${번호}번 문서가 없습니다` });
    return;
  }

  const 글자 = await 본문읽기(req);
  let 데이터;
  try {
```

```

데이터 = JSON.parse(글자 || "{}");
} catch {
  JSON응답(res, 400, { error: "JSON 형식이 아닙니다" });
  return;
}

```

온 것만 바꿉니다. 안 온 것은 그대로 둡니다. ?? 는 "왼쪽이 null 이나 undefined 일 때만 오른쪽" 입니다. (JS자료 09단원)

```

문서.title = 데이터.title ?? 문서.title;
문서.type = 데이터.type ?? 문서.type;

JSON응답(res, 200, 문서);
return;
}

```

```

PUT /documents/3
{"title":"설비점검표(개정)"}

200 {"id":3,"title":"설비점검표(개정)","type":"점검"}

```

type 을 안 보냈는데 “점검” 이 그대로 남아 있는 것을 보세요.

★ 왜 || 가 아니라 ?? 인가 title 을 빈 글자 "" 로 보내면 || 는 이걸 ‘없는 값’ 으로 보고 무시합니다. ?? 는 null/undefined 만 없는 값으로 봅니다. "" 는 통과시킵니다. “일부러 빈 값으로 고치고 싶다” 를 살리려면 ?? 여야 합니다. 숫자 0 을 다룰 때도 똑같은 함정이 있습니다.

지우기 (Delete)

섹션 8

```

if (번호글자 !== undefined && req.method === "DELETE") {
  const 번호 = Number(번호글자);
  const 있나 = 문서들.some((문서) => 문서.id === 번호);

  if (!있나) {
    JSON응답(res, 404, { error: `${번호}번 문서가 없습니다` });
    return;
  }
}

```

filter 는 ‘남길 것’ 을 고릅니다. 지울 것을 빼고 새 배열을 만듭니다.

```

문서들 = 문서들.filter((문서) => 문서.id !== 번호);

```

지웠으니 돌려줄 내용이 없습니다. 204 입니다.

```

내용없음(res);
return;
}

```

DELETE /documents/3

204

★ 문서들을 `let` 으로 선언한 이유가 여기 있습니다. `filter` 는 원본을 안 바꾸고 새 배열을 줍니다. 그 새 배열을 다시 담아야 하므로 `const` 로는 안 됩니다.

`splice` 로 원본에서 빼내는 방법도 있지만, "새로 만들어 갈아 끼우기" 가 실수가 적습니다. (React 에서도 이 방식입니다)

경로는 맞는데 메서드가 다른 경우

섹션 9

```
JSON응답(res, 405, { error: "이 주소에서는 쓸 수 없는 방법입니다" });
```

PATCH /documents

405 {"error":"이 주소에서는 쓸 수 없는 방법입니다"}

```
});
```

```

server.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/documents`);
});

```

지우고 나서 목록을 다시 보면

3번을 지웠으니 처음 두 개만 남아 있습니다.

GET /documents

200 [{"id":1,"title":"작업표준서","type":"표준"}, {"id":2,"title":"검사성적서","type":"성적"}]

다음번호는 4 인 채로 남아 있습니다. 3 으로 되돌리지 않습니다. 지워진 번호를 재활용하면 옛날 기록과 헷갈리기 때문입니다. (02단원 연습문제 14에서 본 것과 같은 이야기입니다)

Postman 으로 순서대로 해 보기

아래를 위에서부터 차례로 눌러 보세요. 서버는 켜 둔 채로 합니다.

1	GET	http://localhost:3000/documents	2건
2	POST	http://localhost:3000/documents	201, id 3
		Body → raw → JSON	
		{ "title": "설비점검표", "type": "점검" }	
3	GET	http://localhost:3000/documents	3건으로 늘어남
4	GET	http://localhost:3000/documents/3	방금 만든 것
5	PUT	http://localhost:3000/documents/3	제목만 바뀜
		{ "title": "설비점검표(개정)" }	
6	DELETE	http://localhost:3000/documents/3	204, 본문 없음
7	GET	http://localhost:3000/documents	다시 2건
8	Ctrl+C 로 서버를 끄고 다시 켜 뒤 1번을 다시		다시 2건

8번이 오늘의 마지막 교훈입니다. 2번에서 만든 문서는 서버를 끄는 순간 사라집니다. 그래서 데이터베이스가 필요합니다.

직접 해 볼 것

◦ 직접 해보기 1

목록에 정렬을 추가해 보세요. ?sort=desc 이면 id 가 큰 것부터 나오게 합니다. (힌트: [...문서들].sort((a, b) => b.id - a.id) 원본을 정렬하면 진짜로 순서가 바뀌어 버립니다. 복사본을 만들어 정렬하세요)

◦ 직접 해보기 2

제목 검색을 추가해 보세요. ?q=표준 이면 title 에 '표준' 이 든 것만 나오게 합니다. (힌트: includes)

◦ 직접 해보기 3

★ 이번 단원의 진짜 과제입니다. 문서들 을 파일에 저장해서, 서버를 켜다 켜도 남게 만드세요.

- 02단원 개념05 의 불러오기/저장하기를 그대로 씁니다
- 서버가 켜질 때 documents.json 을 읽어 문서들 에 담고
- POST / PUT / DELETE 마다 다시 저장합니다
- 파일이 없으면 빈 배열로 시작해야 합니다 (ENOENT)
- 다음번호 도 저장해야 합니다. 안 하면 켜다 켜릴 때 번호가 처음부터 다시 시작해 겹칩니다

다 만들었으면 위 8번 시나리오를 다시 해 보세요.
이번에는 서버를 켜다 켜도 3번 문서가 남아 있어야 합니다.

◦ 직접 해보기 4

지금 이 서버는 두 사람이 동시에 POST 하면 어떻게 될까요? 🖋 3 을 파일로 만들었다면 무슨 일이 생길까요? (02단원 개념05 마지막에 적힌 네 가지 이유를 다시 읽어 보세요)

자주 하는 실수

이런 실수를 합니다

주소에서 꺼낸 번호를 숫자로 안 바꿈

문서들.find((문서) => 문서.id === 번호글자) → 1 === "1" 은 `false` 입니다. 영원히 못 찾습니다. 경로에서 나온 것도 쿼리에서 나온 것도 전부 글자입니다. `Number()` 를 거치세요.

이런 실수를 합니다

find 결과를 확인 안 하고 바로 씀

```
const 문서 = 문서들.find(...);
```

문서.title ← 못 찾았으면 `undefined` 입니다 → `TypeError: Cannot read properties of undefined` → 서버가 죽습니다. find 뒤에는 항상 `if (!문서)` 를 붙이세요.

이런 실수를 합니다

만든 뒤 200 을 돌려줌

틀린 건 아니지만 201 이 정확합니다. Location 헤더도 함께 주세요.

이런 실수를 합니다

204 에 본문을 담음

```
res.writeHead(204); res.end("자웠습시다");
```

→ 204 는 '내용 없음' 이라는 약속이라 이 글자는 버려집니다. 메시지를 주고 싶으면 200 에 담아 보내세요.

이런 실수를 합니다

목록이 비었다고 404 를 줌

빈 배열은 정상입니다. 200 에 [] 를 담아 주세요. 프론트가 "에러다!" 하고 빨간 화면을 띄우게 됩니다.

이런 실수를 합니다

문서들을 `const` 로 선언하고 `filter` 로 삭제하려 함

→ `TypeError: Assignment to constant variable`. 배열 자체를 갈아 끼울 거라면 `let` 이어야 합니다.

이런 실수를 합니다

PUT 에서 받은 데이터를 통째로 덮어씀

문서들[번호] = 데이터 → 보낸 쪽이 title 만 보냈다면 id 와 type 이 통째로 사라집니다. 속성 하나씩 골라 덮으세요.

정리

- 1 CRUD 는 만들기·읽기·고치기·지우기 넷이다. 세상의 거의 모든 서버가 이 넷을 한다.
- 2 주소에서 꺼낸 번호는 글자다. Number 로 바꿔야 find 가 맞는다.
- 3 find 는 못 찾으면 `undefined` 다. 확인하고 없으면 404 를 준다.
- 4 목록은 결과가 비어도 404 가 아니다. 빈 배열에 200 이다. '하나만 보기' 와 결정적으로 다른 점이다.
- 5 새로 만들면 201 을 주고, 만든 것을 그대로 돌려준다. 번호가 몇 번으로 붙었는지 부른 쪽이 알아야 하기 때문이다.
- 6 지우면 204 를 주고 본문을 담지 않는다. 204 는 '내용 없음' 이라는 뜻이다.
- 7 PUT 은 받은 것만 갈아 끼운다. 통째로 덮으면 안 보낸 값이 사라진다. 없으면 원래 값을 쓰도록 ?? 를 쓴다. || 는 0 과 "" 까지 갈아 끼워서 안 된다.
- 8 배열에서 지우려면 `let` 으로 선언해야 한다. filter 는 새 배열을 돌려주기 때문이다.
- 9 메모리에 담은 데이터는 서버를 끄면 사라진다. 07단원에서 파일로 옮긴다.

HTTP 와 주소 다루기

RUN node 연습문제.js

이 파일은 '서버 없이' 푸는 문제들입니다. 서버를 직접 만드는 문제는 연습문제_서버.js 에 따로 있습니다.

규칙 · 위에서부터 순서대로 푸세요. 아래 문제가 위 문제의 답을 씁니다. · 한 문제 풀 때마다 저장하고 실행해서 결과를 확인하세요. · '기대 출력' 과 다르면 다음으로 넘어가지 마세요. · 막히면 개념 파일을 다시 보세요. 정답을 먼저 보지 마세요.

문제 1 주소 뜯어보기

아래 주소에서 경로 / 호스트 / 포트를 각각 출력하세요. 포트가 숫자인지 글자인지도 `typeof` 로 확인해 보세요.

힌트: `new URL(글자)` 로 만들면 `pathname`, `host`, `port` 를 꺼낼 수 있습니다. (개념01)

```
const 주소글자 = "http://localhost:3000/api/documents?page=2&size=10";
```

```
기대 출력  /api/documents
           localhost:3000
           3000
           string
```

여기에 코드를 쓰세요

문제 2 쿼리 꺼내기

문제 1의 주소에서 ① page 값 ② sort 값 (없습니다) ③ sort 가 있는지 없는지 를 각각 출력하세요.

힌트: `searchParams.get` / `searchParams.has`

```
기대 출력  2
           null
           false
```

여기에 코드를 쓰세요

문제 3 페이지 번호를 안전하게 꺼내기

URL 을 받아서 page 값을 '숫자로' 돌려주는 함수를 만드세요. 이름은 페이지번호 로 하세요.

조건: - page 가 없으면 1 - 숫자가 아니면 1 - 1보다 작으면 1

힌트: Number 로 바꾸고, `Number.isInteger` 로 확인합니다.

★ 함정이 하나 숨어 있습니다. page 가 없을 때 get 은 `null` 을 주는데, `Number(null)` 은 `NaN` 이 아니라 0 입니다. 그래서 "숫자인가" 만 확인하면 통과해 버립니다. 범위까지 확인해야 합니다.

기대 출력	3
	1
	1
	1

(아래 네 줄로 시험해 보세요) 페이지번호(`new URL("http://localhost:3000/x?page=3")`) 페이지번호(`new URL("http://localhost:3000/x")`) 페이지번호(`new URL("http://localhost:3000/x?page=abc")`) 페이지번호(`new URL("http://localhost:3000/x?page=0")`)

여기에 코드를 쓰세요

문제 4 같은 이름의 쿼리가 여러 개일 때

아래 주소에서 tag 를 꺼내 보세요. ① get 으로 꺼낸 것 ② getAll 로 꺼낸 것 ③ 없는 이름을 getAll 로 꺼낸 것

셋의 차이를 눈으로 보는 게 목적입니다.

```
const 주소4 = new URL("http://localhost:3000/search?tag=안전&tag=품질&q=점검");
```

기대 출력	안전
-------	----

— '안전', '품질'

[]

여기에 코드를 쓰세요

문제 5 상황에 맞는 상태코드 고르기

아래 여덟 가지 상황에 어떤 상태코드를 돌려줘야 할까요? 상황과 코드를 짝지어 출력하세요.

힌트: 객체에 { "상황": 코드 } 로 적고, `Object.keys` 로 돌면서 찍으면 됩니다.

화살표는 → 입니다. 복사해서 쓰세요.

쓸 수 있는 코드: 200 201 204 400 401 403 404 500

기대 출력	목록을 잘 돌려줌 → 200
	새 문서를 만들 → 201
	지웠고 돌려줄 내용이 없음 → 204
	title 을 안 보냄 → 400
	로그인을 안 함 → 401
	로그인은 했지만 권한이 없음 → 403
	그 번호의 문서가 없음 → 404
	내 코드에서 <code>TypeError</code> 가 남 → 500

여기에 코드를 쓰세요

문제 6 상태코드를 무리로 나누기

상태코드를 받아서 어느 무리인지 글자로 돌려주는 함수 분류 를 만드세요.

200~299 → "성공"
300~399 → "다른 곳으로"
400~499 → "보낸 쪽 잘못"

500 이상 → "서버 잘못" 그 밖 → "알 수 없음"

만든 뒤 [200, 204, 301, 404, 500] 을 map 으로 돌려 " / " 로 이어서 한 줄로 출력하세요.

기대 출력 성공 / 성공 / 다른 곳으로 / 보낸 쪽 잘못 / 서버 잘못

여기에 코드를 쓰세요

문제 7 경로를 조각내기

경로 글자를 받아 '/' 로 나눈 조각 배열을 돌려주는 함수 조각내기 를 만드세요. 빈 조각은 버려야 합니다.

힌트: `split("/")` 한 뒤 `filter` 로 빈 글자를 버립니다.

`filter(Boolean)` 이라고 쓰면 됩니다.

기대 출력

— 'documents'

— 'documents', '12'

— 'documents', '12', 'files'

[]

(시험할 값: `"/documents"`, `"/documents/12"`, `"/documents/12/files"`, `"/"`)

여기에 코드를 쓰세요

문제 8 경로를 분석하는 함수

경로를 받아 { resource, id } 모양의 객체를 돌려주는 함수 경로분석 을 만드세요.

- resource 는 첫 조각. 없으면 null - id 는 두 번째 조각을 '숫자로' 바꾼 것. 없으면 null

힌트: 문제 7의 조각내기를 그대로 쓰면 됩니다.

?? 는 "왼쪽이 없으면 오른쪽" 입니다. (JS자료 09단원)

★ 속성 이름은 영어로 쓰세요. 한글로 쓰면 출력에 따옴표가 붙어 다르게 보입니다.

```
기대 출력  { resource: 'documents', id: null }
           { resource: 'documents', id: 12 }
           { resource: null, id: null }
```

(id 가 12 로 따옴표 없이 나와야 합니다. '12' 로 나오면 Number 를 안 한 것입니다)

여기에 코드를 쓰세요

문제 9 헤더는 소문자입니다

아래 headers 객체에서 ① "Content-Type" 으로 찾은 값 ② "content-type" 으로 찾은 값 을 출력해 차이를 보세요.

그다음, 대소문자를 아무렇게나 줘도 찾아 주는 함수 헤더값(headers, 이름) 을 만드세요.

```
const headers = {
  host: "localhost:3000",
  "content-type": "application/json; charset=utf-8",
  "user-agent": "PostmanRuntime/7.36.0",
};
```

```
기대 출력  undefined
           application/json; charset=utf-8
           application/json; charset=utf-8
           PostmanRuntime/7.36.0
```

(셋째 줄은 헤더값(headers, "Content-Type"), 넷째 줄은 헤더값(headers, "USER-AGENT") 입니다)

여기에 코드를 쓰세요

문제 10 본문이 JSON 인지 확인하기

headers 를 받아 본문이 JSON 인지 `true/false` 로 돌려주는 함수 `JSON본문인가` 를 만드세요.

★ 두 가지를 조심하세요. ① 실제로 오는 값은 `"application/json; charset=utf-8"` 입니다.

`===` 로 비교하면 `false` 가 됩니다. 들어 있는지를 봐야 합니다.

② 헤더가 아예 없을 수도 있습니다. 이때 터지지 않아야 합니다.

```
기대 출력    true
              true
              false
              false
```

(시험할 값)

```
{ "content-type": "application/json; charset=utf-8" }
{ "content-type": "application/json" }
{ "content-type": "text/plain" }
{ }
```

여기에 코드를 쓰세요

문제 11 빠진 값 찾기

데이터와 '반드시 있어야 할 이름들' 을 받아, 빠진 이름만 배열로 돌려주는 함수 `빠진값찾기(데이터, 필수들)` 를 만드세요.

그다음 그 결과로 상태코드를 정해 출력하세요. 빠진 게 없으면 → 201 빠진 게 있으면 → 400 과 함께 빠진 이름들을 심표로 이어서

기대 출력

— 'type', 'author'

[] 400 type 을(를) 넣어 주세요

(시험할 값) 빠진값찾기({ title: "작업표준서" }, ["title", "type", "author"]) 빠진값찾기({ title: "a", type: "b", author: "c" }, ["title", "type", "author"]) 빠진값찾기({ title: "작업표준서" }, ["title", "type"]) → 이걸로 상태코드 결정

여기에 코드를 쓰세요

문제 12 주소 설계 고치기

아래는 잘못 설계된 주소들입니다. REST 규칙에 맞게 고치세요. 고친 것을 배열에 담고 한 줄씩 출력하면 됩니다.

① GET /getDocuments 문서 목록 보기 ② GET /createDocument 문서 새로 만들기 ③ GET /getDocument?id=3 3번 문서 하나 보기 ④ GET /deleteDocument?id=3 3번 문서 지우기 ⑤ GET /getDocumentsByType?type=표준 표준 종류만 보기

규칙 · 주소에 동사를 쓰지 않습니다. 무엇을 할지는 메서드가 말합니다. · 자원 이름은 복수형 명사입니다. (documents) · 하나를 꼭 집을 때는 경로에, 걸러 볼 때는 쿼리에 씁니다.

기대 출력 GET /documents
 POST /documents
 GET /documents/3
 DELETE /documents/3
 GET /documents?type=표준

여기에 코드를 쓰세요

문제 13 이 코드는 왜 안 될까요

아래 두 줄의 주석을 지우고 실행하면 에러가 납니다. 실행해서 에러를 직접 본 다음, 아래 빈칸을 채우세요.

```
const 나쁜주소 = new URL("/documents?page=2");
console.log(나쁜주소.pathname);
```

에러 이름

왜 이런 에러가 났나: _____

어떻게 고치나 (코드로): _____

서버 코드에서는 왜 이 문제가 늘 생기나: _____ (힌트: req.url 에는 무엇이 들어 있었나요? 개념03 섹션 2를 다시 보세요)

고친 코드를 아래에 쓰고 실행해 보세요.

```
기대 출력    /documents
             http://localhost:3000/documents?page=2
```

여기에 코드를 쓰세요

설비 관리 API 만들기

RUN node 연습문제_서버.js

고칠 때마다 Ctrl + C 로 끄고 다시 켜야 반영됩니다.

개념05 에서 만든 문서 API 를 흉내 내어, 이번에는 '설비' API 를 만듭니다. 아래 일곱 문제를 위에서부터 순서대로 채우세요.

확인은 Postman 으로 합니다. (개념04 아래쪽에 쓰는 법이 있습니다) 한 문제 만들 때마다 서버를 다시 켜고 Postman 으로 눌러 보세요. 일곱 개를 다 만들고 한 번에 확인하려 하지 마세요. 어디가 틀렸는지 못 찾습니다.

만들 주소는 이렇습니다.

GET	/health	서버가 살아 있나
GET	/equipments	목록
GET	/equipments?line=A	A라인만
GET	/equipments/1	하나만
POST	/equipments	새로 등록
PATCH	/equipments/1	상태만 바꾸기

DELETE /equipments/1 지우기

```
const http = require("http");

const PORT = process.env.PORT || 3000;
```

준비 (여기는 그대로 두세요) —————

```
let 설비들 = [
  { id: 1, name: "컨베이어", line: "A", status: "가동" },
  { id: 2, name: "프레스", line: "B", status: "정지" },
];

let 다음번호 = 3;

const 상태값들 = ["가동", "정지"];

function JSON응답(res, 상태코드, 데이터) {
  res.writeHead(상태코드, { "Content-Type": "application/json; charset=utf-8" });
  res.end(JSON.stringify(데이터));
}
```

```

}

function 본문읽기(req) {
  return new Promise((resolve, reject) => {
    let 쌓은글자 = "";
    req.setEncoding("utf-8");
    req.on("data", (조각) => (쌓은글자 += 조각));
    req.on("end", () => resolve(쌓은글자));
    req.on("error", reject);
  });
}

```

본문을 읽고 JSON 으로 바꾸는 것까지 한 번에 합니다. 형식이 틀리면 `null` 을 돌려줍니다. 부르는 쪽에서 400 을 내면 됩니다.

```

async function JSON본문(req) {
  const 글자 = await 본문읽기(req);
  try {
    return JSON.parse(글자 || "{}");
  } catch {
    return null;
  }
}

const server = http.createServer(async (req, res) => {
  console.log(`${req.method} ${req.url}`);

  const 주소 = new URL(req.url, `http://${req.headers.host}`);
  const 경로 = 주소.pathname;

```

문제 1 /health 만들기

GET /health 에 200 과 { status: "ok" } 를 돌려주세요.

```
기대 출력    GET /health → 200 {"status":"ok"}
```

힌트: JSON응답(res, 200, { status: "ok" }) 한 줄이면 됩니다.

응답한 뒤 `return` 을 꼭 붙이세요.

여기에 코드를 쓰세요

준비 (여기도 그대로 두세요)

```
const 조각들 = 경로.split("/").filter(Boolean);
const 첫조각 = 조각들[0];
const 번호글자 = 조각들[1];

if (첫조각 !== "equipments") {
  JSON응답(res, 404, { error: "그런 주소가 없습니다" });
  return;
}
```

여기부터는 첫조각이 반드시 equipments 입니다.

```
/equipments      → 번호글자 는 undefined
/equipments/1    → 번호글자 는 "1"
```

문제 2 목록 보기 + 라인으로 걸러 보기

GET /equipments 에 설비들 전부를 돌려주세요. ?line=A 가 붙어 오면 그 라인 것만 걸러서 돌려주세요.

```
기대 출력   GET /equipments      → 200 설비 2건
            GET /equipments?line=A → 200 컨베이어 1건
            GET /equipments?line=Z → 200 []      * 404 가 아닙니다
```

힌트: 조건은 (번호글자 === undefined && req.method === "GET") 입니다.

쿼리는 주소.searchParams.get("line") 으로 꺼냅니다. 없으면 null 입니다.

여기에 코드를 쓰세요

문제 3 하나만 보기

GET /equipments/1 에 1번 설비를 돌려주세요.

세 가지 경우를 모두 처리해야 합니다.

```
숫자가 아닌 번호 → 400 { error: "번호는 숫자여야 합니다" }  
그 번호가 없음 → 404 { error: "99번 설비가 없습니다" }
```

찾음 → 200 설비 객체

```
기대 출력  GET /equipments/1 → 200  
            {"id":1,"name":"컨베이어","line":"A","status":"가동"}  
            GET /equipments/99 → 404  
            GET /equipments/abc → 400
```

★ 순서가 중요합니다. 숫자인지 먼저 보고, 그다음에 찾으세요. 순서를 바꾸면 /equipments/abc 에 404 가 나옵니다. “없는 설비” 와 “잘못 쓴 주소” 는 다른 문제입니다.

힌트: Number(번호글자) 로 바꾸고 Number.isInteger 로 확인합니다.

find 는 못 찾으면 undefined 를 줍니다. 반드시 if (!설비) 를 붙이세요.

여기에 코드를 쓰세요

문제 4 새로 등록하기

POST /equipments 로 온 본문을 읽어 새 설비를 만드세요.

```
- JSON 형식이 아니면 → 400 { error: "JSON 형식이 아닙니다" }
```

- name 이나 line 이 없으면 → 400 빠진 이름을 알려 주세요 - 잘 왔으면 → 201 만든 설비를 그대로 돌려주세요

새 설비의 모양:

```
{ id: 다음번호, name: 받은name, line: 받은line, status: "정지" }
```

```
기대 출력  POST /equipments { "name": "용접로봇" }
           → 400 { "error": "line 을(를) 넣어 주세요" }
```

```
POST /equipments { "name": "용접로봇", "line": "C" }
```

```
→ 201 { "id": 3, "name": "용접로봇", "line": "C", "status": "정지" }
```

★ status 는 보낸 값을 쓰지 말고 무조건 "정지" 로 시작하세요. 보낸 쪽이 { "status": "폭발" } 을 넣어도 데이터가 안 망가져야 합니다. 규칙은 서버가 쥐고 있어야 합니다.

힌트: `const 데이터 = await JSON본문(req);` ← `await` 를 빠뜨리지 마세요

빠진 것 모으기는 연습문제.js 문제 11 과 같습니다.

"name", "line" .filter((키) ⇒ !데이터[키])

만든 뒤 다음번호 += 1 과 설비들.push(새설비) 를 잊지 마세요.

여기에 코드를 쓰세요

문제 5 상태만 바꾸기

PATCH /equipments/3 으로 온 { "status": "가동" } 을 반영하세요.

- 그 번호가 없으면 → 404

```
- status 가 가동/정지가 아니면 → 400 { error: "status 는 가동 또는 정지여야 합니다" }
```

- 잘 왔으면 → 200 바뀐 설비를 돌려주세요

기대 결과 (문제 4에서 3번을 만든 뒤에 해 보세요):

```
PATCH /equipments/3 { "status": "폭발" }
```

```
→ 400
```

```
PATCH /equipments/3 { "status": "가동" }
```

```
→ 200 {"id":3,"name":"용접로봇","line":"C","status":"가동"}
```

★ PUT 이 아니라 PATCH 인 이유 PUT 전체를 통째로 바꾼다 (안 보낸 값은 사라지는 게 원칙) PATCH 일부만 바꾼다 "상태만 바꾼다" 니까 PATCH 가 어울립니다.

힌트: 정해진 값만 받으려면 상태값들.includes(데이터.status) 를 쓰세요.

find 로 찾은 객체의 속성을 바꾸면 배열 안의 것도 함께 바뀝니다.
(객체는 주소를 들고 있기 때문입니다 - JS자료 07단원)

여기에 코드를 쓰세요

문제 6 지우기

DELETE /equipments/3 으로 그 설비를 지우세요.

- 그 번호가 없으면 → 404 - 지웠으면 → 204 ★ 본문은 보내지 않습니다

```
기대 출력  DELETE /equipments/3 → 204, 본문 없음
           DELETE /equipments/3 → 404 (이미 지웠으니)
```

힌트: 있는지 확인은 some 이 편합니다.

지우기는 설비들 = 설비들.filter((설비) => 설비.id !== 번호) 입니다.
204 는 JSON응답 을 쓰지 말고 직접 쓰세요.
res.writeHead(204);
res.end();

여기에 코드를 쓰세요

문제 7 여기까지 안 걸린 요청

위 어디에도 안 걸렸다면, 주소는 맞는데 메서드가 다른 경우입니다. 405 와 { error: "이 주소에서는 쓸 수 없는 방법입니다" } 를 돌려주세요.

기대 출력 PUT /equipments → 405

★ 왜 404 가 아닌가 /equipments 는 있는 주소입니다. PUT 을 안 만들었을 뿐입니다. 404 를 주면 상대가 "주소를 잘못 썼나?" 하고 엉뚱한 데를 찾습니다.

여기에 코드를 쓰세요

```
});

server.listen(PORT, () => {
  console.log(`설비 관리 API 가 켜졌습니다. http://localhost:${PORT}/equipments`);
});
```

다 만들었으면 이 순서대로 눌러 보세요

```
1 GET    /health          200 {"status":"ok"}
2 GET    /equipments      2건
3 GET    /equipments?line=A 1건 (컨베이어)
4 GET    /equipments?line=Z 0건 - 200 과 [] 입니다
5 GET    /equipments/1    컨베이어 하나
6 GET    /equipments/99 404
7 GET    /equipments/abc  400
8 POST   /equipments      400 - line 을 안 넣었으니
        { "name": "용접로봇" }
```

9 POST /equipments 201, id 3, status 는 "정지"

```
{ "name": "용접로봇", "line": "C" }
```

10 PATCH /equipments/3 400

```
{ "status": "폭발" }
```

11 PATCH /equipments/3 200, status 가 "가동" 으로

```

    { "status": "가동" }
12 DELETE /equipments/3 204, 본문 없음
13 DELETE /equipments/3 404
14 PUT    /equipments          405

```

4번과 6번의 차이를 꼭 눈으로 확인하세요. 둘 다 “없다” 인데 하나는 200, 하나는 404 입니다. 왜 그런지 말로 설명할 수 있으면 이 단원은 끝난 것입니다.

잘 안 될 때 보는 곳

브라우저가 계속 빙빙 돌니다 → 그 갈래에서 `res.end` 를 안 불렀습니다.

Cannot set headers after they are sent → 응답 뒤에 `return` 을 빠뜨려서 두 번 보내려 한 것입니다.

고쳤는데 그대로입니다 → 서버를 다시 안 켰습니다. Ctrl + C 후 다시 node 로 실행하세요.

EADDRINUSE → 앞서 켜 서버가 아직 살아 있습니다. 그 터미널에서 Ctrl + C 를 누르세요.

```

터미널을 닫아 버렸다면 아래로 찾아서 끕니다.
netstat -ano | findstr :3000
taskkill /PID <번호> /F

```

본문이 `undefined` 입니다 → `await` 를 빠뜨렸거나, Postman 에서 Body → raw → JSON 을 안 골랐습니다.

한글이 깨집니다 → Content-Type 에 charset=utf-8 이 빠졌습니다.

아무리 해도 404 만 나옵니다 → `console.log(경로, 번호글자, req.method)` 를 맨 위에 찍어 보세요.

내가 생각한 값과 실제 값이 다른 경우가 대부분입니다.

Express 시작하기

OPEN 04_Express_시작하기

04

```
const app = express();  
app.get("/", (req, res) => {  
  res.send("안녕하세요. Express 입니다.");  
  res.json({ status: "ok" });  
});
```

01 Express 시작하기

02 라우팅 (주소와 함수를 짝지어 등록하기)

03 요청에서 값 꺼내기 (params · query · body · headers)

04 응답 보내기와 에러 처리

05 nodemon 과 파일 나누기 (express.Router)

- 연습문제

- 서버 연습문제

Express 시작하기

RUN node 개념01_Express_시작하기.js

그러면 Ctrl + C

03단원에서 맨손으로 서버를 만들어 봤습니다. 그리고 불편한 점 네 가지를 봤습니다.

① if 가 계속 늘어난다 ② writeHead 와 `JSON.stringify` 를 반복한다 ③ `/documents/12` 에서 12 를 직접 잘라 내야 한다 ④ POST 본문을 읽는 코드가 길다

Express 는 이 넷을 전부 해결해 줍니다. 이번 단원이 끝나면 03단원 개념05 의 200줄짜리 API 가 40줄이 됩니다.

Express 는 무엇인가

섹션 1

Express 는 Node 의 http 모듈을 감싼 '도구' 입니다. 새로운 언어도, 새로운 서버도 아닙니다. 여전히 안에서는 http 가 돕니다.

그래서 03단원에서 배운 것이 하나도 안 버려집니다. 요청·응답·상태코드·헤더는 그대로입니다. 쓰는 방법만 짧아집니다.

```
http    "요청이 오면 이 함수 하나가 다 처리해라"
Express "이 주소로 이 방법의 요청이 오면 이 함수를 불러라"
```

이 차이 하나가 전부입니다.

설치

섹션 2

Express 는 내장 모듈이 아닙니다. npm 으로 받아야 합니다. (01단원 개념03)

```
npm install express
```

이 자료는 맨 위 폴더(백엔드자료)에 이미 설치해 두었습니다. Node 는 require 할 때 `node_modules` 를 위쪽 폴더로 계속 거슬러 올라가며 찾습니다. 그래서 04단원 폴더 안에 `node_modules` 가 없어도 잘 찾습니다.

여러분이 새 프로젝트를 만들 때는 이렇게 합니다.


```
mkdir 내프로젝트
cd 내프로젝트
npm init -y
npm install express
```

```
const express = require("express");
```

require 의 결과가 '함수' 입니다. 그래서 불러서 앱을 만듭니다.

```
const app = express();

const PORT = process.env.PORT || 3000;
```

가장 작은 서버

섹션 3

03단원의 이 코드가

```
const server = http.createServer((req, res) => {

  const 주소 = new URL(req.url, `http://${req.headers.host}`);
  if (주소.pathname === "/") {
    res.writeHead(200, { "Content-Type": "text/plain; charset=utf-8" });
    res.end("안녕하세요");
    return;
  }
  ...

});
```

이렇게 줄어듭니다.

```
app.get("/", (req, res) => {
  res.send("안녕하세요. Express 입니다.");
});
```

```
GET /

200 안녕하세요. Express 입니다.
```

한 줄씩 뜯어봅니다.

app.get(...)	GET 요청만 받겠다
"/"	이 주소로 올 때만

(req, res) => { ... } 그때 이 함수를 실행해라 res.send(...) 이 글자를 돌려줘라

writeHead 가 사라졌습니다. Content-Type 도, charset 도, return 도 없습니다. Express 가 알아서 붙여 줍니다.

★ 이렇게 '주소와 함수를 짝지어 등록하는 것' 을 라우팅(routing)이라고 합니다. 길 안내라는 뜻입니다. "이 주소로 온 요청은 이쪽으로 보내라" 입니다.

JSON 돌려주기

섹션 4

```
app.get("/health", (req, res) => {
  res.json({ status: "ok" });
});
```

GET /health

200 {"status":"ok"}

res.json 하나로 세 가지가 한꺼번에 됩니다.

① JSON.stringify 를 대신 해 줍니다 ② Content-Type: application/json 을 붙여 줍니다 ③ charset=utf-8 까지 붙여 줍니다 ← 한글이 안 깨집니다

03단원의 이 세 줄이

```
res.writeHead(200, { "Content-Type": "application/json; charset=utf-8" });
res.end(JSON.stringify({ status: "ok" }));
return;
```

한 줄이 되었습니다.

```
app.get("/documents", (req, res) => {
  res.json([
    { id: 1, title: "작업표준서" },
    { id: 2, title: "검사성적서" },
  ]);
});
```

GET /documents

200 [{"id":1,"title":"작업표준서"},{"id":2,"title":"검사성적서"}]

배열도 그대로 넣으면 됩니다.

send 와 json 의 차이

섹션 5

```
app.get("/send-text", (req, res) => {
  res.send("그냥 글자입니다");
});
```

GET /send-text

200 그냥 글자입니다

```
app.get("/send-object", (req, res) => {
  res.send({ a: 1 }); // 객체를 넣으면 res.json 과 똑같이 동작합니다
});
```

GET /send-object

200 {"a":1}

정리하면

```
res.send(글자)   → text/html 로 보냅니다
res.send(객체)   → json 으로 보냅니다 (res.json 을 대신 불러 줍니다)
res.json(객체)   → 언제나 json 으로 보냅니다
```

★ API 를 만들 때는 res.json 만 쓰세요. send 는 상황에 따라 달라져서, 읽는 사람이 결과를 예측하기 어렵습니다. “여기는 JSON 을 준다” 를 코드로 분명히 말하는 편이 낫습니다.

상태코드 붙이기

섹션 6

```
app.get("/created", (req, res) => {
  res.status(201).json({ id: 3, message: "만들었습니다" });
});
```

GET /created

201 {"id":3,"message":"만들었습니다"}

res.status(201) 은 상태코드만 정해 두고 자기 자신을 돌려줍니다. 그래서 뒤에 .json(...) 을 이어 붙일 수 있습니다. 이것 체이닝이라고 합니다.

```
res.status(201).json({ ... })
```

↑ 코드만 정함 ↑ 실제로 보냄

status 만 부르고 끝내면 아무것도 안 보내집니다. 브라우저가 계속 기다립니다. 반드시 뒤에 json / send / end 중 하나가 와야 합니다.

내용 없이 상태코드만 보낼 때는 sendStatus 를 씁니다.

```
app.get("/no-content", (req, res) => {
  res.sendStatus(204);
});
```

GET /no-content

204

res.status(204) 와 res.sendStatus(204) 는 다릅니다. 이름이 비슷해서 자주 헷갈립니다.

res.status(204)	코드만 정하고 아직 안 보냄 (뒤에 .json 등이 와야 함)
res.sendStatus(204)	코드를 정하고 바로 보냄 (여기서 끝)

등록한 순서대로 찾습니다

섹션 7

Express 는 요청이 오면 위에서부터 차례로 내려오며 맞는 것을 찾습니다. 처음 맞는 것 하나를 실행하고 멈춥니다.

```
app.get("/order", (req, res) => {
  res.send("첫 번째");
});

app.get("/order", (req, res) => {
  res.send("두 번째"); // 여기까지 오지 않습니다
});
```

GET /order

200 첫 번째

★ 이 성질 때문에 생기는 사고가 있습니다. "모든 주소" 를 받는 규칙을 맨 위에 적으면, 그 아래 것들이 전부 무시됩니다. 넓은 규칙은 항상 아래, 좁은 규칙은 위에 두세요. 개념02 에서 자세히 봅니다.

★ 주소에 한글을 쓰면 안 됩니다

섹션 8

아래 라우트는 아무리 해도 걸리지 않습니다. 등록은 됐는데 실행이 안 됩니다.

```
app.get("/한글주소", (req, res) => {
  res.send("여기는 절대 실행되지 않습니다");
});
```

GET /한글주소

404

왜 그럴까요?

브라우저는 주소에 한글이 있으면 보내기 전에 암호 같은 글자로 바꿉니다. 이것 퍼센트 인코딩이라고 합니다.

내가 친 주소 /한글주소
실제로 나가는 것 /%ED%95%9C%EA%B8%80%EC%A3%BC%EC%86%8C

Express 는 이 '바뀐 글자' 를 그대로 놓고 라우트와 비교합니다. "/한글주소" 와 "/%ED%95%9C..." 는 당연히 다릅니다. 그래서 404 입니다.

브라우저 주소창에는 /한글주소 라고 예쁘게 보이니까 "분명 주소는 맞는데 왜 404 지?" 하고 한참 헤매게 됩니다.

그래서 규칙이 하나 생깁니다.

★ 라우트 주소는 영어 소문자와 하이픈(-)만 씁니다.

/documents /equipment-logs /users/:id

한글은 '값' 에만 씁니다. 값은 Express 가 알아서 되돌려 줍니다.

/documents?type=표준 → req.query.type 은 "표준" 으로 잘 들어옵니다
/search/작업표준서 → req.params 도 잘 들어옵니다 (개념02)

03단원에서는 이 문제가 없었습니다. 우리가 URL 로 분석해서 pathname 을 꺼내 썼기 때문입니다. 그때는 Node 가 알아서 되돌려 준 글자를 비교했습니다.

이 파일에서도 처음에는 /만들었다치고, /내용없음 처럼 한글로 썼다가 전부 404 가 나서 영어로 바꿨습니다. 정말 흔한 사고입니다.

없는 주소는 Express 가 알아서 404

섹션 9

03단원에서는 맨 아래에 404 를 직접 적었습니다. Express 는 아무 규칙에도 안 걸리면 알아서 404 를 줍니다. 안 적어도 됩니다.

GET /없는주소

404

★ 그런데 그 404 의 내용은 HTML 입니다.

```
<!DOCTYPE html> ... <pre>Cannot GET /없는주소</pre> ...
```

브라우저로 보면 괜찮지만, JSON 을 기대하던 프론트엔드는 "JSON 인 줄 알았는데 HTML 이 왔다" 며 에러를 냅니다.

그래서 API 서버는 404 도 JSON 으로 직접 만들어 줍니다. 개념04 에서 합니다.

서버 켜기

섹션 10

```
app.listen(PORT, () => {  
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);  
});
```

http 의 server.listen 과 완전히 같습니다. createServer 를 우리가 안 불렀을 뿐, Express 가 안에서 대신 부릅니다.

같은 서버, 두 가지 방법

http	Express
------	---------

서버 만들기 createServer `express()` 주소 나누기 if 문 `app.get("/주소", ...)` 메서드 나누기 if (req.method) `app.get / app.post ...` JSON 보내기 `writeHead + stringify + end res.json(...)` 상태코드 `writeHead(201, ...)` `res.status(201)` 한글 안 깨지게 `charset=utf-8` 직접 붙이기 자동 응답 뒤 `return` 반드시 필요 안 해도 됨* 없는 주소 맨 아래에 직접 자동 404

* 라우트 함수 하나가 끝나면 Express 가 다음으로 안 넘어갑니다.

그래도 `res.json` 뒤에 코드가 더 있으면 그건 실행됩니다.
실수를 막으려면 `return res.json(...)` 처럼 붙이는 습관이 좋습니다.

직접 해 볼 것

서버를 켜고 브라우저에서 아래를 열어 보세요.

`http://localhost:3000/` `http://localhost:3000/health` `http://localhost:3000/documents`
`http://localhost:3000/created`

<code>http://localhost:3000/없는주소</code>	← HTML 404 를 눈으로 확인
<code>http://localhost:3000/한글주소</code>	← 라우트를 만들었는데도 404

▹ 직접 해보기 1

/about 을 만들어 자기소개를 res.json 으로 돌려주세요.

▹ 직접 해보기 2

섹션 7 의 두 /order 중 위엿것을 지우고 다시 켜 보세요. 이번에는 무엇이 나오나요?

▹ 직접 해보기 3

res.status(201) 만 쓰고 .json(...) 을 지워 보세요. 브라우저가 어떻게 되나요? 왜 그럴까요?
(확인했으면 반드시 되돌리세요)

▹ 직접 해보기 4

03단원 개념03 파일과 이 파일을 나란히 열어 보세요. 같은 일을 하는 부분끼리 짝지어 보고, 줄 수를 세어 보세요.

▹ 직접 해보기 5

/한글주소 를 브라우저 F12 → Network 탭을 켜 놓고 열어 보세요. 요청 목록에서 실제로 나간 주소가 어떻게 생겼는지 확인하세요. 그다음 라우트를 "/korean" 으로 바꾸고 다시 켜서 되는지 보세요.

자주 하는 실수

이런 실수를 합니다

express() 를 안 부름

```
const app = express; ← 괄호가 없습니다
```

→ TypeError: app.get is not a function express 는 '앱을 만들어 주는 함수' 입니다. 불러야 앱이 나옵니다.

이런 실수를 합니다

app.get 을 listen 아래에 씀

listen 아래에 적어도 동작은 합니다. 하지만 읽기 어려워집니다. 라우트는 전부 위에, listen 은 맨 마지막에 두세요. 관례입니다.

이런 실수를 합니다

응답을 두 번 보냄

```
res.json({ a: 1 });  
res.json({ b: 2 });
```

→ Error: Cannot set headers after they are sent to the client 03단원과 똑같은 에러입니다. `return` 을 붙이면 막을 수 있습니다.

이런 실수를 합니다

res.send 와 res.json 을 섞어 씀

틀린 건 아니지만, 어떤 주소는 문자열을 어떤 주소는 객체를 주게 됩니다. 프론트에서 받는 코드가 주소마다 달라져야 합니다. json 으로 통일하세요.

이런 실수를 합니다

Cannot find module 'express'

npm install express 를 안 했거나, 다른 폴더에서 실행한 것입니다. 이 자료는 백엔드자료 폴더 아래에서 실행해야 찾을 수 있습니다.

이런 실수를 합니다

파일을 고치고 새로고침만 함

03단원과 똑같습니다. 서버를 다시 켜야 합니다. 개념05 에서 이걸 자동으로 해 주는 nodemon 을 씁니다.

이런 실수를 합니다

라우트 주소를 한글로 씀

app.get("/설비목록", ...) 은 등록은 되지만 절대 실행되지 않습니다. 에러도 안 나고 그냥 404 만 나와서 원인을 찾기 아주 어렵습니다. 주소는 영어로, 한글은 값에만. (섹션 8)

정리

- 1 Express 는 03단원에서 손으로 하던 네 가지를 대신해 준다. 갈래 나누기 · 응답 만들기 · 주소에서 값 꺼내기 · 본문 읽기.
- 2 npm i express 로 받고, `express()` 로 app 을 만들고, `app.listen` 으로 켜다.
- 3 `app.get("/주소", (req, res) => ...)` 하나가 03단원의 if 한 덩어리를 대신한다.
- 4 `res.json` 은 객체를 알아서 글자로 바꾸고 `Content-Type` 까지 붙여 준다. `JSON.stringify` 도 `writeHead` 도 직접 쓰지 않는다.

- 5 `res.send` 는 넣는 것에 따라 달라진다. 글자면 `text/html`, 객체면 `json` 이다. API 는 `res.json` 으로 통일하는 편이 헛갈리지 않는다.
- 6 상태코드는 `res.status(201).json(...)` 처럼 앞에 붙인다. `status` 만 쓰고 끝내면 응답이 안 나가서 브라우저가 계속 기다린다.
- 7 라우트는 등록된 순서대로 찾는다. 먼저 걸리는 것이 이긴다.
- 8 주소에 한글을 쓰면 안 된다. 에러도 없이 404 만 나서 원인을 찾기 아주 어렵다. 주소는 영어로, 한글은 값에만.
- 9 없는 주소는 Express 가 알아서 404 를 준다. 03단원에서 손으로 하던 일이다.

라우팅 (주소와 함수를 짝지어 등록하기)

RUN node 개념02_라우팅.js

그러면 Ctrl + C

03단원에서는 이렇게 갈래를 나눴습니다.

```
if (경로 === "/documents") {

  if (req.method === "GET") { ... }
  if (req.method === "POST") { ... }

}
```

Express 에서는 이렇게 씁니다.

```
app.get("/documents", ...)
app.post("/documents", ...)
```

if 가 사라졌습니다. 이번 파일에서 이 문법을 전부 봅니다.

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;
```

메서드마다 함수가 하나씩

섹션 1

app 에는 메서드 이름과 똑같은 함수가 있습니다.

```
app.get    app.post    app.put
app.patch  app.delete
```

모양은 전부 같습니다. app.메서드(주소, 함수)

```
app.get("/documents", (req, res) => {
  res.json({ 방법: "GET", 뜻: "목록을 봅니다" });
});
```

GET /documents

200 {"방법":"GET","뜻":"목록을 봅니다"}

```
app.post("/documents", (req, res) => {
  res.status(201).json({ 방법: "POST", 뜻: "새로 만듭니다" });
});
```

POST /documents

201 {"방법":"POST","뜻":"새로 만듭니다"}

```
app.delete("/documents", (req, res) => {
  res.json({ 방법: "DELETE", 뜻: "지웁니다" });
});
```

DELETE /documents

200 {"방법":"DELETE","뜻":"지웁니다"}

★ 주소가 같아도 메서드가 다르면 완전히 다른 라우트입니다. 위 세 개는 전부 “/documents” 지만 서로 아무 상관이 없습니다. 03단원에서 if 를 두 겹으로 썼던 것이 여기서서는 그냥 세 줄입니다.

★ delete 는 예약어 아닌가요? 자바스크립트에 delete 연산자가 있긴 하지만, app.delete 처럼 점 뒤에 오는 이름으로는 쓸 수 있습니다. 문제없습니다. (Express 4 에는 app.del 이라는 옛날 이름도 있었는데 5 에서 없어졌습니다)

경로에 값 끼워 넣기 (경로 파라미터)

섹션 2

03단원에서 가장 귀찮았던 부분입니다.

```
const 조각들 = 경로.split("/").filter(Boolean);
const 번호 = Number(조각들[1]);
```

Express 는 주소에 콜론(:)을 붙이면 알아서 꺼내 줍니다.

```
app.get("/documents/:id", (req, res) => {
  res.json({
    받은id: req.params.id,
    타입: typeof req.params.id,
  });
});
```

GET /documents/12

200 {"받은id":"12","타입":"string"}

:id 라고 쓰면 req.params.id 에 들어옵니다. 이름은 마음대로 지어도 됩니다.

"/documents/:번호" → req.params.번호 ← 됩니다. 되지만 권장하지 않습니다
"/documents/:id" → req.params.id

★★ 타입이 "string" 인 것을 꼭 보세요. 주소에서 나온 것은 언제나 글자입니다. 03단원과 똑같습니다.
Express 를 쓴다고 숫자로 바뀌지 않습니다.

문서.id === req.params.id → 1 === "1" → false. 영원히 못 찾습니다
문서.id === Number(req.params.id) → 이렇게 써야 합니다

이 실수는 Express 를 처음 쓰는 사람이 거의 100% 합니다.

파라미터를 여러 개

섹션 3

```
app.get("/lines/:line/equipments/:id", (req, res) => {  
  res.json(req.params);  
});
```

GET /lines/A/equipments/7

200 {"line":"A","id":"7"}

req.params 는 객체입니다. 콜론이 붙은 것들이 전부 들어 있습니다.

★ 주소를 이렇게 겹쳐 쓰는 것은 "A라인에 속한 7번 설비" 라는 뜻입니다. 자원 사이에 포함 관계가 있을 때만 쓰세요. 관계가 없는데 겹쳐 쓰면 주소만 길어지고 좋을 게 없습니다.

값에 한글이 들어오는 것은 괜찮습니다

섹션 4

```
app.get("/search/:word", (req, res) => {  
  res.json({ 검색어: req.params.word, 글자수: req.params.word.length });  
});
```

GET /search/작업표준서

200 {"검색어":"작업표준서","글자수":5}

개념01 섹션 8 에서 “주소에 한글 쓰지 말라” 고 했는데 이걸 왜 될까요?

안 되는 것: `app.get("/한글주소", ...)` ← 주소를 한글로 '정한' 것 되는 것: `app.get("/search/:word", ...)`
 ← 값으로 한글이 '들어온' 것

주소 자체는 영어(/search/:word)입니다. :word 자리에 무엇이 오든 Express 가 받아서 되돌려 줍니다.
 정리하면 — 주소는 영어로 정하고, 한글은 값으로 받으세요.

있어도 되고 없어도 되는 파라미터

섹션 5

Express 5 에서 문법이 바뀐 부분입니다. 종괄호로 감쌉니다.

```
app.get("/reports{/:year}", (req, res) => {
  const 연도 = req.params.year || "전체";
  res.json({ 연도 });
});
```

```
GET /reports/2026
200 {"연도":"2026"}
```

```
GET /reports
200 {"연도":"전체"}
```

하나의 라우트가 두 주소를 모두 받습니다.

★★ Express 4 를 쓴 블로그를 보면 이렇게 나옵니다.

```
app.get("/reports/:year?", ...) ← Express 4 방식
```

Express 5 에서 이렇게 쓰면 서버가 아예 안 켜집니다.

```
TypeError: Unexpected ? at index 14: /reports/:year?
```

물음표가 없어지고 종괄호가 생겼다고 기억하세요.

```
Express 4: /reports/:year?
Express 5: /reports{/:year}
```

슬래시가 종괄호 '}' 에 있는 것에 주의하세요. /reports{year} 가 아닙니다. year 가 없을 때 /reports/ 처럼 슬래시가 남으면 안 되기 때문입니다.

이것도 Express 5 에서 문법이 바뀌었습니다.

```
app.get("/files/*splat", (req, res) => {
  res.json({ 조각들: req.params.splat });
});
```

```
GET /files/2026/03/보고서.pdf
```

```
200 {"조각들":["2026","03","보고서.pdf"]}
```

*splat 이라고 쓰면 나머지 전부가 req.params.splat 에 배열로 들어옵니다. splat 이라는 이름은 마음대로 바뀌어도 됩니다. (*rest, *path ...)

★★ Express 4 에서는 이름 없이 별표만 썼습니다.

```
app.get("/files/*", ...) ← Express 4 방식. 5 에서는 서버가 안 켜집니다
→ TypeError: Missing parameter name at index 8: /files/*
```

별표 뒤에 반드시 이름을 붙여야 합니다.

와일드카드는 파일 경로처럼 '몇 조각이 올지 모를 때' 만 씁니다. 그 밖에는 쓰지 마세요. 다음 섹션의 사고가 납니다.

★ 순서가 전부입니다

Express 는 위에서부터 내려오며 처음 맞는 것 하나만 실행합니다. 그래서 '넓은 것' 을 위에 두면 아래가 전부 죽습니다.

좁은 것 먼저

```
app.get("/orders/new", (req, res) => {
  res.json({ 화면: "새 주문 만들기" });
});
```

```
GET /orders/new
```

```
200 {"화면":"새 주문 만들기"}
```

넓은 것 나중

```
app.get("/orders/:id", (req, res) => {
  res.json({ 화면: "주문 상세", id: req.params.id });
});
```

GET /orders/7

200 {"화면":"주문 상세","id":"7"}

★ 이 둘의 순서를 바꾸면 어떻게 될까요?

app.get("/orders/:id", ...) ← 먼저 등록

app.get("/orders/new", ...) ← 나중

/orders/new 로 요청하면 :id 가 먼저 걸립니다. id 에 "new" 가 들어갑니다. 그러면 "new 번 주문을 찾아라" 가 되어 404 가 납니다. 아래 /orders/new 는 영원히 실행되지 않습니다.

에러도 안 납니다. 그냥 조용히 엉뚱한 곳으로 갑니다. Express 에서 "왜 이 라우트가 안 걸리지?" 의 대부분이 순서 문제입니다.

규칙 하나만 외우세요.

★ 구체적인 주소를 위에, :id 나 *splat 같은 것을 아래에.

▹ 직접 해보기

1

위 두 라우트의 순서를 실제로 바꿔 보고, /orders/new 가 어떻게 되는지 눈으로 확인하세요.
(확인했으면 반드시 되돌리세요)

메서드를 안 가리는 app.all

섹션 8

```
app.all("/ping", (req, res) => {
  res.json({ 받은방법: req.method });
});
```

GET /ping

200 {"받은방법":"GET"}

POST /ping

200 {"받은방법":"POST"}

어떤 메서드로 와도 이 함수가 실행됩니다. 405(그 방법은 안 됩니다)를 직접 만들 때 씁니다. 연습문제에서 써 봅니다.

섹션 10에서 `"/documents"` 를 세 번 반복해서 적었습니다. 주소를 한 번만 적고 메서드를 이어 붙일 수 있습니다.

```
app.route("/notes")
  .get((req, res) => {
    res.json({ 방법: "GET" });
  })
  .post((req, res) => {
    res.status(201).json({ 방법: "POST" });
  })
  .delete((req, res) => {
    res.json({ 방법: "DELETE" });
  });
```

GET /notes

200 {"방법":"GET"}

POST /notes

201 {"방법":"POST"}

DELETE /notes

200 {"방법":"DELETE"}

주소를 한 곳에만 적으니 오타가 줄어듭니다. `"/notes"` 를 `"/memos"` 로 바꿀 때 한 군데만 고치면 됩니다.

꼭 이렇게 써야 하는 건 아닙니다. 취향입니다. 다만 남의 코드에서 자주 보이니 읽을 줄은 알아야 합니다.

없는 메서드로 오면

PATCH /notes

404

`/notes` 에 `patch` 는 안 만들었습니다. Express 는 404 를 줍니다. 405 가 더 정확하지만, Express 는 기본으로 405 를 만들어 주지 않습니다. 필요하다면 `app.all` 로 직접 만들어야 합니다. (연습문제 참고)

```
app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/documents`);
});
```

Express 4 와 5 의 문법 차이 (블로그를 볼 때 주의하세요)

하는 일 Express 4 Express 5 (지금 쓰는 것)

선택 파라미터	/reports/:year?	/reports{/:year}
---------	-----------------	------------------

아무거나 다 받기 /files/* /files/*splat 전체 받기 app.get("*", ...) app.get("/*splat", ...) DELETE 짧은 이름 app.del() 없어짐. app.delete()

4 문법을 5 에 쓰면 '서버가 아예 안 켜집니다'. 그래서 오히려 다행입니다. 조용히 틀리는 것보다 낫습니다.

에러 메시지에 pathToRegexpError 라는 주소가 보이면 "라우트 주소 문법이 틀렸구나" 라고 생각하면 됩니다.

지금 설치된 버전을 확인하려면 터미널에서

```
npm list express
```

직접 해 볼 것

▹ 직접 해보기 1

/equipments/:id/logs 라우트를 만들어 req.params 를 그대로 돌려주게 해 보세요.

▹ 직접 해보기 2

섹션 2 의 라우트를 고쳐서, id 를 숫자로 바꾼 결과와 타입을 함께 돌려주게 해 보세요. { 원본: "12", 숫자: 12, 타입: "number" }

▹ 직접 해보기 3

/files/*splat 에 아주 긴 경로를 넣어 보세요. http://localhost:3000/files/a/b/c/d/e 배열이 몇 개짜리로 나오나요?

▹ 직접 해보기 4

app.get("/files/*", ...) 로 고쳐서 다시 켜 보세요. 어떤 에러가 나나요? 어디를 고치라고 알려 주나요? (확인했으면 반드시 되돌리세요)

자주 하는 실수

이런 실수를 합니다

req.params.id 를 숫자로 안 바꿈

가장 많이 하는 실수입니다. find 가 계속 undefined 를 줍니다. Number(req.params.id) 를 습관으로 만드세요.

이런 실수를 합니다

:id 라우트를 구체적인 주소보다 위에 둬

/orders/new 가 :id 에 걸려서 영원히 실행되지 않습니다. 에러가 안 나서 찾기 어렵습니다. 구체적인 것 먼저.

이런 실수를 합니다

콜론을 빼먹음

app.get("/documents/id", ...) ← 이걸 진짜 'id' 라는 글자입니다 /documents/id 로 와야만 걸립니다.
/documents/12 는 404 입니다.

이런 실수를 합니다

Express 4 문법을 그대로 씀

/reports/:year? 나 /files/* 를 쓰면 서버가 안 켜집니다. 블로그 글이 언제 쓰였는지 확인하세요.

이런 실수를 합니다

req.params 를 req.param 이라고 씀

s 가 빠지면 `undefined` 입니다. Express 4 에는 req.param() 함수가 있었지만 5 에서 없어졌습니다.

이런 실수를 합니다

라우트 주소 앞에 / 를 안 붙임

app.get("documents", ...) → 걸리지 않습니다. 항상 / 로 시작하세요.

정리

- 1 메서드마다 함수가 하나씩이다. `app.get` · `app.post` · `app.put` · `app.delete`. 03단원의 중첩 if 가 통째로 사라진다.
- 2 경로에 값을 끼우려면 `:id` 처럼 콜론을 붙인다. `req.params.id` 로 꺼낸다.
- 3 `req.params` 값은 언제나 글자다. 숫자로 쓸 것이면 `Number` 로 바꾼다.
- 4 값에 한글이 오는 것은 괜찮다. 안 되는 것은 주소 자체에 한글을 쓰는 것이다.
- 5 순서가 전부다. `/documents/new` 는 `/documents/:id` 보다 위에 있어야 한다. 아래에 두면 `new` 가 `:id` 에 먼저 걸려서 영영 안 불린다.
- 6 Express 5 는 문법이 바뀌었다. `:id?` 는 `{/:id}`, `/*` 는 `/*splat` 이다. 블로그 대부분이 4 기준이라 그대로 쓰면 서버가 아예 안 켜진다.
- 7 `app.all` 은 메서드를 안 가리고, `app.route` 는 같은 주소를 한 번에 묶는다.
- 8 경로는 맞는데 그 메서드를 안 만들었으면 Express 는 404 를 준다. 405 가 더 정확하지만 기본으로 만들어 주지 않는다. 필요하면 `app.all` 로 직접 만든다.

요청에서 값 꺼내기 (params · query · body · headers)

RUN node 개념03_요청_읽기.js

그러면 Ctrl + C

클라이언트가 서버에 값을 보내는 길은 네 개뿐입니다.

① 경로 /documents/12 → req.params ② 쿼리 /documents?type=표준 → req.query ③ 본문 POST 로 보낸 JSON → req.body ④ 헤더 Content-Type 같은 것 → req.headers

03단원에서는 ①②를 직접 잘라 냈고, ③을 조각으로 모았습니다. Express 는 넷 다 알아서 담아 줍니다. 꺼내 쓰기만 하면 됩니다.

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;
```

본문을 읽으려면 이 한 줄이 필요합니다

섹션 1

★★ 이 줄이 없으면 req.body 가 영원히 undefined 입니다. Express 를 처음 쓰는 사람의 1번 질문이 “왜 req.body 가 undefined 인가요?” 입니다. 답은 거의 항상 “이 줄을 안 썼다” 입니다.

```
app.use(express.json());
```

이 줄이 하는 일이 03단원 개념04 의 본문읽기 함수 전체입니다.

req.on(“data”, ...) 로 조각 모으기 req.on(“end”, ...) 에서 합치기 JSON.parse 로 객체 만들기 실패하면 400 내기

이 열댓 줄이 app.use(express.json()) 한 줄로 끝납니다.

HTML 폼(<form>)으로 보낸 것도 받으려면 한 줄을 더 씁니다.

```
app.use(express.urlencoded({ extended: true }));
```

★ app.use 는 “모든 요청이 여기를 먼저 지나가게 하라” 는 뜻입니다. 이런 것을 미들웨어라고 부릅니다. 05단원에서 자세히 만듭니다. 지금은 “라우트보다 위에 적어야 한다” 만 기억하세요. 아래에 적으면 라우트가 먼저 실행되어 버려서 소용이 없습니다.

경로에서 (req.params)

섹션 2

```
app.get("/documents/:id/logs/:logId", (req, res) => {
  res.json({
    params: req.params,
    id타입: typeof req.params.id,
  });
});
```

GET /documents/12/logs/5

200 {"params":{"id":"12","logId":"5"},"id타입":"string"}

콜론을 붙인 것들이 전부 req.params 에 들어옵니다. 언제나 글자입니다. 개념02 에서 본 그대로입니다.

쿼리에서 (req.query)

섹션 3

```
app.get("/documents", (req, res) => {
  res.json({
    query: req.query,
    page타입: typeof req.query.page,
  });
});
```

GET /documents?type=표준&page=2

200 {"query":{"type":"표준","page":"2"},"page타입":"string"}

03단원의 이 코드가

```
const 주소 = new URL(req.url, `http://${req.headers.host}`);
const 종류 = 주소.searchParams.get("type");
```

req.query.type 하나로 끝났습니다.

★ 한글이 잘 들어오는 것을 보세요. Express 가 되돌려 줍니다. 주소(라우트)에는 한글을 못 쓰지만, 값에는 얼마든지 쓸 수 있습니다.

GET /documents

200 {"query":{},"page타입":"undefined"}

★ 쿼리가 하나도 없으면 req.query 는 빈 객체 {} 입니다. undefined 가 아닙니다. 그래서 req.query.page 를 바로 꺼내도 터지지 않습니다. undefined 가 나올 뿐입니다.

GET /documents?tag=안전&tag=품질

200 {"query":{"tag":["안전","품질"]},"page타입":"undefined"}

★★ 같은 이름이 두 번 오면 값이 '배열' 이 됩니다. 글자 하나를 기대하고 쓴 코드가 여기서 깨집니다.

```
req.query.tag.toLowerCase()
→ 하나만 오면 잘 됩니다
→ 두 개 오면 배열이라 TypeError. 서버가 죽습니다
```

누군가 주소를 손으로 고쳐서 ?tag=a&tag=b 로 보내면 그걸로 끝입니다. 안전하게 쓰려면 이렇게 합니다.

```
const tag = Array.isArray(req.query.tag) ? req.query.tag[0] : req.query.tag;
```

본문에서 (req.body)

섹션 4

```
app.post("/documents", (req, res) => {
  res.json({
    body: req.body,
    타입: typeof req.body,
  });
});
```

POST /documents

{"title":"작업표준서","count":3}

200 {"body":{"title":"작업표준서","count":3},"타입":"object"}

★ count 가 3 입니다. 따옴표가 없습니다. 본문(JSON)으로 온 것은 타입이 그대로 살아 있습니다. 숫자는 숫자로, true 는 true 로 들어옵니다.

경로·쿼리는 전부 글자, 본문은 타입이 살아 있음. 이 차이가 “왜 어떤 건 Number 를 해야 하고 어떤 건 안 해도 되나” 의 답입니다.

★ 본문이 undefined 로 나오는 두 경우

섹션 5

POST /documents

200 {"타입":"undefined"}

본문을 아예 안 보내면 req.body 는 undefined 입니다. (JSON.stringify 는 undefined 인 속성을 아예 빼고 찍습니다. 그래서 body 가 안 보입니다)

★★ Express 4에서는 이때 빈 객체 {}였습니다. Express 5에서 바뀌었습니다. 그래서 옛날 블로그를 보고 이렇게 쓰면 서버가 죽습니다.

```
const { title } = req.body;
→ TypeError: Cannot destructure property 'title' of 'req.body' as it is undefined.
```

안전하게 쓰려면

```
const { title } = req.body || {};
```

또는 아예 없으면 400을 내보내면 됩니다. 아래 섹션 7에서 합니다.

```
POST /documents form:name=hong&age=20
200 {"body":{"name":"hong","age":"20"},"타입":"object"}
```

이건 HTML 폼으로 보낸 경우입니다. express.urlencoded가 받아 줍니다. age가 "20"으로 글자인 것을 보세요. 폼으로 온 것은 전부 글자입니다. JSON과 다릅니다.

★★ 두 번째 경우가 진짜 함정입니다 — Content-Type을 안 맞춘 경우.

Postman에서 Body → raw만 고르고 JSON으로 안 바꾸면 Content-Type이 text/plain으로 나갑니다. express.json()은 "이건 내가 읽을 게 아니네"하고 그냥 지나갑니다. 결과는 req.body가 undefined.

본문을 분명히 보냈는데 undefined라면 99%이 문제입니다. Postman의 Body 탭에서 raw 오른쪽 드롭다운이 JSON인지 확인하세요.

JSON이 깨져서 오면

섹션 6

```
POST /documents
{title:"따옴표없음"}
400
```

03단원에서는 이걸 직접 막았습니다.

```
try { 데이터 = JSON.parse(글자); }
catch { res.writeHead(400); ... }
```

express.json()이 대신 해 줍니다. 우리는 아무것도 안 썼는데 400이 나왔습니다.

★ 다만 지금은 응답 '내용'이 HTML입니다. 브라우저로 보면 에러 화면이 나옵니다. API 서버라면 JSON으로 바꿔 줘야 합니다. 개념04에서 합니다.

```
app.post("/equipments", (req, res) => {
```

① 본문이 아예 없을 수 있으니 `|| {}` 로 받쳐 둡니다

```
const { name, line } = req.body || {};
```

② 필수 값을 확인합니다

```
const 빠진것 = [];
if (!name) 빠진것.push("name");
if (!line) 빠진것.push("line");

if (빠진것.length > 0) {
  return res.status(400).json({ error: `${빠진것.join(", ")} 을(를) 넣어 주세요`
});
}
```

③ 여기부터는 믿고 써도 됩니다

```
res.status(201).json({ name, line });
});
```

POST /equipments

400 {"error": "name, line 을(를) 넣어 주세요"}

POST /equipments

{"name": "용접로봇"}

400 {"error": "line 을(를) 넣어 주세요"}

POST /equipments

{"name": "용접로봇", "line": "C"}

201 {"name": "용접로봇", "line": "C"}

★ `return res.status(...)` 처럼 `return` 을 붙인 이유 Express 는 `return` 을 안 붙여도 다음 라우트로 안 넘어갑니다. 하지만 '이 함수 안의 아래 코드' 는 그대로 실행됩니다. `return` 이 없으면 400 을 보낸 뒤에 201 까지 보내려다 에러가 납니다.

→ Cannot set headers after they are sent to the client

응답 앞에는 습관처럼 `return` 을 붙이세요.

★ 구조 분해로 꺼낸 이유 `req.body.name`, `req.body.line` 을 매번 쓰면 길어집니다. 한 번 꺼내 두면 아래 코드가 짧아집니다. (JS자료 09단원)

헤더에서 (req.headers / req.get)

섹션 8

```
app.get("/headers", (req, res) => {
  res.json({
    "req.get 은 대문자로 찾아도 됨": req.get("User-Agent") !== undefined,
    "req.headers 는 대문자로 못 찾음": req.headers["User-Agent"] !== undefined,
    "req.headers 는 소문자로 찾음": req.headers["user-agent"] !== undefined,
  });
});
```

GET /headers

```
200 {"req.get 은 대문자로 찾아도 됨":true,"req.headers 는 대문자로 못 찾음":false,"r
eq.headers 는 소문자로 찾음":true}
```

03단원에서 본 대소문자 문제입니다. `req.headers` 는 Node 그대로라 전부 소문자입니다. `req.get(이름)` 은 대소문자를 안 가립니다. 이쪽이 안전합니다.

자주 쓰는 헤더

<code>req.get("Content-Type")</code>	본문이 무슨 형식인가
<code>req.get("Authorization")</code>	로그인 증표 – PART 4 에서 씁니다
<code>req.get("User-Agent")</code>	어떤 프로그램이 보냈나

어디에 담아야 하나

섹션 9

```
app.get("/decide", (req, res) => {
  res.json({
    경로: "그것 하나를 꼭 집는 값. /documents/12 의 12",
    쿼리: "있어도 되고 없어도 되는 값. 걸러 보기·정렬·쪽 번호",
    본문: "새로 만들거나 고칠 내용. 길고 구조가 있는 값",
    헤더: "내용이 아니라 '요청에 대한 정보'. 형식·인증",
  });
});
```

GET /decide

```
200 {"경로":"그것 하나를 꼭 짚는 값. /documents/12 의 12","쿼리":"있어도 되고 없어도 되는 값. 걸러 보기·정렬·쪽 번호","본문":"새로 만들거나 고칠 내용. 길고 구조가 있는 값","헤더":"내용이 아니라 '요청에 대한 정보'. 형식·인증"}
```

헛갈릴 때 판단 기준

“이게 없으면 무슨 요청인지 알 수 없다” → 경로 “없어도 요청은 성립한다” → 쿼리 “글자가 길다 / 구조가 있다 / 비밀스럽다” → 본문

마지막이 중요합니다. 비밀번호를 쿼리에 담으면 안 됩니다. 주소는 서버 기록에 그대로 남습니다. 브라우저 기록에도 남고, 남에게 링크를 보낼 때 딸려 갑니다.

```
app.listen(PORT, () => {  
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/documents`);  
});
```

03단원과 나란히 놓고 보기

하는 일 03단원 (http) Express

경로에서 값 split("/").filter(Boolean)[1] req.params.id 쿼리에서 값 new URL(...).searchParams.get("q") req.query.q 본문 읽기 Promise + on("data") + on("end") req.body JSON 로 바꾸기 JSON.parse + try/catch 자동 깨진 JSON 직접 400 자동 400 폼 데이터 직접 파싱 req.body

03단원의 서론 줄이 Express 에서는 세 줄입니다. 그런데 사라진 게 아니라 express.json() 안에 들어 있을 뿐입니다. 안에서 무슨 일이 벌어지는지 알기 때문에, 안 될 때 어디를 볼지 알 수 있습니다.

직접 해 볼 것

▫ 직접 해보기 1

맨 위의 app.use(express.json()) 을 주석 처리하고 POST /documents 를 보내 보세요. req.body 가 어떻게 되나요? (확인했으면 반드시 되돌리세요)

▫ 직접 해보기 2

app.use(express.json()) 을 맨 아래(listen 바로 위)로 옮겨 보세요. 이번에는 어떻게 되나요? 왜 그럴까요?

▫ 직접 해보기 3

Postman 에서 Body → raw 를 고르되 JSON 으로 바꾸지 말고 POST /documents 를 보내 보세요. 무엇이 나오나요?

▫ 직접 해보기 4

/documents 에 정렬을 추가해 보세요. ?sort=desc 면 { 정렬: "내림차순" } 을 돌려주고, 없으면 { 정렬: "기본" } 을 돌려주게 하세요.

▫ 직접 해보기 5

/documents?tag=a&tag=b 로 보내 보고, tag 가 하나든 여럿이든 항상 '배열' 로 다루는 코드를 써 보세요. (힌트: [].concat(req.query.tag || []))

04

자주 하는 실수

이런 실수를 합니다

app.use(express.json()) 을 안 씀

req.body 가 undefined 입니다. 1번 원인입니다.

이런 실수를 합니다

app.use(express.json()) 을 라우트 아래에 씀

위에서부터 실행되므로 라우트가 먼저 걸립니다. 역시 undefined 입니다. 미들웨어는 항상 라우트보다 위에.

이런 실수를 합니다

req.body 를 바로 구조 분해함

```
const { title } = req.body;
```

→ 본문을 안 보내면 TypeError 로 서버가 죽습니다. req.body || {} 로 받치거나, 먼저 없는지 확인하세요.

이런 실수를 합니다

req.query 의 값을 숫자로 안 바꿈

req.query.page * 10 은 "2" * 10 = 20 이라 우연히 됩니다. req.query.page + 10 은 "2" + 10 = "210" 이 됩니다. 우연히 되는 코드가 제일 무섭습니다. Number 로 분명히 바꾸세요.

이런 실수를 합니다

같은 이름의 쿼리가 여러 개 올 수 있다는 걸 잊음

글자인 줄 알았는데 배열이라 TypeError 가 납니다.

이런 실수를 합니다

req.params 와 req.query 를 헷갈림

/documents/12 → req.params.id /documents?id=12 → req.query.id 주소 모양이 다릅니다. 콜론을 쓴 것만 params 입니다.

이런 실수를 합니다

비밀번호를 쿼리에 담음

/login?password=1234 는 서버 기록에 그대로 남습니다. 비밀은 반드시 본문(POST)에 담으세요.

정리

- 1 클라이언트가 값을 보내는 길은 넷뿐이다. 경로 req.params · 쿼리 req.query · 본문 req.body · 헤더 req.headers.
- 2 본문을 읽으려면 app.use(express.json()) 한 줄이 반드시 있어야 한다. 그리고 라우트보다 위에 뒤야 한다. 아래에 두면 안 읽힌다.
- 3 req.params 와 req.query 값은 언제나 글자다. 숫자로 쓰려면 Number 로 바꾼다.
- 4 Express 5 는 본문이 없으면 req.body 가 undefined 다(4 는 {} 였다). 바로 구조 분해하면 그 자리에서 터진다. 있는지 먼저 본다.
- 5 JSON 이 깨져서 오면 Express 가 알아서 400 을 낸다. 내가 try/catch 할 일이 아니다.
- 6 같은 이름의 쿼리가 여러 개 오면 배열이 된다. 하나로 알고 쓰면 틀린다.
- 7 비밀번호는 쿼리에 담지 않는다. 주소는 기록에 그대로 남는다. 본문에 담는다.
- 8 어디에 담을지는 이렇게 나눈다. 무엇을 가리키는 값은 경로, 걸러 보는 조건은 쿼리, 만들고 고칠 내용은 본문.

응답 보내기와 에러 처리

RUN node 개념04_응답과_에러처리.js

그러면 Ctrl + C

앞에서 요청을 읽었으니 이제 돌려줄 차례입니다. 그리고 '잘못됐을 때 무엇을 돌려줄지' 가 이 파일의 절반입니다.

잘될 때만 생각하고 만든 서버는 실무에서 하루도 못 버팁니다. 에러 처리를 얼마나 챙겼는지가 초보와 중급을 가릅니다.

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

응답을 보내는 방법들

섹션 1

```
app.get("/ok", (req, res) => {
  res.json({ message: "잘 됐습니다" });
});
```

GET /ok

200 {"message":"잘 됐습니다"}

```
app.get("/created", (req, res) => {
```

만들었으면 201, 그리고 만든 것이 어디 있는지 Location 으로 알려 줍니다.

```
res.status(201).location("/documents/3").json({ id: 3 });
});
```

GET /created

201 {"id":3}

```
app.get("/no-content", (req, res) => {
  res.sendStatus(204); // 내용 없이 상태코드만
});
```

GET /no-content

204

```
app.get("/custom-header", (req, res) => {
```

헤더를 직접 붙일 수도 있습니다.

```
  res.set("X-App-Version", "1.0.0");
  res.json({ ok: true });
});
```

GET /custom-header

200 {"ok":true}

정리

res.json(객체)	JSON 으로 보냅니다. API 는 이것만 쓰면 됩니다
res.status(코드)	코드만 정합니다. 뒤에 .json 등이 와야 합니다
res.sendStatus(코드)	코드만 보내고 끝냅니다
res.location(주소)	Location 헤더를 붙입니다

res.set(이름, 값) 아무 헤더나 붙입니다 res.send(글자) 글자를 그대로 보냅니다 (HTML 페이지 등)

전부 res 를 다시 돌려주기 때문에 점으로 이어 붙일 수 있습니다.

```
res.status(201).location("/x/3").json({ ... })
```

없는 주소를 JSON 으로

섹션 2

개념01 에서 본 것처럼, Express 기본 404 는 HTML 입니다. 프론트엔드가 JSON 을 기대하는데 HTML 이 오면 그쪽에서 에러가 납니다.

그래서 API 서버는 404 도 직접 만듭니다. ★ 반드시 '모든 라우트 아래' 에 적어야 합니다. 위에 적으면 전부 404 가 됩니다.

이 파일에서는 아래쪽 섹션 6 에 있습니다. 거기서 다시 봅니다.

에러가 나면 어떻게 되나

섹션 3

```
app.get("/boom-sync", (req, res) => {
```

일부러 터뜨립니다. 실무에서는 오타나 `undefined` 때문에 저절로 납니다.

```
const 없는것 = undefined;
res.json(없는것.속성); // TypeError
});
```

```
GET /boom-sync
```

```
500 {"error":"서버에서 문제가 생겼습니다"}
```

라우트 안에서 에러가 나면 Express 가 잡아서 '에러 처리기' 로 보냅니다. 서버는 죽지 않습니다. 그 요청 하나만 500 이 되고 다음 요청은 정상입니다.

★ 03단원의 맨손 서버였다면 여기서 서버가 통째로 죽었습니다. Express 를 쓰는 큰 이유 중 하나입니다.

```
app.get("/boom-async", async (req, res) => {
  await new Promise((resolve) => setTimeout(resolve, 10));
  throw new Error("비동기 안에서 터졌습니다");
});
```

```
GET /boom-async
```

```
500 {"error":"서버에서 문제가 생겼습니다"}
```

★★ 이게 Express 5 의 가장 큰 선물입니다.

Express 4 에서는 `async` 함수 안의 에러를 못 잡았습니다. 응답이 영원히 안 오고 브라우저가 계속 돌았습니다. 그래서 라우트마다 이렇게 감싸야 했습니다.

```
app.get("/x", async (req, res, next) => {
  try { ... } catch (err) { next(err); }
});
```

Express 5 는 알아서 잡아 줍니다. `try/catch` 를 안 써도 됩니다. 옛날 블로그에 `try/catch` 가 잔뜩 있는 이유가 이것입니다.

내가 만든 에러 던지기

섹션 4

상태코드를 붙인 에러를 만드는 작은 도구입니다. 이런 걸 하나 만들어 두면 라우트가 아주 깔끔해집니다.

```
function HttpError(코드, 메시지) {
  const 에러 = new Error(메시지);
  에러.status = 코드; // 에러 객체에 속성을 더 붙일 수 있습니다 return 에러;
}

const 문서들 = [
  { id: 1, title: "작업표준서" },
  { id: 2, title: "검사성적서" },
];

app.get("/documents/:id", (req, res) => {
  const 번호 = Number(req.params.id);

  if (!Number.isInteger(번호)) {
    throw HttpError(400, "번호는 숫자여야 합니다");
  }

  const 문서 = 문서들.find((문서) => 문서.id === 번호);

  if (!문서) {
    throw HttpError(404, `${번호}번 문서가 없습니다`);
  }

  res.json(문서);
});
```

GET /documents/1

200 {"id":1,"title":"작업표준서"}

GET /documents/abc

400 {"error":"번호는 숫자여야 합니다"}

GET /documents/99

404 {"error":"99번 문서가 없습니다"}

★ throw 를 쓰니 라우트가 짧아졌습니다.

res.status(404).json({ error: ... }) 를 매번 쓰는 것과 비교해 보세요.

“무엇이 잘못됐는지” 만 말하고, “어떻게 응답할지” 는 에러 처리기에 맡깁니다.

응답 모양이 바뀌면 에러 처리기 한 곳만 고치면 됩니다. 라우트가 서른 개여도 마찬가지입니다.

next로 넘기기

섹션 5

```
app.get("/next-error", (req, res, next) => {
```

throw 대신 next(에러)를 써도 똑같습니다.

```
  next(HttpError(403, "권한이 없습니다"));
});
```

```
GET /next-error
403 {"error": "권한이 없습니다"}
```

throw와 next(에러) 중 무엇을 쓰나 보통은 throw가 짧고 읽기 좋습니다. 콜백 '안'에서 에러가 났을 때는 throw가 안 통합니다. 그때 next를 씁니다.

```
fs.readFile("x.txt", (err, data) => {
  if (err) return next(err); ← 여기서 throw 하면 Express가 못 잡습니다
  ...
});
```

fs.promises와 await를 쓰면 이런 일이 없습니다. 그래서 promises를 씁니다.

못 찾은 주소 (라우트 다음, 에러 처리기 앞)

섹션 6

여기까지 내려왔다는 것은 위의 어떤 라우트에도 안 걸렸다는 뜻입니다. 주소를 안 적었으니 '모든 요청'이 여기로 옵니다.

★ req.path는 인코딩된 그대로입니다. 한글이 %EC%97%86...으로 보입니다. 기록으로 남길 때 읽을 수가 없으니 되돌려 줍니다.

```
function 읽기좋은주소(경로) {
  try {
    return decodeURIComponent(경로);
  } catch {
```

%ZZ처럼 망가진 글자가 오면 decodeURIComponent는 URIError를 던집니다. 여기서 안 막으면 '없는 주소' 하나 때문에 500이 납니다.

```
return 경로;
}
}

app.use((req, res) => {
  res.status(404).json({
    error: "그런 주소가 없습니다",
    요청: `${req.method} ${읽기좋은주소(req.path)}`,
  });
});
```

GET /없는주소

```
404 {"error":"그런 주소가 없습니다","요청":"GET /없는주소"}
```

PATCH /ok

```
404 {"error":"그런 주소가 없습니다","요청":"PATCH /ok"}
```

★★ req.path 를 그냥 쓰면 이렇게 나옵니다.

```
"요청": "GET /%EC%97%86%EB%8A%94%EC%A3%BC%EC%86%8C"
```

개념01 섹션 8 에서 본 퍼센트 인코딩입니다. Express 는 라우트를 이 상태로 비교하기 때문에 되돌리지 않습니다. (req.params 와 req.query 는 되돌려 줍니다. 이 둘만 예외입니다)

사람이 읽을 기록에 넣을 때만 decodeURIComponent 로 되돌리세요. 그리고 반드시 try/catch 로 감싸세요. 이건 진짜로 터지는 함수입니다.

★ 두 번째 확인을 보세요. /ok 는 있는 주소인데 PATCH 는 안 만들었습니다. 405 가 더 정확하지만 Express 는 404 를 줍니다. 405 가 필요하다면 app.route 나 app.all 로 직접 만들어야 합니다.

에러 처리기 (가장 마지막)

섹션 7

★★ 인자가 '네 개' 입니다. 이게 에러 처리기의 표시입니다. Express 는 인자 개수를 세어서 "아, 이건 에러용이구나" 하고 구별합니다. next 를 안 써도 반드시 적어야 합니다. 빼면 그냥 미들웨어가 되어 버립니다.

```
app.use((err, req, res, next) => {
```

① 터미널에는 전부 남깁니다. 원인을 찾으려면 이게 있어야 합니다.

```
console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
```

② 본문이 깨진 JSON 이면 express.json() 이 이 에러를 만듭니다.

```
if (err.type === "entity.parse.failed") {
  return res.status(400).json({ error: "JSON 형식이 아닙니다" });
}
```

③ 내가 만든 에러면 status 가 붙어 있습니다.

```
const 코드 = err.status || 500;
```

④ 500 일 때는 자세한 내용을 밖으로 내보내지 않습니다.

```
if (코드 === 500) {
  return res.status(500).json({ error: "서버에서 문제가 생겼습니다" });
}

res.status(코드).json({ error: err.message });
});
```

POST /documents

{깨진:JSON}

400 {"error":"JSON 형식이 아닙니다"}

★ ④번이 중요합니다 — 왜 500 의 내용을 숨기나

에러 메시지에에는 파일 경로, 테이블 이름, 쿼리문이 그대로 들어 있습니다. 그걸 그대로 돌려주면 공격하려는 사람에게 지도를 그려 주는 셈입니다.

```
"Cannot read properties of undefined (reading 'password')"
```

```
"connect ECONNREFUSED 10.0.1.23:5432"
```

400·404 는 '보낸 쪽이 고칠 수 있는 것' 이라 알려 주는 게 맞습니다. 500 은 '내 잘못' 이라 상대가 알아도 할 수 있는 게 없습니다.

대신 터미널에는 전부 남깁니다. 그래야 내가 고칠 수 있습니다. 운영 서버에서는 이 기록을 파일로 남깁니다. PART 4 에서 합니다.

★ Express 기본 에러 처리기는 어떤가 우리가 이 app.use 를 안 적어도 Express 가 500 을 돌려주긴 합니다. 그런데 개발 중에는 에러 스택 전체를 HTML 로 그대로 보여 줍니다. NODE_ENV=production 으로 켜면 "Internal Server Error" 만 나옵니다. 어차피 JSON 으로 주고 싶으니 직접 만드는 게 낫습니다.

```
app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/ok`);
});
```

★ 순서 정리 — 이 순서를 어기면 전부 어긋납니다

① app.use(express.json()) 미들웨어 — 맨 위 ② app.get / app.post ... 라우트들 ③ app.use((req, res) => ...) 404 — 라우트 다음 ④ app.use((err, ...) => ...) 에러 처리기 — 맨 마지막, 인자 네 개

흔한 사고 ③을 ②보다 위에 두면 → 모든 요청이 404 ④를 ③보다 위에 두면 → 에러가 404 처리기에 먼저 걸림 ④의 인자를 세 개로 쓰면 → 에러 처리기로 인식이 안 됨. 기본 HTML 이 나옴

이 네 줄 순서를 외워 두세요. 앞으로 만드는 모든 서버가 이 모양입니다.

직접 해 볼 것

- 직접 해보기 1

/documents/:id 에서 throw 대신 res.status(404).json(...) 을 직접 쓰도록 고쳐 보세요. 두 방식 중 어느 쪽이 읽기 좋은가요?
- 직접 해보기 2

에러 처리기의 인자를 세 개(err, req, res)로 줄여 보세요. /boom-sync 가 어떻게 되나요? (확인했으면 반드시 되돌리세요)
- 직접 해보기 3

404 처리기를 라우트들보다 위로 옮겨 보세요. 무슨 일이 생기나요? (확인했으면 되돌리세요)
- 직접 해보기 4

에러 처리기의 500 갈래에서 err.message 를 그대로 돌려주도록 바꿔 보고, /boom-sync 응답에 어떤 정보가 새어 나오는지 보세요. (확인했으면 반드시 되돌리세요)
- 직접 해보기 5

HttpError 를 써서 /documents 에 POST 를 만들어 보세요. title 이 없으면 400, 있으면 201 입니다.
- 직접 해보기 6

터미널을 보세요. 에러가 날 때마다 [에러] 로 시작하는 줄이 남습니다. 이 줄에 시각을 함께 남기려면 어떻게 할까요?

자주 하는 실수

이런 실수를 합니다

에러 처리기 인자를 세 개로 씀

(err, req, res) 로 쓰면 Express 가 알아보지 못합니다. next 를 안 쓰더라도 네 개를 다 적으세요.

이런 실수를 합니다

404 처리기를 라우트 위에 씌

모든 요청이 404 가 됩니다. 라우트는 실행조차 안 됩니다.

이런 실수를 합니다

응답 뒤에 `return` 을 안 붙임

에러 처리기 안에서 특히 위험합니다. 400 을 보내고 500 을 또 보내려 합니다. → Cannot set headers after they are sent to the client

이런 실수를 합니다

500 에 `err.message` 를 그대로 담음

내부 사정이 밖으로 새어 나갑니다. 500 은 뭉뚱그려 답하세요.

이런 실수를 합니다

터미널에 아무것도 안 남김

응답만 예쁘게 만들고 `console.error` 를 안 쓰면 나중에 "500 이 났다" 는 것만 알고 원인을 못 찾습니다.

이런 실수를 합니다

Express 4 처럼 라우트마다 `try/catch` 를 두름

Express 5 에서는 필요 없습니다. 코드만 길어집니다. 다만 '에러를 잡아서 다른 에러로 바꿔 던질 때' 는 여전히 씁니다.

정리

- 1 잘될 때만 생각하고 만든 서버는 실무에서 하루도 못 버틴다. 에러 처리를 얼마나 챙겼는지가 초보와 중급을 가른다.
- 2 순서가 정해져 있다. 라우트들 → 404 처리기 → 에러 처리기 순이고 바꿀 수 없다.
- 3 404 처리기를 라우트보다 위에 두면 전부 404 가 된다.
- 4 에러 처리기는 인자가 네 개다. (err, req, res, next). 세 개로 쓰면 Express 가 에러 처리기로 안 보고 그냥 미들웨어로 본다.
- 5 내가 만든 에러는 던지거나 next(err) 로 넘긴다. 넘기면 에러 처리기가 받는다.
- 6 Express 5 는 async 함수 안에서 난 에러도 알아서 에러 처리기로 보낸다. 4 처럼 라우트마다 try/catch 를 두를 필요가 없다.
- 7 500 응답에 err.message 를 그대로 담지 않는다. 안쪽 사정이 밖으로 나간다. 밖에는 짧게, 터미널에는 자세히 남긴다.
- 8 터미널에 아무것도 안 남기면 나중에 원인을 못 찾는다. 반드시 남긴다.

nodemon 과 파일 나누기 (express.Router)

```
RUN node 개념05_nodemon과_파일_나누기.js
```

또는 npx nodemon 개념05_nodemon과_파일_나누기.js

그러면 Ctrl + C

이 파일은 혼자가 아닙니다. 옆의 routes 폴더를 함께 보세요.

04_Express_시작하기/

```
개념05_nodemon과_파일_나누기.js ← 지금 이 파일
routes/
  documents.js
  equipments.js
```

1부 — nodemon

지금까지 코드를 고칠 때마다 이걸 반복했습니다.

Ctrl + C 로 끄고 → 위쪽 화살표 → Enter

하루에 백 번쯤 합니다. 정말 번거롭습니다. nodemon 은 파일이 바뀌면 알아서 서버를 다시 켜 주는 도구입니다.

설치

```
npm install --save-dev nodemon
```

--save-dev 를 붙이는 이유

```
nodemon 은 '개발할 때만' 필요합니다.
서버에 올릴 때는 필요 없습니다. 그래서 devDependencies 에 넣습니다.
(01단원 개념03 에서 본 그 구분입니다)
```

이 자료는 맨 위 폴더에 이미 설치해 두었습니다.

실행

```
npx nodemon 개념05_nodemon과_파일_나누기.js
```

npx 는 "node_modules 에 설치된 명령을 실행해라" 라는 뜻입니다.

설치는 했는데 명령어를 못 찾을 때 npx 를 앞에 붙이면 됩니다.

켜 두고 파일을 아무거나 고친 뒤 저장해 보세요. 터미널에 이렇게 나옵니다.

nodemon · restarting due to changes...

nodemon · starting node 개념05_...js

package.json 에 등록해 두면 더 짧아집니다

```
"scripts": {
```

```
  "dev": "nodemon server.js",  
  "start": "node server.js"
```

```
}
```

npm run dev	개발할 때
npm start	서버에 올렸을 때

실무 프로젝트는 거의 다 이 두 개가 있습니다. 처음 보는 프로젝트를 받으면 package.json 의 scripts 부터 보세요.

nodemon 이 못 고쳐 주는 것

① package.json 을 고쳤을 때는 다시 켜야 할 때가 있습니다 ② 문법 에러가 나면 그냥 멈춥니다. 고치고 저장하면 다시 살아납니다 ③ 켜져 있는 상태에서 브라우저만 새로고침하면 소용없습니다

→ 터미널에 restarting 이 났는지 꼭 확인하세요

★ “고쳤는데 반영이 안 돼요” 의 절반은

nodemon 이 아예 안 켜져 있는 경우입니다. node 로 켜 놓고 착각한 것입니다.

2부 — 파일 나누기


```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

왜 나누나

섹션 1

지금까지는 라우트를 전부 한 파일에 적었습니다. 자원이 세 개만 되어도 이렇게 됩니다.

```
app.get("/documents", ...)      20줄
app.get("/documents/:id", ...)  20줄
app.post("/documents", ...)     25줄
app.delete("/documents/:id", ...) 15줄
app.get("/equipments", ...)     20줄
```

...

500줄짜리 파일이 됩니다. 고칠 곳을 찾는 데만 한참 걸립니다. 여러 명이 같이 만들면 같은 파일을 동시에 고쳐서 충돌이 납니다.

그래서 '자원별로 파일을 나눕니다'. 문서 관련은 routes/documents.js, 설비 관련은 routes/equipments.js 로.

라우터 데려오기

섹션 2

```
const documentsRouter = require("./routes/documents");
const equipmentsRouter = require("./routes/equipments");
```

★ "/" 로 시작하는 것에 주의하세요.

```
./routes/documents → 내 폴더 아래 routes 폴더의 documents.js
routes/documents  → node_modules 에서 찾습니다. 없으니 에러
```

(01단원 개념02 에서 본 그 규칙입니다)

★ 확장자 .js 는 생략할 수 있습니다. 붙여도 됩니다.

```
app.use("/documents", documentsRouter);
app.use("/equipments", equipmentsRouter);
```

이 두 줄이 하는 일

"/documents 로 시작하는 요청은 documentsRouter 에게 넘겨라"

documents.js 안에는 router.get("/") 라고만 적혀 있습니다. 앞에 "/documents" 가 붙어서 최종 주소가 완성됩니다.

```
app.use("/documents", ...) + router.get("/")      → GET /documents
app.use("/documents", ...) + router.get("/:id")    → GET /documents/:id
```

★ 이 방식의 진짜 장점 나중에 주소를 /api/documents 로 바꾸고 싶다면 여기 한 줄만 고치면 됩니다.

```
app.use("/api/documents", documentsRouter);
```

documents.js 는 한 글자도 안 고쳐도 됩니다.

잘 붙었는지 확인

```
app.get("/health", (req, res) => {
  res.json({ status: "ok" });
});
```

GET /health

200 {"status":"ok"}

GET /documents

200 [{"id":1,"title":"작업표준서"}, {"id":2,"title":"검사성적서"}]

GET /documents/1

200 {"id":1,"title":"작업표준서"}

GET /documents/99

404 {"error":"99번 문서가 없습니다"}

POST /documents

{ "title": "신규문서" }

201 { "id": 3, "title": "신규문서" }**GET** /documents**200** [{ "id": 1, "title": "작업표준서" }, { "id": 2, "title": "검사성적서" }, { "id": 3, "title": "신규문서" }]**DELETE** /documents/3**204****GET** /equipments**200** [{ "id": 1, "name": "컨베이어", "line": "A" }, { "id": 2, "name": "프레스", "line": "B" }]**GET** /equipments?line=A**200** [{ "id": 1, "name": "컨베이어", "line": "A" }]**GET** /equipments/1**200** { "id": 1, "name": "컨베이어", "line": "A" }

라우트를 다른 파일에 적었는데도 똑같이 동작합니다. 옮겼을 뿐 달라진 것은 없습니다.

404 와 에러 처리기는 여전히 맨 아래

섹션 5

```
app.use((req, res) => {
  res.status(404).json({ error: "그런 주소가 없습니다" });
});
```

GET /없는주소**404** { "error": "그런 주소가 없습니다" }

```
app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);

  if (err.type === "entity.parse.failed") {
    return res.status(400).json({ error: "JSON 형식이 아닙니다" });
  }
});
```

Express 개념 확인

RUN node 연습문제.js

이 파일은 서버 없이 푸는 문제들입니다. 실제로 서버를 만드는 문제는 연습문제_서버.js 에 있습니다. 이쪽을 먼저 푸세요. 여기서 헛갈리는 게 있으면 서버에서도 헛갈립니다.

문제 1 Express 4 문법을 5 로 고치기

Express 4 로 쓰인 라우트 주소를 5 문법으로 바꿔 주는 함수 버전5로 를 만드세요.

/:이름? → {/:이름} (선택 파라미터) 끝의 * → *splat (와일드카드) 그 밖 → 그대로

힌트: replace 와 정규식을 씁니다. 정규식은 아직 안 배웠으니

아래 완성 코드를 그대로 쓰면 됩니다.

```
.replace(/\(/(:[a-zA-Z]+\)\?/g, "{/$1}")
```

```
.replace(/\*/, "*splat")
```

\$1 은 괄호로 잡은 부분을 그대로 가져다 쓰는 표시입니다.

(시험할 값을 이 순서로 넣으세요) "/reports/:year?" "/files/*" "/documents/:id" "/a/:b?/c/*"

```
기대 출력  /reports{/:year}
           /files/*splat
           /documents/:id
           /a{/:b}/c/*splat
```

(세 번째는 바꿀 게 없어서 그대로 나와야 합니다. “안 건드려야 할 것을 안 건드리는가” 도 함께 확인하는 것입니다)

여기에 코드를 쓰세요

문제 2 req.params 의 함정을 눈으로 보기

아래는 Express 가 주는 req.params 를 흉내 낸 것입니다. find 로 문서를 찾아 두 번 출력하세요.

① params.id 를 그대로 써서 찾기 ② Number 로 바꿔서 찾기

결과가 왜 다른지 설명할 수 있어야 합니다.

```
const 문서들 = [
  { id: 1, title: "작업표준서" },
  { id: 2, title: "검사성적서" },
];

const params = { id: "1" }; // ← Express 가 주는 모양. 값이 글자입니다
```

```
기대 출력      undefined
                { id: 1, title: '작업표준서' }
```

(첫 줄이 왜 `undefined` 인지 꼭 생각해 보세요. 에러가 안 나는 게 핵심입니다)

여기에 코드를 쓰세요

문제 3 라우트 등록 순서 정하기

아래 라우트들을 '등록해야 할 순서'로 정렬해 출력하세요. 구체적인 것(콜론이 적은 것)이 먼저 와야 합니다.

힌트: 콜론 개수를 세는 함수를 만들고 sort에 넘기세요.

```
"/orders".match(/:/g) 는 못 찾으면 null 입니다. || [] 로 받치세요.
sort 는 원본을 바꿉니다. [...배열] 로 복사한 뒤 정렬하세요.
```

```
const 라우트들 = ["/orders/:id", "/orders/new", "/orders", "/orders/:id/items"];
```

```
기대 출력
```

```
[ '/orders/new', '/orders', '/orders/:id', '/orders/:id/items' ]
```

여기에 코드를 쓰세요

문제 4 어디에 답아야 하나

아래 여섯 가지 값은 경로·쿼리·본문·헤더 중 어디에 답아야 할까요? 상황과 답을 짝지어 출력하세요.

3번 문서 하나 보기 표준 종류만 걸러 보기 몇 쪽을 볼지 새 문서의 제목과 내용 비밀번호 로그인 증표

기대 출력 3번 문서 하나 보기 → 경로
표준 종류만 걸러 보기 → 쿼리
몇 쪽을 볼지 → 쿼리
새 문서의 제목과 내용 → 본문
비밀번호 → 본문
로그인 증표 → 헤더

여기에 코드를 쓰세요

문제 5 상태코드 고르기

Express 라우트에서 아래 상황일 때 어떤 상태코드를 쓸까요?

res.json 으로 목록을 돌려줌 새 문서를 만들고 돌려줌 지웠고 돌려줄 내용이 없음 req.body 에 title 이 없음 그 번호의 문서를 못 찾음 라우트 안에서 TypeError 가 남

기대 출력 res.json 으로 목록을 돌려줌 → 200
새 문서를 만들고 돌려줌 → 201
지웠고 돌려줄 내용이 없음 → 204
req.body 에 title 이 없음 → 400
그 번호의 문서를 못 찾음 → 404
라우트 안에서 TypeError 가 남 → 500

여기에 코드를 쓰세요

문제 6 적는 순서

아래 네 줄을 '파일에 적어야 할 순서' 대로 배열에 담고, 앞에 번호를 붙여 출력하세요.

`app.use((err, req, res, next) => 에러 응답)`

```
app.use('/documents', documentsRouter)
app.use(express.json())
app.use((req, res) => 404 응답)
```

힌트: `forEach` 의 두 번째 인자가 0부터 시작하는 번호입니다.

기대 출력

1. `app.use(express.json())`
2. `app.use('/documents', documentsRouter)`
3. `app.use((req, res) => 404 응답)`
4. `app.use((err, req, res, next) => 에러 응답)`

여기에 코드를 쓰세요

문제 7 이 코드는 왜 안 될까요

아래처럼 라우트를 등록했습니다.

```
app.get("/orders/:id", (req, res) => { ... });
app.get("/orders/new", (req, res) => { ... });
```

`/orders/new` 로 요청하면 무슨 일이 생기나: _____

에러가 나나: _____

어떻게 고치나: _____

어떤 규칙으로 기억하면 되나: _____

내 라우트가 걸리는지 확인하는 방법: _____

문제 8 req.body 가 undefined 입니다

POST 요청에 분명히 JSON 을 담아 보냈는데 req.body 가 `undefined` 입니다. 원인이 될 만한 것 세 가지를 적고, 배열에 담아 출력하세요.

① _____ ② _____ ③ _____

그리고 아래도 답해 보세요.

Express 4 와 5 에서 '본문을 아예 안 보냈을 때' req.body 가 어떻게 다른가:

이 차이 때문에 어떤 코드가 깨지나: _____

원인을 찾으려면 라우트 맨 앞에서 무엇을 찍어 보면 되나: _____

기대 출력 `express.json()` 을 안 씀
 `express.json()` 을 라우트 아래에 씀
 Content-Type 이 application/json 이 아님

여기에 코드를 쓰세요

설비 관리 API 를 Express 로 다시 만들기

RUN npx nodemon 연습문제_서버.js

(nodemon 으로 켜 두면 저장할 때마다 알아서 다시 켜집니다)

★ 이 문제의 목적 03단원 연습문제_서버_정답.js 와 '완전히 같은 API' 를 Express 로 만드는 것입니다. 결과가 한 글자도 달라지면 안 됩니다.

다 만든 뒤 두 파일의 줄 수를 세어 보세요. 무엇이 사라졌는지 눈으로 확인하는 것이 이 문제의 진짜 목적입니다.

만들 주소

GET	/health	서버가 살아 있나
GET	/equipments	목록
GET	/equipments?line=A	A라인만
GET	/equipments/1	하나만
POST	/equipments	새로 등록
PATCH	/equipments/1	상태만 바꾸기

DELETE /equipments/1 지우기 그 밖 405 또는 404

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;
```

★ 이 한 줄이 없으면 req.body 가 영원히 undefined 입니다. (개념03)

```
app.use(express.json());
```

준비 (여기는 그대로 두세요)

```
let 설비들 = [
  { id: 1, name: "컨베이어", line: "A", status: "가동" },
  { id: 2, name: "프레스", line: "B", status: "정지" },
];

let 다음번호 = 3;

const 상태값들 = ["가동", "정지"];
```

상태코드가 붙은 에러를 만드는 도구입니다. (개념04 섹션 4)

```
function HttpError(코드, 메시지) {  
  const 에러 = new Error(메시지);  
  에러.status = 코드;  
  return 에러;  
}
```

문제 0 공통 함수 만들기

번호글자를 받아 설비를 돌려주는 함수 설비찾기 를 만드세요.

- 숫자로 못 바꾸면 throw HttpError(400, "번호는 숫자여야 합니다") - 그 번호가 없으면 throw HttpError(404, `\${번호}번 설비가 없습니다) - 찾으면 그 설비를 return

★ 이 함수를 먼저 만드는 이유 문제 3·5·6 에서 똑같은 코드가 세 번 필요합니다. 03단원에서 그 반복을 눈으로 봤을 것입니다. 이번엔 처음부터 묶고 갑니다.

힌트: Number(번호글자) 로 바꾸고 Number.isInteger 로 확인합니다.

throw 한 에러는 맨 아래 에러 처리기가 받아 응답으로 바꿔 줍니다.
라우트 안에서는 try/catch 를 쓸 필요가 없습니다.

여기에 코드를 쓰세요

문제 1 /health

GET /health 에 200 과 { status: "ok" } 를 돌려주세요.

기대 출력 GET /health → 200 {"status":"ok"}

여기에 코드를 쓰세요

문제 2 목록 + 라인으로 걸러 보기

GET /equipments 에 설비들을 돌려주세요. ?line=A 가 붙어 오면 그 라인 것만 걸러서 돌려주세요.

```
기대 출력  GET /equipments      → 200 2건
            GET /equipments?line=A → 200 컨베이어 1건
            GET /equipments?line=Z → 200 []      ★ 404 가 아닙니다
```

힌트: 03단원의 new URL(...).searchParams.get("line") 이

Express 에서는 req.query.line 한 조각입니다.

여기에 코드를 쓰세요

문제 3 하나만 보기

GET /equipments/:id 에 그 설비를 돌려주세요.

```
기대 출력  GET /equipments/1    → 200
            {"id":1,"name":"컨베이어","line":"A","status":"가동"}
            GET /equipments/abc  → 400 {"error":"번호는 숫자여야 합니다"}
            GET /equipments/99   → 404 {"error":"99번 설비가 없습니다"}
```

★ 문제 0의 설비찾기 를 쓰면 이 라우트는 한 줄입니다. 길어졌다면 설비찾기 안에 들어갔어야 할 코드를
여기 쓴 것입니다.

여기에 코드를 쓰세요

문제 4 새로 등록하기

POST /equipments 로 온 본문을 읽어 새 설비를 만드세요.

- name 이나 line 이 없으면 → 400, 빠진 이름을 알려 주기 - 잘 왔으면 → 201, 만든 설비를 돌려주기 - Location 헤더도 붙이기 → /equipments/3

새 설비의 모양:

```
{ id: 다음번호, name, line, status: "정지" }
```

```
기대 출력    POST /equipments (본문 없음)
              → 400 {"error":"name, line 을(를) 넣어 주세요"}
              POST /equipments { "name": "용접로봇" }
              → 400 {"error":"line 을(를) 넣어 주세요"}
              POST /equipments { title: "따옴표없음" } ← 일부러 깨뜨린 JSON
              → 400 {"error":"JSON 형식이 아닙니다"} ← 우리가 안 만들어도 나옵니다
              POST /equipments { "name": "용접로봇", "line": "C" }
              → 201 {"id":3,"name":"용접로봇","line":"C","status":"정지"}
```

★ 세 번째를 보세요. 깨진 JSON 은 우리가 막지 않아도 됩니다. `express.json()` 이 잡아서 에러로 넘기고, 에러 처리기가 400 으로 바꿉니다. 03단원의 `try/catch` 가 통째로 필요 없어졌습니다.

```
힌트: const { name, line } = req.body || {};
```

```
|| {} 를 빼먹으면 본문 없이 왔을 때 500 이 납니다. 400 이어야 하는데요.
Location 은 res.status(201).location("/equipments/3").json(...) 입니다.
```

여기에 코드를 쓰세요

문제 5 상태만 바꾸기

PATCH /equipments/:id 로 온 { "status": "가동" } 을 반영하세요.

- 그 번호가 없으면 → 404 - status 가 가동/정지가 아니면 → 400 - 잘 왔으면 → 200, 바뀐 설비를 돌려주기

기대 결과 (문제 4에서 3번을 만든 뒤): PATCH /equipments/3 { "status": "폭발" } → 400 PATCH /equipments/99 { "status": "가동" } → 404 PATCH /equipments/3 { "status": "가동" } → 200, status 가 "가동"

★ 순서가 중요합니다. 먼저 찾고, 그다음에 값을 검사하세요. 반대로 하면 없는 설비에 "status 가 틀렸다" 는 엉뚱한 답이 나옵니다.

힌트: 상태값들.includes(status) 로 정해진 값만 받습니다.

find 로 찾은 객체의 속성을 바꾸면 배열 안의 것도 함께 바뀝니다.
배열에 다시 넣는 코드는 필요 없습니다.

여기에 코드를 쓰세요

문제 6 지우기

DELETE /equipments/:id 로 그 설비를 지우세요.

기대 출력 DELETE /equipments/3 → 204, 본문 없음
 DELETE /equipments/3 → 404 (이미 지웠으니)

힌트: 설비찾기 를 쓰면 “없으면 404” 가 저절로 됩니다.

지우기는 설비들 = 설비들.filter(...) 입니다.
204 는 res.sendStatus(204) 한 줄입니다.

★ filter 안의 인자 이름을 '설비' 로 하면 바깥 변수와 겹칩니다. '하나' 처럼 다른 이름을 쓰세요.

여기에 코드를 쓰세요

문제 7 405 만들기

Express 는 기본으로 405 를 안 만들어 줍니다. app.all 로 직접 만듭니다.

기대 출력 PUT /equipments → 405 {"error": "이 주소에서는 쓸 수 없는 방법입니다"}

★ 반드시 위의 모든 /equipments 라우트 '아래' 에 두세요. 맨 위에 두면 GET 도 POST 도 전부 405 가 됩니다. 왜 그런지 설명할 수 있어야 합니다. (개념02 섹션 7)

여기에 코드를 쓰세요

문제 8 없는 주소를 JSON 으로

어떤 라우트에도 안 걸리면 404 와 { error: "그런 주소가 없습니다" } 를 주세요.

기대 출력 GET /없는주소 → 404 {"error":"그런 주소가 없습니다"}

★ Express 기본 404 는 HTML 입니다. JSON 을 기대하는 프론트가 깨집니다. 그래서 API 서버는 직접 만들어 줍니다.

힌트: 주소 없이 app.use((req, res) ⇒ { ... }) 입니다.

여기에 코드를 쓰세요

문제 9 에러 처리기

맨 마지막에 에러 처리기를 만드세요. ★ 인자가 네 개여야 합니다.

① 터미널에 남기기 `console.error` 로 메서드·경로·메시지

② 깨진 JSON 이면 400 {"error":"JSON 형식이 아닙니다"}

(err.type 이 "entity.parse.failed" 입니다)

③ err.status 가 있으면 그 코드와 err.message

④ 없으면 (=500) {"error":"서버에서 문제가 생겼습니다"}

★ err.message 를 그대로 내보내면 안 됩니다

★ ④를 왜 숨기나 에러 메시지에는 파일 경로·테이블 이름 같은 내부 사정이 들어 있습니다. 400·404 는 보낸 쪽이 고칠 수 있으니 알려 주고, 500 은 내 잘못이라 상대가 알아도 소용없습니다. 대신 터미널에는

남깁니다.

힌트: `app.use((err, req, res, next) => { ... })`

`next` 를 안 쓰더라도 네 개를 다 적어야 에러 처리기로 인식됩니다.

여기에 코드를 쓰세요

```
app.listen(PORT, () => {
  console.log(`설비 관리 API 가 켜졌습니다. http://localhost:${PORT}/equipments`);
});
```

다 만들었으면 이 순서대로 눌러 보세요

```
1 GET    /health                200 2 GET    /equipments
2건
3 GET    /equipments?line=A     1건
4 GET    /equipments?line=Z     200 과 []
5 GET    /equipments/1          컨베이어
6 GET    /equipments/99 404 7 GET    /equipments/abc        400 8 POST
/equipments { "name": "용접로봇" } 400 9 POST    /equipments {
"name": "용접로봇", "line": "C" } 201, id 3
10 PATCH /equipments/3 { "status": "폭발" } 400
11 PATCH /equipments/3 { "status": "가동" } 200
12 DELETE /equipments/3 204
13 DELETE /equipments/3 404
14 PUT    /equipments            405
15 Ctrl+C 후 다시 켜고 2번을 다시 2건 - 3번은 사라짐
```

★ 03단원 때 눌러 본 것과 결과가 한 글자도 다르면 안 됩니다. 03단원 정답 서버를 다른 포트로 함께 켜 놓고 나란히 비교해도 좋습니다.

```
PORT=3001 node ../03_HTTP와_서버/연습문제_서버_정답.js
(윈도우 PowerShell 에서는 $env:PORT=3001; node ...)
```

다 만든 뒤 답해 보세요

① 03단원 파일과 이 파일은 각각 몇 줄인가요? ____줄 → ____줄

② 03단원에는 있었는데 여기서 통째로 사라진 코드 세 가지는?

③ 03단원 서버는 라우트 안에서 `TypeError` 가 나면 어떻게 됐나요? 이 서버는 어떻게 되나요?

④ 에러 응답을 만드는 코드가 03단원에는 몇 군데 있었고, 여기에는 몇 군데 있나요?

⑤ 응답 모양을 `{ error: ... }` 에서 `{ message: ... }` 로 바꾸려면 각각 몇 군데를 고쳐야 하나요?

잘 안 될 때 보는 곳

`req.body` 가 `undefined` → `app.use(express.json())` 을 안 썼거나 라우트 아래에 썼습니다. → Postman 에서 Body → raw → JSON 을 골랐는지 확인하세요.

라우트를 만들었는데 404 → 주소에 한글을 썼거나 (개념01 섹션 8) → 위쪽에 `:id` 나 `app.all` 이 먼저 걸렸습니다 (개념02 섹션 7) → 라우트 맨 앞에 `console.log` 를 넣어 걸리는지 확인하세요.

설비를 못 찾음 (계속 404) → `req.params.id` 를 Number 로 안 바꿨습니다. `1 === "1"` 은 `false` 입니다.

에러를 던졌는데 HTML 이 나옴 → 에러 처리기 인자가 네 개가 아닙니다. → 또는 404 처리기가 에러 처리기보다 아래에 있습니다.

`Cannot set headers after they are sent` → 응답을 두 번 보냈습니다. `res.status(...).json(...)` 앞에 `return` 을 붙이세요.

`TypeError: Missing parameter name / Unexpected ?` → Express 4 라우트 문법을 썼습니다. (개념02 마지막 표)

`EADDRINUSE` → 앞서 켜 서버가 살아 있습니다. `Ctrl + C` 로 끄세요.

```
터미널을 닫아 버렸다면 netstat -ano | findstr :3000 → taskkill /PID <번호> /F
```

04단원 · routes/documents.js

문서 관련 라우트만 모아 둔 파일

이 파일은 직접 실행하지 않습니다.

개념05_nodemon과_파일_나누기.js 가 불러다 씁니다.

```
const express = require("express");
```

★ `express()` 가 아니라 `express.Router()` 입니다. Router 는 '작은 app' 이라고 생각하면 됩니다. app 처럼 `get / post / use` 를 다 쓸 수 있지만, 혼자서는 서버가 못 됩니다. 누군가(app)가 데려다 붙여 줘야 동작합니다.

```
const router = express.Router();
```

데이터는 일단 여기 둡니다. PART 4 에서 데이터베이스로 옮깁니다.

```
let 문서들 = [
  { id: 1, title: "작업표준서" },
  { id: 2, title: "검사성적서" },
];

let 다음번호 = 3;
```

★★ 주소를 쓸 때 `"/documents"` 를 적지 않습니다. 여기서는 `/` 하나만 적습니다. 앞에 붙일 `"/documents"` 는 데려가는 쪽(app)이 정합니다.

```
app.use("/documents", router) + router.get("/") → GET /documents
app.use("/documents", router) + router.get("/:id") → GET /documents/:id
```

이렇게 하면 나중에 주소를 `/api/documents` 로 바꿀 때 이 파일은 하나도 안 고쳐도 됩니다. 부르는 쪽 한 줄만 고치면 끝입니다.

```

router.get("/", (req, res) => {
  res.json(문서들);
});

router.get("/:id", (req, res) => {
  const 번호 = Number(req.params.id);
  const 문서 = 문서들.find((문서) => 문서.id === 번호);

  if (!문서) {
    return res.status(404).json({ error: `${번호}번 문서가 없습니다` });
  }

  res.json(문서);
});

router.post("/", (req, res) => {
  const { title } = req.body || {};

  if (!title) {
    return res.status(400).json({ error: "title 을 넣어 주세요" });
  }

  const 새문서 = { id: 다음번호, title };
  다음번호 += 1;
  문서들.push(새문서);

  res.status(201).json(새문서);
});

router.delete("/:id", (req, res) => {
  const 번호 = Number(req.params.id);
  const 있나 = 문서들.some((문서) => 문서.id === 번호);

  if (!있나) {
    return res.status(404).json({ error: `${번호}번 문서가 없습니다` });
  }

  문서들 = 문서들.filter((문서) => 문서.id !== 번호);
  res.sendStatus(204);
});

```

★ 만든 라우터를 내보냅니다. 01단원에서 배운 `module.exports` 그대로입니다.

```
module.exports = router;
```

내보내는 것을 잊으면 부르는 쪽에서 이렇게 됩니다. `TypeError: Router.use() requires a middleware function but got a Object` "라우터를 달라고 했는데 빈 객체가 왔다" 는 뜻입니다.

설비 관련 라우트만 모아 둔 파일

documents.js 와 완전히 같은 모양입니다.

자원이 하나 늘 때마다 이런 파일이 하나씩 늘어납니다.

```
const express = require("express");

const router = express.Router();

const 설비들 = [
  { id: 1, name: "컨베이어", line: "A" },
  { id: 2, name: "프레스", line: "B" },
];

router.get("/", (req, res) => {
  const 라인 = req.query.line;
  const 결과 = 라인 ? 설비들.filter((설비) => 설비.line === 라인) : 설비들;

  res.json(결과);
});

router.get("/:id", (req, res) => {
  const 번호 = Number(req.params.id);
  const 설비 = 설비들.find((설비) => 설비.id === 번호);

  if (!설비) {
    return res.status(404).json({ error: `${번호}번 설비가 없습니다` });
  }

  res.json(설비);
});

module.exports = router;
```

★ 두 파일에 똑같은 모양이 반복되는 게 보이나요? 목록 / 하나 / 만들기 / 지우기는 어느 자원이든 똑같습니다. 07단원(저장소로 묶기)에서 이 반복을 어떻게 다루는지 봅니다.

미들웨어

OPEN 05_미들웨어

```
const app = express();  
app.use(express.json());  
app.use((req, res, next) => {  
  req.기록 = ["첫 번째 미들웨어"];
```

01 미들웨어란 무엇인가

02 쓸모 있는 미들웨어 직접 만들기

03 인증과 권한 (401 과 403)

04 검증을 미들웨어로 빼기

05 morgan 과 미들웨어 파일 나누기

- 연습문제

- 서버 연습문제

미들웨어란 무엇인가

RUN node 개념01_미들웨어란_무엇인가.js

그러면 Ctrl + C

04단원에서 이미 미들웨어를 썼습니다. 이름만 몰랐을 뿐입니다.

app.use(express.json()) ← 미들웨어 app.use((req, res) ⇒ 404) ← 미들웨어 app.use((err, ...) ⇒ 에러) ← 미들웨어

이번 단원에서 이걸 직접 만듭니다. 로그 남기기, 로그인 확인, 권한 확인, 요청 시간 재기 — 실무 서버 코드의 절반이 미들웨어입니다.

미들웨어는 '거쳐 가는 함수' 입니다

섹션 1

요청이 라우트에 도착하기 전에 들르는 곳이라고 생각하세요.

요청 → 미들웨어1 → 미들웨어2 → 라우트 → 응답

공항 검색대와 비슷합니다. 비행기(라우트)를 타기 전에 표 확인, 짐 검사, 보안 검색을 차례로 거칩니다. 어느 한 곳에서 걸리면 비행기까지 못 갑니다.

미들웨어 함수는 이렇게 생겼습니다.

```
(req, res, next) => { ... }
```

요청 응답 다음으로 넘기는 함수

라우트 함수와 딱 하나 다릅니다. 세 번째 인자 next 가 있습니다.

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

next 를 부르면 다음으로 갑니다

섹션 2

req 는 요청 하나가 끝날 때까지 계속 따라다니는 객체입니다. 여기에 값을 붙여 두면 다음 미들웨어와 라우트에서 꺼내 쓸 수 있습니다.

```
app.use((req, res, next) => {
  req.기록 = ["첫 번째 미들웨어"];
  next(); // * 이걸 불러야 다음으로 넘어갑니다
});

app.use((req, res, next) => {
  req.기록.push("두 번째 미들웨어");
  next();
});

app.get("/order", (req, res) => {
  req.기록.push("라우트");
  res.json({ 순서: req.기록 });
});
```

GET /order

200 {"순서":["첫 번째 미들웨어","두 번째 미들웨어","라우트"]}

적은 순서 그대로 실행됐습니다.

★ req 에 값을 붙이는 것이 미들웨어의 핵심 쓸모입니다. “이 요청을 보낸 사람이 누구인지” 를 미들웨어가 알아내서 req.user 에 붙여 두면, 라우트 서른 개가 전부 req.user 를 꺼내 쓸 수 있습니다. 같은 코드를 서른 번 안 써도 됩니다. 개념03 에서 실제로 만듭니다.

★ 이름은 마음대로 지어도 됩니다. req.기록, req.user, req.시작시각 ... 다만 Express 가 이미 쓰는 이름 (req.body, req.query 등)은 피하세요.

next 를 안 부르면 거기서 멈춥니다

섹션 3

```
app.use("/blocked", (req, res, next) => {
  res.status(403).json({ error: "여기서 막았습니다" });
});
```

next() 를 안 불렀습니다. 아래 라우트는 실행되지 않습니다.

```
});

app.get("/blocked", (req, res) => {
  res.json({ 도달: "여기는 절대 실행되지 않습니다" });
});
```

GET /blocked

403 {"error":"여기서 막았습니다"}

라우트를 분명히 만들었는데 실행이 안 됐습니다. 미들웨어가 응답을 보내고 끝냈기 때문입니다.

★ 이게 '막는다' 는 뜻입니다. 로그인 안 한 사람을 여기서 돌려보내면, 아래 라우트는 로그인 확인을 안 해도 됩니다.

★★ 그런데 응답도 안 보내고 next 도 안 부르면? 요청이 영원히 멈춥니다. 브라우저가 계속 빙빙 돌립니다. 에러도 안 납니다. 미들웨어를 만들 때 가장 많이 하는 실수입니다.

미들웨어 안의 모든 갈래는 셋 중 하나로 끝나야 합니다.

- | | |
|------------------|---------------|
| ① next() 를 부른다 | → 다음으로 보낸다 |
| ② 응답을 보낸다 | → 여기서 끝낸다 |
| ③ next(에러) 를 부른다 | → 에러 처리기로 보낸다 |

if 를 썼다면 else 쪽도 확인하세요. 한쪽만 next() 를 부르는 실수가 흔합니다.

app.use 에 주소를 주면 그 아래만

섹션 4

```
app.use("/admin", (req, res, next) => {
  req.관리자영역 = true;
  next();
});

app.get("/admin/users", (req, res) => {
  res.json({ 관리자영역: req.관리자영역 === true });
});
```

GET /admin/users

200 {"관리자영역":true}

```
app.get("/public/info", (req, res) => {
  res.json({ 관리자영역: req.관리자영역 === true });
});
```

GET /public/info

200 {"관리자영역":false}

app.use("/admin", ...) 은 "/admin 으로 시작하는 요청만" 이라는 뜻입니다.


```

/admin          걸립니다
/admin/users    걸립니다
/admin/x/y/z    걸립니다
/administrator  * 안 걸립니다 (조각 단위로 봅니다)
/public/info    안 걸립니다
  
```

주소를 안 주면 모든 요청에 걸립니다. 로그 남기기처럼 전부에 해야 하는 일은 주소를 안 줍니다. 로그인 확인처럼 일부만 해야 하는 일은 주소를 줍니다.

라우트 하나에만 붙이기

섹션 5

app.use 말고, 라우트를 등록할 때 함수를 여러 개 넘겨도 됩니다.

```

function 표시남기기(req, res, next) {
  req.표시 = "이 라우트에만 붙었습니다";
  next();
}

app.get("/one-route", 표시남기기, (req, res) => {
  res.json({ 표시: req.표시 });
});
  
```

GET /one-route

200 {"표시": "이 라우트에만 붙었습니다"}

```

app.get("/other-route", (req, res) => {
  res.json({ 표시: req.표시 ?? "안 붙었습니다" });
});
  
```

GET /other-route

200 {"표시": "안 붙었습니다"}

app.메서드(주소, 미들웨어1, 미들웨어2, ..., 최종처리)

몇 개를 넘겨도 됩니다. 앞에서부터 차례로 실행되고, 마지막 함수가 응답을 보냅니다.

사실 마지막 함수도 미들웨어입니다. next 를 안 쓸 뿐입니다. Express 에는 '미들웨어' 라는 한 종류만 있습니다. 라우트 함수는 "next 를 안 쓰고 응답을 보내는 미들웨어" 일 뿐입니다.

```
function 확인1(req, res, next) {
  req.통과 = ["확인1"];
  next();
}

function 확인2(req, res, next) {
  req.통과.push("확인2");
  next();
}

const 검사세트 = [확인1, 확인2];

app.get("/checked", 검사세트, (req, res) => {
  res.json({ 통과: req.통과 });
});
```

GET /checked

200 {"통과":["확인1","확인2"]}

자주 같이 쓰는 미들웨어들은 배열로 묶어 두면 편합니다. `app.get("/a", 검사세트, ...)` `app.get("/b", 검사세트, ...)`

express.json() 도 미들웨어였습니다

```
app.post("/echo", (req, res) => {
  res.json({ 받은것: req.body });
});
```

POST /echo

{"a":1}

200 {"받은것":{"a":1}}

`app.use(express.json())` 이 하는 일을 이제 알 수 있습니다.

① 본문 조각을 모은다 ② `JSON.parse` 로 객체를 만든다 ③ `req.body` 에 붙인다 ← 섹션 2에서 우리가 `req` 기록을 붙인 것과 같습니다 ④ `next()` 를 부른다

04단원에서 “라우트보다 위에 써야 한다” 고 한 이유도 분명해졌습니다. 아래에 쓰면 라우트가 먼저 실행되어, `req.body` 를 붙일 기회가 없습니다.

★ `express.json()` 에 괄호가 붙은 이유 `express.json` 은 '미들웨어를 만들어 주는 함수' 입니다. 불러야 미들웨어가 나옵니다.

```
app.use(express.json)    ← 괄호 없음. 함수 자체를 넘김. 안 됩니다
app.use(express.json())  ← 불러서 나온 미들웨어를 넘김. 맞습니다
```

옵션을 줄 수 있어서 이렇게 만들어져 있습니다.

```
express.json({ limit: "1mb" })  본문 크기 제한
```

```
app.use((req, res) => {
  res.status(404).json({ error: "그런 주소가 없습니다" });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  const 코드 = err.status || 500;
  res.status(코드).json({ error: 코드 === 500 ? "서버에서 문제가 생겼습니다" :
err.message });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/order`);
});
```

미들웨어를 등록하는 네 가지 방법

`app.use(함수)` 모든 요청 `app.use("/admin", 함수)` `/admin` 으로 시작하는 요청 `app.get("/x", 함수, 최종처리)` `/x` 로 오는 GET 하나 `app.get("/x", [함수1, 함수2], ...)` 여러 개를 배열로

어느 것이든 `(req, res, next)` 모양은 같습니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 2의 두 번째 미들웨어에서 `next()` 를 지워 보세요. `/order` 로 요청하면 어떻게 되나요? 브라우저 탭이 어떤 상태가 되는지 보세요. (확인했으면 반드시 되돌리세요)

▫ 직접 해보기 2

섹션 2의 두 미들웨어 순서를 바꿔 보세요. 무슨 에러가 나나요? 왜 그럴까요? (힌트: 두 번째 것이 먼저 오면 `req.기록` 이 아직 없습니다)

▫ 직접 해보기 3

모든 요청에 대해 터미널에 “요청 들어옴: GET /order” 를 찍는 미들웨어를 만들어 보세요. 맨 위에 두어야 합니다.

▫ 직접 해보기 4

/admin 미들웨어를 /administrator 로 요청해 보세요. 걸리나요? 왜 그럴까요?

▫ 직접 해보기 5

섹션 3의 미들웨어에서 응답을 보낸 ‘뒤에’ `next()` 를 불러 보세요. 터미널에 어떤 에러가 나나요? (확인했으면 반드시 되돌리세요)

자주 하는 실수

이런 실수를 합니다

`next()` 를 안 부름

요청이 영원히 멈춥니다. 에러도 안 나서 제일 찾기 어렵습니다. 브라우저 탭이 계속 도는데 터미널은 조용하면 이걸 의심하세요.

이런 실수를 합니다

`if` 의 한쪽에서만 `next()` 를 부름

```
if (조건) { next(); }
```

→ 조건이 `false` 면 아무 일도 안 일어나고 멈춥니다. `else` 쪽도 반드시 끝을 내세요.

이런 실수를 합니다

응답을 보내고 `next()` 도 부름

```
res.json(...); next();
```

→ 다음 미들웨어가 또 응답하려다 에러가 납니다. → Cannot set headers after they are sent to the client 응답했으면 거기서 끝입니다. `return` 을 붙이는 습관이 좋습니다.

이런 실수를 합니다

미들웨어를 라우트 아래에 등록

라우트가 먼저 걸려서 미들웨어가 실행되지 않습니다. “미들웨어는 위, 라우트는 아래” 를 기억하세요.

이런 실수를 합니다

next 를 **next()** 로 안 부르고 **next** 라고만 씀

함수 이름만 적은 것입니다. 아무 일도 안 일어납니다. 괄호를 붙이세요.

이런 실수를 합니다

express.json 에 괄호를 안 붙임

`app.use(express.json)` → 미들웨어가 아니라 '만드는 함수' 를 등록한 것입니다. 이 함수는 `next()` 를 부르지 않아서, [실수 1]과 똑같이 모든 요청이 응답 없이 영영 멈춥니다. 에러도 안 나서 원인을 찾기 어렵습니다.

정리

- 1 미들웨어는 요청이 라우트에 닿기 전에 거쳐 가는 함수다. (`req`, `res`, `next`) 셋을 받는다.
- 2 `next()` 를 불러야 다음으로 간다. 안 부르면 거기서 멈추고 응답이 영영 안 나간다.
- 3 `if` 의 한쪽에서만 `next()` 를 부르면 나머지 길에서 멈춘다. 모든 갈래를 확인한다.
- 4 응답을 보냈으면 `next()` 를 부르지 않는다. 두 번 보내려다 에러가 난다.
- 5 미들웨어는 라우트보다 위에 등록한다. 아래에 두면 아예 안 거친다.
- 6 `app.use("/주소", 미들웨어)` 로 그 아래만 걸 수 있고, 라우트 하나에만 붙일 수도 있다.
- 7 여러 개를 배열로 묶어 두면 그대로 재사용할 수 있다.
- 8 `express.json()` 도 미들웨어였다. 괄호를 붙여서 '만들어진 미들웨어' 를 넘겨야 한다.

쓸모 있는 미들웨어 직접 만들기

RUN node 개념02_직접_만들어_보기.js

그러면 Ctrl + C

★ 이 파일은 터미널을 함께 보면서 해야 합니다.

응답보다 터미널에 찍히는 내용이 더 중요합니다.

실무 서버에 거의 항상 들어가는 미들웨어 네 가지를 직접 만듭니다.

① 요청 기록 남기기 ② 응답까지 걸린 시간 재기 ③ 요청마다 번호 붙이기 ④ 응답에 헤더 붙이기

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

요청 기록 남기기

섹션 1

서버가 무슨 요청을 받았는지 안 남기면, 문제가 생겼을 때 아무것도 알 수 없습니다. “어제 오후에 에러가 났대요” 라는 말만 듣고 원인을 찾아야 합니다.

```
app.use((req, res, next) => {
  const 시각 = new Date().toISOString(); // 2026-08-13T05:12:33.123Z
  모양 console.log(`${시각} ${req.method} ${req.originalUrl}`);
  next();
});
```

★ req.url 이 아니라 req.originalUrl 을 썼습니다.

req.url	미들웨어 위치에 따라 앞부분이 잘려 있을 수 있습니다
req.originalUrl	브라우저가 보낸 주소 그대로입니다

app.use("/admin", ...) 안에서는 req.url 이 "/users" 로만 보입니다. 기록에는 "/admin/users" 를 남겨야 하니 originalUrl 이 맞습니다.

★ toISOString 을 쓴 이유 컴퓨터마다 시간 표기가 다르면 기록을 모아서 볼 수가 없습니다. ISO 형식은 어디서나 같은 모양이라 정렬도 됩니다. 끝의 Z 는 세계 표준시(UTC)라는 뜻입니다. 한국 시간보다 9시간 이릅니다.

걸린 시간 재기

섹션 2

“어느 주소가 느린가” 를 알아야 고칠 곳을 찾을 수 있습니다.

★ 문제가 하나 있습니다. 미들웨어는 ‘응답을 보내기 전’ 에 실행됩니다. 그런데 걸린 시간은 ‘응답을 보낸 뒤’ 에야 알 수 있습니다. 여기서 시간을 재고 `next()` 를 부르면, 그 뒤로는 우리 손을 떠납니다.

그래서 “응답이 끝나면 알려 줘” 를 `res` 에 부탁해 둡니다.

```
app.use((req, res, next) => {
  const 시작 = Date.now();
```

`res` 도 이벤트를 낼 수 있습니다. `finish` 는 ‘응답을 다 보냈다’ 는 뜻입니다.

```
res.on("finish", () => {
  const 걸린시간 = Date.now() - 시작;
  console.log(`  L ${res.statusCode} - ${걸린시간}ms`);
});

next();
});
```

11단원(JS자료)의 `addEventListener` 와 같은 모양입니다. “이 일이 생기면 이 함수를 실행해 줘” 입니다.

★ `res.on("finish")` 안에서는 응답을 보낼 수 없습니다. 이미 다 보낸 뒤니까요. 기록만 남길 수 있습니다.

```
app.get("/slow", async (req, res) => {
  await new Promise((resolve) => setTimeout(resolve, 200));
  res.json({ 설명: "일부러 0.2초 기다렸습니다. 터미널의 ms 를 보세요" });
});
```

```
GET /slow
```

```
200 {"설명":"일부러 0.2초 기다렸습니다. 터미널의 ms 를 보세요"}
```

```
app.get("/fast", (req, res) => {
  res.json({ 설명: "바로 응답합니다" });
});
```

```
GET /fast
```

```
200 {"설명":"바로 응답합니다"}
```

두 주소를 눌러 보고 터미널의 `ms` 를 비교하세요. `/slow` 는 200ms 언저리, `/fast` 는 0~2ms 가 나옵니다.

서버는 여러 요청을 동시에 처리합니다. 기록이 뒤섞이면 “이 줄과 저 줄이 같은 요청인가?” 를 알 수 없습니다. 그래서 요청마다 번호를 붙입니다.

```
let 요청번호 = 0;

app.use((req, res, next) => {
  요청번호 += 1;
  req.요청번호 = 요청번호; // * req 에 붙여 두면 아래 어디서든 쓸 수 있습니다 next();
});

app.get("/request-id", (req, res) => {
  console.log(`  이 요청의 번호는 ${req.요청번호} 입니다`);

  res.json({
    타입: typeof req.요청번호,
    번호가붙었나: req.요청번호 > 0,
  });
});
```

GET /request-id

200 {"타입":"number","번호가붙었나":true}

번호 자체는 요청을 몇 번 보냈느냐에 따라 달라집니다. 그래서 여기서는 ‘붙었는지’ 만 확인했습니다. 실제 번호는 터미널에서 보세요.

★ 실무에서는 숫자 대신 무작위 글자를 씁니다. 서버를 여러 대 돌리면 번호가 겹치기 때문입니다.

```
req.요청번호 = crypto.randomUUID();
```

crypto 는 Node 내장 모듈입니다. require("crypto") 로 가져옵니다.

응답에 헤더 붙이기

```
app.use((req, res, next) => {
  res.set("X-Request-Id", String(req.요청번호));
  res.set("X-App-Name", "equipment-api"); // * 영어여야 합니다. 아래 설명을
  보세요 next();
});

app.get("/header-echo", (req, res) => {
```


res.get 으로 내가 붙인 헤더를 다시 읽어 볼 수 있습니다.

```
res.json({
  붙었나: res.get("X-Request-Id") !== undefined,
  앱이름: res.get("X-App-Name"),
});
```

GET /header-echo

200 {"붙었나":true,"앱이름":"equipment-api"}

★ X- 로 시작하는 이름 표준에 없는, 우리가 만든 헤더라는 표시입니다. 관례입니다. 요청 번호를 헤더로 돌려주면, 사용자가 "이 번호로 문의합니다" 라고 할 수 있습니다. 그 번호로 서버 기록을 찾으면 무슨 일이 있었는지 바로 알 수 있습니다.

★ 헤더는 응답을 보내기 '전' 에만 붙일 수 있습니다. res.json 뒤에 res.set 을 부르면 에러가 납니다. 그래서 이 미들웨어가 라우트보다 위에 있습니다.

05

2: ★★ 헤더 값에 한글을 넣으면 서버가 500 을 냅니다

섹션 4

```
app.get("/header-korean", (req, res) => {
  let 던진에러 = null;

  try {
    res.set("X-Test", "설비관리");
  } catch (에러) {
    던진에러 = 에러;
  }

  res.json({
    한글헤더가되나: 던진에러 === null,
    에러이름: 던진에러 ? 던진에러.name : null,
    에러코드: 던진에러 ? 던진에러.code : null,
  });
});
```

GET /header-korean

200 {"한글헤더가되나":false,"에러이름":"TypeError","에러코드":"ERR_INVALID_CHAR"}

에러 메시지는 이렇습니다. TypeError: Invalid character in header content ["X-Test"]

왜 안 되나 HTTP 헤더는 영어와 숫자, 몇몇 기호만 담을 수 있게 정해져 있습니다. 아주 오래전에 만들어진 약속이라 한글이 들어갈 자리가 없습니다. 본문(body)은 아무 글자나 담을 수 있습니다. 헤더만 그렇습니다.

개념01 에서 본 것과 같은 이야기입니다. 정리하면

라우트 주소 한글 × (조용히 404) 헤더 이름·값 한글 × (500 으로 터짐) 쿼리 값 한글 ○ 경로 파라미터 한글 ○ 본문(JSON) 한글 ○ 응답 본문 한글 ○

★ 이 실수는 try/catch 로 감싸지 않으면 500 이 됩니다. 미들웨어에서 이러면 그 서버의 '모든 요청' 이 500 이 됩니다. 실제로 이 파일을 처음 만들 때 그렇게 됐습니다.

꼭 한글을 헤더로 보내야 한다면 인코딩해서 보냅니다.

```
res.set("X-App-Name", encodeURIComponent("설비관리API"));
```

→ X-App-Name: %EC%84%A4%EB%B9%84%EA%B4%80%EB%A6%ACAPI 받는 쪽에서 decodeURIComponent 로 되돌립니다. 번거로우니 웬만하면 본문에 담으세요.

★ req.query 는 고칠 수 없습니다 (Express 5)

섹션 5

```
app.use((req, res, next) => {
```

검색어 앞뒤 공백을 없애 두려고 이렇게 써 봅니다.

```
if (typeof req.query.q === "string") {
  req.query.q = req.query.q.trim(); // ← 아무 일도 안 일어납니다
}
next();
});

app.get("/search", (req, res) => {
  res.json({
    받은값: req.query.q,
    공백이지워졌나: req.query.q === "표준",
  });
});
```

```
GET /search?q=%20%20표준%20%20
```

```
200 {"받은값":" 표준 ","공백이지워졌나":false}
```

★★ 에러가 안 났는데 값도 안 바뀌었습니다.

Express 5 에서 req.query 는 '읽을 때마다 새로 만들어지는 값' 입니다. 우리가 바꾼 것은 그 순간 만들어진 복사본이고, 라우트에서 다시 읽으면 또 새로 만들어집니다. 고친 게 사라집니다.

```
req.query = {...} 로 통째로 바뀌도 마찬가지입니다. 조용히 무시됩니다.
```

(Express 4 에서는 바뀌었습니다. 그래서 옛날 코드가 여기서 깨집니다)

에러가 안 나기 때문에 이걸 정말 찾기 어렵습니다. “분명히 trim 했는데 왜 공백이 남아 있지?” 로 한참 헤매게 됩니다.

그럼 어떻게 하나 — 다른 이름에 답습니다.

```
app.use((req, res, next) => {
  req.정리된쿼리 = {};

  for (const 키 of Object.keys(req.query)) {
    const 값 = req.query[키];
    req.정리된쿼리[키] = typeof 값 === "string" ? 값.trim() : 값;
  }

  next();
});

app.get("/search2", (req, res) => {
  res.json({
    원본: req.query.q,
    정리됨: req.정리된쿼리.q,
  });
});
```

```
GET /search2?q=%20%20표준%20%20
200 {"원본":"  표준  ","정리됨":"표준"}
```

req 에 새 이름으로 붙이는 것은 잘 됩니다. 우리가 만든 속성이니까요. 못 바꾸는 것은 Express 가 관리하는 req.query 뿐입니다.

★ req.params 와 req.body 는 바꿀 수 있으나 req.body 는 우리가 만든 것이나 마찬가지로 바뀌어도 됩니다. req.params 도 바뀌긴 하지만, 헛갈리니 건드리지 마세요. “받은 것은 읽기만, 바꾼 것은 새 이름으로” 를 규칙으로 삼으면 안전합니다.

미들웨어를 함수로 빼서 이름 붙이기

섹션 6

지금까지는 app.use 안에 함수를 바로 적었습니다. 밖으로 빼면 이름이 생겨서 읽기 좋아지고, 다른 파일에서도 쓸 수 있습니다.

```
function 느린요청경고(기준ms) {
```

★ 미들웨어를 ‘만들어 주는’ 함수입니다. 옵션을 받아서 미들웨어를 돌려줍니다. express.json({...}) 과 같은 모양입니다.

```
return function (req, res, next) {
  const 시작 = Date.now();
```

```

res.on("finish", () => {
  const 걸린시간 = Date.now() - 시작;
  if (걸린시간 > 기준ms) {
    console.log(`  * 느립니다: ${req.originalUrl} - ${걸린시간}ms`);
  }
});

next();
};
}

app.use(느린요청경고(100)); // 100ms 를 넘으면 경고

app.get("/slow2", async (req, res) => {
  await new Promise((resolve) => setTimeout(resolve, 150));
  res.json({ 설명: "터미널에 느리다는 경고가 나옵니다" });
});

```

GET /slow2

200 {"설명":"터미널에 느리다는 경고가 나옵니다"}

★ 왜 함수를 돌려주게 만드나 기준을 밖에서 정하고 싶어서입니다.

```

app.use(느린요청경고(100));   개발 중에는 100ms
app.use(느린요청경고(1000)); 운영에서는 1초

```

`express.json({ limit: "1mb" })` 도 정확히 같은 구조입니다.

이제 왜 괄호를 붙여야 하는지 알 수 있습니다.

```

app.use((req, res) => {
  res.status(404).json({ error: "그런 주소가 없습니다" });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  const 코드 = err.status || 500;
  res.status(코드).json({ error: 코드 === 500 ? "서버에서 문제가 생겼습니다" :
err.message });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/fast`);
  console.log("요청을 보내면서 이 터미널을 함께 보세요.");
});

```

터미널에 이렇게 나옵니다

2026-08-13T05:12:33.101Z GET /fast

```
L 200 - 1ms
```

2026-08-13T05:12:38.442Z GET /slow

```
L 200 - 203ms
```

2026-08-13T05:12:41.887Z GET /slow2

```
L ★ 느립니다: /slow2 - 152ms
L 200 - 152ms
```

이 몇 줄이 실무에서 문제를 찾는 출발점입니다. “느리다” 는 민원이 들어오면 이 기록부터 봅니다.

직접 해 볼 것

▫ 직접 해 보기 1

섹션 1의 기록에 요청 번호도 함께 남겨 보세요. (힌트: 번호를 붙이는 미들웨어를 기록보다 위로 옮겨야 합니다)

▫ 직접 해 보기 2

응답 상태코드가 400 이상일 때만 터미널에 ★ 를 붙여 보세요. (힌트: `res.on("finish")` 안에서 `res.statusCode` 를 봅니다)

▫ 직접 해 보기 3

POST 요청일 때 본문도 함께 기록하는 미들웨어를 만들어 보세요. 다 만든 뒤, 이걸 실무에서 그대로 쓰면 왜 위험할지 생각해 보세요. (힌트: 회원가입 요청의 본문에는 무엇이 들어 있나요?)

▫ 직접 해 보기 4

섹션 5의 `/search` 를 Express 4 라고 생각하고 코드를 읽어 보세요. Express 4 였다면 공백이지워졌나 가 `true` 였을 것입니다. 이런 차이를 미리 모르면 어떻게 될지 생각해 보세요.

▫ 직접 해 보기 5

느린요청경고 를 라우트 하나에만 붙여 보세요. `app.get("/slow2", 느린요청경고(100), (req, res) => ...)`

자주 하는 실수

이런 실수를 합니다

응답 뒤에 헤더를 붙임

```
res.json(...); res.set(...);
```

→ Cannot set headers after they are sent to the client 헤더는 응답 전에만 붙일 수 있습니다.

이런 실수를 합니다

res.on("finish") 안에서 응답을 보내려 함

이미 끝난 뒤라 아무 일도 안 일어나거나 에러가 납니다. 기록만 남기세요.

이런 실수를 합니다

req.query 를 고치려 함

Express 5에서는 조용히 무시됩니다. 새 이름에 담으세요.

이런 실수를 합니다

미들웨어 만드는 함수를 괄호 없이 넘김

app.use(느린요청경고) ← 미들웨어가 아니라 '만드는 함수'를 넘긴 것입니다 Express가 이걸 미들웨어로 알고 실행하면 req가 기준ms(100) 자리에 들어갑니다. app.use(느린요청경고(100)) 처럼 반드시 불러서 넘기세요.

이런 실수를 합니다

기록에 비밀번호를 남김

req.body를 통째로 찍으면 비밀번호가 터미널과 로그 파일에 그대로 남습니다. 실무에서 실제로 사고가 나는 부분입니다. 남길 것을 골라서 남기세요.

이런 실수를 합니다

req.url로 기록을 남김

app.use("/admin", ...) 안에서는 앞부분이 잘려 있습니다. 기록에는 req.originalUrl을 쓰세요.

정리

- 1 기록 남기기·시간 재기·요청 번호 붙이기처럼 모든 요청에 공통인 일이 미들웨어의 자리다.
- 2 응답이 끝난 뒤에 할 일은 `res.on("finish")` 에 건다. 이미 끝난 응답이므로 거기서 또 보내려 하면 안 된다.
- 3 헤더는 응답을 보내기 전에 붙여야 한다. 보낸 뒤에는 늦다.
- 4 헤더 값에 한글을 넣으면 서버가 500 을 낸다. 헤더에 실리는 것은 영어로 쓴다.
- 5 `req.query` 는 Express 5 에서 고칠 수 없다. 에러도 안 나고 값만 조용히 사라진다. 다듬은 값이 필요하면 `req` 에 새 이름으로 담는다.
- 6 미들웨어를 만들어 주는 함수는 반드시 불러서 넘긴다 — `app.use(느린요청경고(100))`. 괄호 없이 넘기면 `req` 가 기준 `ms(100)` 자리에 들어간다.
- 7 기록에 `req.body` 를 통째로 찍으면 비밀번호가 그대로 남는다. 남길 것을 골라서 남긴다.
- 8 기록에는 `req.url` 이 아니라 `req.originalUrl` 을 쓴다. `app.use("/admin", ...)` 안에서는 앞부분이 잘려 있다.

인증과 권한 (401 과 403)

RUN node 개념03_인증과_권한.js

그러면 Ctrl + C

미들웨어가 제일 빛나는 곳이 여기입니다.

라우트가 서른 개인데 전부 “로그인했나?” 를 확인해야 한다면 같은 코드를 서른 번 쓸 수는 없습니다.

미들웨어 하나를 앞에 세워 두면 됩니다.

★ 여기서 만드는 것은 ‘홍내’ 입니다. 진짜 로그인(비밀번호·토큰·만료시간)은 PART 4 의 11단원에서 배웁니다. 지금은 “미들웨어로 막는다” 는 구조만 익힙니다.

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

function HttpError(코드, 메시지) {
  const 에러 = new Error(메시지);
  에러.status = 코드;
  return 에러;
}
```

인증과 권한은 다릅니다

섹션 1

두 단어가 비슷해 보이지만 완전히 다른 질문입니다.

인증(Authentication)	너 누구니?	→ 아니면 401
권한(Authorization)	너 이거 해도 되니?	→ 아니면 403

회사 건물로 비유하면

401	출입증이 없어서 로비도 못 들어감
403	출입증은 있는데 서버실 문이 안 열림

★ 이 둘을 바꿔 쓰면 상대가 엉뚱한 곳을 고칩니다. 401 을 받으면 “다시 로그인해야겠다” 고 생각합니다. 403 을 받으면 “관리자에게 권한을 달라고 해야겠다” 고 생각합니다. 403 이어야 하는데 401 을 주면,

로그인만 계속 다시 하게 됩니다.

가짜 사용자 목록

섹션 2

진짜 서버라면 데이터베이스에서 찾습니다. 지금은 객체에 적어 둡니다.

```
const 키별사용자 = {
  "key-user-1": { name: "김민준", role: "user" },
  "key-admin-1": { name: "이서연", role: "admin" },
};
```

인증 미들웨어

섹션 3

```
function 인증(req, res, next) {
```

로그인 증표는 Authorization 헤더에 담아 보내는 것이 약속입니다.

```
const 헤더값 = req.get("Authorization");

if (!헤더값) {
  throw HttpError(401, "로그인이 필요합니다");
}
```

"Bearer key-user-1" 모양입니다. 앞의 Bearer 는 '이 증표를 가진 자' 라는 뜻입니다.

```
const [방식, 키] = 헤더값.split(" ");

if (방식 !== "Bearer" || !키) {
  throw HttpError(401, "Authorization 형식이 올바르지 않습니다");
}

const 사용자 = 키별사용자[키];

if (!사용자) {
  throw HttpError(401, "증표가 올바르지 않습니다");
}
```

★ 여기가 핵심입니다. 찾아낸 사용자를 req 에 붙여 둡니다. 이제 아래 모든 라우트가 req.user 를 그냥 꺼내 쓸 수 있습니다.

```
req.user = 사용자;

next();
}
```

★ 왜 헤더에 담나 쿼리(?key=...)에 담으면 서버 접속 기록과 브라우저 방문 기록에 남습니다. 링크를 복사해 보내면 증표까지 딸려 갑니다. 헤더는 기록에 안 남습니다. 비밀은 헤더나 본문에 담으세요.

권한 미들웨어

섹션 4

```
function 관리자만(req, res, next) {
```

이 미들웨어는 인증 뒤에 와야 합니다. req.user 가 있어야 하니까요.

```
  if (req.user.role !== "admin") {
    throw HttpError(403, "관리자만 할 수 있습니다");
  }

  next();
}
```

★ 인증과 권한을 왜 따로 만드나 합쳐 두면 “로그인만 하면 되는 주소” 에서도 관리자 검사를 하게 됩니다. 하나씩 나눠 두면 필요한 것만 골라 붙일 수 있습니다.

```
app.get("/me", 인증, ...)           로그인만
app.delete("/users/:id", 인증, 관리자만, ...) 로그인 + 관리자
```

아무나 볼 수 있는 주소

섹션 5

```
app.get("/public", (req, res) => {
  res.json({ 설명: "누구나 볼 수 있습니다" });
});
```

```
GET /public
```

```
200 {"설명":"누구나 볼 수 있습니다"}
```

미들웨어를 안 붙였으니 증표 없이도 됩니다.

로그인해야 볼 수 있는 주소

섹션 6

```
app.get("/me", 인증, (req, res) => {
  res.json({ 이름: req.user.name, 역할: req.user.role });
});
```

GET /me

401 {"error": "로그인이 필요합니다"}

GET /me

Authorization: key-user-1

401 {"error": "Authorization 형식이 올바르지 않습니다"}

(Bearer 없이 키만 보낸 경우입니다. 형식이 틀렸습니다)

GET /me

Authorization: Bearer no-such-key

401 {"error": "증표가 올바르지 않습니다"}

★ 시험용 키를 전부 영어로 쓴 이유 헤더 값에는 한글을 담을 수 없습니다. 개념02 섹션 4-2 에서 본 그대로입니다. 보낼 때도 마찬가지로, 한글 증표를 만들면 요청 자체가 안 나갑니다. 증표·키·토큰은 언제나 영어와 숫자로 만드세요.

GET /me

Authorization: Bearer key-user-1

200 {"이름": "김민준", "역할": "user"}

GET /me

Authorization: Bearer key-admin-1

200 {"이름": "이서연", "역할": "admin"}

★ 라우트 안에는 인증 코드가 한 줄도 없습니다. "req.user 는 이미 있다" 고 믿고 쓰면 됩니다. 미들웨어가 없으면 여기까지 못 오니까요.

관리자만 할 수 있는 일

섹션 7

```
app.delete("/users/:id", 인증, 관리자만, (req, res) => {
  res.json({ 지운사람: req.params.id, 지시한사람: req.user.name });
});
```

DELETE /users/7

401 {"error": "로그인이 필요합니다"}

DELETE /users/7

Authorization: Bearer key-user-1

403 {"error": "관리자만 할 수 있습니다"}

DELETE /users/7

Authorization: Bearer key-admin-1

200 {"지운사람": "7", "지시한사람": "이서연"}

★ 세 줄을 나란히 보세요. 증표 없음 → 401 (너 누구니) 증표는 있으나 user → 403 (누군지 알겠는데 안 돼) 증표가 admin → 200 (통과)

이 세 갈래가 인증·권한의 전부입니다.

여러 라우트에 한꺼번에 붙이기

섹션 8

라우트마다 인증 을 적는 것도 금방 지겨워집니다. /admin 아래 전체를 한 번에 막을 수 있습니다.

```
app.use("/admin", 인증, 관리자만);

app.get("/admin/stats", (req, res) => {
  res.json({ 통계: "관리자만 봅니다", 보는사람: req.user.name });
});

app.get("/admin/logs", (req, res) => {
  res.json({ 기록: "여기도 관리자만 " });
});
```

GET /admin/stats

401 {"error": "로그인이 필요합니다"}

GET /admin/stats

Authorization: Bearer key-user-1

403 {"error": "관리자만 할 수 있습니다"}

GET /admin/stats

Authorization: Bearer key-admin-1

200 {"통계": "관리자만 봅니다", "보는사람": "이서연"}

```
GET /admin/logs
Authorization: Bearer key-admin-1
```

```
200 {"기록":"여기도 관리자만"}
```

app.use 에 미들웨어를 여러 개 넘길 수 있습니다. 순서대로 실행됩니다.

★★ 이 방식의 진짜 장점 나중에 /admin/users 를 새로 만들 때, 인증을 붙이는 걸 '잊어버릴' 수가 없습니다. 이미 위에서 통째로 막아 뒀으니까요.

라우트마다 붙이는 방식은 하나만 빠뜨려도 구멍이 됩니다. 보안은 "빠뜨릴 수 없게 만드는 것" 이 요령입니다.

순서를 틀리면

섹션 9

관리자만 을 인증 보다 먼저 두면 어떻게 될까요?

```
app.use("/admin", 관리자만, 인증);
```

관리자만 안에서 req.user.role 을 읽는데 req.user 가 아직 없습니다. → TypeError: Cannot read properties of undefined (reading 'role') → 500 입니다. 401 이어야 하는데요.

미들웨어는 '앞에서 만든 것을 뒤에서 쓴다' 는 관계가 있습니다. 순서를 함부로 바꾸면 안 됩니다.

★ 방어적으로 만들려면 관리자만 안에서 이렇게 확인할 수 있습니다.

```
if (!req.user) throw HttpError(401, "로그인이 필요합니다");
```

순서를 잘못 짜도 500 대신 401 이 나옵니다. "앞 미들웨어가 제대로 돌아갔는지" 를 확인하는 습관은 좋습니다.

```
app.use((req, res) => {
  res.status(404).json({ error: "그런 주소가 없습니다" });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  const 코드 = err.status || 500;
  res.status(코드).json({ error: 코드 === 500 ? "서버에서 문제가 생겼습니다" :
err.message });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/public`);
});
```

Postman 으로 확인하기

증표는 헤더에 담습니다. 주소창으로는 보낼 수 없습니다.

- 1 Postman 에서 주소를 넣고
- 2 주소칸 아래의 Headers 탭을 누릅니다
- 3 Key 에 Authorization

Value 에 Bearer key-user-1

4. Send

또는 Authorization 탭에서 Type 을 Bearer Token 으로 고르고 Token 칸에 key-user-1 만 넣어도 됩니다. Postman 이 Bearer 를 붙여 줍니다.

눌러 볼 것

GET	/public		증표 없이 → 200
GET	/me		증표 없이 → 401
GET	/me	key-user-1	→ 200 김민준

DELETE /users/7 key-user-1 → 403 DELETE /users/7 key-admin-1 → 200

GET	/admin/stats	key-user-1	→ 403
GET	/admin/stats	key-admin-1	→ 200

401 과 403 이 언제 갈리는지 손으로 확인하는 것이 목적입니다.

직접 해 볼 것

▫ 직접 해보기 1

키별사용자 에 자기 이름으로 사용자를 하나 추가하고 그 키로 /me 를 불러 보세요.

▫ 직접 해보기 2

역할이 "manager" 인 사용자를 추가하고, "user 는 안 되고 manager 와 admin 은 되는" 미들웨어를 만들어 보세요. (힌트: ["manager", "admin"].includes(req.user.role))

▫ 직접 해보기 3

섹션 8의 app.use("/admin", 인증, 관리자만) 에서 관리자만 을 빼 보세요. user 키로 /admin/stats 가 되나요? (확인했으면 되돌리세요)

▫ 직접 해보기 4

섹션 9 처럼 순서를 바꿔 보고 500 이 나는 것을 확인하세요. 그다음 관리자만 안에 req.user 확인을 넣어 401 이 나오게 고쳐 보세요.

▫ 직접 해보기 5

인증 미들웨어에서 req.user 대신 req.사용자 라는 이름을 써 보세요. 잘 되나요? 그런데 왜 다들 req.user 라고 쓸까요? (힌트: 남이 만든 미들웨어를 가져다 쓸 때 이름이 맞아야 합니다)

▫ 직접 해보기 6

/me 를 부를 때마다 터미널에 누가 불렀는지 남겨 보세요. 개념02 의 기록 미들웨어에 req.user 를 더하면 됩니다. (힌트: 기록 미들웨어가 인증보다 위에 있으면 req.user 가 아직 없습니다)

자주 하는 실수

이런 실수를 합니다

401 과 403 을 바꿔 씀

증표가 없거나 틀림 → 401 증표는 맞는데 자격이 모자람 → 403 받는 쪽이 무엇을 고쳐야 할지가 달라집니다.

이런 실수를 합니다

권한 미들웨어를 인증보다 먼저 둠

req.user 가 아직 없어서 500 이 납니다.

이런 실수를 합니다

라우트마다 인증을 붙이다가 하나를 빠뜨림

그 하나가 구멍이 됩니다. app.use 로 통째로 막는 편이 안전합니다.

이런 실수를 합니다

증표를 쿼리에 담음

/me?key=key-admin-1 은 서버 기록에 그대로 남습니다. 헤더에 담으세요.

이런 실수를 합니다

헤더 이름을 대문자로 직접 찾을

req.headers["Authorization"] 은 undefined 입니다. req.get("Authorization") 을 쓰면 대소문자를 안 가립니다.

이런 실수를 합니다

split(" ") 결과를 확인 안 함

"Bearer" 만 보내면 키가 `undefined` 입니다. 그대로 객체에서 찾으면 `undefined` 가 나와 401 이 되긴 하지만, 형식이 틀린 것과 키가 틀린 것은 다른 문제입니다. 나눠서 답하세요.

정리

- 1 인증은 "누구냐", 권한은 "그걸 해도 되냐" 다. 다른 일이다.
- 2 상태코드도 다르다. 누구인지 모르면 401, 누구인지는 아는데 자격이 없으면 403 이다.
- 3 순서가 정해져 있다. 인증 다음에 권한이다. 뒤집으면 누구인지도 모르는 채로 자격을 따지게 된다.
- 4 라우트마다 하나씩 붙이면 언젠가 하나를 빠뜨린다. 구간에 한꺼번에 거는 편이 안전하다.
- 5 증표는 쿼리에 담지 않는다. 주소는 기록에 그대로 남는다. 헤더에 담는다.
- 6 헤더는 `req.get("Authorization")` 으로 꺼낸다. 소문자로 직접 찾지 않는다.
- 7 `split(" ")` 결과는 확인하고 쓴다. 형식이 다르게 오면 `undefined` 가 나온다.

검증을 미들웨어로 빼기

RUN node 개념04_검증_미들웨어.js

그러면 Ctrl + C

04단원 연습문제에서 이런 코드를 썼습니다.

```
const 빠진것 = [];
if (!name) 빠진것.push("name");
if (!line) 빠진것.push("line");
if (빠진것.length > 0) throw HttpError(400, ...);
```

라우트가 열 개면 이 코드가 열 번 나옵니다. 게다가 "길이가 2 이상인가", "A/B/C 중 하나인가" 까지 넣으면 라우트가 검증 코드로 뒤덮입니다.

검증을 미들웨어로 빼면 라우트는 '진짜 하는 일' 만 남습니다.

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

function HttpError(코드, 메시지) {
  const 에러 = new Error(메시지);
  에러.status = 코드;
  return 에러;
}
```

가장 단순한 검증 미들웨어

섹션 1

필수 값 이름들을 받아서, 그것들이 본문에 있는지 확인하는 미들웨어를 만듭니다.

```
function 필수값(...키들) {
```

★ ...키들은 "인자를 몇 개 주든 배열로 받겠다" 는 뜻입니다. (JS자료 09단원) 필수값("name", "line") → 키들은 ["name", "line"]

```
return (req, res, next) => {
  const 본문 = req.body || {};
}
```

morgan 과 미들웨어 파일 나누기

RUN node 개념05_morgan과_파일_나누기.js

그러면 Ctrl + C

★ 이 파일도 터미널을 함께 보세요.

이 파일은 혼자가 아닙니다. 옆 폴더들을 함께 보세요.

05_미들웨어/

개념05_morgan과_파일_나누기.js ← 지금 이 파일

- middlewares/
 - 기록.js 요청 번호 + 기록
 - 인증.js 누구인지 확인 + 권한
 - 검증.js 값의 모양 확인
- utils/
 - HttpError.js 상태코드가 붙은 에러

지금까지 만든 미들웨어를 전부 파일로 옮겼습니다. 이 파일에는 '무엇을 어디에 붙일지' 만 남습니다.

```
const express = require("express");
const morgan = require("morgan");

const { 번호붙이기, 요청기록 } = require("./middlewares/기록");
const { 인증, 역할확인 } = require("./middlewares/인증");
const { 필수값, 검증 } = require("./middlewares/검증");
const { HttpError } = require("./utils/HttpError");

const app = express();
const PORT = process.env.PORT || 3000;
```

morgan — 남이 만든 기록 미들웨어

섹션 1

개념02 에서 기록 미들웨어를 직접 만들었습니다. 같은 일을 하는 도구가 이미 있습니다. morgan 입니다.

```
npm install morgan
```

왜 직접 만들어 보고 나서 도구를 배우나 morgan 이 무슨 일을 하는지 이미 알기 때문에, 안 될 때 어디를 볼지 압니다. 도구부터 배우면 "마법처럼 되는 것" 이 되어 버립니다.

morgan 은 형식 이름을 받습니다.

```

morgan("dev")      개발용. 짧고 색이 붙습니다
morgan("tiny")     제일 짧습니다
morgan("combined") 운영용. 접속 IP, 브라우저 정보까지 전부

```

형식을 직접 적을 수도 있습니다. : 로 시작하는 것이 자리표시자입니다.

내가 만든 자리표시자도 추가할 수 있습니다.

```

morgan.token("user", (req) => (req.user ? req.user.name : "-"));
morgan.token("reqid", (req) => (req.요청번호 ? `#${req.요청번호}` : "-"));

```

★ 자리표시자 이름은 영어로 쓰세요. 값에는 한글이 들어가도 됩니다. 터미널에 찍는 것이라 헤더와 달리 한글이 괜찮습니다.

```

app.use(번호붙이기); // morgan 보다 먼저 실행되어야 :reqid 를 채울 수 있습니다
app.use(morgan(":reqid :method :url :status :response-time ms - :user"));

app.use(express.json());

```

★ 우리가 만든 요청기록 대신 morgan 을 쓰고 있습니다. 둘 다 켜면 같은 내용이 두 번 나옵니다. 요청기록이 어떻게 생겼는지 보고 싶으면 아래 줄의 주석을 지우세요.

```
app.use(요청기록);
```

공개 주소

섹션 2

```

app.get("/health", (req, res) => {
  res.json({ status: "ok" });
});

```

```

GET /health
200 {"status":"ok"}

```

터미널에 이렇게 나옵니다. #1 GET /health 200 2.431 ms - - 맨 끝의 - 는 로그인을 안 했다는 뜻입니다. (:user 자리)

로그인이 필요한 주소

섹션 3

```

app.get("/me", 인증, (req, res) => {
  res.json({ 이름: req.user.name, 역할: req.user.role });
});

```

GET /me

401 {"error":"로그인이 필요합니다"}

GET /me

Authorization: Bearer key-user-1

200 {"이름":"김민준","역할":"user"}

두 번째 요청은 터미널에 이렇게 나옵니다. #3 GET /me 200 1.204 ms - 김민준

★ 누가 무엇을 했는지가 기록에 남습니다. 인증 미들웨어가 req.user 를 붙여 뒀기 때문입니다. morgan 은 응답이 끝난 뒤에 찍기 때문에, 그때는 req.user 가 이미 있습니다.

역할까지 확인하는 주소

섹션 4

```
app.delete("/users/:id", 인증, 역할확인("admin"), (req, res) => {  
  res.json({ 지운사람: req.params.id, 지시한사람: req.user.name });  
});
```

DELETE /users/7

Authorization: Bearer key-user-1

403 {"error":"admin 만 할 수 있습니다"}

DELETE /users/7

Authorization: Bearer key-admin-1

200 {"지운사람":"7","지시한사람":"이서연"}

개념03의 관리자만 이 역할확인("admin")이 되었습니다. 나중에 manager도 허용하려면 역할확인("admin", "manager")로 끝납니다.

```
app.get("/reports", 인증, 역할확인("admin", "user"), (req, res) => {  
  res.json({ 보고서: "둘 다 볼 수 있습니다", 보는사람: req.user.name });  
});
```

GET /reports

Authorization: Bearer key-user-1

200 {"보고서":"둘 다 볼 수 있습니다","보는사람":"김민준"}

검증까지 없기

섹션 5

```
const 설비규칙 = [
  { 키: "name", 필수: true, 타입: "string", 최소길이: 2, 최대길이: 20 },
  { 키: "line", 필수: true, 허용: ["A", "B", "C"] },
];

app.post("/equipments", 인증, 역할확인("admin"), 검증(설비규칙), (req, res) => {
  res.status(201).json({ 등록됨: req.body.name, 등록자: req.user.name });
});
```

POST /equipments

{"name":"용접로봇","line":"A"}

401 {"error":"로그인이 필요합니다"}**POST** /equipments

Authorization: Bearer key-user-1

{"name":"용접로봇","line":"A"}

403 {"error":"admin 만 할 수 있습니다"}**POST** /equipments

Authorization: Bearer key-admin-1

{"name":"용","line":"Z"}

400 {"error":"입력값이 올바르지 않습니다","항목":[{"키":"name","이유":"2글자 이상이
어야 합니다"}, {"키":"line","이유":"A, B, C 중 하나여야 합니다"}]}**POST** /equipments

Authorization: Bearer key-admin-1

{"name":"용접로봇","line":"A"}

201 {"등록됨":"용접로봇","등록자":"이서연"}

★★ 이 한 줄을 다시 보세요.

```
app.post("/equipments", 인증, 역할확인("admin"), 검증(설비규칙), (req, res) => {...})
```

누구니 자격 있니

모양 맞니

진짜 하는 일

라우트 함수 안에는 '진짜 하는 일' 만 두 줄 남았습니다. 검사 코드는 한 줄도 없습니다. 전부 앞에서 끝냈습니다.

이게 미들웨어를 배우는 이유입니다. 실무 서버의 라우트는 대부분 이 모양입니다.

★ 순서가 중요합니다 인증 → 역할확인 → 검증 순입니다. 검증을 먼저 하면, 로그인도 안 한 사람에게 "name 이 짧습니다" 라고 알려 주게 됩니다. 들어올 자격부터 보고, 내용은 나중에 봅니다.

404 와 에러 처리기

섹션 6

```
app.use((req, res) => {
  res.status(404).json({ error: "그런 주소가 없습니다" });
});
```

GET /없는주소

```
404 {"error":"그런 주소가 없습니다"}
```

```
app.use((err, req, res, next) => {
  console.error(`  ✖️ ${req.요청번호} [에러] ${err.message}`);

  if (err.type === "entity.parse.failed") {
    return res.status(400).json({ error: "JSON 형식이 아닙니다" });
  }

  const 코드 = err.status || 500;

  if (코드 === 500) {
    return res.status(500).json({ error: "서버에서 문제가 생겼습니다" });
  }

  res.status(코드).json({ error: err.message });
});
```

★ 에러 기록에도 요청 번호를 넣었습니다. morgan 의 기록과 짝을 맞춰 볼 수 있습니다. "#5 요청이 500 이 났고, 그 이유가 이것" 을 한눈에 알 수 있습니다.

```
app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/health`);
});
```

폴더 구조 정리

프로젝트/

server.js	라우트 연결 + listen
middlewares/	거쳐 가는 것들
기록.js	
인증.js	
검증.js	
utils/	여기저기서 쓰는 작은 도구
HttpError.js	
routes/	주소별 라우트 (04단원 개념05)

나누는 기준 여러 라우트가 함께 쓰는가 → middlewares 여러 파일이 함께 쓰는 작은 함수인가 → utils
특정 자원만 다루는가 → routes

★ 실무에서 파일 이름은 보통 영어로 씁니다 middlewares/auth.js, middlewares/validate.js 처럼요.
이 자료는 처음 배우는 사람이 읽기 쉽도록 한글로 썼습니다. 회사에 가면 영어 이름을 쓰게 될 텐데, 구조는 똑같습니다. 파일 '이름' 은 한글이어도 됩니다. 안 되는 건 라우트 주소와 헤더입니다.

morgan 형식 정리

```
:method      GET
:url          /equipments?line=A
:status       201
:response-time 1.234 (ms 는 직접 붙여야 합니다)
:res[content-length] 응답 크기
:remote-addr   보낸 쪽 IP
:user-agent    브라우저 정보
```

미리 만들어진 묶음

```
morgan("dev")      :method :url :status :response-time ms - :res[content-length]
morgan("combined") 운영용 표준 형식. 접속 기록 분석 도구들이 이걸 읽습니다
```

★ 운영에서는 파일로 남깁니다 터미널에만 찍으면 서버를 켜다 켜 때 사라집니다. morgan 에 stream 옵션을 주면 파일에 쌓입니다. PART 4 에서 합니다.

직접 해 볼 것

▫ 직접 해보기 1

morgan 형식을 "dev" 로 바꿔 보세요. app.use(morgan("dev")) 무엇이 달라지나요? 색이 붙나요?

▫ 직접 해보기 2

app.use(번호붙이기) 를 morgan 아래로 옮겨 보세요. :reqid 자리가 어떻게 되나요? 왜 그럴까요?
(확인했으면 되돌리세요)

▫ 직접 해보기

3

응답 크기를 함께 남기도록 형식을 고쳐 보세요. (힌트: `:res[content-length]`)

▫ 직접 해보기

4

`middlewares/기록.js`의 요청기록 을 켜 보세요. `morgan` 과 무엇이 다른가요? 어느 쪽이 읽기 좋나요?

▫ 직접 해보기

5

`middlewares/검증.js`를 서버 없이 시험해 보세요. 새 파일을 만들고 `const { 한줄검사 } = require("../middlewares/검증"); console.log(한줄검사({ 필수: true }, undefined));` 미들웨어를 파일로 빼면 이렇게 따로 시험할 수 있습니다.

▫ 직접 해보기

6

`middlewares/인증.js`에 `manager` 역할 사용자를 추가하고, `/reports`를 역할확인("admin", "manager")로 바꿔 보세요. `user` 키로는 403이 나와야 합니다.

자주 하는 실수

이런 실수를 합니다

morgan을 번호붙이기보다 위에 둬

`:reqid`가 -로만 나옵니다. 아직 `req.요청번호`가 없기 때문입니다.

이런 실수를 합니다

morgan에 괄호를 안 붙임

`app.use(morgan)` ← 미들웨어를 만드는 함수를 그대로 넘긴 것입니다. `app.use(morgan("dev"))`처럼 불러서 넘기세요.

이런 실수를 합니다

require 경로에서 `../`를 빠뜨림

`middlewares/인증.js`에서 `utils`를 부르려면 `../utils/HttpError`입니다. 한 칸 위로 올라갔다가 내려가야 합니다.

이런 실수를 합니다

미들웨어 순서를 인증보다 검증 먼저로 둬

로그인도 안 한 사람에게 "name이 짧습니다"를 알려 주게 됩니다. 자격을 먼저, 내용을 나중에.

이런 실수를 합니다**module.exports 를 = router 처럼 통째로 덮어씀**

{ 인증, 역할확인 } 처럼 여러 개를 내보낼 때는 객체로 묶으세요. 하나만 내보내면 받는 쪽에서 구조 분해가 안 됩니다.

이런 실수를 합니다**기록에 비밀번호나 증표를 남김**

morgan("combined") 는 referrer 와 user-agent 헤더만 남기고 Authorization 은 남기지 않지만, 직접 만든 기록에서 req.headers 를 통째로 찍으면 증표가 그대로 남습니다.

정리

- 1 morgan 은 남이 만든 기록 미들웨어다. 직접 만든 기록기를 걷어내고 이걸 쓴다.
- 2 morgan 은 요청 번호를 붙이는 미들웨어보다 아래에 둔다. 위에 두면 번호가 아직 없는 채로 기록된다.
- 3 morgan 도 괄호를 붙여서 넘긴다 — app.use(morgan("dev")).
- 4 미들웨어 순서는 기록 → 인증 → 권한 → 검증 → 라우트다. 검증을 인증보다 먼저 두면 누구인지 모른 채 값부터 따지게 된다.
- 5 파일을 나눌 때 require 경로의 ../ 를 빠뜨리지 않는다.
- 6 여러 개를 내보낼 때는 { 인증, 역할확인 } 처럼 객체로 묶는다. 하나만 내보내면 받는 쪽에서 구조 분해가 안 된다.
- 7 404 처리기와 에러 처리기는 나눈 뒤에도 여전히 맨 아래다.
- 8 기록에 비밀번호나 증표를 남기지 않는다.

미들웨어 개념 확인

RUN node 연습문제.js

서버 없이 푸는 문제들입니다. 여기서 만든 함수들을 연습문제_서버.js 에서 그대로 씁니다. 이쪽을 먼저 푸세요.

문제 1 파일로 뺀 함수를 불러다 시험하기

middlewares/검증.js 의 한줄검사 를 불러와서 아래 값들로 시험하세요.

힌트: `const { 한줄검사 } = require("./middlewares/검증");`

./ 로 시작해야 내 폴더에서 찾습니다. (01단원 개념02)

시험할 것 (이 순서로) ({ 필수: true }, undefined) ({ 필수: false }, undefined) ({ 타입: "number" }, "3")
({ 허용: ["A", "B"] }, "Z") ({ 최소길이: 2 }, "김") ({ 최소: 1, 최대: 100 }, 200) ({ 타입: "string", 최소길이: 2 }, "용접로봇")

기대 출력 필수입니다
 null
 number 이어야 합니다 (지금은 string)
 A, B 중 하나여야 합니다
 2글자 이상이어야 합니다
 100 이하여야 합니다
 null

(null 이 '문제 없음' 이라는 뜻입니다)

여기에 코드를 쓰세요

문제 2 규칙 표로 전체 검증하기

작업지시 규칙을 만들고, 데이터 하나를 통째로 검사하는 함수를 만드세요.

규칙 (작업지시규칙 이라는 이름으로)

```
title    필수, 글자, 2~30글자
line     필수, A/B/C 중 하나
priority 선택, 숫자, 1~5
```

함수 전체검증(규칙들, 데이터) 문제가 있는 것들을 [{ 키, 이유 }] 배열로 돌려줍니다. 없으면 빈 배열.

함수 결과짜기(제목, 데이터) 문제가 없으면 "제목: 통과" 문제가 있으면 한 줄씩 "제목: 키 — 이유"

★ 객체를 그대로 `console.log` 하지 마세요.

`node` 는 한글 속성 이름에 따옴표를 붙여서 보기 어렵습니다.

글자로 만들어 찍으세요. (— 는 그대로 복사해 쓰세요)

시험할 것 (이 순서로) 결과짜기("빈 것", {}) 결과짜기("라인 틀림", { title: "벨트 교체", line: "Z" })

결과짜기("우선순위 범위 밖", { title: "벨트 교체", line: "A", priority: 9 }) 결과짜기("정상", { title: "벨트 교체", line: "A", priority: 3 }) 결과짜기("우선순위 없음", { title: "벨트 교체", line: "A" })

```
기대 출력    빈 것: title - 필수입니다
              빈 것: line - 필수입니다
              라인 틀림: line - A, B, C 중 하나여야 합니다
              우선순위 범위 밖: priority - 5 이하여야 합니다
              정상: 통과
              우선순위 없음: 통과
```

(마지막 두 개가 둘 다 통과인 이유를 설명할 수 있어야 합니다)

여기에 코드를 쓰세요

문제 3 401 인가 403 인가

사용자와 허용 역할 목록을 받아 상태코드를 돌려주는 함수 역할판단 을 만드세요.

사용자가 없으면 401 있는데 역할이 허용 목록에 없으면 403 그 밖 200

시험할 것 (이 순서로) (`null`, ["admin"]) ({ role: "user" }, ["admin"]) ({ role: "admin" }, ["admin"]) ({ role: "manager" }, ["admin", "manager"])

```
기대 출력    401
              403
              200
              200
```

(왜 401 을 먼저 확인해야 하는지 생각해 보세요)

여기에 코드를 쓰세요

문제 4 Authorization 헤더 읽기

헤더값을 받아 증표(키)만 뽑아 주는 함수 증표읽기 를 만드세요. 형식이 틀리면 `null` 을 돌려줍니다.

헤더가 없으면 `null` "Bearer " 로 시작 안 하면 `null` "Bearer" 뒤에 키가 없으면 `null` 맞으면 키만

힌트: `split(" ")` 로 나누면 ["Bearer", "key-user-1"] 이 됩니다.

"Bearer" 만 오면 두 번째가 `undefined` 입니다. 이것도 막아야 합니다.

시험할 것 (이 순서로) `undefined` "key-user-1" "Bearer" "Bearer key-user-1"

```
기대 출력    null
              null
              null
              key-user-1
```

여기에 코드를 쓰세요

문제 5 적는 순서

아래 여섯 줄을 '파일에 적어야 할 순서' 대로 배열에 담고 번호를 붙여 출력하세요.

`app.use((err, req, res, next) => 에러 응답)`

```
app.get("/me", 인증, ...)
app.use(express.json())
app.use((req, res) => 404 응답)
```

app.use(번호붙이기)

```
app.use(morgan(...))
```

기대 출력

1. app.use(번호붙이기)
2. app.use(morgan(...))
3. app.use(express.json())
4. app.get("/me", 인증, ...)
5. app.use((req, res) => 404 응답)
6. app.use((err, req, res, next) => 에러 응답)

(1과 2의 순서에 정답이 하나만 있는 건 아닙니다. 왜 이 순서인지 생각해 보세요)

여기에 코드를 쓰세요

문제 6 상황별 상태코드

아래 여섯 상황에 어떤 상태코드를 줘야 할까요?

Authorization 헤더가 아예 없음 Bearer 없이 키만 보냄 없는 키를 보냄 user 가 관리자 전용 주소에 들어옴 admin 이 관리자 전용 주소에 들어옴 로그인 했는데 남의 글을 지우려 함

기대 출력

Authorization 헤더가 아예 없음 → 401
 Bearer 없이 키만 보냄 → 401
 없는 키를 보냄 → 401
 user 가 관리자 전용 주소에 들어옴 → 403
 admin 이 관리자 전용 주소에 들어옴 → 200
 로그인 했는데 남의 글을 지우려 함 → 403

(마지막 줄은 미들웨어에서 막을 수 있을까요? 왜 그럴까요?)

여기에 코드를 쓰세요

문제 7 next() 를 안 부르면

미들웨어에서 `next()` 도 안 부르고 응답도 안 보내면

무슨 일이 생기나: _____

에러가 나나: _____

어떻게 알아채나 (세 가지): ① _____ ② _____

_____ ③ _____

어떻게 막나: _____

특히 어떤 코드를 조심해야 하나: _____

문제 8 req.query 를 못 고치는 이유

Express 5 에서 `req.query.q = "고친값"` 이라고 쓰면

무슨 일이 일어나나: _____

왜 그런가: _____

왜 특히 위험한가: _____

어떻게 하나: _____

규칙 한 줄로 정리하면: _____

들어온 값의 모양을 확인한다

개념04 에서 만든 것을 파일로 옮겼습니다.

```
const { HttpError } = require("../utils/HttpError");
```

필수 값이 있는지만 본다

```
function 필수값(...키들) {
  return (req, res, next) => {
    const 본문 = req.body || {};
    const 빠진것 = 키들.filter((키) => 본문[키] === undefined || 본문[키] === "");

    if (빠진것.length > 0) {
      return next(HttpError(400, `${빠진것.join(", ")} 을(를) 넣어 주세요`));
    }

    next();
  };
}
```

규칙 하나를 값 하나에 대고 확인한다. 문제가 없으면 null.

```
function 한줄검사(규칙, 값) {
  if (값 === undefined || 값 === "") {
    return 규칙.필수 ? "필수입니다" : null;
  }

  if (규칙.타입 && typeof 값 !== 규칙.타입) {
    return `${규칙.타입} 이어야 합니다 (지금은 ${typeof 값})`;
  }

  if (규칙.허용 && !규칙.허용.includes(값)) {
    return `${규칙.허용.join(", ")} 중 하나여야 합니다`;
  }

  if (규칙.최소길이 !== undefined && 값.length < 규칙.최소길이) {
    return `${규칙.최소길이}글자 이상이어야 합니다`;
  }

  if (규칙.최대길이 !== undefined && 값.length > 규칙.최대길이) {
```

```

}

if (규칙.타입 && typeof 값 !== 규칙.타입) {
  return `${규칙.타입} 이어야 합니다 (지금은 ${typeof 값})`;
}

if (규칙.허용 && !규칙.허용.includes(값)) {
  return `${규칙.허용.join(", ")} 중 하나여야 합니다`;
}

if (규칙.최소길이 !== undefined && 값.length < 규칙.최소길이) {
  return `${규칙.최소길이}글자 이상이어야 합니다`;
}

if (규칙.최대길이 !== undefined && 값.length > 규칙.최대길이) {
  return `${규칙.최대길이}글자 이하여야 합니다`;
}

if (규칙.최소 !== undefined && 값 < 규칙.최소) {
  return `${규칙.최소} 이상이어야 합니다`;
}

if (규칙.최대 !== undefined && 값 > 규칙.최대) {
  return `${규칙.최대} 이하여야 합니다`;
}

return null;
}

```

규칙 표를 받아 미들웨어를 만든다

```

function 검증(규칙들) {
  return (req, res, next) => {
    const 본문 = req.body || {};
    const 항목 = [];

    for (const 규칙 of 규칙들) {
      const 이유 = 한줄검사(규칙, 본문[규칙.키]);

      if (이유) {
        항목.push({ 키: 규칙.키, 이유 });
      }
    }

    if (항목.length > 0) {
      return res.status(400).json({ error: "입력값이 올바르지 않습니다", 항목 });
    }
  }
}

```


05단원 · middlewares/기록.js

요청 기록 남기기

개념02 에서 만든 것을 파일로 옮겼습니다.

```
let 요청번호 = 0;
```

요청마다 번호를 붙입니다.

```
function 번호붙이기(req, res, next) {
  요청번호 += 1;
  req.요청번호 = 요청번호;
  res.set("X-Request-Id", String(요청번호));
  next();
}
```

요청과 걸린 시간을 터미널에 남깁니다.

```
function 요청기록(req, res, next) {
  const 시작 = Date.now();

  console.log(`#${req.요청번호} ${req.method} ${req.originalUrl}`);

  res.on("finish", () => {
    const 걸린시간 = Date.now() - 시작;
    const 표시 = res.statusCode >= 400 ? "*" : " ";
    console.log(`  ${표시}#${req.요청번호} ${res.statusCode} - ${걸린시간}ms`);
  });

  next();
}
```

★ 여러 개를 내보낼 때는 객체로 묶습니다. (01단원 개념02)

```
module.exports = { 번호붙이기, 요청기록 };
```

부르는 쪽에서는 이렇게 씁니다.

```
const { 번호붙이기, 요청기록 } = require("../middlewares/기록");
```

작업지시 API (미들웨어 총동원)

RUN npx nodemon 연습문제_서버.js

★ 이 문제의 목적 라우트 함수 안에서 '검사 코드'를 전부 없애는 것입니다. 로그인 확인·권한 확인·숫자 변환·찾기·404를 모두 미들웨어로 밀어냅니다.

다 만들고 나서 라우트 함수들을 보세요. 대부분 한두 줄이어야 합니다. 길다면 미들웨어로 뺄 것이 남은 것입니다.

만들 주소

GET	/health	누구나
GET	/orders	로그인만 (?line= / ?status= 로 걸러 보기)
GET	/orders/:id	로그인만
POST	/orders	admin 만
PATCH	/orders/:id	로그인만 (상태만 바꾸기)

DELETE /orders/:id admin 만 그 밖 405 또는 404

쓸 수 있는 증표 Bearer key-user-1 김민준 (user) Bearer key-admin-1 이서연 (admin)

```
const express = require("express");
const morgan = require("morgan");
```

★ 05단원에서 만든 미들웨어를 그대로 가져다 씁니다.

```
const { 번호붙이기 } = require("./middlewares/기록");
const { 인증, 역할확인 } = require("./middlewares/인증");
const { 검증 } = require("./middlewares/검증");
const { HttpError } = require("./utils/HttpError");

const app = express();
const PORT = process.env.PORT || 3000;
```

문제 1 기본 미들웨어 없기

아래 세 가지를 순서대로 등록하세요.

① 번호붙이기 ② morgan — 형식은 ":reqid :method :url :status :response-time ms - :user" ③ express.json()

morgan 의 :reqid 와 :user 는 기본에 없습니다. 직접 만들어야 합니다.

```
morgan.token("user", (req) => ...)
morgan.token("reqid", (req) => ...)
```

★ 번호붙이기가 morgan 보다 먼저여야 :reqid 가 채워집니다. 왜일까요?

기대 결과 (터미널): #1 GET /health 200 2.431 ms - - #3 GET /orders 200 1.204 ms - 김민준

여기에 코드를 쓰세요

준비 (여기는 그대로 두세요)

```
let 작업지시들 = [
  { id: 1, title: "벨트 교체", line: "A", priority: 2, status: "대기", 등록자: "김민준" },
  { id: 2, title: "베어링 점검", line: "B", priority: 4, status: "완료", 등록자: "이서연" },
];

let 다음번호 = 3;

const 상태값들 = ["대기", "진행", "완료"];

const 작업지시규칙 = [
  { 키: "title", 필수: true, 타입: "string", 최소길이: 2, 최대길이: 30 },
  { 키: "line", 필수: true, 허용: ["A", "B", "C"] },
  { 키: "priority", 필수: false, 타입: "number", 최소: 1, 최대: 5 },
];
```

문제 2 :id 를 숫자로 바꾸는 미들웨어

app.param 으로 ":id" 를 가로채서 - 숫자가 아니면 next(400, "번호는 숫자여야 합니다")) - 맞으면 req.번호 에 숫자로 담고 next()

★ 이렇게 해 두면 :id 를 쓰는 라우트가 전부 혜택을 받습니다. Number(req.params.id) 를 다시 쓸 일이 없어집니다.

힌트: app.param("id", (req, res, next, 값) => { ... })

인자가 네 개입니다. 마지막이 그 파라미터의 값입니다.

여기에 코드를 쓰세요

문제 3 작업지시를 찾아 붙이는 미들웨어

함수 작업지시찾기(req, res, next) 를 만드세요. - req.번호 로 작업지시들 에서 찾습니다 - 없으면 next(404, `${req.번호}`번 작업지시가 없습니다)) - 있으면 req.작업 에 담고 next()

★ 이 함수 하나를 GET·PATCH·DELETE 세 곳이 나눠 씁니다. 404 메시지를 바꾸고 싶으면 여기 한 곳만 고치면 됩니다.

여기에 코드를 쓰세요

문제 4 공개 주소

GET /health 에 200 과 { status: "ok" } 를 돌려주세요. 증표 없이 됩니다.

기대 출력 GET /health → 200 {"status":"ok"}

여기에 코드를 쓰세요

문제 5 목록 (로그인만 하면 볼 수 있음)

GET /orders 에 목록을 돌려주세요. ?line= 과 ?status= 로 걸러 볼 수 있어야 합니다. 둘 다 붙을 수도 있습니다.

기대 출력	GET /orders	(증표 없이) → 401
	GET /orders	user → 2건
	GET /orders?line=A	user → 1건 (벨트 교체)
	GET /orders?status=완료	user → 1건 (베어링 점검)
	GET /orders?line=C	user → 200 과 []

힌트: 조건을 겹쳐 쓰려면 `let` 으로 시작해 `filter` 를 이어 붙입니다.

```
let 결과 = 작업지시들;
if (line) 결과 = 결과.filter(...);
if (status) 결과 = 결과.filter(...);
```

여기에 코드를 쓰세요

문제 6 하나만 보기

GET /orders/:id 에 그 작업지시를 돌려주세요.

기대 출력	GET /orders/1	user → 200	벨트 교체
	GET /orders/99	user → 404	
	GET /orders/abc	user → 400	

★ 문제 2:3 을 제대로 만들었다면 이 라우트는 한 줄입니다. 길어졌다면 미들웨어로 갔어야 할 코드가 여기 남아 있는 것입니다.

여기에 코드를 쓰세요

문제 7 새로 등록 (admin 만)

POST /orders 로 새 작업지시를 만드세요. 인증 → 역할확인("admin") → 검증(작업지시규칙) 순으로 미들웨어를 붙입니다.

새 작업지시의 모양:

```
{ id: 다음번호, title, line, priority: 안 주면 3, status: "대기", 등록자: 로그인한 사람 }
```

```
기대 출력   POST /orders (증표 없이)           → 401
              POST /orders user { "title":"모터 점검","line":"C" } → 403
              POST /orders admin { "title":"모","line":"Z" }       → 400, 항목 두 개
              POST /orders admin { "title":"모터 점검","line":"C" } → 201, id 3
```

★★ 등록자를 본문에서 받지 마세요. req.user.name 을 쓰세요. 보낸 쪽이 정하게 두면 남의 이름으로 등록할 수 있습니다. “누가 했는지” 는 언제나 서버가 정합니다.

힌트: priority 기본값은 req.body.priority ?? 3 입니다.

|| 3 으로 쓰면 0 을 보냈을 때 3 이 되어 버립니다.

여기에 코드를 쓰세요

문제 8 상태 바꾸기 (로그인만 하면 됨)

PATCH /orders/:id 로 status 만 바꾸세요.

```
기대 출력   PATCH /orders/3 user { "status":"이상" } → 400
              {"error":"status 는 대기, 진행, 완료 중 하나여야 합니다"}
              PATCH /orders/3 user { "status":"진행" } → 200, status 가 "진행"
```

★ 여기서는 검증 미들웨어를 안 써도 됩니다. 상태값들 은 이 라우트에서만 쓰는 규칙입니다. “두 곳 이상에서 쓰이면 빼낸다” 정도로 생각하세요.

여기에 코드를 쓰세요

문제 9 지우기 (admin 만)

DELETE /orders/:id 로 지우세요. 204 를 돌려줍니다.

```
기대 출력  DELETE /orders/3 user → 403
            DELETE /orders/3 admin → 204
            DELETE /orders/3 admin → 404 (이미 지웠으니)
```

★★ 미들웨어 순서를 조심하세요. 인증 → 역할확인 → 작업지시찾기 순입니다.

찾기를 먼저 하면 어떻게 될까요? 403 을 받을 사람이 404 를 통해 “그 번호는 있구나” 를 알게 됩니다. 사소해 보이지만 이런 것이 정보를 흘리는 통로가 됩니다. 자격이 없는 사람에게는 ‘있는지 없는지’ 도 알려 주지 않습니다.

여기에 코드를 쓰세요

문제 10 405 · 404 · 에러 처리기

① `app.all("/orders", ...)` 로 405 를 만드세요. (모든 `/orders` 라우트 아래에) ② 주소 없는 `app.use` 로 404 를 JSON 으로 만드세요. ③ 인자 네 개짜리 에러 처리기를 맨 마지막에 만드세요. - 터미널에 요청 번호와 함께 남기기 - `err.type` 이 “`entity.parse.failed`” 면 400 “JSON 형식이 아닙니다” - `err.status` 가 있으면 그 코드와 `err.message` - 없으면 500 “서버에서 문제가 생겼습니다”

```
기대 출력  PUT /orders admin → 405
            GET /없는주소 → 404
```

여기에 코드를 쓰세요

```
app.listen(PORT, () => {
  console.log(`작업지시 API 가 켜졌습니다. http://localhost:${PORT}/health`);
});
```

다 만들었으면 이 순서대로 눌러 보세요

Postman 의 Headers 에 Authorization: Bearer key-user-1 또는 key-admin-1

```

1 GET    /health                200
(중표 없이)                401
2건
4 GET    /orders?line=A         user      1건
5 GET    /orders?line=C         user      200 과 []
6 GET    /orders/1             user      벨트 교체
7 GET    /orders/99            user      404
user      400
403
    { "title": "모터 점검", "line": "C" }
10 POST   /orders                admin     201, id 3
11 PATCH  /orders/3              user      { "status": "진행" } 200
12 DELETE /orders/3              user      403
13 DELETE /orders/3              admin     204
14 PUT    /orders                admin     405

```

★ 터미널을 함께 보세요. 이렇게 남아야 합니다.

#10 POST /orders 201 3.121 ms - 이서연 #12 DELETE /orders/3 403 1.004 ms - 김민준

이 기록만 봐도 “김민준이 지우려다 막혔다” 를 알 수 있습니다.

다 만든 뒤 답해 보세요

① 라우트 함수 중 가장 긴 것이 몇 줄인가요? 3줄이 넘는다면, 그 안의 무엇을 미들웨어로 뺄 수 있을까요?

② 작업지시찾기 를 쓰는 라우트가 몇 개인가요? 이걸 안 만들었다면 같은 코드를 몇 번 썼을까요?

③ 문제 9에서 작업지시찾기 를 역할확인보다 먼저 두면 user 가 DELETE /orders/99 를 보냈을 때 무엇이 나오나요? 실제로 순서를 바꿔서 확인해 보세요. (확인 후 되돌리세요)

④ 404 메시지를 “찾을 수 없습니다” 로 바꾸려면 몇 군데를 고쳐야 하나요?

⑤ 이 서버에 ‘불량기록’ API 를 추가한다면 middlewares 폴더에서 무엇을 그대로 쓸 수 있나요?

잘 안 될 때 보는 곳

요청이 멈추고 응답이 안 옴 → 어느 미들웨어가 `next()` 를 안 불렀습니다. → 미들웨어마다 `console.log` 를 넣어 어디까지 갔는지 보세요.

`req.user` 가 `undefined` → 인증 미들웨어를 안 붙였거나, 다른 미들웨어보다 뒤에 붙였습니다.

`req.번호` 가 `undefined` → `app.param` 의 이름이 라우트의 `:id` 와 다릅니다. → `app.param("id")` 는 `:id` 에만 걸립니다.

500 이 나는데 401 이어야 함 → 역할확인이 인증보다 먼저 실행되어 req.user 를 못 읽은 것입니다.

morgan 의 :reqid 가 계속 - → 번호붙이기가 morgan 보다 아래에 있습니다.

morgan 의 :user 가 계속 - → 정상입니다. 로그인이 필요 없는 주소이거나 401 이 난 요청입니다.

Cannot find module './middlewares/기록' → 이 파일과 middlewares 폴더가 같은 폴더에 있어야 합니다. → ./ 를 빠뜨리면 node_modules 에서 찾습니다.

누구인지 확인하고 권한을 본다

개념03 에서 만든 것을 파일로 옮겼습니다.

```
const { HttpError } = require("../utils/HttpError");
```

진짜 서버라면 데이터베이스에서 찾습니다. PART 4 에서 바꿉니다.

```
const 키별사용자 = {
  "key-user-1": { name: "김민준", role: "user" },
  "key-admin-1": { name: "이서연", role: "admin" },
};

function 인증(req, res, next) {
  const 헤더값 = req.get("Authorization");

  if (!헤더값) {
    return next(HttpError(401, "로그인이 필요합니다"));
  }

  const [방식, 키] = 헤더값.split(" ");

  if (방식 !== "Bearer" || !키) {
    return next(HttpError(401, "Authorization 형식이 올바르지 않습니다"));
  }

  const 사용자 = 키별사용자[키];

  if (!사용자) {
    return next(HttpError(401, "증표가 올바르지 않습니다"));
  }

  req.user = 사용자;
  next();
}
```

로그인은 했는지도 함께 확인합니다. 순서를 잘못 짜도 500 대신 401 이 나가게 하려는 것입니다. (개념03 섹션 9)

```
function 역할확인(...허용역할들) {
  return (req, res, next) => {
    if (!req.user) {
```

상태코드가 붙은 에러 만들기

여러 파일에서 쓰니까 여기로 뺐습니다.

```
function HttpError(코드, 메시지) {  
  const 에러 = new Error(메시지);  
  에러.status = 코드;  
  return 에러;  
}  
  
module.exports = { HttpError };
```

★ utils 라는 이름 “어디에도 딱 안 맞지만 여러 곳에서 쓰는 작은 도구들” 을 모으는 폴더입니다.

관례입니다. helpers, lib 라고 부르는 곳도 있습니다.

★ 주의 utils 에 아무거나 다 넣으면 나중에 ‘잡동사니 폴더’ 가 됩니다. “이게 정말 어디에도 안 맞나?” 를 한번 생각하고 넣으세요.

REST API 설계

OPEN 06_REST_API_설계

06

```
const app = express();
app.use(express.json());
const 라우터 = express.Router();
if (!설비) {
```

01 자원과 주소 설계

02 목록 다루기 (페이징 · 정렬 · 부분 응답)

03 검색과 필터

04 응답 형식 통일하기

05 API 문서와 버전 관리

- 연습문제

- 서버 연습문제

자원과 주소 설계

RUN node 개념01_자원과_주소_설계.js

그러면 Ctrl + C

지금까지 주소를 그때그때 지어서 썼습니다. 이제 규칙을 정합니다.

왜 규칙이 필요한가 주소는 한 번 공개하면 못 바꿉니다. 프론트엔드가 그 주소를 쓰고 있고, 다른 회사가 쓰고 있을 수도 있습니다. 나중에 고치려면 옛 주소와 새 주소를 둘 다 유지해야 합니다.

그래서 처음에 잘 지어야 합니다. 다행히 규칙이 정해져 있습니다. 이것 REST 라고 부릅니다.

```
const express = require("express");
const { 설비목록, 점검기록목록 } = require("./data/설비데이터");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

자원이란 무엇인가

섹션 1

자원(resource)은 '이 시스템이 다루는 것' 입니다.

설비 관리 시스템 설비, 점검기록, 작업지시, 사용자

쇼핑몰	상품, 주문, 장바구니, 리뷰, 회원
게시판	글, 댓글, 회원

주소를 지을 때는 '무엇을 할까' 가 아니라 '무엇에 대해서인가' 를 먼저 정합니다.

× 설비를 등록한다, 설비를 조회한다, 설비를 삭제한다 ← 할 일 목록 ○ 설비 ← 자원 하나

자원 하나가 정해지면 주소는 두 개뿐입니다.

/equipments	설비 전부 (컬렉션)
/equipments/3	설비 하나 (개별)

나머지는 메서드가 말합니다.

다섯 가지 기본 주소

섹션 2

```
const 라우터 = express.Router();

라우터.get("/equipments", (req, res) => {
  res.json({ 설명: "목록", 개수: 설비목록().length });
});

라우터.get("/equipments/:id", (req, res) => {
  const 설비 = 설비목록().find((설비) => 설비.id === Number(req.params.id));

  if (!설비) {
    return res.status(404).json({ error: "그 설비가 없습니다" });
  }

  res.json(설비);
});

라우터.post("/equipments", (req, res) => {
  res.status(201).json({ 설명: "새로 만들기" });
});

라우터.patch("/equipments/:id", (req, res) => {
  res.json({ 설명: "일부만 고치기", id: Number(req.params.id) });
});

라우터.delete("/equipments/:id", (req, res) => {
  res.sendStatus(204);
});
```

06

주소 앞에 /api/v1 을 붙입니다

섹션 3

```
app.use("/api/v1", 라우터);
```

```
GET /api/v1/equipments
```

```
200 {"설명":"목록","개수":12}
```

```
GET /api/v1/equipments/3
```

```
200 {"id":3,"name":"프레스 1호","line":"B","status":"가동","설치일":"2019-07-02"}
```

```
POST /api/v1/equipments
```

```
201 {"설명":"새로 만들기"}
```

PATCH /api/v1/equipments/3

200 {"설명":"일부만 고치기","id":3}

DELETE /api/v1/equipments/3

204

★ /api 를 붙이는 이유 나중에 같은 서버에서 HTML 화면도 줘야 할 때가 옵니다.

GET /equipments 사람이 보는 HTML 페이지

GET /api/v1/equipments 프로그램이 쓰는 JSON

섞이면 "이 주소가 HTML 인가 JSON 인가" 를 매번 확인해야 합니다. 처음부터 나눠 두면 그럴 일이 없습니다.

★ /v1 을 붙이는 이유 주소는 한 번 공개하면 못 바꾼다고 했습니다. 그런데 언젠가는 꼭 바꿔야 할 일이 생깁니다.

그때 /api/v2/equipments 를 새로 만들고, v1 은 그대로 둡니다. 기존에 쓰던 곳은 안 깨지고, 새로 만드는 곳은 v2 를 씁니다. 준비가 되면 v1 을 없앱니다.

v1 없이 시작하면 나중에 붙일 데가 없어서 주소가 지저분해집니다. 처음부터 붙여 두세요. 붙이는 비용은 0 입니다.

포함 관계는 주소로 나타냅니다

섹션 4

"1번 설비의 점검기록" 은 어떻게 쓸까요?

× /checkLogs?equipmentId=1 됩니다. 되지만 관계가 안 보입니다 ○ /equipments/1/logs 1번 설비에 딸린 것이 분명합니다

```
라우터.get("/equipments/:id/logs", (req, res) => {  
  const 설비번호 = Number(req.params.id);  
  const 기록들 = 점검기록목록().filter((기록) => 기록.equipmentId === 설비번호);  
  
  res.json({ 설비번호, 개수: 기록들.length, ids: 기록들.map((기록) => 기록.id) });  
});
```

GET /api/v1/equipments/1/logs

200 {"설비번호":1,"개수":3,"ids":[1,2,5]}

GET /api/v1/equipments/7/logs

200 {"설비번호":7,"개수":0,"ids":[]}


```
라우터.get("/equipments/:id/logs/:logId", (req, res) => {
  const 기록 = 점검기록목록().find(
    (기록) =>
      기록.id === Number(req.params.logId) && 기록.equipmentId ===
      Number(req.params.id)
  );

  if (!기록) {
    return res.status(404).json({ error: "그 점검기록이 없습니다" });
  }

  res.json(기록);
});
```

```
GET /api/v1/equipments/1/logs/2
200 {"id":2,"equipmentId":1,"result":"이상","점검일":"2026-08-08","담당자":"이서연"}
```

```
GET /api/v1/equipments/3/logs/2
404 {"error":"그 점검기록이 없습니다"}
```

★ 두 번째 확인이 중요합니다. 2번 기록은 분명히 있습니다. 그런데 3번 설비의 것이 아닙니다. 주소가 “3번 설비의 2번 기록” 이라고 했으니, 그런 것은 없는 게 맞습니다.

logId 만 보고 찾으면 남의 설비 기록이 보입니다. 중첩 주소를 만들 때는 반드시 ‘위쪽 번호까지’ 확인하세요. 실무에서 정말 자주 나는 사고입니다. 남의 주문서가 보이는 식으로요.

★ 얼마나 깊이 중첩해도 되나 두 단계까지만 하세요.

```
/equipments/1/logs/2      ○
/lines/A/equipments/1/logs/2 x 너무 깊니다
```

깊어지면 이렇게 나눕니다.

```
/logs/2      기록 하나는 그냥 최상위로
/equipments/1/logs  목록만 중첩으로
```

동사가 꼭 필요할 때

섹션 5

“주소에 동사를 쓰지 마라” 고 했지만 예외가 있습니다. CRUD 로 표현이 안 되는 ‘행동’ 이 있습니다.

설비를 점검한다 만들기? 고치기? 애매합니다 주문을 취소한다 지우기가 아닙니다. 기록은 남아야 합니다 비밀번호를 바꾼다 고치기지만 보통과 다릅니다

```
라우터.post("/equipments/:id/checks", (req, res) => {
```

점검을 '실행' 하면 점검기록이 하나 '생깁니다'. 그러면 이걸 '점검기록을 만드는 것' 입니다. 명사로 표현할 수 있습니다.

```
res.status(201).json({ 설명: "점검을 실행해 기록을 하나 만들었습니다" });
});
```

```
POST /api/v1/equipments/3/checks
```

```
201 {"설명":"점검을 실행해 기록을 하나 만들었습니다"}
```

```
라우터.post("/orders/:id/cancel", (req, res) => {
```

이걸 명사로 바꾸기 어렵습니다. 이럴 때는 동사를 씁니다.

```
res.json({ 설명: "취소했습니다", id: Number(req.params.id) });
});
```

```
POST /api/v1/orders/7/cancel
```

```
200 {"설명":"취소했습니다","id":7}
```

★ 순서대로 생각하세요. ① 명사로 표현할 수 있나? → /equipments/1/checks (점검기록을 만든다)

② 상태를 바꾸는 것인가? → PATCH /orders/7 { "status": "취소" }

③ 둘 다 아니면 동사를 쓴다 → POST /orders/7/cancel

③을 쓸 때는 반드시 POST 입니다. GET 으로 만들면 안 됩니다. GET 은 "가져오기만 한다" 는 약속이라, 브라우저가 미리 불러 볼 수도 있습니다. 주소창에 넣거나 링크를 걸면 주문이 취소되어 버립니다.

나쁜 주소 모아 보기

섹션 6

```
app.get("/design-guide", (req, res) => {
  res.json({
    나쁜: [
      "GET /getEquipments",
      "GET /equipment/3",
      "POST /createEquipment",
      "GET /deleteEquipment?id=3",
      "POST /equipments/3/update",
      "GET /equipmentsByLine?line=A",
    ],
  });
});
```

```

    좋음: [
      "GET /api/v1/equipments",
      "GET /api/v1/equipments/3",
      "POST /api/v1/equipments",
      "DELETE /api/v1/equipments/3",
      "PATCH /api/v1/equipments/3",
      "GET /api/v1/equipments?line=A",
    ],
  });
});

```

GET /design-guide

```

200 {"나뭇":["GET /getEquipments","GET /equipment/3","POST /createEquipment","GET /deleteEquipment?id=3","POST /equipments/3/update","GET /equipmentsByLine?line=A"],"좋음":["GET /api/v1/equipments","GET /api/v1/equipments/3","POST /api/v1/equipments","DELETE /api/v1/equipments/3","PATCH /api/v1/equipments/3","GET /api/v1/equipments?line=A"]}

```

한 줄씩 무엇이 틀렸는지

/getEquipments	동사가 들어갔습니다. GET 이 이미 '가져온다' 입니다
/equipment/3	단수형입니다. 컬렉션에서 하나를 고르는 것이니 복수형
/createEquipment	동사 + 메서드가 GET/POST 어느 쪽인지도 안 보입니다
/deleteEquipment?id=3	★ 가장 위험합니다. GET 으로 지우고 있습니다
/equipments/3/update	고치기는 PATCH 가 이미 있습니다
/equipmentsByLine	걸러 보기는 쿼리로. 조건마다 주소를 만들면 끝이 없습니다

```

app.use((req, res) => {
  res.status(404).json({ error: "그런 주소가 없습니다" });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  const 코드 = err.status || 500;
  res.status(코드).json({ error: 코드 === 500 ? "서버에서 문제가 생겼습니다" : err.message });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/api/v1/equipments`);
});

```

주소 설계 규칙 정리

① 자원은 복수형 명사 /equipments (/equipment ×) ② 동사는 메서드가 말한다
GET/POST/PATCH/DELETE ③ 하나는 경로로 /equipments/3 ④ 걸러 보기는 쿼리로 /equipments?
line=A ⑤ 포함 관계는 중첩으로 /equipments/1/logs (두 단계까지) ⑥ 앞에 /api/v1 을 붙인다 ⑦ 영어
소문자와 하이픈만 /work-orders (/작업지시 ×, /workOrders 도 피함)

⑦의 하이픈 주소는 대소문자를 구별하는 곳도 있고 안 하는 곳도 있어서 /workOrders 는 위험합니다.
/work-orders 로 쓰세요.

그리고 04단원에서 본 것 — 주소에 한글을 쓰면 조용히 404 가 납니다.

직접 해 볼 것

▫ 직접 해보기 1

아래 요구사항의 주소와 메서드를 지어 보세요.

- ① 모든 작업지시 목록
- ② A라인 작업지시만
- ③ 3번 작업지시 하나
- ④ 3번 작업지시에 댓글 달기
- ⑤ 3번 작업지시의 댓글 목록
- ⑥ 3번 작업지시를 완료 처리
- ⑦ 5번 댓글 지우기

▫ 직접 해보기 2

/api/v1/equipments/1/logs/2 를 만들었습니다. /api/v1/logs/2 도 함께 만들면 어떤 문제가 생길까요?
(힌트: 같은 것을 두 주소로 가리키게 됩니다)

▫ 직접 해보기 3

섹션 4의 logId 확인에서 equipmentId 조건을 빼 보세요. /api/v1/equipments/3/logs/2 가 어떻게
되나요? 이게 왜 사고인지 설명해 보세요. (확인 후 되돌리세요)

▫ 직접 해보기 4

POST /orders/:id/cancel 을 GET 으로 바꿔 보세요. 브라우저 주소창에 넣으면 어떻게 되나요? 이게 왜
위험한지 생각해 보세요. (확인 후 되돌리세요)

▫ 직접 해보기 5

여러분이 만들고 싶은 서비스의 자원을 세 개 적고, 각각의 다섯 가지 기본 주소를 써 보세요.

자주 하는 실수

이런 실수를 합니다

주소에 동사를 넣음

/getX, /createX, /updateX, /deleteX 메서드가 이미 그 말을 하고 있습니다.

이런 실수를 합니다

GET 으로 데이터를 바꿈

/deleteEquipment?id=3 은 링크를 누르는 것만으로 지워집니다. 검색 엔진이 링크를 따라가다 데이터를 다 지운 사고가 실제로 있었습니다. GET 은 몇 번을 불러도 아무것도 안 바뀌어야 합니다.

이런 실수를 합니다

자원을 단수형으로 씀

/equipment/3 보다 /equipments/3 입니다. “설비 목록에서 3번” 이라고 읽으면 복수형이 자연스럽습니다.

이런 실수를 합니다

중첩 주소에서 위쪽 번호를 확인 안 함

/equipments/3/logs/2 에서 logId 만 보면 남의 기록이 보입니다.

이런 실수를 합니다

조건마다 새 주소를 만듦

/equipmentsByLine, /equipmentsByStatus, /equipmentsByLineAndStatus ... 조건이 늘 때마다 주소가 두 배씩 늘어납니다. 쿼리를 쓰세요.

이런 실수를 합니다

/api/v1 을 안 붙이고 시작함

나중에 붙이려면 기존 주소를 전부 바꿔야 합니다. 처음에 붙이는 것은 공짜입니다.

정리

- 1 주소는 '무엇' 을 가리키고, '무엇을 하는지' 는 메서드가 말한다.
- 2 그래서 주소에 동사를 넣지 않는다. /getDocuments 가 아니라 GET /documents 다.
- 3 자원은 복수형으로 쓴다. /document 가 아니라 /documents.
- 4 기본은 다섯 가지다 — 목록·하나 보기·만들기·고치기·지우기.
- 5 주소 앞에 /api/v1 을 붙이고 시작한다. 나중에 붙이려면 이미 쓰고 있는 쪽을 전부 고쳐야 한다.
- 6 포함 관계는 주소로 나타낸다 — /documents/12/comments. 이때 위쪽 번호(12)가 진짜 있는지 반드시 먼저 확인한다.
- 7 GET 으로 데이터를 바꾸지 않는다. GET 은 봐도 아무 일이 없어야 한다.
- 8 조건마다 새 주소를 만들지 않는다. 조건은 쿼리로 받는다.

목록 다루기 (페이징 · 정렬 · 부분 응답)

RUN node 개념02_목록_페이징과_정렬.js

그러면 Ctrl + C

지금까지 목록을 이렇게 돌려줬습니다.

```
res.json(설비들);
```

설비가 12개일 때는 괜찮습니다. 그런데 10만 개라면?

① 서버가 10만 개를 전부 메모리에 올립니다 ② JSON 으로 바꾸면 수십 MB 가 됩니다 ③ 그걸 통째로 보냅니다. 몇 초가 걸립니다 ④ 받는 쪽도 10만 개를 화면에 그리려다 멈춥니다

그래서 '조금씩 나눠서' 줍니다. 이것 페이징이라고 합니다.

```
const express = require("express");
const { 설비목록 } = require("../data/설비데이터");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

쪽 번호 정하기

섹션 1

쿼리로 받습니다. 없을 수도 있고, 이상한 값이 올 수도 있습니다.

```
const 기본쪽크기 = 10;
const 최대쪽크기 = 100;

function 쪽정보(query) {
```

Number(undefined) 는 NaN, Number(null) 은 0 입니다. 둘 다 막아야 합니다.

```
let page = Number(query.page);
let limit = Number(query.limit);

if (!Number.isInteger(page) || page < 1) page = 1;
if (!Number.isInteger(limit) || limit < 1) limit = 기본쪽크기;
```

★ 상한을 반드시 두세요 ?limit=999999 로 보내면 결국 전부 주게 됩니다. 페이징한 의미가 없어집니다.

```

if (limit > 최대쪽크기) limit = 최대쪽크기;

return { page, limit };
}

```

★ 왜 잘못된 값을 에러로 안 만들고 기본값으로 바꾸나 ?page=abc 는 보통 오타입니다. 400 을 내는 것도 맞지만, 목록 조회는 “일단 뭐라도 보여 주는” 편이 쓰기 좋습니다. POST 로 데이터를 만들 때와는 판단이 다릅니다. 틀린 값으로 뭔가를 ‘만드는’ 것은 400 이어야 합니다.

정렬 조건 읽기

섹션 2

?sort=name 이름 오름차순 ?sort=-id 번호 내림차순 (앞에 - 를 붙이면 반대) ?sort=line,-id 라인 오름차순, 같으면 번호 내림차순

```

function 정렬조건(sort) {
  if (!sort) return [];

  return sort
    .split(",")
    .map((조각) => 조각.trim())
    .filter(Boolean)
    .map((조각) =>
      조각.startsWith("-")
        ? { 키: 조각.slice(1), 방향: -1 }
        : { 키: 조각, 방향: 1 }
    );
}

function 비교하기(a, b, 조건들) {
  for (const { 키, 방향 } of 조건들) {
    const 왼쪽 = a[키];
    const 오른쪽 = b[키];

```

글자끼리는 localeCompare 로 비교합니다. 이유는 섹션 6-2 에 있습니다. numeric: true 는 “설비2호” 와 “설비10호” 를 숫자로 보게 합니다.


```
if (typeof 왼쪽 === "string" && typeof 오른쪽 === "string") {
  const 결과 = 왼쪽.localeCompare(오른쪽, "ko", { numeric: true });
  if (결과 !== 0) return 결과 * 방향;
  continue;
}

if (왼쪽 < 오른쪽) return -1 * 방향;
if (왼쪽 > 오른쪽) return 1 * 방향;
}

return 0; // 전부 같으면 순서를 안 바꿉니다
}
```

원하는 속성만 골라 주기

섹션 3

목록에서 이름만 필요한 화면도 있습니다. 그런데 설비 하나에 속성이 스무 개라면, 열아홉 개를 헛되이 보내는 셈입니다.

?fields=id,name → { id, name } 만

```
function 필드고르기(항목, fields) {
  if (!fields) return 항목;

  const 결과 = {};

  for (const 키 of fields.split(",").map((조각) => 조각.trim())) {
    if (키 in 항목) {
      결과[키] = 항목[키];
    }
  }

  return 결과;
}
```

★ 없는 이름을 달라고 하면? 그냥 무시합니다. 에러를 낼 수도 있지만, “달라고 한 것 중 있는 것만 준다” 가 다루기 쉽습니다.

다 합치기

섹션 4

```
app.get("/api/v1/equipments", (req, res) => {
  const { page, limit } = 쪽정보(req.query);
  const 조건들 = 정렬조건(req.query.sort);

  let 결과 = 설비목록();
```

① 정렬을 먼저 합니다

```
if (조건들.length > 0) {
  결과 = [...결과].sort((a, b) => 비교하기(a, b, 조건들));
}

const total = 결과.length;
```

② 그다음 자릅니다

```
const 시작 = (page - 1) * limit;
결과 = 결과.slice(시작, 시작 + limit);
```

③ 마지막에 속성을 고릅니다

```
const data = 결과.map((설비) => 필드고르기(설비, req.query.fields));

res.json({
  data,
  meta: {
    page,
    limit,
    total,
    totalPages: Math.ceil(total / limit),
  },
});
```

```
GET /api/v1/equipments?limit=3&fields=id
```

```
200 {"data":[{"id":1},{"id":2},{"id":3}],"meta":{"page":1,"limit":3,"total":12,"totalPages":4}}
```

```
GET /api/v1/equipments?page=2&limit=3&fields=id
```

```
200 {"data":[{"id":4},{"id":5},{"id":6}],"meta":{"page":2,"limit":3,"total":12,"totalPages":4}}
```

```
GET /api/v1/equipments?page=4&limit=3&fields=id
```

```
200 {"data":[{"id":10},{"id":11},{"id":12}],"meta":{"page":4,"limit":3,"total":12,"totalPages":4}}
```

```
GET /api/v1/equipments?page=9&limit=3&fields=id
```

```
200 {"data":[],"meta":{"page":9,"limit":3,"total":12,"totalPages":4}}
```

★ 마지막 쪽을 넘어가면 빈 배열입니다. 404 가 아닙니다. "9쪽을 달라" 는 요청은 성공했고, 그 쪽이 비어 있을 뿐입니다.

```
GET /api/v1/equipments?limit=2&fields=id,name
```

```
200 {"data":[{"id":1,"name":"컨베이어 1호"},{"id":2,"name":"컨베이어 2호"}],"meta":{"page":1,"limit":2,"total":12,"totalPages":6}}
```

이상한 값이 올 때

섹션 5

```
GET /api/v1/equipments?page=abc&limit=2&fields=id
```

```
200 {"data":[{"id":1}, {"id":2}], "meta":{"page":1, "limit":2, "total":12, "totalPages":6}}
```

page=abc 는 1쪽으로 왔습니다.

```
GET /api/v1/equipments?page=2&limit=999&fields=id
```

```
200 {"data":[], "meta":{"page":2, "limit":100, "total":12, "totalPages":1}}
```

★ limit=999 를 100 으로 깎았습니다. 상한이 없으면 페이지징한 의미가 없습니다.

```
GET /api/v1/equipments?page=0&limit=-5&fields=id,line
```

```
200 {"data":[{"id":1,"line":"A"}, {"id":2,"line":"A"}, {"id":3,"line":"B"}, {"id":4,"line":"B"}, {"id":5,"line":"C"}, {"id":6,"line":"C"}, {"id":7,"line":"A"}, {"id":8,"line":"B"}, {"id":9,"line":"C"}, {"id":10,"line":"C"}], "meta":{"page":1, "limit":10, "total":12, "totalPages":2}}
```

page=0 과 limit=-5 를 기본값으로 되돌렸습니다. 이것 안 막으면 slice(-10, ...) 같은 게 되어 엉뚱한 결과가 나옵니다.

정렬 확인

섹션 6

```
GET /api/v1/equipments?sort=-id&limit=3&fields=id
```

```
200 {"data":[{"id":12}, {"id":11}, {"id":10}], "meta":{"page":1, "limit":3, "total":12, "totalPages":4}}
```

```
GET /api/v1/equipments?sort=line,-id&limit=4&fields=id,line
```

```
200 {"data":[{"id":11, "line":"A"}, {"id":7, "line":"A"}, {"id":2, "line":"A"}, {"id":1, "line":"A"}], "meta":{"page":1, "limit":4, "total":12, "totalPages":3}}
```

A라인끼리 모이고, 그 안에서 번호가 큰 것부터입니다. 조건을 심표로 이어 쓰면 “앞엇것이 같을 때 뒤엇것으로” 가 됩니다.

```
GET /api/v1/equipments?sort=name&limit=3&fields=id,name
```

```
200 {"data":[{"id":7,"name":"검사기 1호"}, {"id":8,"name":"검사기 2호"}, {"id":5,"name":"용접로봇 1호"}], "meta":{"page":1,"limit":3,"total":12,"totalPages":4}}
```

검사기 → 용접로봇 → 적재로봇 → 컨베이어 → 포장기 → 프레스 순입니다. ㄱ, ㄴ, ㄷ, ㄹ, ㅁ 순서니까 우리가 아는 사전 순서가 맞습니다.

★ 정렬을 자르기 '전' 에 하는 이유 자르고 정렬하면 그 쪽 안에서만 정렬됩니다. 1쪽이 3,1,2 / 2쪽이 6,4,5 처럼 되어 전혀 정렬이 안 된 결과가 나옵니다. 순서는 언제나 전체를 기준으로 정하고, 그다음에 자릅니다.

2: ★ localeCompare 를 꼭 써야 하나 (실제로 재 봤습니다)

섹션 6

흔히 “한글은 localeCompare 로 정렬해야 한다” 고 합니다. 정말 그럴까요? 네 가지 경우를 실제로 정렬해 봤습니다.

```
app.get("/api/v1/sort-compare", (req, res) => {
  const 묶음 = {
    한글만: ["프레스", "검사기", "용접로봇", "컨베이어"],
    영어대소문자: ["banana", "Apple", "cherry", "Date"],
    한글영어섞임: ["프레스", "Press", "검사기", "ABC"],
    숫자가든이름: ["설비2호", "설비10호", "설비1호"],
  };

  const 결과 = {};

  for (const [제목, 값들] of Object.entries(묶음)) {
    결과[제목] = {
      localeCompare: [...값들].sort((a, b) => a.localeCompare(b, "ko")),
      그냥비교: [...값들].sort((a, b) => (a < b ? -1 : a > b ? 1 : 0)),
      numeric컴: [...값들].sort((a, b) => a.localeCompare(b, "ko", { numeric: true })),
    };
  }

  res.json(결과);
});
```

GET /api/v1/sort-compare

```
200 {"한글만":{"localeCompare":["검사기","용접로봇","컨베이어","프레스"],"그냥비교":["검사기","용접로봇","컨베이어","프레스"],"numericکم":["검사기","용접로봇","컨베이어","프레스"]},"영어대소문자":{"localeCompare":["Apple","banana","cherry","Date"],"그냥비교":["Apple","Date","banana","cherry"],"numericکم":["Apple","banana","cherry","Date"]},"한글영어섞임":{"localeCompare":["검사기","프레스","ABC","Press"],"그냥비교":["ABC","Press","검사기","프레스"],"numericکم":["검사기","프레스","ABC","Press"]},"숫자가든이름":{"localeCompare":["설비10호","설비1호","설비2호"],"그냥비교":["설비10호","설비1호","설비2호"],"numericکم":["설비1호","설비2호","설비10호"]}}
```

재 본 결과를 정리하면 이렇습니다.

① 한글만 있으면 — 두 방법이 완전히 같습니다

검사기 / 용접로봇 / 컨베이어 / 프레스

유니코드에서 완성형 한글(가~힉)이 이미 ㄱ~ㅎ 순서로 배치돼 있기 때문입니다.

그래서 그냥 < 로 비교해도 사전 순서가 나옵니다.

"한글은 localeCompare 를 꼭 써야 한다" 는 말은 이 경우엔 사실이 아닙니다.

② 영어 대소문자가 섞이면 — 달라집니다

localeCompare : Apple / banana / cherry / Date ← 사전 순서
그냥 비교 : Apple / Date / banana / cherry ← 대문자가 전부 먼저

대문자 A~Z 의 번호가 소문자 a~z 보다 앞이라 이렇게 됩니다.

영어가 들어오면 localeCompare 가 필요합니다.

③ 한글과 영어가 섞이면 — 어느 쪽이 먼저인지가 달라집니다

localeCompare : 한글 먼저
그냥 비교 : 영어 먼저

둘 다 틀린 건 아닙니다. "우리 화면에서 무엇이 먼저여야 하나" 를 정하는 문제입니다.

④ 이름에 숫자가 들어가면 — 둘 다 틀립니다 ★

설비10호 / 설비1호 / 설비2호

"1" 다음에 "0" 이 오는지 "호" 가 오는지로 비교하기 때문입니다.

글자로만 보면 "10" 이 "2" 보다 앞입니다.

{ numeric: true } 를 주면 숫자 부분을 숫자로 봅니다.

설비1호 / 설비2호 / 설비10호 ← 이게 사람이 기대하는 순서입니다

★ 결론 그냥 localeCompare(다른값, "ko", { numeric: true }) 를 쓰세요. 한글만 있으면 어차피 결과가 같고, 나머지 경우는 이쪽이 맞습니다. 이 파일의 비교하기 함수가 그렇게 되어 있습니다.

★ 하나 더 — 눈에 안 보이는 함정 같은 '가' 인데 컴퓨터가 다르게 보는 경우가 있습니다.

```
'가'.normalize("NFD") → 글자 수가 2 입니다 (ㄱ + ㅏ 로 쪼개짐)
쪼개진 것 === '가' → false
쪼개진 것.localeCompare('가', 'ko') → 0 (같다고 봅니다)
```

맥에서 만든 파일 이름이 이렇게 쪼개진 형태로 옵니다. 윈도우에서 만든 것과 === 로 비교하면 다르다고 나옵니다. 파일 업로드를 다룰 때 실제로 만나게 됩니다. (09단원) 그때는 normalize("NFC") 로 맞춰 놓고 비교합니다.

★ 정렬 없는 페이지는 위험합니다

섹션 7

```
app.get("/api/v1/why-sort", (req, res) => {
  res.json({
    문제:
      "정렬을 안 하면 데이터베이스가 매번 다른 순서로 줄 수 있습니다. " +
      "1쪽에서 본 항목이 2쪽에서 또 나오거나, 어떤 항목은 아예 안 보입니다.",
    해결: "페이징을 쓸 때는 항상 정렬 기준을 함께 정하세요. 보통 id 나 만든
시각입니다.",
  });
});
```

GET /api/v1/why-sort

```
200 {"문제":"정렬을 안 하면 데이터베이스가 매번 다른 순서로 줄 수 있습니다. 1쪽에서
본 항목이 2쪽에서 또 나오거나, 어떤 항목은 아예 안 보입니다.", "해결":"페이징을
쓸 때는 항상 정렬 기준을 함께 정하세요. 보통 id 나 만든 시각입니다."}
```

지금은 배열이라 순서가 늘 같습니다. 그래서 이 문제가 안 보입니다. 데이터베이스로 바꾸는 순간(PART 4) 진짜로 일어납니다.

★ 기본 정렬을 정해 두는 것이 안전합니다.

```
const 조건들 = 정렬조건(req.query.sort || "id");
```

이러면 sort 를 안 줘도 항상 같은 순서가 나옵니다.

```
app.use((req, res) => {
  res.status(404).json({ error: "그런 주소가 없습니다" });
});
```

```
app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  const 코드 = err.status || 500;
  res.status(코드).json({ error: 코드 === 500 ? "서버에서 문제가 생겼습니다" :
err.message });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/api/v1/equipments?
limit=3`);
});
```

왜 { data, meta } 로 감싸나

배열을 그냥 주면

— {...}, {...}, {...}

전체가 몇 개인지, 지금 몇 쪽인지 알 방법이 없습니다. 화면에 "1 / 4 쪽" 을 그릴 수가 없습니다.

그래서 감쌉니다.

```
{ "data": [...], "meta": { "page": 1, "limit": 3, "total": 12, "totalPages": 4 } }
```

★ 한 번 감싸면 되돌리기 어렵습니다 목록 주소 중 어떤 건 배열, 어떤 건 감싼 것이면 프론트가 매번 헛갈립니다. 처음부터 전부 감싸거나, 전부 안 감싸거나 하나로 통일하세요. 개념04 에서 이 이야기를 더 합니다.

★ 감싸는 대신 헤더로 주는 방식도 있습니다 X-Total-Count: 12 처럼요. 깔끔하지만 프론트가 헤더를 읽어야 합니다. 그리고 CORS 를 쓰면 헤더를 따로 열어 줘야 보입니다. (08단원) 초보에게는 본문에 담은 편이 덜 헛갈립니다.

페이징 방식 두 가지

① 쪽 번호 방식 (지금 만든 것)

```
?page=2&limit=10
화면에 1 2 3 4 쪽 버튼을 그릴 수 있습니다. 게시판에 어울립니다.
단점: 보는 도중에 새 데이터가 들어오면 항목이 밀려 겹쳐 보입니다.
```

② 커서 방식

```
?after=1234&limit=10 "1234번 다음 것부터 10개"
무한 스크롤에 어울립니다. 밀림 문제가 없습니다.
단점: "5쪽으로 가기" 를 못 합니다.
```

지금은 ①만 알면 충분합니다. SNS 처럼 계속 새 글이 올라오는 서비스를 만들 때 ②를 찾아보세요.

직접 해 볼 것

▫ 직접 해보기 1

meta 에 hasNext 를 추가해 보세요. 다음 쪽이 있으면 true, 없으면 false 입니다. (힌트: page < totalPages)

▫ 직접 해보기 2

기본 정렬을 id 로 정해 보세요. 정렬조건(req.query.sort || "id") sort 없이 요청해도 meta 가 그대로인지 확인하세요.

▫ 직접 해보기 3

섹션 4에서 정렬과 자르기 순서를 바꿔 보세요. ?sort=-id&page=2&limit=3 이 어떻게 되나요? (확인 후 반드시 되돌리세요)

▫ 직접 해보기 4

최대쪽크기를 5 로 줄이고 ?limit=50 을 보내 보세요. meta.limit 이 얼마로 나오나요?

▫ 직접 해보기 5

?fields=id,없는속성 으로 요청해 보세요. 어떻게 되나요? 에러를 내는 게 나올까요, 무시하는 게 나올까요?

▫ 직접 해보기 6

비교하기 에서 localeCompare 줄을 지우고 ?sort=name 을 다시 보내 보세요. 순서가 달라지나요? (달라지지 않습니다. 왜 그런지 섹션 6-2 를 다시 읽으세요)

그다음 data/설비데이터.js 의 이름 하나를 "Press 1호" 로 바꾸고 다시 정렬해 보세요. 이번에는 달라지나요? (확인 후 전부 되돌리세요)

▫ 직접 해보기 7

data/설비데이터.js 의 이름을 설비1호 · 설비2호 · 설비10호 로 바꾸고 ?sort=name 을 보내 보세요. { numeric: true } 를 지우면 순서가 어떻게 되나요? (확인 후 되돌리세요)

자주 하는 실수

이런 실수를 합니다

limit 에 상한을 안 둬

?limit=999999 하나로 서버가 멈출 수 있습니다. 반드시 상한을 두세요.

이런 실수를 합니다

page 를 그대로 믿음

?page=0 이나 ?page=-1 이 오면 slice 가 이상하게 동작합니다. Number 로 바꾸고 범위까지 확인하세요.

이런 실수를 합니다

자르고 나서 정렬

그 쪽 안에서만 정렬됩니다. 전체가 뒤죽박죽이 됩니다.

이런 실수를 합니다

정렬 없이 페이징

지금은 잘 돌아갑니다. 데이터베이스로 바꾸면 항목이 겹치거나 사라집니다.

이런 실수를 합니다

total 을 자른 뒤에 셈

결과.slice(...) 뒤에 length 를 세면 항상 limit 이 나옵니다. total 은 반드시 자르기 전에 세세요.

이런 실수를 합니다

sort 를 원본 배열에 그대로 씀

설비목록().sort(...) 는 원본을 바꿉니다.

...결과 · .sort(...) 로 복사본을 정렬하세요.

정리

- 1 목록은 통째로 주지 않는다. page 와 limit 으로 잘라서 준다.
- 2 limit 에는 반드시 상한을 둔다. 안 두면 limit=999999 한 번에 서버가 넘어간다.
- 3 page 를 그대로 믿지 않는다. 0 도 음수도 글자도 올 수 있다.
- 4 순서는 정렬 다음 자르기다. 자르고 나서 정렬하면 그 쪽 안에서만 정렬된다.
- 5 total 은 자르기 전에 센다. 자른 뒤에 세면 언제나 limit 이 나온다.
- 6 sort 는 원본 배열을 바꾼다. slice() 로 복사한 뒤 정렬한다.
- 7 정렬 없는 페이징은 위험하다. 순서가 보장되지 않아 같은 항목이 두 쪽에 나오거나 아예 빠질 수 있다.
- 8 완성형 한글은 유니코드 순서가 이미 사전 순서라 < 로도 정렬된다. localeCompare 가 정말 필요한 것은 영어 대소문자와 한영 혼합이고, 이름 속 숫자(설비10호)는 { numeric: true } 를 줘야 맞는다.

검색과 필터

RUN node 개념03_검색과_필터.js

그러면 Ctrl + C

목록을 나눠 주는 법(개념02)을 배웠으니, 이제 골라 주는 법입니다.

필터	정해진 값과 정확히 같은 것만	?line=A
검색	글자가 들어 있는 것을 찾기	?q=로봇

둘은 비슷해 보이지만 다르게 다뤄야 합니다.

```
const express = require("express");
const { 설비목록 } = require("../data/설비데이터");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

필터 — 정확히 같은 것

섹션 1

line=A 는 "line 이 정확히 A 인 것" 입니다. 이런 건 === 로 비교합니다.

```
app.get("/api/v1/filter-basic", (req, res) => {
  const { line, status } = req.query;

  let 결과 = 설비목록();

  if (line) 결과 = 결과.filter((설비) => 설비.line === line);
  if (status) 결과 = 결과.filter((설비) => 설비.status === status);

  res.json({ 개수: 결과.length, ids: 결과.map((설비) => 설비.id) });
});
```

GET /api/v1/filter-basic

200 {"개수":12,"ids":[1,2,3,4,5,6,7,8,9,10,11,12]}

```
GET /api/v1/filter-basic?line=A
```

```
200 {"개수":4,"ids":[1,2,7,11]}
```

```
GET /api/v1/filter-basic?line=A&status=가동
```

```
200 {"개수":3,"ids":[1,7,11]}
```

```
GET /api/v1/filter-basic?line=Z
```

```
200 {"개수":0,"ids":[]}
```

★ 조건을 겹칠 때는 결과를 계속 좁혀 나갑니다. `let` 으로 시작해서 조건마다 `filter` 를 이어 붙이면 됩니다. 조건이 없으면 `if` 를 통과하지 않으니 자동으로 무시됩니다.

★ 결과가 0건이어도 404 가 아닙니다. "조건에 맞는 목록을 달라" 는 요청은 성공했습니다. 200 과 빈 배열입니다. 05단원부터 계속 나오는 이야기인데, 정말 많이 틀립니다.

여러 값 중 하나

섹션 2

06

"A라인이거나 B라인" 은 어떻게 받을까요? 심표로 이어서 받는 것이 흔한 방법입니다.

?line=A,B

```
app.get("/api/v1/filter-multi", (req, res) => {
  const { line } = req.query;

  let 결과 = 설비목록();

  if (line) {
    const 라인들 = line.split(",").map((조각) => 조각.trim());
    결과 = 결과.filter((설비) => 라인들.includes(설비.line));
  }

  res.json({ 개수: 결과.length, ids: 결과.map((설비) => 설비.id) });
});
```

```
GET /api/v1/filter-multi?line=A
```

```
200 {"개수":4,"ids":[1,2,7,11]}
```

```
GET /api/v1/filter-multi?line=A,B
```

```
200 {"개수":8,"ids":[1,2,3,4,7,8,11,12]}
```

GET /api/v1/filter-multi?line=A,%20B

200 {"개수":8,"ids":[1,2,3,4,7,8,11,12]}

★ 세 번째를 보세요. A, B 처럼 쉼표 뒤에 공백이 있어도 됩니다. `trim()` 을 안 하면 " B" 를 찾게 되어 B라인이 하나도 안 나옵니다. 사람이 손으로 주소를 칠 수도 있으니 공백은 미리 지워 두세요.

★ ?line=A&line=B 방식도 있습니다 Express 는 이걸 배열 ["A","B"] 로 줍니다. (04단원 개념03) 다만 하나만 오면 배열이 아니라 글자라서, 받는 쪽이 매번 확인해야 합니다. 쉼표 방식이 다루기 쉬워서 이 자료에서는 이쪽을 씁니다.

검색 — 글자가 들어 있는 것

섹션 3

이름에 '로봇' 이 든 것을 찾는다면 === 로는 안 됩니다. includes 로 "들어 있나" 를 봅니다.

```
app.get("/api/v1/search-basic", (req, res) => {
  const { q } = req.query;

  let 결과 = 설비목록();

  if (q) {
    결과 = 결과.filter((설비) => 설비.name.includes(q));
  }

  res.json({ 검색어: q ?? null, 개수: 결과.length, ids: 결과.map((설비) => 설비.id)
});
```

GET /api/v1/search-basic?q=로봇

200 {"검색어":"로봇","개수":4,"ids":[5,6,11,12]}

GET /api/v1/search-basic?q=1호

200 {"검색어":"1호","개수":6,"ids":[1,3,5,7,9,11]}

GET /api/v1/search-basic?q=없는말

200 {"검색어":"없는말","개수":0,"ids":[]}

GET /api/v1/search-basic

200 {"검색어":null,"개수":12,"ids":[1,2,3,4,5,6,7,8,9,10,11,12]}

★ 검색어가 없으면 전부 돌려줍니다. “검색어를 넣으라” 며 400 을 내는 것도 방법이지만, 같은 주소로 목록도 보고 검색도 하려면 이쪽이 편합니다.

★ 검색에서 꼭 챙겨야 할 세 가지

섹션 4

```
app.get("/api/v1/search-good", (req, res) => {
```

① 앞뒤 공백 지우기 사용자가 실수로 공백을 넣거나, 복사·붙여넣기하면 따라옵니다.

```
const q = (req.query.q ?? "").trim();
```

② 빈 검색어는 검색 안 한 것으로 ?q= 로 보내면 빈 글자입니다. "" 는 모든 글자에 들어 있으므로 includes("") 는 항상 true 입니다. 걸러 내나 마나가 됩니다.

```
if (!q) {
  const 전부 = 설비목록();
  return res.json({ 검색어: null, 개수: 전부.length, ids: 전부.map((설비) =>
    설비.id) });
}
```

③ 대소문자 무시 Robot 으로 찾든 robot 으로 찾든 나와야 합니다.

```
const 소문자q = q.toLowerCase();

const 결과 = 설비목록().filter((설비) =>
  설비.name.toLowerCase().includes(소문자q)
);

res.json({ 검색어: q, 개수: 결과.length, ids: 결과.map((설비) => 설비.id) });
});
```

```
GET /api/v1/search-good?q=%20%20로봇%20%20
```

```
200 {"검색어":"로봇","개수":4,"ids":[5,6,11,12]}
```

```
GET /api/v1/search-good?q=
```

```
200 {"검색어":null,"개수":12,"ids":[1,2,3,4,5,6,7,8,9,10,11,12]}
```

```
GET /api/v1/search-good?q=%20
```

```
200 {"검색어":null,"개수":12,"ids":[1,2,3,4,5,6,7,8,9,10,11,12]}
```

★ 두 번째와 세 번째를 비교해 보세요. ?q= 도, ?q=(공백) 도 전부 ‘검색 안 함’ 으로 처리했습니다. trim 을 먼저 하고 비어 있는지 확인했기 때문입니다. 순서가 중요합니다.

★ 04단원 개념03 에서 본 것 — req.query 는 못 고칩니다. req.query.q = req.query.q.trim() 은 조용히 무시됩니다. 그래서 여기서는 새 변수 q 에 담았습니다.

대소문자 무시가 실제로 되는지 확인합니다.

```
app.get("/api/v1/search-case", (req, res) => {
  const q = (req.query.q ?? "").trim().toLowerCase();
  const 목록 = ["Robot Arm", "robot hand", "ROBOT LEG", "컨베이어"];

  res.json({
    대소문자무시: 목록.filter((이름) => 이름.toLowerCase().includes(q)),
    그냥비교: 목록.filter((이름) => 이름.includes(req.query.q ?? "")),
  });
});
```

GET /api/v1/search-case?q=robot

```
200 {"대소문자무시":["Robot Arm","robot hand","ROBOT LEG"],"그냥비교":["robot hand"]}
```

그냥 비교하면 소문자로 쓴 것 하나만 나옵니다. 사용자는 세 개가 다 나오길 기대합니다.

여러 속성에서 한꺼번에 찾기

섹션 5

보통 검색창은 하나입니다. 이름이든 라인이든 어디서든 찾아 줘야 합니다.

```
app.get("/api/v1/search-fields", (req, res) => {
  const q = (req.query.q ?? "").trim().toLowerCase();

  if (!q) {
    return res.json({ 개수: 0, ids: [], 안내: "검색어를 넣어 주세요" });
  }
});
```

찾아볼 속성들을 정해 둡니다.

```
const 검색대상 = ["name", "line", "status"];

const 결과 = 설비목록().filter((설비) =>
  검색대상.some((키) => String(설비[키]).toLowerCase().includes(q))
);

res.json({ 개수: 결과.length, ids: 결과.map((설비) => 설비.id) });
```

```
GET /api/v1/search-fields?q=점검중
```

```
200 {"개수":2,"ids":[4,10]}
```

```
GET /api/v1/search-fields?q=프레스
```

```
200 {"개수":2,"ids":[3,4]}
```

```
GET /api/v1/search-fields?q=c
```

```
200 {"개수":4,"ids":[5,6,9,10]}
```

★ some 은 “하나라도 맞으면 true” 입니다. (JS자료 08단원) every 를 쓰면 “전부 맞아야” 가 되어 아무것도 안 나옵니다.

★ String(설비[키]) 로 감싼 이유 숫자 속성을 검색 대상에 넣으면 toLowerCase 가 없어서 터집니다. 글자로 바꿔 두면 무엇이 와도 안전합니다.

★ 세 번째를 보세요. c 한 글자로 C라인 네 개가 나왔습니다. 짧은 검색어는 엉뚱한 것까지 걸립니다. 실무에서는 “두 글자 이상” 같은 제한을 두기도 합니다.

검색과 필터와 페이징을 함께

섹션 6

```
app.get("/api/v1/equipments", (req, res) => {
  const q = (req.query.q ?? "").trim().toLowerCase();
  const { line, status } = req.query;

  let 결과 = 설비목록();
```

① 검색 (넓게 찾기)

```
if (q) {
  결과 = 결과.filter((설비) =>
    ["name", "line", "status"].some((키) => String(설비
[키]).toLowerCase().includes(q)
  )
);
}
```

② 필터 (정확히 좁히기)

```

if (line) {
  const 라인들 = line.split(",").map((조각) => 조각.trim());
  결과 = 결과.filter((설비) => 라인들.includes(설비.line));
}
if (status) 결과 = 결과.filter((설비) => 설비.status === status);

```

③ 개수는 자르기 전에 셉니다

```
const total = 결과.length;
```

④ 페이징

```

let page = Number(req.query.page);
let limit = Number(req.query.limit);
if (!Number.isInteger(page) || page < 1) page = 1;
if (!Number.isInteger(limit) || limit < 1) limit = 10;
if (limit > 100) limit = 100;

const 시작 = (page - 1) * limit;
const data = 결과.slice(시작, 시작 + limit).map((설비) => ({ id: 설비.id }));

res.json({
  data,
  meta: { page, limit, total, totalPages: Math.ceil(total / limit) },
});
});

```

GET /api/v1/equipments?q=로봇&limit=2

```
200 {"data":[{"id":5}, {"id":6}], "meta": {"page":1, "limit":2, "total":4, "totalPages":2}}
```

GET /api/v1/equipments?q=로봇&limit=2&page=2

```
200 {"data":[{"id":11}, {"id":12}], "meta": {"page":2, "limit":2, "total":4, "totalPages":2}}
```

GET /api/v1/equipments?q=로봇&line=A

```
200 {"data":[{"id":11}], "meta": {"page":1, "limit":10, "total":1, "totalPages":1}}
```

GET /api/v1/equipments?status=점검중

```
200 {"data":[{"id":4}, {"id":10}], "meta": {"page":1, "limit":10, "total":2, "totalPages":1}}
```


★★ 순서가 중요합니다. 걸러 내기 → 개수 세기 → 자르기.

total 을 자른 뒤에 세면 항상 limit 이하가 나옵니다. 화면에 “전체 4건” 이 아니라 “전체 2건” 이 표시됩니다. 실무에서 정말 많이 나오는 버그입니다.

★ 검색을 먼저 하고 필터를 나중에 한 이유 순서를 바꿔도 결과는 같습니다. filter 는 겹쳐 쓰면 교집합이니까요. 다만 데이터가 많아지면, 많이 걸러 내는 조건을 먼저 두는 게 빠릅니다.

```
app.use((req, res) => {
  res.status(404).json({ error: "그런 주소가 없습니다" });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  const 코드 = err.status || 500;
  res.status(코드).json({ error: 코드 === 500 ? "서버에서 문제가 생겼습니다" :
err.message });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/api/v1/equipments?q=로봇`);
});
```

검색과 필터, 무엇을 쿼리 이름으로 쓸까

?q=	검색어. 관례적으로 q 또는 search 를 씁니다
?line=A	속성 이름을 그대로. 무엇을 거르는지 분명합니다

?status=가동 ?line=A,B 여러 값은 쉼표로 ?page= ?limit= 페이지징

?sort=-id	정렬
?fields=id,name	원하는 속성만

이 이름들은 거의 표준처럼 쓰입니다. 남이 쓰던 이름을 그대로 쓰면 설명할 것이 줄어듭니다.

★ 하면 안 되는 것 ?filter={"line":"A"} JSON 을 쿼리에 넣는 방식입니다. 되긴 하는데 주소가 지저분해지고, 인코딩 때문에 사고가 잦습니다. 조건이 아주 복잡한 경우가 아니면 쓰지 마세요.

지금 방식의 한계

여기서 만든 검색은 '전부 꺼내서 훑는' 방식입니다. 설비 12개니까 순식간입니다. 10만 개라면 요청마다 10만 번을 훑습니다.

그래서 진짜 서버는 데이터베이스에게 시킵니다.

SELECT * FROM equipments WHERE name LIKE '%로봇%' LIMIT 10 OFFSET 0

데이터베이스는 이런 걸 빠르게 하도록 만들어진 프로그램입니다. (PART 4)

★ 그래도 지금 배운 것이 그대로 쓰입니다 “검색어를 trim 한다”, “빈 검색어는 무시한다”, “대소문자를 맞추다”, “total 은 자르기 전에 센다” — 전부 데이터베이스를 써도 똑같습니다. 도구가 바뀌어도 규칙은 안 바뀝니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 4의 `trim()` 을 지우고 `?q=%20%20로봇` 을 보내 보세요. 몇 건이 나오나요? (확인 후 되돌리세요)

▫ 직접 해보기 2

섹션 4의 빈 검색어 확인을 지우고 `?q=` 를 보내 보세요. `includes("")` 가 무엇을 돌려주는지 생각해 보세요.

▫ 직접 해보기 3

섹션 6에서 `total` 을 `slice` 뒤에 세도록 바꿔 보세요. `?q=로봇&limit=2` 의 `meta` 가 어떻게 되나요? 화면에 “전체 몇 건” 을 그린다면 무엇이 잘못될까요? (확인 후 반드시 되돌리세요)

▫ 직접 해보기 4

검색어가 한 글자면 400 을 내도록 바꿔 보세요. 두 글자 이상만 검색하게 하는 것입니다.

▫ 직접 해보기 5

`?installedAfter=2022-01-01` 을 만들어 그 날짜 이후에 설치된 설비만 나오게 해 보세요. (힌트: 날짜가 “2021-03-15” 모양이라 글자 비교로도 됩니다. 왜 될까요? 다른 모양이었다면 어땠을까요?)

▫ 직접 해보기 6

섹션 5의 `some` 을 `every` 로 바꿔 보세요. 몇 건이 나오나요? 왜 그럴까요? (확인 후 되돌리세요)

자주 하는 실수

이런 실수를 합니다

빈 검색어를 안 거름

`""`.includes 는 언제나 `true` 라서 전부 통과합니다. “검색했는데 왜 다 나오지?” 의 원인입니다.

이런 실수를 합니다

trim 을 안 함

복사·붙여넣기하면 공백이 따라옵니다. 아무것도 안 나옵니다. 사용자는 “분명히 맞게 썼는데” 라고 생각합니다.

이런 실수를 합니다

대소문자를 안 맞춤

양쪽 다 toLowerCase 해야 합니다. 한쪽만 하면 소용없습니다.

이런 실수를 합니다

total 을 자른 뒤에 섬

화면의 전체 건수와 쪽 수가 전부 틀립니다.

이런 실수를 합니다

결과가 0건일 때 404 를 줌

목록 조회는 0건도 성공입니다. 200 과 빈 배열입니다.

이런 실수를 합니다

쉼표로 나눈 뒤 trim 을 안 함

?line=A, B 에서 " B " 를 찾게 되어 B라인이 하나도 안 나옵니다.

이런 실수를 합니다

숫자 속성에 toLowerCase 를 부름

→ TypeError. String() 으로 감싸 두세요.

정리

- 1 필터는 '정확히 같은 것', 검색은 '글자가 들어 있는 것' 이다.
- 2 검색에서 꼭 행길 것이 셋이다 — 빈 검색어 거르기 · trim · 대소문자 맞추기.
- 3 빈 검색어를 안 거르면 전부가 걸린다. 검색이 아니라 목록이 되어 버린다.
- 4 대소문자는 양쪽 다 toLowerCase 로 맞춘다. 숫자 속성에 toLowerCase 를 부르면 터지니 글자인지 먼저 본다.
- 5 여러 값 중 하나는 쉼표로 받아 나눈다. 나눈 뒤 trim 을 빼먹지 않는다.
- 6 total 은 여기서도 자르기 전에 센다.
- 7 결과가 0건인 것은 404 가 아니다. 빈 배열에 200 이다. 잘 찾아봤는데 없는 것이지, 주소가 틀린 것이 아니다.

응답 형식 통일하기

RUN node 개념04_응답_형식_통일.js

그러면 Ctrl + C

지금까지 응답을 그때그때 지어서 보냈습니다. 모아 놓고 보면 이렇습니다.

`{...}, {...}` · 목록

<code>{"id":1,"title":"..."}</code>	하나
<code>{"data":[...],"meta":{"..."}}</code>	페이지장한 목록

`{"error":"그런 설비가 없습니다"}` 에러 `{"error":"입력값이 올바르지 않습니다","항목":[...]}` 검증 실패
프론트엔드가 이걸 받으면 어떻게 될까요?

```
const 답 = await res.json();
// 목록이면 답 자체가 배열
// 페이지장이면 답.data
// 에러면 답.error 인데, 어떤 건 글자고 어떤 건 객체
```

주소마다 다르게 써야 합니다. 주소가 서른 개면 서른 번 다릅니다. 그래서 형식을 하나로 정합니다.

```
const express = require("express");
const { 설비목록 } = require("../data/설비데이터");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

형식 정하기

섹션 1

이 자료에서 쓸 형식입니다. 정답이 하나만 있는 건 아닙니다. 중요한 것은 '정하고 끝까지 지키는 것' 입니다.

성공	<code>{ "data": ... , "meta": {...} }</code>	meta 는 있을 때만
실패	<code>{ "error": { "code": "...", "message": "...", "details": ... } }</code>	

규칙 하나 성공 응답에는 error 가 절대 없고, 실패 응답에는 data 가 절대 없습니다. 그래서 프론트는 이렇게만 쓰면 됩니다.

```
if (답.error) { 에러표시(답.error.message); return; }
화면그리기(답.data);
```

주소가 서른 개여도 이 두 줄로 끝납니다.

성공 응답 도우미

섹션 2

```
function 보내기(res, data, meta) {
  const 몸통 = { data };

  if (meta) {
    몸통.meta = meta;
  }

  res.json(몸통);
}

function 만들었음(res, data, 주소) {
  res.status(201).location(주소).json({ data });
}

app.get("/api/v1/equipments", (req, res) => {
  const 전부 = 설비목록();
  const data = 전부.slice(0, 3).map((설비) => ({ id: 설비.id, name: 설비.name }));

  보내기(res, data, { page: 1, limit: 3, total: 전부.length, totalPages: 4 });
});
```

GET /api/v1/equipments

```
200 {"data":[{"id":1,"name":"컨베이어 1호"}, {"id":2,"name":"컨베이어 2호"}, {"id":3,"name":"프레스 1호"}], "meta":{"page":1,"limit":3,"total":12,"totalPages":4}}
```

```
app.get("/api/v1/equipments/:id", (req, res, next) => {
  const 설비 = 설비목록().find((설비) => 설비.id === Number(req.params.id));

  if (!설비) {
    return next(에러.없음("설비"));
  }

  보내기(res, 설비); // meta 없이
});
```

GET /api/v1/equipments/3

```
200 {"data":{"id":3,"name":"프레스 1호","line":"B","status":"가동","설치일":"2019-07-02"}}
```

★ 하나를 줄 때도 data 로 감쌉니다. 감싸는 게 번거로워 보이지만, 프론트 코드가 한 가지로 통일됩니다. 나중에 meta 를 붙일 일이 생겨도 형식이 안 깨집니다.

에러 클래스 만들기

섹션 3

지금까지는 이렇게 썼습니다.

```
function HttpError(코드, 메시지) {
```

```
  const 에러 = new Error(메시지);
  에러.status = 코드;
  return 에러;
```

```
}
```

이제 코드(code)와 자세한 내용(details)까지 담아야 하니 class 로 만듭니다.

```
class AppError extends Error {
  constructor(status, code, message, details) {
    super(message); // Error 가 message 를 담당합니다 this.name = "AppError";
    this.status = status; // HTTP 상태코드 - 404 this.code = code; // 우리가 정한
    코드 - "NOT_FOUND" this.details = details; // 필요하면 자세한 내용
  }
}
```

★ extends Error 를 왜 하나 Error 를 물려받으면 throw 로 던질 수 있고 스택도 자동으로 붙습니다. 그냥 객체를 던져도 되긴 하지만, 에러 처리 도구들이 Error 를 기대합니다.

★ super(message) 를 빼먹으면 → ReferenceError: Must call super constructor ... 물려받은 클래스는 자기 것을 만들기 전에 부모부터 만들어야 합니다.

자주 쓰는 에러를 미리 만들어 둡니다.

```
const 에러 = {
  없음: (무엇) => new AppError(404, "NOT_FOUND", `${무엇}을(를) 찾을 수 없습니다`),

  검증실패: (항목들) => new AppError(400, "VALIDATION_FAILED", "입력값이 올바르지 않습니다", 항목들),

  로그인필요: () => new AppError(401, "UNAUTHENTICATED", "로그인이 필요합니다"),

  권한없음: (무엇) => new AppError(403, "FORBIDDEN", `${무엇} 권한이 없습니다`),

  중복: (무엇) => new AppError(409, "DUPLICATED", `이미 있는 ${무엇}입니다`),
};
```

GET /api/v1/equipments/99

404 {"error":{"code":"NOT_FOUND","message":"설비을(를) 찾을 수 없습니다"}}

★ “설비을(를)” 이 어색합니다. 조사 문제입니다. 받침이 있으면 ‘을’, 없으면 ‘를’ 인데 프로그램은 그걸 모릅니다. 그래서 ‘을(를)’ 로 씁니다. 한국어 서비스에서 흔히 보는 표기입니다. 제대로 하려면 받침을 계산해야 합니다. 연습문제에서 만들어 봅니다.

06

에러 코드를 왜 따로 두나

섹션 4

```
app.get("/api/v1/why-code", (req, res) => {
  보내기(res, {
    이유1: "메시지는 바뀌어도 코드는 안 바뀝니다. 프론트가 코드로 판단하면 안  
깨집니다.",
    이유2: "영어·일본어 화면을 만들 때, 프론트가 코드를 보고 자기 언어 메시지를  
고릅니다.",
    이유3: "'없음' 인지 '권한없음' 인지를 상태코드만으로는 구별이 안 될 때가  
있습니다.",
    나쁜예: 'if (답.error.message === "설비을(를) 찾을 수 없습니다")',
    좋은예: 'if (답.error.code === "NOT_FOUND")',
  });
});
```

GET /api/v1/why-code

200 {"data":{"이유1":"메시지는 바뀌어도 코드는 안 바뀝니다. 프론트가 코드로 판단하면 안 깨집니다.", "이유2":"영어·일본어 화면을 만들 때, 프론트가 코드를 보고 자기 언어 메시지를 고릅니다.", "이유3":"'없음' 인지 '권한없음' 인지를 상태코드만으로는 구별이 안 될 때가 있습니다.", "나쁜예":"if (답.error.message === \"설비을(를) 찾을 수 없습니다\")", "좋은예":"if (답.error.code === \"NOT_FOUND\")"}}

★ 메시지로 판단하면 안 되는 이유 “설비를 찾을 수 없습니다” 를 “해당 설비가 없습니다” 로 다듬는 순간 프론트의 if 문이 조용히 `false` 가 됩니다. 에러가 안 납니다. 맞춤법 하나 고쳤다가 기능이 멈추는 일이 실제로 일어납니다.

★ 코드는 영어 대문자에 밀줄로 씁니다 `NOT_FOUND`, `VALIDATION_FAILED`, `DUPLICATED` 관례입니다. 한글로 쓰면 안 되는 건 아니지만 남들이 다 이렇게 씁니다.

검증 실패는 자세히

섹션 5

```
app.post("/api/v1/equipments", (req, res, next) => {
  const { name, line } = req.body || {};
  const 항목 = [];

  if (!name) 항목.push({ 키: "name", 이유: "필수입니다" });
  else if (name.length < 2) 항목.push({ 키: "name", 이유: "2글자 이상이어야 합니다"
});

  if (!line) 항목.push({ 키: "line", 이유: "필수입니다" });
  else if (!["A", "B", "C"].includes(line))
    항목.push({ 키: "line", 이유: "A, B, C 중 하나여야 합니다" });

  if (항목.length > 0) {
    return next(에러.검증실패(항목));
  }

  만들었음(res, { id: 13, name, line }, "/api/v1/equipments/13");
});
```

POST /api/v1/equipments

```
400 {"error":{"code":"VALIDATION_FAILED","message":"입력값이 올바르지 않습니다","details":[{"키":"name","이유":"필수입니다"}, {"키":"line","이유":"필수입니다"}]}}
```

POST /api/v1/equipments

```
{ "name": "용", "line": "Z" }
```

```
400 {"error":{"code":"VALIDATION_FAILED","message":"입력값이 올바르지 않습니다","details":[{"키":"name","이유":"2글자 이상이어야 합니다"}, {"키":"line","이유":"A, B, C 중 하나여야 합니다"}]}}
```

POST /api/v1/equipments

```
{ "name": "용접로봇", "line": "A" }
```

```
201 {"data":{"id":13,"name":"용접로봇","line":"A"}}
```


★ details 가 있으니 프론트가 입력칸마다 빨간 글씨를 붙일 수 있습니다. message 만 주면 “입력값이 올바르지 않습니다” 하나만 띄우게 됩니다. 사용자는 어디가 틀렸는지 모릅니다.

★ 05단원에서는 { error, 항목 } 이라는 다른 모양이었습니다. 여기서는 error 안으로 넣어 형식을 통일했습니다. “실패 응답의 정보는 전부 error 안에” 라는 규칙이 생긴 것입니다.

다른 에러들

섹션 6

```
app.get("/api/v1/secret", (req, res, next) => {
  next(에러.로그인필요());
});
```

GET /api/v1/secret

401 {"error":{"code":"UNAUTHENTICATED","message":"로그인이 필요합니다"}}

```
app.delete("/api/v1/equipments/:id", (req, res, next) => {
  next(에러.권한없음("삭제"));
});
```

DELETE /api/v1/equipments/3

403 {"error":{"code":"FORBIDDEN","message":"삭제 권한이 없습니다"}}

```
app.post("/api/v1/users", (req, res, next) => {
  next(에러.중복("아이디"));
});
```

POST /api/v1/users

409 {"error":{"code":"DUPLICATED","message":"이미 있는 아이디입니다"}}

★ 409 Conflict “지금 상태와 안 맞는다” 는 뜻입니다. 아이디 중복이 대표적입니다. 400 을 써도 되지만, 409 면 프론트가 “중복” 이라고 바로 알 수 있습니다.

```
app.get("/api/v1/boom", (req, res) => {
  const 없는것 = undefined;
  res.json(없는것.속성); // 일부러 터뜨립니다
});
```

GET /api/v1/boom

500 {"error":{"code":"INTERNAL_ERROR","message":"서버에서 문제가 생겼습니다"}}

★ 예상 못 한 에러도 같은 형식으로 나갑니다. TypeError 메시지는 밖으로 안 나갑니다. 터미널에만 남습니다.

404 와 에러 처리기

섹션 7

```
app.use((req, res, next) => {
  next(new AppError(404, "ROUTE_NOT_FOUND", "그런 주소가 없습니다"));
});
```

```
GET /api/v1/없는주소
404 {"error":{"code":"ROUTE_NOT_FOUND","message":"그런 주소가 없습니다"}}
```

★ 없는 자원(NOT_FOUND)과 없는 주소(ROUTE_NOT_FOUND)를 코드로 구별했습니다. 둘 다 404지만 프론트가 해야 할 일이 다릅니다. 앞은 “그 설비가 지워졌나 보다”, 뒤는 “내가 주소를 잘못 썼다”입니다.

```
app.use((err, req, res, next) => {
```

① 터미널에는 전부 남깁니다

```
console.error(`[에러] ${req.method} ${req.path} - ${err.name}: ${err.message}`);
```

② 본문 JSON 이 깨진 경우 (express.json 이 만든 에러)

```
if (err.type === "entity.parse.failed") {
  return res.status(400).json({
    error: { code: "INVALID_JSON", message: "JSON 형식이 아닙니다" },
  });
}
```

③ 우리가 만든 에러

```
if (err instanceof AppError) {
  const 몸통 = { code: err.code, message: err.message };
```

details 가 있을 때만 넣습니다. 없는데 넣으면 null 이 지저분하게 남습니다.

```
if (err.details !== undefined) {
  몸통.details = err.details;
}

return res.status(err.status).json({ error: 몸통 });
}
```

④ 그 밖 — 예상 못 한 에러

```
res.status(500).json({
  error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" },
});
});
```

POST /api/v1/equipments

{깨진:JSON}

400 {"error":{"code":"INVALID_JSON","message":"JSON 형식이 아닙니다"}}

★ `err instanceof AppError` 로 구별하는 것이 핵심입니다. “내가 일부러 만든 에러”와 “예상 못 한 에러”를 나눠야 합니다. 앞은 메시지를 그대로 보여 줘도 되고, 뒤는 감춰야 합니다.

★ 스택은 절대 응답에 넣지 마세요 `err.stack`에는 파일 경로가 전부 들어 있습니다. 서버 폴더 구조가 그대로 노출됩니다. 터미널에만 남기세요.

```
app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/api/v1/equipments`);
});
```

프론트엔드에서 쓰는 모습

async function 설비가져오기(id) {

```
const res = await fetch(`/api/v1/equipments/${id}`);
const 답 = await res.json();
```

```
if (답.error) {
  if (답.error.code === "NOT_FOUND") {
    화면에("그 설비는 삭제되었습니다");
  } else if (답.error.code === "UNAUTHENTICATED") {
    로그인화면으로();
  } else {
    화면에(답.error.message);
  }
  return;
}
```

그리기(답.data);

}

주소가 서른 개여도 이 모양 하나로 전부 처리됩니다. 형식을 통일한 대가로 얻는 것이 이것입니다.

★ `res.ok` 로 판단해도 됩니다 `fetch` 의 `res.ok` 는 상태코드가 200~299 면 `true` 입니다. 다만 `error` 필드를 보는 편이 더 분명합니다. 둘 다 봐도 됩니다.

응답에 넣을 것과 넣지 말 것

넣을 것

<code>code</code>	프론트가 판단할 기준
<code>message</code>	사람이 읽을 한 줄
<code>details</code>	입력값 오류처럼 자세히 알려 줄 것
<code>requestId</code>	문의할 때 쓸 번호 (05단원의 요청 번호)

넣지 말 것

<code>stack</code>	파일 경로가 그대로 노출됩니다
<code>쿼리문</code>	데이터베이스 구조가 노출됩니다
<code>내부 IP·포트</code>	"connect ECONNREFUSED 10.0.1.23:5432"
<code>사용자 비밀번호</code>	당연합니다. 에러 메시지에 딸려 나가는 경우가 실제로 있습니다

★ 판단 기준 "이걸 보고 상대가 무엇을 고칠 수 있나?" 고칠 수 있으면 알려 주고(400~404), 못 고치면 감춥니다(500).

직접 해 볼 것

▫ 직접 해보기 1

에러 응답에 `requestId` 를 넣어 보세요. 05단원의 번호붙이기 미들웨어를 가져와 `req.요청번호` 를 쓰면 됩니다. 터미널 기록과 같은 번호가 나오는지 확인하세요.

▫ 직접 해보기 2

에러 처리기에서 `err instanceof AppError` 확인을 지우고 `/api/v1/boom` 을 불러 보세요. 무엇이 나오나요? (확인 후 반드시 되돌리세요)

▫ 직접 해보기 3

500 응답에 `err.message` 를 담아 보고, `/api/v1/boom` 이 무엇을 알려 주는지 보세요. 이 정보로 공격자가 무엇을 알 수 있을까요? (확인 후 반드시 되돌리세요)

▫ 직접 해보기 4

에러 목록에 '너무 많이 요청함' 을 추가해 보세요. 상태코드는 429, 코드는 `TOO_MANY_REQUESTS` 입니다.

▫ 직접 해보기 5

성공 응답도 `{ success: true, data: ... }` 로 바꿔 보세요. 프론트 코드가 더 편해지나요, 더 번거로워지나요? (정답은 없습니다. 왜 그렇게 생각하는지가 중요합니다)

▹ 직접 해보기

6

“설비율(률)” 문제를 고쳐 보세요. 받침이 있으면 ‘을’, 없으면 ‘를’ 입니다. (힌트: 마지막 글자의 코드에서 (코드 - 44032) % 28 이 0이면 받침이 없습니다. 연습문제에서 자세히 만듭니다)

자주 하는 실수

이런 실수를 합니다

주소마다 응답 모양이 다름

프론트가 주소마다 다르게 써야 합니다. 형식을 하나로 정하세요.

이런 실수를 합니다

성공인데 error 필드가 있음

{ data: [...], error: null } 같은 모양입니다. 틀린 건 아니지만, “error 가 있으면 실패” 규칙이 깨집니다. 없으면 아예 넣지 마세요.

이런 실수를 합니다

에러 코드 없이 메시지만 줌

프론트가 메시지 글자를 비교하게 됩니다. 맞춤법만 고쳐도 깨집니다.

이런 실수를 합니다

500 에 상세 내용을 담음

내부 구조가 밖으로 새어 나갑니다.

이런 실수를 합니다

super(message) 를 빼먹음

→ ReferenceError. class 를 물려받을 때는 부모부터 만들어야 합니다.

이런 실수를 합니다

instanceof 대신 err.status 유무로 판단

남이 만든 미들웨어의 에러에도 status 가 붙어 있을 수 있습니다. 내 에러인지 확실히 알려면 instanceof 를 쓰세요.

정리

- 1 응답 모양을 하나로 정해 두면 받는 쪽 코드가 단순해진다. 주소마다 다르면 부르는 쪽이 매번 다르게 읽어야 한다.
- 2 성공에는 `error` 를 넣지 않고, 실패에는 `data` 를 넣지 않는다.
- 3 에러에는 사람이 읽을 메시지와 함께 기계가 볼 '코드' 를 준다. 메시지는 언제든지 바뀌지만 코드는 안 바뀌므로, 프론트는 코드로 갈래를 나눈다.
- 4 에러 클래스를 만들어 두면 던지는 쪽이 짧아진다. `super(message)` 를 빼먹지 않는다.
- 5 검증 실패는 자세히 준다. 어느 필드가 왜 틀렸는지 알려 줘야 화면에 표시할 수 있다.
- 6 500 에는 상세 내용을 담지 않는다. 안쪽 사정이 밖으로 나간다.
- 7 내가 만든 에러인지는 `instanceof` 로 가린다. `err.status` 가 있는지로 가리면 남이 던진 에러가 섞여 들어온다.
- 8 404 처리기와 에러 처리기는 여전히 맨 아래다.

API 문서와 버전 관리

RUN node 개념05_API_문서와_버전.js

그러면 Ctrl + C

주소도 정했고 응답 형식도 정했습니다. 그런데 프론트엔드는 그걸 어떻게 알까요?

문서가 없으면 이렇게 됩니다.

프론트: "설비 목록 주소가 뭐예요?" 백엔드: "/api/v1/equipments 요" 프론트: "페이징 되나요?" 백엔드: "page 랑 limit 있어요" 프론트: "없는 설비 요청하면 뭐가 와요?" 백엔드: "404 요... 아 잠깐, 형식이 좀 바뀌었는데"

이 대화를 매일 합니다. 문서 한 장이면 안 해도 됩니다.

```
const express = require("express");
const { 설비목록 } = require("../data/설비데이터");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

06

문서에 무엇을 적나

섹션 1

주소 하나마다 여섯 가지를 적으면 충분합니다.

① 메서드와 주소 GET /api/v1/equipments ② 무엇을 하는가 설비 목록을 쪽 단위로 돌려줍니다 ③ 받는 것 page, limit, line, q (각각 필수인지, 기본값은 뭔지) ④ 주는 것 예시 응답을 통째로 ⑤ 나올 수 있는 에러 400 VALIDATION_FAILED, 404 NOT_FOUND ⑥ 예시 요청 curl 이나 주소 한 줄

★ ④가 제일 중요합니다. 말로 설명하는 것보다 실제 응답 한 덩어리를 붙여 두는 편이 훨씬 낫습니다. 프론트는 그걸 그대로 복사해서 가짜 데이터로 씁니다.

문서를 코드 옆에 두기

섹션 2

```
const 문서 = {
  이름: "설비 관리 API",
  버전: "1.0.0",
  기준주소: "/api/v1",
```

```

{
  주소: "GET /api/v1/equipments",
  설명: "설비 목록을 쪽 단위로 돌려줍니다",
  쿼리: {
    page: "쪽 번호. 기본 1",
    limit: "한 쪽 개수. 기본 10, 최대 100",
    line: "라인으로 거르기. A / B / C",
    q: "이름·라인·상태에서 검색",
  },
  응답예시: { data: [{ id: 1, name: "컨베이어 1호" }], meta: { page: 1, total: 12 } },
},
에러: ["없음"],
},
{
  주소: "GET /api/v1/equipments/:id",
  설명: "설비 하나를 돌려줍니다",
  쿼리: {},
  응답예시: { data: { id: 1, name: "컨베이어 1호", line: "A", status: "가동" } },
  에러: ["404 NOT_FOUND"],
},
],
};

app.get("/api/v1/docs", (req, res) => {
  res.json({ data: 문서 });
});

```

GET /api/v1/docs

```

200 {"data":{"이름":"설비 관리 API","버전":"1.0.0","기준주소":"/api/v1","주소들":
[{"주소":"GET /api/v1/equipments","설명":"설비 목록을 쪽 단위로 돌려줍니다","쿼리":{"page":"쪽 번호. 기본 1","limit":"한 쪽 개수. 기본 10, 최대 100","line":"라인으로 거르기. A / B / C","q":"이름·라인·상태에서 검색"},"응답예시":{"data":[{"id":1,"name":"컨베이어 1호"}],"meta":{"page":1,"total":12}},"에러":["없음"]},{주소":"GET /api/v1/equipments/:id","설명":"설비 하나를 돌려줍니다","쿼리":{"응답예시":{"data":{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"},"에러":["404 NOT_FOUND"]}}}]}

```

★ 문서를 주소로 주면 좋은 점 서버만 켜져 있으면 누구나 최신 문서를 봅니다. 따로 파일을 주고받을 필요가 없습니다.

★ 나쁜 점 코드를 고치고 이 객체를 안 고치면 문서가 거짓말을 합니다. “틀린 문서는 없는 문서보다 나쁩니다.” 정말입니다.

★ 그래서 실무에서는 OpenAPI(Swagger)를 씁니다 문서를 정해진 형식으로 적으면 자동으로 화면을 그려 주고, 그 화면에서 바로 요청을 보내 볼 수도 있습니다. Postman 없어요. 코드에서 문서를 뽑아내는

도구도 있어서 어긋날 일이 줄어듭니다. 지금은 이름만 알아 두세요. 처음부터 이걸 붙이면 배울 게 너무 많아집니다.

★ 지금 단계에서는 README.md 로 충분합니다 이 폴더의 README.md 를 열어 보세요. 같은 내용을 글로 적어 두었습니다. 프로젝트 폴더에 README.md 한 장만 있어도 사람이 훨씬 덜 물어봅니다.

언제 버전을 올리나

섹션 3

```
app.get("/api/v1/versioning", (req, res) => {
  res.json({
    data: {
      올려야하는변경: [
        "응답에서 필드를 뺌",
        "필드 이름을 바꿈 (name → title)",
        "필드 타입을 바꿈 (숫자 → 글자)",
        "주소를 바꾸거나 없앴",
        "필수 입력값을 추가함",
        "응답 형식을 통째로 바꿈 (배열 → { data })",
      ],
      안올려도되는변경: [
        "응답에 필드를 추가함",
        "선택 입력값을 추가함",
        "새 주소를 만듦",
        "에러 메시지 글자를 다듬음",
        "속도를 개선함",
      ],
    },
  });
});
```

GET /api/v1/versioning

```
200 {"data":{"올려야하는변경":["응답에서 필드를 뺌","필드 이름을 바꿈 (name → titl
e)","필드 타입을 바꿈 (숫자 → 글자)","주소를 바꾸거나 없앴","필수 입력값을 추가
함","응답 형식을 통째로 바꿈 (배열 → { data })"],"안올려도되는변경":["응답에 필
드를 추가함","선택 입력값을 추가함","새 주소를 만듦","에러 메시지 글자를 다듬
음","속도를 개선함"]}}
```

★ 판단 기준 한 줄 “지금 쓰고 있는 프론트 코드가 이 변경으로 깨지나?” 깨지면 버전을 올립니다. 안 깨지면 그냥 고칩니다.

★ 필드를 ‘추가’ 하는 건 왜 안 깨지나 프론트는 자기가 쓰는 필드만 꺼내 씁니다. 모르는 필드는 무시합니다. 그래서 추가는 안전하고, 빼는 것은 위험합니다.

다만 프론트가 응답을 통째로 화면에 그리고 있다면 추가도 티가 납니다. 그런 경우는 미리 알려 주는 게 좋습니다.

★ 에러 '메시지' 를 다듬는 건 안전한데, '코드' 를 바꾸는 건 위험합니다 개념04 에서 코드를 따로 둔 이유가 여기서 드러납니다.

v1 과 v2 를 함께 돌리기

섹션 4

```
const v1 = express.Router();
const v2 = express.Router();
```

v1 — 옛날 형식. 배열을 그대로 줍니다.

```
v1.get("/equipments", (req, res) => {
```

이 주소는 곧 없앨 예정이라고 알려 줍니다.

```
  res.set("Deprecation", "true");
  res.set("Sunset", "Wed, 31 Dec 2026 00:00:00 GMT");
  res.set("Link", '</api/v2/equipments>; rel="successor-version"');

  const 목록 = 설비목록()
    .slice(0, 2)
    .map((설비) => ({ id: 설비.id, name: 설비.name }));

  res.json(목록);
});
```

v2 — 새 형식. { data, meta } 로 감쌉니다.

```
v2.get("/equipments", (req, res) => {
  const 전부 = 설비목록();
  const data = 전부.slice(0, 2).map((설비) => ({ id: 설비.id, name: 설비.name }));

  res.json({ data, meta: { page: 1, limit: 2, total: 전부.length, totalPages: 6 } });
});

app.use("/api/v1", v1);
app.use("/api/v2", v2);
```

GET /api/v1/equipments

200 [{"id":1,"name":"컨베이어 1호"}, {"id":2,"name":"컨베이어 2호"}]

GET /api/v2/equipments

200 {"data":[{"id":1,"name":"컨베이어 1호"}, {"id":2,"name":"컨베이어 2호"}], "meta": {"page":1, "limit":2, "total":12, "totalPages":6}}

★ 같은 데이터를 두 형식으로 주고 있습니다. 기존 프론트는 v1 을 그대로 씁니다. 안 깨집니다. 새로 만드는 화면은 v2 를 씁니다. 전부 옮기고 나면 v1 을 없앱니다.

★ Deprecation 과 Sunset 헤더 “이 주소는 곧 없어집니다”, “언제까지만 씁니다” 를 기계가 읽을 수 있게 알리는 헤더입니다. Postman 의 Headers 탭에서 확인해 보세요.

물론 헤더만 붙여 놓으면 아무도 안 봅니다. 문서에 적고, 쓰는 사람에게 직접 알려야 합니다. 헤더는 거들 뿐입니다.

★ 언제 v1 을 없애나 접속 기록을 보고 결정합니다. (05단원의 morgan 기록이 여기서 쓰입니다) v1 요청이 한 달째 0건이면 없애도 됩니다. “아무도 안 쓰겠지” 하고 지웠다가 사고 나는 일이 있습니다. 기록으로 확인하세요.

두 버전이 코드를 나눠 쓰는 법

섹션 5

v1 과 v2 가 완전히 다른 코드면 고칠 곳이 두 배가 됩니다. 진짜 일하는 부분은 한 곳에 두고, 모양만 다르게 만듭니다.

```
function 설비들가져오기() {
```

진짜 일하는 곳. 나중에 데이터베이스가 될 자리입니다.

```
    return 설비목록()
        .slice(0, 2)
        .map((설비) => ({ id: 설비.id, name: 설비.name }));
}

v1.get("/equipments-shared", (req, res) => {
    res.json(설비들가져오기()); // v1 모양
});

v2.get("/equipments-shared", (req, res) => {
    res.json({ data: 설비들가져오기() }); // v2 모양
});
```

```
GET /api/v1/equipments-shared
```

```
200 [{"id":1,"name":"컨베이어 1호"}, {"id":2,"name":"컨베이어 2호"}]
```

```
GET /api/v2/equipments-shared
```

```
200 {"data":[{"id":1,"name":"컨베이어 1호"}, {"id":2,"name":"컨베이어 2호"}]}
```

★ 설비를 가져오는 방법이 바뀌면 함수 하나만 고칩니다. 두 버전이 자동으로 함께 고쳐집니다. ‘모양을 만드는 곳’ 과 ‘일을 하는 곳’ 을 나누는 것 — 07단원의 주제입니다.

```

app.use((req, res) => {
  res.status(404).json({ error: { code: "ROUTE_NOT_FOUND", message: "그런 주소가 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(err.status || 500).json({
    error: { code: err.code || "INTERNAL_ERROR", message: err.status ? err.message : "서버에서 문제가 생겼습니다" },
  });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/api/v1/docs`);
});

```

문서 한 장 예시 (README.md 에 이렇게 씁니다)

GET /api/v1/equipments

설비 목록을 쪽 단위로 돌려줍니다.

| 쿼리 | 필수 | 기본 | 설명 | | ----- | ---- | ---- | ----- | | page | 아니오 | 1 | 쪽
번호 | | limit | 아니오 | 10 | 한 쪽 개수 (최대 100) | | line | 아니오 | - | A / B / C | | q | 아니오 | - | 이름·
라인·상태에서 검색 |

요청 GET /api/v1/equipments?line=A&limit=2

응답 200 { "data": [{ "id": 1, "name": "컨베이어 1호" },

```

"meta": { "page": 1, "limit": 2, "total": 4, "totalPages": 2 } }

```

에러 없음. 조건에 맞는 것이 없으면 data 가 빈 배열입니다.

표 하나에 예시 두 개. 이 정도면 프론트가 안 물어봅니다.

직접 해 볼 것

▫ 직접 해보기

1

이 폴더의 README.md 를 열어 보고, POST /api/v1/equipments 문서를 직접 추가해 보세요.

▫ 직접 해보기

2

문서 객체에 POST 주소를 추가하고 /api/v1/docs 로 확인하세요.

▫ 직접 해보기 3

Postman 으로 GET /api/v1/equipments 를 부르고 Headers 탭에서 Deprecation 과 Sunset 을 찾아보세요.

▫ 직접 해보기 4

아래 변경들이 버전을 올려야 하는지 판단해 보세요.

- ① 설비 응답에 '마지막점검일' 을 추가한다
- ② 설비의 line 을 "A" 에서 { code: "A", name: "1라인" } 으로 바꾼다
- ③ ?sort= 를 새로 지원한다
- ④ 설비 등록에 '담당자' 를 필수로 받는다
- ⑤ 404 메시지를 "설비를 찾을 수 없습니다" 로 다듬는다
- ⑥ 404 코드를 NOT_FOUND 에서 EQUIPMENT_NOT_FOUND 로 바꾼다

▫ 직접 해보기 5

v2 에 '설치일' 필드를 추가해 보세요. 이걸 버전을 또 올려야 할까요?

▫ 직접 해보기 6

설비들 가져오기 함수를 고쳐서 3개를 돌려주게 하세요. v1 과 v2 가 함께 바뀌는지 확인하세요.

자주 하는 실수

이런 실수를 합니다

문서를 안 쓰고 "코드 보면 안다" 고 함

프론트는 서버 코드를 볼 수 없거나, 봐도 어디를 봐야 할지 모릅니다.

이런 실수를 합니다

문서를 고치는 걸 잊음

틀린 문서는 없는 문서보다 나쁩니다. 믿고 만들었다가 다 틀리니까요. 응답 형식을 바꿀 때 문서도 함께 고치는 습관을 들이세요.

이런 실수를 합니다

버전 없이 시작함

나중에 불일 데가 없습니다. 처음부터 /api/v1 로 시작하세요.

이런 실수를 합니다

필드를 조용히 없앴

프론트가 어느 날 갑자기 `undefined` 를 만납니다. 없애기 전에 알리고, 기간을 주고, 기록으로 확인하세요.

이런 실수를 합니다

버전마다 코드를 통째로 복사

고칠 곳이 두 배가 됩니다. 일하는 부분은 함께 쓰고 모양만 나누세요.

이런 실수를 합니다

v2 를 만들고 v1 을 바로 없앴

쓰던 곳이 전부 멈춥니다. 기록으로 0건을 확인하고 없애세요.

정리

- 1 문서에는 주소·메서드·받는 값·돌려주는 모양·날 수 있는 에러를 적는다.
- 2 “코드 보면 안다” 는 안 된다. 부르는 쪽이 코드를 못 볼 수도 있다.
- 3 문서는 코드 옆에 둔다. 멀리 두면 고치는 것을 잊는다.
- 4 버전은 처음부터 붙이고 시작한다. 나중에 붙이려면 쓰던 곳을 전부 고쳐야 한다.
- 5 버전을 올리는 때는 ‘이미 쓰던 것이 깨질 때’ 다. 필드를 더하는 것은 안 깨지므로 버전을 올리지 않는다.
- 6 필드를 조용히 없애지 않는다. 쓰고 있던 쪽이 그날로 터진다.
- 7 버전마다 코드를 통째로 복사하지 않는다. 달라지는 부분만 나누고 나머지는 같이 쓴다.
- 8 v2 를 냈다고 v1 을 바로 없애지 않는다. 옮길 시간을 준다.

REST API 설계

RUN node 연습문제.js

서버 없이 푸는 문제들입니다. 여기서 만든 함수들을 연습문제_서버.js 에서 그대로 씁니다.

문제 1 조사 붙이기

개념04 의 에러 메시지가 "설비(을) 찾을 수 없습니다" 로 어색했습니다. 받침에 따라 조사를 골라 주는 함수를 만드세요.

만들 함수 두 개 받침있나(글자) → true / false / null(한글이 아님) 조사붙이기(단어, 받침있을때, 받침없을때) → 단어 + 알맞은 조사

★ 한글 한 글자의 번호는 이렇게 정해져 있습니다.

$$44032 + (\text{초성번호} \times 21 \times 28) + (\text{중성번호} \times 28) + \text{종성번호}$$

앞의 두 덩어리가 전부 28 의 배수라서, (코드 - 44032) % 28 이 곧 종성(받침) 번호입니다. 0 이면 받침이 없습니다.

완성형 한글은 가(44032) ~ 힉(55203) 입니다. 이 범위 밖이면 한글이 아닙니다.

힌트: 글자.charCodeAt(0) 으로 번호를 얻습니다.

단어[단어.length - 1] 이 마지막 글자입니다.
한글이 아니면 "API(을/를)" 처럼 둘 다 보여 주세요.

시험할 것 (이 순서로) ("설비", "을", "를") ("점검기록", "을", "를") ("작업지시", "이", "가") ("사용자", "은", "는") ("로봇", "은", "는") ("API", "을", "를")

기대 출력	설비를
	점검기록을
	작업지시가
	사용자는
	로봇은
	API(을/를)

여기에 코드를 쓰세요

문제 2 페이징 계산

쿼리에서 안전하게 page 와 limit 을 꺼내는 함수와, meta 를 만들어 주는 함수를 만드세요.

쪽정보(query, 기본 = 10, 최대 = 100) → { page, limit }

- page 가 정수가 아니거나 1보다 작으면 1
- limit 이 정수가 아니거나 1보다 작으면 기본값
- limit 이 최대를 넘으면 최대로 깎기

쪽메타(total, page, limit) → { page, limit, total, totalPages }

★ 함정: `Number(null)` 은 0 이고 `Number(undefined)` 는 `NaN` 입니다. 0 은 정수라서 `Number.isInteger` 를 통과합니다. 범위까지 봐야 합니다.

시험할 것 (이 순서로) 쪽정보({}) 쪽정보({ page: "3", limit: "5" }) 쪽정보({ page: "abc" }) 쪽정보({ limit: "999" }) 쪽정보({ page: "0", limit: "-5" }) 쪽메타(12, 1, 5) 쪽메타(0, 1, 10)

```
기대 출력    { page: 1, limit: 10 }
              { page: 3, limit: 5 }
              { page: 1, limit: 10 }
              { page: 1, limit: 100 }
              { page: 1, limit: 10 }
              { page: 1, limit: 5, total: 12, totalPages: 3 }
              { page: 1, limit: 10, total: 0, totalPages: 0 }
```

(마지막 줄의 totalPages 가 0 입니다. 화면에 어떻게 보일지 생각해 보세요)

여기에 코드를 쓰세요

문제 3 정렬 조건 읽기

?sort= 값을 조건 배열로 바꾸는 함수 정렬조건 을 만드세요.

```
"name"      → [{ 키: "name", 방향: 1 }]
"-id"       → [{ 키: "id", 방향: -1 }]
"line,-id"  → [{ 키: "line", 방향: 1 }, { 키: "id", 방향: -1 }]
```

없거나 빈 값 → []

그리고 결과를 사람이 읽기 좋게 찍는 함수 조건찍기(sort) 를 만드세요.

★ 공백이 섞여 와도 되어야 합니다. "line, -id " 도 잘 돌아가야 합니다. trim 을 안 하면 " -id" 가 되어 - 를 못 알아봅니다. 에러가 안 나고 조용히 정렬만 안 됩니다.

힌트: split(",") → map(trim) → filter(Boolean) → map(조건으로)

조건찍기 는 "line 오름, id 내림" 처럼 만들어 찍으세요.
객체를 그대로 찍으면 한글 키에 따옴표가 붙어 보기 어렵습니다.

시험할 것 (이 순서로) undefined, "name", "-id", "line,-id", "line, -id ", ""

```
기대 출력    (없음) → 정렬 안 함
              "name" → name 오름
              "-id"  → id 내림
              "line,-id" → line 오름, id 내림
              "line, -id " → line 오름, id 내림
              ""     → 정렬 안 함
```

여기에 코드를 쓰세요

문제 4 원하는 속성만 고르기

?fields=id,name 처럼 받아서 그 속성만 남기는 함수 필드고르기(항목, fields) 를 만드세요.

fields 가 없거나 빈 값이면 → 항목을 그대로 없는 이름을 달라고 하면 → 무시

힌트: 키 in 항목 으로 있는지 확인합니다.

```
const 설비 = { id: 1, name: "컨베이어 1호", line: "A", status: "가동" };
```

시험할 것 (이 순서로) (설비, "id,name") (설비, undefined) (설비, "id, 없는것") (설비, "")

```
기대 출력    { id: 1, name: '컨베이어 1호' }  
              { id: 1, name: '컨베이어 1호', line: 'A', status: '가동' }  
              { id: 1 }  
              { id: 1, name: '컨베이어 1호', line: 'A', status: '가동' }
```

여기에 코드를 쓰세요

문제 5 주소 설계 고치기

아래 주소들을 REST 규칙에 맞게 고쳐 배열에 담고 한 줄씩 출력하세요.

① GET /getEquipmentList 설비 목록 ② GET /equipment/3 3번 설비 하나 ③ POST /addEquipment 설비 등록 ④ POST /modifyEquipment?id=3 3번 설비 상태 바꾸기 ⑤ GET /removeEquipment?id=3 3번 설비 지우기 ⑥ GET /searchEquipment?line=A&st=가동 A라인 가동 중인 설비 ⑦ GET /getLogsOfEquipment?eqId=3 3번 설비의 점검기록 목록 ⑧ POST /addLog 3번 설비에 점검기록 추가

규칙: 복수형 명사 · 동사 금지 · 하나는 경로 · 거르기는 쿼리 ·

포함 관계는 중첩 · 앞에 /api/v1

```
기대 출력    GET /api/v1/equipments  
              GET /api/v1/equipments/3  
              POST /api/v1/equipments  
              PATCH /api/v1/equipments/3  
              DELETE /api/v1/equipments/3  
              GET /api/v1/equipments?line=A&status=가동  
              GET /api/v1/equipments/3/logs  
              POST /api/v1/equipments/3/logs
```

(⑧가 왜 특히 위험했는지 설명할 수 있어야 합니다)

여기에 코드를 쓰세요

문제 6 버전 올릴 때 하나

아래 변경마다 "올려야 함" / "안 올려도 됨" 을 찍지어 출력하세요.

판단 기준 한 줄: "지금 쓰고 있는 프론트 코드가 이 변경으로 깨지나?"

기대 출력 설비 응답에 마지막점검일을 추가 → 안 올려도 됨
 line 을 글자에서 객체로 바꿈 → 올려야 함
 ?sort= 를 새로 지원 → 안 올려도 됨
 설비 등록에 담당자를 필수로 추가 → 올려야 함
 404 메시지 글자를 다듬음 → 안 올려도 됨
 404 코드를 NOT_FOUND 에서 EQUIPMENT_NOT_FOUND 로 바꿈 → 올려야 함
 응답을 배열에서 { data } 로 감쌘 → 올려야 함
 새 주소 /api/v1/lines 를 만들 → 안 올려도 됨

(메시지는 안 깨지는데 코드는 깨지는 이유를 설명할 수 있어야 합니다)

여기에 코드를 쓰세요

문제 7 응답 만들기

성공·실패 응답을 만들어 주는 함수 두 개를 만드세요.

성공(data, meta) → { data } 또는 { data, meta }

실패(code, message, details) → { error: { code, message } } 또는 details 포함

★ meta 와 details 는 있을 때만 넣으세요. { data: [...], meta: null } 은 지저분하고, 프론트 확인도 번거로워집니다.

★ 출력은 `JSON.stringify` 로 찍으세요. `console.log` 로 객체를 그냥 찍으면 실제로 나가는 모양과 달라 보입니다.

시험할 것 (이 순서로) 성공([1, 2]) 성공([1, 2], { total: 2 }) 성공({ id: 1 }) 실패("NOT_FOUND", "설비를 찾을 수 없습니다") 실패("VALIDATION_FAILED", "입력값이 올바르지 않습니다", [{ 키: "name", 이유: "필수입니다" }])

```
기대 출력  {"data":[1,2]}
            {"data":[1,2],"meta":{"total":2}}
            {"data":{"id":1}}
            {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}
            {"error":{"code":"VALIDATION_FAILED","message":"입력값이 올바르지
            않습니다","details":[{"키":"name","이유":"필수입니다"}]}}
```

여기에 코드를 쓰세요

문제 8 total 은 언제 세나

목록을 만들 때 total 을 slice 뒤에 세면

무엇이 잘못되나: _____

화면에서 어떻게 보이냐: _____

왜 못 알아채냐: _____

올바른 순서를 다섯 단계로 적어 보세요: ① _____ ② _____

_____ ③ _____ ④ _____

_____ ⑤ _____

정렬을 자르기 전에 해야 하는 이유도 같은 이야기입니다. 왜일까요:

06단원에서 함께 쓰는 연습용 데이터

개념 파일마다 데이터를 다시 적으면 관리가 안 됩니다.

한 곳에 두고 require 해서 씁니다.

★ 진짜 서버라면 이 자리에 데이터베이스가 옵니다. 07단원에서 파일로, PART 4 에서 데이터베이스로 바꿉니다. 그때 이 파일만 같이 끼우면 나머지는 그대로 둘 수 있습니다. 그래서 데이터를 한 곳에 모아 두는 것입니다.

```
const 설비들 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동", 설치일: "2021-03-15" },
  { id: 2, name: "컨베이어 2호", line: "A", status: "정지", 설치일: "2021-03-15" },
  { id: 3, name: "프레스 1호", line: "B", status: "가동", 설치일: "2019-07-02" },
  { id: 4, name: "프레스 2호", line: "B", status: "점검중", 설치일: "2019-07-02" },
  { id: 5, name: "용접로봇 1호", line: "C", status: "가동", 설치일: "2023-11-20" },
  { id: 6, name: "용접로봇 2호", line: "C", status: "가동", 설치일: "2023-11-20" },
  { id: 7, name: "검사기 1호", line: "A", status: "가동", 설치일: "2022-05-08" },
  { id: 8, name: "검사기 2호", line: "B", status: "정지", 설치일: "2022-05-08" },
  { id: 9, name: "포장기 1호", line: "C", status: "가동", 설치일: "2020-01-30" },
  { id: 10, name: "포장기 2호", line: "C", status: "점검중", 설치일: "2020-01-30" },
  { id: 11, name: "적재로봇 1호", line: "A", status: "가동", 설치일: "2024-02-11" },
  { id: 12, name: "적재로봇 2호", line: "B", status: "가동", 설치일: "2024-02-11" },
];

const 점검기록들 = [
  { id: 1, equipmentId: 1, result: "정상", 점검일: "2026-08-01", 담당자: "김민준" },
  { id: 2, equipmentId: 1, result: "이상", 점검일: "2026-08-08", 담당자: "이서연" },
  { id: 3, equipmentId: 3, result: "정상", 점검일: "2026-08-02", 담당자: "김민준" },
  { id: 4, equipmentId: 5, result: "정상", 점검일: "2026-08-05", 담당자: "박지훈" },
  { id: 5, equipmentId: 1, result: "정상", 점검일: "2026-08-12", 담당자: "김민준" },
];
```

★ 복사본을 돌려주는 함수를 함께 둡니다. 원본을 그대로 내보내면, 어느 파일에서 sort 를 한 번 하는 순간 다른 파일의 순서까지 바뀝니다. 찾기 아주 어려운 버그가 됩니다.

```
function 설비목록() {
  return 설비들.map((설비) => ({ ...설비 }));
}

function 점검기록목록() {
  return 점검기록들.map((기록) => ({ ...기록 }));
}

module.exports = { 설비목록, 점검기록목록 };
```

★ { ...설비 } 로 한 겹 더 복사한 이유 설비들.map((설비) => 설비) 는 '같은 객체' 를 담은 새 배열입니다. 배열은 새것이지만 안의 객체는 원본과 같아서, 속성을 바꾸면 원본도 바뀝니다. { ...설비 } 는 객체까지 복사합니다. (JS자료 09단원)

문서대로 API 만들기

RUN npx nodemon 연습문제_서버.js

★★ 이 문제는 지금까지와 다릅니다.

무엇을 만들지는 이 파일에 안 적혀 있습니다. 같은 폴더의 README.md 를 열어서, 거기 적힌 대로 만드세요.

실무에서 일하는 방식이 이렇습니다. “설비 API 만들어 주세요” 가 아니라 문서를 받고, 그대로 만듭니다. 응답 예시가 한 글자라도 다르면 프론트가 깨집니다.

순서 ① README.md 를 처음부터 끝까지 읽습니다 ② 아래 TODO 를 위에서부터 채웁니다 ③ 하나 만들 때마다 README 의 ‘요청’ 을 Postman 으로 그대로 보냅니다 ④ 응답이 README 의 ‘응답’ 과 똑같은지 글자 단위로 봅니다

```
const express = require("express");
const { 설비목록, 점검기록목록 } = require("../data/설비데이터");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

준비 (여기는 그대로 두세요)

★ 서버가 켜질 때 한 번만 가져옵니다. 라우트 안에서 매번 설비목록() 을 부르면 추가·삭제한 것이 사라집니다.

```
let 설비들 = 설비목록();
const 점검기록들 = 점검기록목록();

let 다음번호 = 13;

const 라인값들 = ["A", "B", "C"];
const 상태값들 = ["가동", "정지", "점검중"];

const 키별사용자 = {
  "key-user-1": { name: "김민준", role: "user" },
  "key-admin-1": { name: "이서연", role: "admin" },
};
```

문제 1 연습문제.js 에서 만든 함수 옮겨 오기

연습문제.js 에서 만든 아래 함수들을 여기로 가져오세요.

받침있나(글자) 조사붙이기(단어, 받침있을때, 받침없을때) 쪽정보(query, 기본, 최대) 정렬조건(sort)

비교하기(a, b, 조건들) ← 개념02 섹션 2 에 있습니다 필드고르기(항목, fields)

★ 비교하기 는 localeCompare(다른값, "ko", { numeric: true }) 를 쓰세요. 왜인지는 개념02 섹션 6-2 에 재 본 결과가 있습니다.

여기에 코드를 쓰세요

문제 2 에러 클래스와 에러 목록

AppError 클래스를 만들고, 자주 쓸 에러들을 미리 만들어 두세요.

```
class AppError extends Error {
```

```
  constructor(status, code, message, details) { ... }
```

```
}
```

에러.없음("설비")	→ 404 NOT_FOUND	"설비를 찾을 수 없습니다"
에러.검증실패(항목들)	→ 400 VALIDATION_FAILED	"입력값이 올바르지 않습니다"
에러.로그인필요()	→ 401 UNAUTHENTICATED	"로그인이 필요합니다"

에러.권한없음("이 작업을 할") → 403 FORBIDDEN "이 작업을 할 권한이 없습니다" 에러.주소없음() → 404 ROUTE_NOT_FOUND "그런 주소가 없습니다"

★ 없음 메시지에 조사붙이기 를 쓰세요. "설비(를) 찾을 수 없습니다" 가 아니라 "설비를 찾을 수 없습니다" 여야 합니다.

★ super(message) 를 빼먹으면 ReferenceError 가 납니다. 물려받은 클래스는 부모부터 만들어야 합니다.

여기에 코드를 쓰세요

문제 3 응답 도우미

보내기(res, data, meta) 를 만드세요. meta 가 없으면 { data } 만, 있으면 { data, meta } 를 보냅니다.

★ meta 가 없을 때 meta: null 을 넣지 마세요. 아예 빼세요.

여기에 코드를 쓰세요

문제 4 미들웨어 네 개

① 인증(req, res, next) Authorization: Bearer <키> 를 읽어 req.user 에 담습니다. 헤더가 없거나 형식이 틀리거나 없는 키면 401 입니다.

② 역할확인(...허용역할들) req.user.role 이 허용 목록에 없으면 403 입니다.

③ app.param("id", ...) :id 를 숫자로 바꿔 req.번호 에 담습니다. 숫자가 아니면 400 VALIDATION_FAILED 이고 details 는

{ 키: "id", 이유: "숫자여야 합니다" } · 입니다.

④ 설비찾기(req, res, next) req.번호 로 설비를 찾아 req.설비 에 담습니다. 없으면 404 입니다.

★ ④는 GET·PATCH·DELETE·logs 네 곳이 함께 씁니다. 꼭 만드세요.

여기에 코드를 쓰세요

문제 5 GET /api/v1/equipments

README 의 표대로 page · limit · sort · line · status · q · fields 를 전부 지원하세요.

★★ 순서를 반드시 지켜세요. ① 검색(q) ② 필터(line, status) ③ 정렬(sort) ④ total 세기 ⑤ 자르기 ⑥ 필드 고르기

④를 ⑤ 뒤에 두면 전체 건수가 틀립니다. ③을 ⑤ 뒤에 두면 그 쪽 안에서만 정렬됩니다.

기대 결과 (README 의 예시와 같아야 합니다) ?limit=2&fields=id

```
→ {"data":[{"id":1},{"id":2}], "meta":{"page":1, "limit":2, "total":12, "totalPages":6}}
```

?line=A&limit=2&sort=-id&fields=id,name

```
→ {"data":[{"id":11, "name":"적재로봇 1호"}, {"id":7, "name":"검사기 1호"}],  
  "meta":{"page":1, "limit":2, "total":4, "totalPages":2}}
```

?q=로봇&fields=id → id 5, 6, 11, 12 ?line=A,B&status=정지&fields=id → id 2, 8

```
?line=Z&fields=id          → 200 과 [], total 0, totalPages 0  
?page=99&limit=5&fields=id → 200 과 [], page 99, totalPages 3
```

여기에 코드를 쓰세요

문제 6 GET /api/v1/equipments/:id

```
기대 출력   /3    → 200 프레스 1호  
            /abc  → 400 VALIDATION_FAILED, details 에 id  
            /99   → 404 NOT_FOUND, "설비를 찾을 수 없습니다"
```

★ 문제 4를 제대로 만들었다면 이 라우트는 한 줄입니다.

여기에 코드를 쓰세요

문제 7 GET /api/v1/equipments/:id/logs

그 설비의 점검기록 목록입니다. 페이징도 되어야 합니다.

```
기대 출력  /1/logs  → 200, 기록 3건 (id 1, 2, 5)
           /7/logs  → 200 과 []      ← 설비는 있는데 기록이 0건
           /99/logs → 404           ← 설비 자체가 없음
```

★★ 이 단원에서 가장 중요한 부분입니다. 위 두 줄의 차이를 만들려면 무엇이 필요할까요? 설비찾기 미들웨어를 안 붙이면 둘 다 200 과 빈 배열이 됩니다. “없는 설비의 기록을 달라” 는 요청에 “0건입니다” 라고 답하는 셈입니다.

여기에 코드를 쓰세요

문제 8 POST /api/v1/equipments

admin 만 할 수 있습니다. README 의 규칙대로 검증하세요.

```
name 필수, 글자, 2~20자
line 필수, A/B/C 중 하나
```

status 는 받지 않습니다. 언제나 “정지” 로 시작합니다

```
기대 출력  증표 없이          → 401
           user 로          → 403
           admin, { "name":"용", "line":"Z" }      → 400, details 두 개
           admin, { "name":"용접로봇 3호", "line":"C", "status":"가동" }
           → 201, {"data":{"id":13, "name":"용접로봇
           3호", "line":"C", "status":"정지"}}
           Location: /api/v1/equipments/13
```

★★ 마지막을 보세요. status: "가동" 을 보냈는데 "정지" 로 만들어져야 합니다. 보낸 값을 그대로 믿으면 규칙이 무너집니다. 규칙은 서버가 짭니다.

여기에 코드를 쓰세요

문제 9 PATCH /api/v1/equipments/:id

로그인하면 누구나 상태를 바꿀 수 있습니다.

```
기대 출력  /13 user { "status":"이상" } → 400, details 에  
            "가동, 정지, 점검중 중 하나여야 합니다"  
            /13 user { "status":"점검중" } → 200, status 가 "점검중"  
            /99 user { "status":"가동" } → 404
```

여기에 코드를 쓰세요

문제 10 DELETE /api/v1/equipments/:id

admin 만 지울 수 있습니다.

```
기대 출력  /13 user → 403  
            /13 admin → 204  
            /13 admin → 404 (이미 지웠으니)
```

★ 미들웨어 순서: 인증 → 역할확인 → 설비찾기 README 에도 적혀 있습니다. "자격이 없는 사람에게는 404 보다 403 이 먼저 나옵니다. 있는지 없는지를 알려 주지 않기 위해서입니다."

여기에 코드를 쓰세요

문제 11 404 와 에러 처리기

① 주소 없는 app.use 로 404 ROUTE_NOT_FOUND 를 만드세요. ② 인자 네 개짜리 에러 처리기를 맨 마지막에 만드세요.

- 터미널에 남기기 - err.type 이 "entity.parse.failed" → 400 INVALID_JSON - err instanceof AppError → err.status, err.code, err.message

(details 가 있을 때만 details 도)

- 그 밖 → 500 INTERNAL_ERROR "서버에서 문제가 생겼습니다"

★ instanceof 로 구별하는 것이 핵심입니다. "내가 일부러 만든 에러" 와 "예상 못 한 에러" 는 다르게 다뤄야 합니다. 앞의 메시지를 보여 줘도 되고, 뒤는 감춰야 합니다.

여기에 코드를 쓰세요

```
app.listen(PORT, () => {
  console.log(`설비 관리 API 가 켜졌습니다.
  http://localhost:${PORT}/api/v1/equipments`);
});
```

문서와 대조하기

README 의 예시 요청을 그대로 보내고, 응답을 글자 단위로 비교하세요.

1	GET	/api/v1/equipments?limit=3	3건 + meta
2	GET	/api/v1/equipments?line=A&limit=2&sort=-id&fields=id,name	11 적재로봇 1호 / 7 검사기 1호
3	GET	/api/v1/equipments?q=로봇	4건
4	GET	/api/v1/equipments?line=Z	200 과 []
5	GET	/api/v1/equipments/3	프레스 1호
6	GET	/api/v1/equipments/abc	400
7	GET	/api/v1/equipments/99	404 "설비를"
8	GET	/api/v1/equipments/1/logs	3건
9	GET	/api/v1/equipments/7/logs	200 과 []
10	GET	/api/v1/equipments/99/logs	404 ← 9와 비교
11	POST	/api/v1/equipments (증표 없이)	401
12	POST	/api/v1/equipments user	403
13	POST	/api/v1/equipments admin { "name":"용","line":"Z" }	400
14	POST	/api/v1/equipments admin 정상 + status:"가동"	201, status 는 "정지"
15	PATCH	/api/v1/equipments/13 user { "status":"점검중" }	200
16	DELETE	/api/v1/equipments/13 user	403
17	DELETE	/api/v1/equipments/13 admin	204
18	DELETE	/api/v1/equipments/13 admin	404

다 만든 뒤 답해 보세요

① 9번과 10번의 응답이 다른 이유는 무엇인가요? 설비찾기 를 안 붙이면 어떻게 되나요?

② 14번에서 status 를 본문으로 받게 하면 어떤 일이 생길까요?

③ 문제 5에서 total 을 slice 뒤에 세면 ?limit=2 일 때 meta 가 어떻게 나오나요? 화면의 쪽 번호 버튼은 어떻게 되나요? _____

④ 16번에서 설비찾기 를 역할확인보다 먼저 두면 user 가 DELETE /api/v1/equipments/99 를 보냈을 때 무엇이 나오나요? 그게 왜 문제인가요? _____

⑤ README 와 코드가 어긋난 곳을 하나라도 찾았나요? 어느 쪽을 고쳤나요? 왜 그쪽을 골랐나요?

잘 안 될 때 보는 곳

meta.total 이 항상 limit 과 같음 → total 을 slice 뒤에 썼습니다. 자르기 전에 세세요.

?sort=-id 인데 정렬이 안 됨 → 자른 뒤에 정렬했거나, trim 을 안 해서 " -id" 가 되었습니다.

?line=A, B 로 보내면 B라인이 안 나옴 → 침표로 나눈 뒤 trim 을 안 했습니다. " B" 를 찾고 있습니다.

/99/logs 가 404 가 아니라 빈 배열 → 설비찾기 미들웨어를 안 붙였습니다.

"설비를(를) 찾을 수 없습니다" 로 나옴 → 조사붙이기 를 안 썼습니다.

에러 응답이 {"error": "..."} 모양 (글자) → error 안에 { code, message } 객체가 들어가야 합니다.
README 를 다시 보세요.

500 인데 400 이어야 함 → req.body 를 || {} 없이 구조 분해했습니다.

설비를 등록했는데 목록에 안 보임 → 라우트 안에서 설비목록() 을 다시 불렀습니다.

그 함수는 매번 새 복사본을 만듭니다. 맨 위의 설비들 변수를 쓰세요.

부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

00단원 연습문제 정답 제조 현장 이해

RUN node 연습문제_정답.js

먼저 스스로 풀어 본 다음에 보세요. 답보다 '왜 그런지' 설명을 꼭 읽으세요.

문제 1

```
const 발전단계 = [
  { 단계: 1, 이름: "기계화", 기록: "종이 장부" },
  { 단계: 2, 이름: "자동화", 기록: "종이 + 엑셀" },
  { 단계: 3, 이름: "정보화", 기록: "엑셀 + 사내 시스템" },
  { 단계: 4, 이름: "지능화", 기록: "전부 데이터" },
];

const 데이터단계 = 발전단계.find((하나) => 하나.기록.endsWith("데이터"));
console.log(데이터단계.이름);
```

출력 지능화

endsWith 는 “그 글자로 끝나는가” 를 봅니다. includes 를 쓰면 “전부 데이터” 만이 아니라 다른 것도 걸릴 수 있습니다.

문제 2

```
const 설비수 = 50;
const 일하는날 = 250;

console.log(설비수 * 일하는날);
```

출력 12500

설비 50대짜리 공장이면 점검일지만 한 해 1만 2천 건입니다. ★ 이 숫자를 보고 나면 “검색이 왜 필요한지” 가 몸으로 이해됩니다. 사람이 1만 2천 건을 뒤질 수는 없습니다.

문제 3

```
const 문서들 = [
  { 이름: "점검일지", 성격: "계속 쌓임" },
  { 이름: "작업표준서", 성격: "거의 안 바뀜" },
  { 이름: "자재출납대장", 성격: "계속 쌓임" },
  { 이름: "MSDS", 성격: "거의 안 바뀜" },
];

console.log(문서들.filter((하나) => 하나.성격 === "계속 쌓임").map((하나) => 하나.이름));
```

출력 ['점검일지', '자재출납대장']

filter 로 고르고 map 으로 이름만 뽑습니다. ★ 이 둘이 검색이 필요한 문서입니다. 나머지는 개수가 적어 통째로 다뤄도 됩니다. (PART 5)

문제 4

```
const 질문들 = [
  { 질문: "5호기 지난달 이상 있었나요?", 종류: "찾기" },
  { 질문: "절단 작업 순서가 어떻게 되나요?", 종류: "설명" },
  { 질문: "3호기 베어링 마지막 교체일은?", 종류: "찾기" },
  { 질문: "이런 불량 전에도 있었나요?", 종류: "찾기" },
  { 질문: "이번 달 베어링 몇 개 나갔죠?", 종류: "계산" },
  { 질문: "이거 피부에 묻으면 어떡하죠?", 종류: "설명" },
];

const 종류별 = {};
for (const 하나 of 질문들) {
  종류별[하나.종류] = (종류별[하나.종류] ?? 0) + 1;
}

console.log(종류별);
```

출력 { '찾기': 3, '설명': 2, '계산': 1 }

?? 0 이 있어야 처음 나오는 종류에서 undefined + 1 = NaN 이 안 됩니다.

★ 키에 따옴표가 붙은 걸 보셨나요? console.log 는 “따옴표 없이 써도 되는 이름인가” 를 **ASCII 로만** 판단합니다. 한글 키는 JS 문법상 멀쩡한데도 따옴표가 붙습니다. ★★ 보기에만 그렇습니다.

JSON.stringify 나 res.json 으로 내보내면 차이가 없습니다.

★★ 이 결과가 곧 기능 목록입니다. 찾기 3 · 설명 2 → AI 로 합니다 계산 1 → ★ SQL 로 합니다. AI 는 세는 걸 못 믿습니다

문제 5

```
const 글가 = "| 4 | 이상 소음·진동 | 주의 | 하부 미세 소음 |";
const 글나 = "오늘 점검했고 이상 없었습니다.";

function 표형인가(글) {
  const 표문자 = (글.match(/[|ㄴㅍㅈㅊㅊㅊ]/g) ?? []).length;
  return 표문자 >= 5;
}

console.log(표형인가(글가));
```

출력 true

```
console.log(표형인가(글나));
```

출력 false

?? [] 가 중요합니다. match 는 못 찾으면 null 을 돌려줍니다. null.length 를 읽으면 터집니다.

★ 글가에는 | 가 5개 있습니다. 딱 기준에 걸립니다. 기준을 6으로 올렸으면 false 가 됐을 것입니다.

★★★ 이 문제를 만들면서 실제로 한 번 틀렸습니다. 처음에는 " | 1 | 유압유 누유 | 양호 | " 를 예제로 썼습니다. | 가 4개입니다. 기준이 5라서 false 가 나왔는데, 기대 출력은 true 라고 적어 뒀습니다. 돌려 보고 알았습니다.

★ 경계값(5)을 기준으로 쓰면서 예제를 눈대중으로 고르면 이렇게 됩니다.

이런 값은 실제 문서로 **세어 보고** 정해야 합니다.

문제 6

```
const 머리 = { 설비: "5호 컨베이어", 점검일: "2026-08-18" };
const 행1 = { 번호: 2, 항목: "벨트 장력", 결과: "주의", 비교: "늘어짐" };
const 행2 = { 번호: 3, 항목: "안전커버", 결과: "양호", 비교: "" };

function 문장으로(머리, 행) {
  const 뒤 = 행.비교 ? `비교: ${행.비교}.` : "";
  return `${머리.점검일} ${머리.설비} 점검 ${행.번호}번 "${행.항목}" 결과는 ${행.결과}.${뒤}`;
}

console.log(문장으로(머리, 행1));
```

출력 2026-08-18 5호 컨베이어 점검 2번 "벨트 장력" 결과는 주의. 비교: 늘어짐.

```
console.log(문장으로(머리, 행2));
```

출력 2026-08-18 5호 컨베이어 점검 3번 "안전커버" 결과는 양호.

★★ 행.비교 ? ... : "" 가 요점입니다. 빈 값을 그냥 이어 붙이면 "결과는 양호. 비교: ." 이 됩니다. 사람이 읽어도 이상하고, AI 가 읽어도 이상합니다.

★ 그리고 두 문장 다 머리(설비명·날짜)를 안에 담고 있습니다. 이 한 줄만 떼어 내도 뜻이 통합니다. 그게 잘 자른 조각입니다.

문제 7

```
const 기록 = {
  설비명: "7호 절단기",
  점검일: "2026-08-18",
  점검자: "한지민",
  사번: "E20240115",
  연락처: "010-1234-5678",
  종합판정: "가동",
};

const 넘길칸 = ["설비명", "점검일", "종합판정"];

function 골라내기(원본, 허용칸) {
  const 결과 = {};
  for (const 칸 of 허용칸) {
```

```

    }
    return 결과;
  }

  console.log(JSON.stringify(골라내기(기록, 넘길칸)));

```

출력 {"설비명": "7호 절단기", "점검일": "2026-08-18", "종합판정": "가동"}

이름·사번·연락처가 빠졌습니다. 그런데 답하는 데 아무 지장이 없습니다.

★★★ 칸 in 원본 을 확인하는 이유 허용칸에 적어 뒀는데 원본에 없는 칸이 있을 수 있습니다. 그냥 넣으면 {"근무조": undefined} 가 됩니다. JSON.stringify 는 undefined 를 빼 주지만, 다른 곳에서 문제가 됩니다.

문제 8

```

const 기록들 = [
  { 설비명: "3호 프레스", 점검일: "2026-08-18", 조치사항: "" },
  { 설비명: "5호 컨베이어", 점검일: "", 조치사항: "" },
  { 설비명: "", 점검일: "2026-08-17", 조치사항: "확인 요망" },
  { 설비명: "7호 절단기", 점검일: "2026-08-18", 조치사항: "" },
];

const 꼭필요한칸 = ["설비명", "점검일"];

const 문제있는것 = 기록들.filter((하나) => 꼭필요한칸.some((칸) => !하나[칸]));

console.log(문제있는것.length);

```

출력 2

some 은 “하나라도 만족하면 true” 입니다. !하나[칸] 은 빈 문자열도 걸러 냅니다. "" 는 거짓이니까요.

★★ 조치사항이 빈 것은 안 셧습니다. 그게 맞습니다. 조치할 게 없었던 것이니 정상입니다.

★ “빈 칸을 다 문제로 세면” 정상 기록까지 문제로 잡힙니다.

꼭 필요한 칸을 따로 정해 두는 이유입니다.

문제 9

```

const 불편목록 = [
  { 불편: "찾는 데 오래 걸린다", 걸리는시간: 20, 판정: "ㅇ 잘 맞음" },

```

```
{ 불편: "신입이 모른다", 걸리는시간: 15, 판정: "○ 잘 맞음" },
  { 불편: "옛 개정판을 본다", 걸리는시간: 30, 판정: "△ 반쯤" },
  { 불편: "반장님께 물어본다", 걸리는시간: 10, 판정: "○ 잘 맞음" },
  { 불편: "야간에 물어볼 사람 없음", 걸리는시간: 60, 판정: "○ 잘 맞음" },
];

const 합 = 불편목록
  .filter((하나) => 하나.판정.startsWith("○"))
  .reduce((더한값, 하나) => 더한값 + 하나.걸리는시간, 0);

console.log(합);
```

출력 105

20 + 15 + 10 + 60 = 105 분입니다. "옛 개정판" 30분은 빠졌습니다.

★★ reduce 의 마지막 0 을 빼먹으면 안 됩니다. 빼면 첫 번째 요소(객체)가 시작값이 되어 "[object Object]20" 이 나옵니다.

★★★ 이 숫자가 **제안서에 들어갈 근거**입니다. "시 로 105분/사람/일 을 줄일 수 있습니다" 는 판단할 수 있는 말입니다. "편해집니다" 는 판단할 수 없습니다.

손으로 · 푸는 문제 — 정답

문제 10

(1) 왜 이런 일이 생기나

****회사마다 문서를 다르게 부르기 때문****입니다.
 "점검일지" 를 그 회사에서는 "일상점검표" 나 "체크시트" 라고 부릅니다.
 담당자는 "점검일지" 라는 이름의 문서를 찾아보고 없다고 답한 것입니다.

★ 담당자가 불친절한 게 아닙니다. 우리가 잘못 물은 것입니다.

(2) 어떻게 물었어야 하나

이름이 아니라 ****내용과 상황****으로 물어야 합니다.

- × "점검일지 주세요"
- "설비 점검하시고 매일 적으시는 게 있나요?
 체크시트나 일상점검표라고 부르실 수도 있는데요."

★★ 더 좋은 방법: ****실물을 보여 달라고**** 하세요.
 "지금 쓰시는 양식 한 장만 보여 주시겠어요?"
 말로 주고받는 것보다 훨씬 빠르고 정확합니다.

문제 11

(1) 왜 뻘나

****글자를 뽑아내기 어려워서****입니다.

- .hwp 는 한글 전용 형식입니다. 다룰 수 있는 라이브러리가 마땅치 않고, 있어도 안정적이지 않습니다.
- 스캔 사진은 ****글자가 아예 없습니다.**** 그림입니다.
OCR(글자 인식)을 따로 붙여야 하는데, 그건 별개의 큰 작업입니다.

★ 정해진 기간에 끝내려면 ****안 되는 것을 정하는 것****도 설계입니다.
다 하려다 아무것도 못 끝내는 게 제일 흔한 실패입니다.

(2) “우리는 .hwp 만 쓰는데요” 라고 하면

★ 이걸 ****범위 결정을 다시 해야 하는 상황****입니다. 그냥 넘기면 안 됩니다.

할 수 있는 선택

- ① ****PDF 로 내보내 달라고 합니다.**** 한글에서 PDF 저장이 됩니다.
가장 현실적입니다. 현장에 부담이 적습니다.
- ② .hwp 변환을 범위에 넣고 ****다른 기능을 뺍니다.**** 시간은 늘지 않습니다.
- ③ 1차는 PDF 로 하고, .hwp 는 다음 단계로 미룹니다.

**** 절대 하면 안 되는 것: “일단 해 보죠” 하고 넘어가기.**
나중에 안 되면 그때는 되돌릴 시간이 없습니다.

문제 12

왜 위험한가

“뺄 칸” 목록으로 만들었기 때문입니다.

지금은 기록에 점검자·사번만 있으니 잘 됩니다. 그런데 나중에 누가 기록에 칸을 하나 더 넣으면 어떻게 될까요?

```
기록.연락처 = "010-1234-5678";
```

뺄 칸 목록에 “연락처” 가 없으니 **그대로 나갑니다.** 코드는 안 터집니다. 아무도 모릅니다.

★★★ 이런 사고는 조용히 납니다. 에러가 나면 차라리 낫습니다. 잘 도는데 개인정보만 새는 게 제일 나쁩니다.

어떻게 바꾸나

“넣을 칸” 목록으로 바꿉니다.

```
const 넘길칸 = ["설비명", "점검일", "종합판정"];
const 넘길것 = {};
for (const 칸 of 넘길칸) {
  if (칸 in 기록) 넘길것[칸] = 기록[칸];
}
```

이러면 새 칸이 생겨도 **기본으로 안 나갑니다**. 내보내려면 목록에 일부러 적어야 합니다.

★ 보안에서 흔히 쓰는 원칙입니다. “막을 것을 정하기” 보다 “허용할 것을 정하기” 가 안전합니다. PART 5에서 다시 만납니다.

문제 13

(1) 뭐라고 답하나

아니요. 반대입니다.

AI 는 정리를 대신 해 주지 않습니다.

오히려 **문서가 정리돼 있어야 AI 가 제대로 답합니다.**

반장님 표현을 빌리면 이렇게 설명할 수 있습니다.

"AI 는 아주 부지런한 신입 같은 겁니다.

주는 문서는 다 읽고 답해 줍니다.

그런데 옛날 버전을 주면 옛날 내용으로 답합니다.

'이건 옛날 거네' 를 알아서 판단하지는 못합니다."

(2) 그대로 두면 무슨 일이 생기나

· v2 와 v3 가 둘 다 검색됩니다. **점수도 거의 같습니다.** 내용이 비슷하니깐요.

· 운이 나쁘면 v2(옛것)가 1등으로 나와 그걸 근거로 답합니다.

· 둘 다 넘기면 "v2 에는 이렇고 v3 에는 이렇습니다" 같은 답이 나옵니다.

*** 제일 나쁜 것: **출처가 제대로 붙어 있습니다.**

사용자는 근거가 있으니 믿습니다. 그게 폐지본인 줄 모릅니다.

★ PART 5 에서 이걸 실제로 재 보고 확인합니다.

(3) 먼저 해야 할 일

- ① ****폐지된 문서를 치웁니다.**** 지우는 게 제일 확실합니다.
- ② 못 지우면 ****폐지 표시****를 달고 검색에서 거릅니다.
- ③ 문서마다 ****개정일과 판(버전)****을 기록합니다.
- ④ ★ 문서를 고치면 ****색인도 같이 고쳐지게**** 만들어 둡니다.
사람이 기억해서 하는 일로 두면 반드시 빠집니다.

**** ①~③은 **사람이 해야 하는 일**입니다. 개발로 못 때웁니다.**
프로젝트를 시작할 때 이걸 누가 언제 할지 정해 두세요.
안 정하면 개발이 끝나도 서비스가 못 씁니다.

01단원 연습문제 정답 Node 시작하기

RUN node 연습문제_정답.js

01

먼저 스스로 풀어 본 다음에 보세요. 답보다 '왜 그런지' 설명을 꼭 읽으세요.

문제 1

```
console.log(typeof window, typeof document, typeof alert);
```

출력 undefined undefined undefined

셋 다 브라우저가 주던 것입니다. Node 에는 화면이 없어서 없습니다.

문제 2

```
const prices = [1500, 4000, 2500, 6000];

console.log(prices.filter((price) => price >= 3000));
```

출력 [4000, 6000]

filter 는 브라우저에서 쓰던 것 그대로입니다. Node 로 왔다고 새 문법을 배우는 게 아닙니다.

문제 3

```
console.log(typeof process.platform === "string");
```

출력 true

윈도우면 "win32", 맥이면 "darwin", 리눅스면 "linux" 입니다. 나중에 EC2 에 올리면 "linux" 가 나옵니다.

문제 4

```
const args = process.argv.slice(2);
console.log(args);
```

출력 []

slice(2) 로 앞의 두 개(Node 위치, 파일 위치)를 잘라냈습니다.

node 연습문제_정답.js 커피 3 으로 실행하면 ['커피', '3'] 이 나옵니다. 숫자 3을 넘겼는데 '3' 이라는 문자열입니다. 터미널에서 넘어오는 값은 언제나 문자열입니다.

문제 5

```
const calc = require("./계산기");
console.log(calc.TAX_RATE);
```

출력 0.1

./ 가 있으면 "내가 만든 파일" 입니다. .js 는 생략해도 됩니다.

문제 6

```
const { add, addTax } = require("./계산기");

console.log(add(100, 200));
```

출력 300

```
console.log(addTax(20000));
```

출력 22000

실무에서 가장 많이 보는 형태입니다. `const express = require("express")` 처럼 통째로 받기도 하고,

```
const { Router } = require("express")
```

 처럼 필요한 것만 꺼내기도 합니다.

문제 7

```
const { SECRET_KEY } = require("./계산기");
console.log(SECRET_KEY);
```

출력 undefined

에러가 아닙니다. 조용히 `undefined` 입니다. 그래서 더 헛갈립니다. 계산기.js 의 `module.exports` 에 `SECRET_KEY` 가 없기 때문입니다.

“분명 만들었는데 `undefined` 가 나온다” 면 그 파일의 `module.exports` 를 먼저 확인하세요.

문제 8

```
const path = require("path");
console.log(path.join("uploads", "2026", "report.pdf"));
```

출력 uploads\2026\report.pdf

./ 없이 이름만 썼습니다. 내 파일이 아니라 Node 내장 모듈이니까요.

왜 직접 “uploads/2026/report.pdf” 라고 안 쓰나 윈도우는 \ 를, 리눅스는 / 를 씁니다. 내 컴퓨터(윈도우)에서 되던 코드가 EC2(리눅스)에서 깨질 수 있습니다. `path.join` 은 그 차이를 알아서 맞춰 줍니다. 경로를 다룰 때는 항상 `path` 를 쓰세요.

문제 9

```
const pkg = require("./package.json");
console.log(pkg.name, pkg.version);
```

출력 01-node-start 1.0.0

CommonJS에서는 JSON 파일도 `require` 로 읽을 수 있습니다. 읽으면 그냥 객체가 됩니다.

문제 10

```
console.log(Object.keys(pkg.scripts));
```

출력 ['start', 'hello']

여기에 적힌 이름으로 `npm run` 이름 을 실행할 수 있습니다. 팀원이 “어떻게 실행해요?” 라고 물으면 여기를 보라고 하면 됩니다.

문제 11

```
const dayjs = require("dayjs");
console.log(dayjs("2026-03-05").format("YYYY년 M월 D일"));
```

출력 2026년 3월 5일

`npm i dayjs` 로 설치했기 때문에 ./ 없이 이름만 써도 찾아냅니다. `node_modules` 안을 뒤져 줍니다.

MM 과 M 의 차이 MM → 03 (두 자리로 맞춤) M → 3 상황에 맞게 고르면 됩니다.

문제 12

```
try {
  console.log(없는변수);
} catch (error) {
  console.log(error.name);
}
```

출력 ReferenceError

```
}
```

error.name 은 종류, error.message 는 설명입니다. 터미널에 빨강게 뜨는 것은 이 둘을 합친 모양입니다.

문제 13

```
const order1 = { id: 1, customer: { city: "부산" } };
const order2 = { id: 2 };

console.log(order1.customer?.city || "정보 없음");
```

출력 부산

```
console.log(order2.customer?.city || "정보 없음");
```

출력 정보 없음

?. 가 없으면 어떻게 되나 order2.customer 는 `undefined` 입니다. `undefined.city` 를 읽으려다 `TypeError` 로 프로그램이 죽습니다.

?. 는 "왼쪽이 없으면 더 들어가지 말고 `undefined` 를 내놔라" 입니다. `||` 는 그 `undefined` 를 "정보 없음" 으로 바꿔 줍니다. 역할이 달라서 둘 다 필요합니다.

★ 백엔드에서 이 패턴을 매일 씁니다. 요청으로 들어온 데이터, 데이터베이스에서 꺼낸 데이터에는 비어 있는 자리가 항상 있습니다.

문제 14

```
function safeFixed(value) {
  try {
    return value.toFixed(2);
  } catch (error) {
    console.log("실패 이유:", error.message);
    return "계산 불가";
  }
}

console.log(safeFixed(3.14159));
```

출력 3.14

```
console.log(safeFixed("문자열"));
```

출력 실패 이유: value.toFixed is not a function
계산 불가

```
console.log(safeFixed(null));
```

출력 실패 이유: Cannot read properties of null (reading 'toFixed')
계산 불가

출력 순서에 주의하세요 `console.log(safeFixed("문자열"))` 은 안쪽 함수가 먼저 실행됩니다. 그래서 함수 안의 "실패 이유:" 가 먼저 찍히고, 돌려준 "계산 불가" 가 나중에 찍힙니다.

에러 종류가 두 개인 것에 주목하세요 "문자열" → `TypeError: value.toFixed is not a function` (그런 메소드가 없다) `null` → `TypeError: Cannot read properties of null` (아예 값이 없다) 원인은 다르지만 처리 방식은 같습니다.

`catch` 를 비워 두면 안 되는 이유

```
catch (error) {}
```

라고만 하면 에러가 조용히 사라집니다.

나중에 "왜 계산 불가가 나오지?" 를 절대 못 찾습니다. 최소한 `message` 는 남기세요.

문제 15

```
const wrong = require("계산기");
```

이 코드는 검사에서 걸립니다

왜:

./ 를 빼먹었습니다.

./ 가 없으면 Node 는 "내가 만든 파일" 이 아니라

"npm 으로 설치한 패키지" 를 node_modules 에서 찾습니다.

거기에 '계산기' 라는 패키지는 없으니 못 찾습니다.

고치는 방법:

```
const right = require("./계산기");
```

이 에러를 보면 확인할 것 두 가지

- ① ./ 로 시작하나? → 내 파일인데 빠뜨린 것
- ② npm install 을 했나? → 패키지인데 설치를 안 한 것

Error: Cannot find module '계산기'

02단원 연습문제 정답 파일 다루기

RUN node 연습문제_정답.js

먼저 스스로 풀어 본 다음에 보세요. 설명을 꼭 읽으세요.

```
const fs = require("fs");
const path = require("path");

const 작업폴더 = path.join(__dirname, "연습_작업");
fs.rmSync(작업폴더, { recursive: true, force: true });
fs.mkdirSync(작업폴더, { recursive: true });
```

문제 1

```
console.log(path.join("uploads", "2026", "03", "보고서.pdf"));
```

출력 uploads\2026\03\보고서.pdf

(윈도우 기준입니다. 맥·리눅스는 uploads/2026/03/보고서.pdf 로 나옵니다) 조각이 몇 개든 상관없습니다. 구분자는 path 가 알아서 골라 줍니다.

문제 2

```
const 파일명1 = "작업표준서.PDF";

console.log(path.extname(파일명1).toLowerCase() === ".pdf");
```

출력 true

toLowerCase() 를 빼면 ".PDF" === ".pdf" 라서 false 가 나옵니다. 사용자가 어떤 대소문자로 올릴지 모릅니다. 항상 낮춰서 비교하세요. 점을 빼먹고 "pdf" 와 비교하는 것도 흔한 실수입니다.

문제 3

```
const 확장자1 = path.extname(파일명1);
console.log(path.basename(파일명1, 확장자1));
```

출력 작업표준서

basename(이름, 땀확장자) 입니다. 두 번째 인자를 ".pdf" 로 하면 대문자 ".PDF" 는 안 떨어집니다.
extname 이 준 것을 그대로 넘기는 게 안전합니다.

문제 4

```
const 샘플경로 = path.join(__dirname, "샘플_문서.txt");
const 내용 = fs.readFileSync(샘플경로, "utf-8");

console.log(내용.split("\n")[0]);
```

출력 작업표준서

path.join(__dirname, ...) 을 쓴 이유 그냥 "샘플_문서.txt" 라고 쓰면 터미널 위치에 따라 ENOENT 가 납니다. __dirname 은 이 파일이 있는 폴더라서 어디서 실행하든 같습니다.

문제 5

```
console.log(내용.split("\n").filter((line) => line !== "").length);
```

출력 4

파일 끝에 \n 이 있어서 split 하면 마지막에 빈 문자열이 하나 생깁니다. filter 로 걸러 내야 4가 나옵니다. 안 거르면 5입니다.

문제 6

```
const 메모경로 = path.join(작업폴더, "메모.txt");

fs.writeFileSync(메모경로, "첫 줄\n", "utf-8");
console.log(fs.existsSync(메모경로));
```

출력 true

문제 7

```
fs.appendFileSync(메모경로, "둘째 줄\n", "utf-8");
fs.appendFileSync(메모경로, "셋째 줄\n", "utf-8");

const 메모내용 = fs.readFileSync(메모경로, "utf-8");
console.log(메모내용.trim().split("\n").length);
```

출력 3

writeFileSync 를 썼다면 앞의 내용이 사라져 1이 나옵니다. 이어 쓰는 것은 appendFileSync 입니다. trim() 은 파일 끝의 \n 때문에 생기는 빈 줄을 없애려고 붙였습니다.

문제 8

```
try {
  fs.readFileSync(path.join(작업폴더, "없는파일.txt"), "utf-8");
} catch (error) {
  console.log(error.code);
}
```

출력 ENOENT

```
}
```

try/catch 로 감쌌기 때문에 프로그램이 안 죽고 계속 갑니다. 감싸지 않았다면 여기서 멈추고 아래 문제들이 실행되지 않습니다.

문제 9

```
const 깊은폴더 = path.join(작업폴더, "docs", "2026", "03");

fs.mkdirSync(깊은폴더, { recursive: true });
console.log(fs.existsSync(깊은폴더));
```

출력 true

recursive 없이 하면 docs 가 없어서 ENOENT 가 납니다. 한 단계씩 세 번 만들 필요 없습니다.

문제 10

```
fs.writeFileSync(path.join(작업폴더, "a.txt"), "A", "utf-8");
fs.writeFileSync(path.join(작업폴더, "b.pdf"), "B", "utf-8");
fs.writeFileSync(path.join(작업폴더, "c.txt"), "C", "utf-8");

const txt목록 = fs
  .readdirSync(작업폴더)
  .filter((name) => path.extname(name) === ".txt")
  .sort();

console.log(txt목록);
```

```
출력      [ 'a.txt', 'c.txt', '메모.txt' ]
```

문제 6에서 만든 메모.txt 도 .txt 라서 함께 나옵니다. 한글이 알파벳 뒤에 오는 것은 정렬 규칙 때문입니다. readdirSync 는 '이름' 만 줍니다. 전체 경로가 아니라는 점에 주의하세요.

문제 11

```
const 폴더만 = fs
  .readdirSync(작업폴더, { withFileTypes: true })
  .filter((entry) => entry.isDirectory())
  .map((entry) => entry.name);

console.log(폴더만);
```

```
출력      [ 'docs' ]
```

withFileTypes: true 를 주면 이름 대신 '정보가 담긴 객체' 가 나옵니다. statSync 를 파일마다 부르는 것보다 훨씬 빠릅니다.

문제 12

```
const 설비들 = [
  { id: 1, name: "컨베이어", line: "A" },
  { id: 2, name: "프레스", line: "B" },
];

const 설비파일 = path.join(작업폴더, "설비.json");

fs.writeFileSync(설비파일, JSON.stringify(설비들, null, 2), "utf-8");

const 읽은설비 = JSON.parse(fs.readFileSync(설비파일, "utf-8"));
console.log(읽은설비[1].name);
```

출력 프레스

stringify 를 빼먹으면 "[object Object]" 가 저장되어 내용이 사라집니다. parse 를 빼먹으면 문자열이라 [1] 이 두 번째 '글자' 가 됩니다.

문제 13

```
function makeSavePath(원본이름, 날짜) {
  const 확장자 = path.extname(원본이름).toLowerCase();
  const 이름 = path.basename(원본이름, path.extname(원본이름));

  const 년 = 날짜.getFullYear();
  const 월 = String(날짜.getMonth() + 1).padStart(2, "0");
  const 일 = String(날짜.getDate()).padStart(2, "0");

  const 폴더 = path.join(작업폴더, String(년), 월);
  fs.mkdirSync(폴더, { recursive: true });

  return path.join(폴더, `${이름}_${년}${월}${일}${확장자}`);
}

const 저장경로 = makeSavePath("검사성적서.XLSX", new Date(2026, 2, 5));

console.log(path.relative(__dirname, 저장경로));
```

출력 연습_작업\2026\03\검사성적서_20260305.xlsx

(윈도우 기준입니다. 맥·리눅스는 / 로 이어집니다)

```
console.log(fs.existsSync(path.dirname(저장경로)));
```

출력 true

두 가지를 눈여겨보세요.

① extname 을 두 번 부릅니다

basename 에 넘길 때는 '원래 대소문자' 여야 떨어집니다. (.XLSX)
파일명에 붙일 때는 소문자로 바꿔 씁니다. (.xlsx)
순서를 바꾸면 이름이 "검사성적서.XLSX" 처럼 안 떨어집니다.

② 폴더를 먼저 만들고 경로를 돌려줍니다

경로만 돌려주고 폴더를 안 만들면, 나중에 쓸 때 ENOENT 가 납니다.
"경로를 만드는 함수" 가 폴더까지 책임지는 게 안전합니다.

path.relative(기준, 경로) 는 "기준에서 본 상대 경로" 를 줍니다. 컴퓨터마다 다른 앞부분을 빼고 보려고 썼습니다.

★ 09단원에서 multer 로 파일 업로드를 만들 때 이 함수를 거의 그대로 씁니다.

문제 14

비동기라서 async 함수로 감쌌습니다. 이 함수는 파일 맨 아래에서 부릅니다. 그래서 출력이 가장 마지막에 나옵니다.

```
async function main() {
  async function 불러오기() {
    try {
      const 글자 = await fs.promises.readFile(설비파일, "utf-8");
      return JSON.parse(글자);
    } catch (error) {
      if (error.code === "ENOENT") {
        return []; // 파일이 없으면 빈 목록
      }
      throw error; // 그 밖의 에러는 그대로 넘김
    }
  }

  async function 저장하기(목록) {
    await fs.promises.writeFile(설비파일, JSON.stringify(목록, null, 2), "utf-8");
  }

  async function 추가하기(name, line) {
    const 목록 = await 불러오기();
```

03단원 연습문제 정답 HTTP와 서버

```
RUN node 연습문제_정답.js
```

먼저 스스로 풀어 본 다음에 보세요. 설명을 꼭 읽으세요.

문제 1

<pre>const 주소글자 = "http://localhost:3000/api/documents?page=2&size=10"; const 주소 = new URL(주소글자); console.log(주소.pathname);</pre>	
출력	/api/documents
코드	▶ 출력
<pre>console.log(주소.host);</pre>	▶ localhost:3000
<pre>console.log(주소.port);</pre>	▶ 3000
<pre>console.log(typeof 주소.port);</pre>	▶ string

port 가 문자열인 것에 주의하세요. URL 에서 나오는 것은 거의 다 글자입니다. host 는 포트를 포함하고, hostname 은 포함하지 않습니다. 주소.host → localhost:3000 주소.hostname → localhost

문제 2

코드	▶ 출력
<pre>console.log(주소.searchParams.get("page"));</pre>	▶ 2
<pre>console.log(주소.searchParams.get("sort"));</pre>	▶ null
<pre>console.log(주소.searchParams.has("sort"));</pre>	▶ false

없는 쿼리는 null 입니다. undefined 가 아닙니다. “있는지 없는지” 만 알고 싶으면 has 가 더 분명합니다.
위의 2 는 숫자가 아니라 글자 “2” 입니다. console.log 는 글자를 찍을 때 따옴표를 안 붙여서 똑같이
보입니다. 배열 안에 넣어서 찍어 보면 구별됩니다. console.log([주소.searchParams.get("page")])

문제 3

```
function 페이지번호(주소) {
  const 값 = 주소.searchParams.get("page");
  const 번호 = Number(값);

  if (!Number.isInteger(번호) || 번호 < 1) {
    return 1; // 이상하면 그냥 1쪽으로
  }
  return 번호;
}

console.log(페이지번호(new URL("http://localhost:3000/x?page=3")));
```

출력 3

```
console.log(페이지번호(new URL("http://localhost:3000/x")));
```

출력 1

```
console.log(페이지번호(new URL("http://localhost:3000/x?page=abc")));
```

출력 1

```
console.log(페이지번호(new URL("http://localhost:3000/x?page=0")));
```

출력 1

★ 여기 함정이 하나 있습니다. page 가 없으면 get 은 null 을 줍니다. 그런데 Number(null) 은 0 입니다. NaN 이 아닙니다! 그래서 Number.isInteger(0) 은 true 가 되어 통과해 버립니다.

막아 주는 것은 뒤의 번호 < 1 조건입니다. 이게 없으면 0쪽을 보여 주려다 목록이 텅 비는 버그가 납니다. 실제로 아주 흔한 버그입니다.

Number("") → 0 (빈 글자도 0입니다)

```
Number(null) → 0
Number("abc") → NaN
Number(undefined) → NaN
```

“숫자로 바꿨으니 됐다” 가 아니라 “범위까지 확인했다” 여야 안전합니다.

문제 4

```
const 주소4 = new URL("http://localhost:3000/search?tag=안전&tag=품질&q=점검");
```

```
console.log(주소4.searchParams.get("tag"));
```

출력 안전

```
console.log(주소4.searchParams.getAll("tag"));
```

출력 ['안전', '품질']

```
console.log(주소4.searchParams.getAll("없는것"));
```

출력 []

같은 이름이 여러 번 올 수 있습니다. 체크박스를 여러 개 고른 경우입니다. get → 첫 번째 하나만 getAll → 전부 배열로. 없으면 빈 배열 (null 이 아닙니다)

여러 개가 올 수 있는 값이라면 처음부터 getAll 을 쓰세요. get 만 쓰면 두 번째부터 조용히 사라져서 원인을 찾기 아주 어렵습니다.

문제 5

```
const 상황별코드 = {
  "목록을 잘 돌려줌": 200,
  "새 문서를 만들": 201,
  "지웠고 돌려줄 내용이 없음": 204,
  "title 을 안 보냄": 400,
  "로그인을 안 함": 401,
  "로그인은 했지만 권한이 없음": 403,
  "그 번호의 문서가 없음": 404,
  "내 코드에서 TypeError 가 남": 500,
};

for (const 상황 of Object.keys(상황별코드)) {
  console.log(`${상황} → ${상황별코드[상황]}`);
}
```

출력 목록을 잘 돌려줌 → 200
 새 문서를 만들 → 201
 지웠고 돌려줄 내용이 없음 → 204
 title 을 안 보냄 → 400
 로그인 안 함 → 401
 로그인 했지만 권한이 없음 → 403
 그 번호의 문서가 없음 → 404
 내 코드에서 TypeError 가 남 → 500

헛갈리기 쉬운 두 쌍만 다시 봅니다.

401 vs 403

401 = 너 누구니? (증표를 안 냈다)
 403 = 누군지는 알겠는데, 너는 안 된다 (권한이 없다)

400 vs 500

400 = 네가 잘못 보냈다
 500 = 내가 잘못 짰다
 이걸 바꿔 쓰면 상대가 원인을 엉뚱한 데서 찾습니다.

문제 6

```
function 분류(코드) {
  if (코드 >= 200 && 코드 < 300) return "성공";
  if (코드 >= 300 && 코드 < 400) return "다른 곳으로";
  if (코드 >= 400 && 코드 < 500) return "보낸 쪽 잘못";
  if (코드 >= 500) return "서버 잘못";
  return "알 수 없음";
}

console.log([200, 204, 301, 404, 500].map(분류).join(" / "));
```

출력 성공 / 성공 / 다른 곳으로 / 보낸 쪽 잘못 / 서버 잘못

첫 자리 숫자만 보면 됩니다. 2로 시작하면 성공, 4면 보낸 쪽, 5면 서버. 세 자리를 다 외울 필요 없습니다.
 자주 쓰는 여덟 개만 알면 충분합니다.

문제 7

```
function 조각내기(경로) {
  return 경로.split("/").filter(Boolean);
}

console.log(조각내기("/documents"));
```

```
출력      [ 'documents' ]
```

```
console.log(조각내기("/documents/12"));
```

```
출력      [ 'documents', '12' ]
```

```
console.log(조각내기("/documents/12/files"));
```

```
출력      [ 'documents', '12', 'files' ]
```

```
console.log(조각내기("/"));
```

```
출력      []
```

filter(Boolean) 이 없으면 맨 앞에 빈 글자가 들어옵니다. `"/documents".split("/")` → `[ '', 'documents' ]`

Boolean 을 넘긴다는 게 이상해 보이면 이렇게 읽으세요. `filter((조각) ⇒ Boolean(조각))` ← 빈 글자는 `false` 라서 버려집니다 짧게 쓴 것뿐입니다. (JS자료 08단원)

문제 8

```
function 경로분석(경로) {
  const 조각 = 경로.split("/").filter(Boolean);

  return {
    resource: 조각[0] ?? null,
    id: 조각[1] ? Number(조각[1]) : null,
  };
}

console.log(경로분석("/documents"));
```

```
출력      { resource: 'documents', id: null }
```

```
console.log(경로분석("/documents/12"));
```

```
출력      { resource: 'documents', id: 12 }
```

```
console.log(경로분석("/"));
```

```
출력      { resource: null, id: null }
```

id 가 12 (따옴표 없음) 인 것을 보세요. Number 로 바꿨기 때문입니다. resource 는 'documents' (따옴표 있음) 라 글자입니다. 콘솔에서 따옴표만 봐도 글자인지 숫자인지 알 수 있습니다.

★ 속성 이름을 왜 영어로 썼나

node 는 객체를 찍을 때 한글 속성 이름에 따옴표를 붙입니다.

```
{ '자원': 'documents' }
```

틀린 건 아니지만 지저분해 보여서, 객체 키는 영어로 쓰는 편이 낫습니다. 실무에서도 키는 영어로 씁니다. 프론트와 주고받는 이름이니깐요.

★ 04단원에서 Express 를 쓰면 이 함수가 통째로 사라집니다. app.get("/documents/:id", ...) 라고만 쓰면 알아서 꺼내 줍니다.

문제 9

```
const headers = {  
  host: "localhost:3000",  
  "content-type": "application/json; charset=utf-8",  
  "user-agent": "PostmanRuntime/7.36.0",  
};
```

```
console.log(headers["Content-Type"]);
```

```
출력      undefined
```

```
console.log(headers["content-type"]);
```

```
출력      application/json; charset=utf-8
```

```
function 헤더값(headers, 이름) {  
  return headers[이름.toLowerCase()];  
}
```

```
console.log(헤더값(headers, "Content-Type"));
```

```
출력      application/json; charset=utf-8
```

```
console.log(헤더값(headers, "USER-AGENT"));
```

출력 PostmanRuntime/7.36.0

Node 는 받은 헤더 이름을 전부 소문자로 바꿔서 답습니다. 브라우저가 Content-Type 이라고 보내도 content-type 으로 들어옵니다.

헤더가 `undefined` 로 나오면 없는 게 아니라 이름을 잘못 찾은 것일 수 있습니다. 무엇이 들어왔는지 통째로 찍어 보는 게 가장 빠릅니다.

```
console.log(req.headers)
```

문제 10

```
function JSON본문인가(headers) {
  const 형식 = headers["content-type"] || "";
  return 형식.includes("application/json");
}

console.log(JSON본문인가({ "content-type": "application/json; charset=utf-8" }));
```

출력 true

```
console.log(JSON본문인가({ "content-type": "application/json" }));
```

출력 true

```
console.log(JSON본문인가({ "content-type": "text/plain" }));
```

출력 false

```
console.log(JSON본문인가({}));
```

출력 false

★ `===` 로 비교하면 안 되는 이유 실제로 오는 값은 `"application/json; charset=utf-8"` 인 경우가 많습니다. `===` `"application/json"` 은 여기서 `false` 가 됩니다. `includes` 로 "들어 있나" 를 봐야 합니다.

★ `|| ""` 를 붙인 이유 헤더가 아예 없으면 `undefined` 입니다. `undefined.includes(...)` 는 `TypeError` 로 서버를 죽입니다. 빈 글자로 바꿔 두면 그냥 `false` 가 나옵니다.

문제 11

```
function 빠진값찾기(데이터, 필수들) {  
  return 필수들.filter((키) => !데이터[키]);  
}  
  
console.log(빠진값찾기({ title: "작업표준서" }, ["title", "type", "author"]));
```

출력 ['type', 'author']

```
console.log(빠진값찾기({ title: "a", type: "b", author: "c" }, ["title", "type",  
"author"]));
```

출력 []

```
const 빠진것 = 빠진값찾기({ title: "작업표준서" }, ["title", "type"]);  
  
if (빠진것.length === 0) {  
  console.log(201);  
} else {  
  console.log(`400 ${빠진것.join(", ")} 을(를) 넣어 주세요`);  
}
```

출력 400 type 을(를) 넣어 주세요

★ 왜 “첫 번째로 빠진 것” 이 아니라 전부 모으나 하나씩 알려 주면 사용자가 고치고 → 또 틀리고 → 또 고치고를 반복합니다. 빠진 것을 한 번에 다 알려 주는 게 훨씬 친절합니다.

★ !데이터[키] 의 한계 숫자 0 이나 빈 글자 "" 를 ‘안 보낸 것’ 으로 봅니다. 수량: 0 을 보냈는데 “수량을 넣어 주세요” 가 나오면 이것 때문입니다. 정확히 하려면 데이터[키] === undefined 로 봐야 합니다. 지금은 title 같은 글자만 다루니 이대로 두고, 함정만 기억해 두세요.

문제 12

```
const 고친주소 = [
  "GET /documents",
  "POST /documents",
  "GET /documents/3",
  "DELETE /documents/3",
  "GET /documents?type=표준",
];

for (const 줄 of 고친주소) {
  console.log(줄);
}
```

출력

```
GET /documents
POST /documents
GET /documents/3
DELETE /documents/3
GET /documents?type=표준
```

규칙 세 가지만 지키면 됩니다.

① 주소에는 동사를 쓰지 않습니다

무엇을 할지는 메서드가 말합니다. 주소는 '무엇을' 만 말합니다.
/getDocuments x GET /documents o

② 자원 이름은 복수형 명사입니다

/document x /documents o
한 개를 가리킬 때도 /documents/3 입니다. /document/3 이 아닙니다.

③ 하나를 꼭 집을 때는 경로, 걸러 볼 때는 쿼리입니다

/documents/3 3번 문서 하나 (없으면 404)
/documents?type=표준 조건에 맞는 것들 (없으면 200 + [])

이 규칙을 따르는 주소 설계를 REST 라고 부릅니다. 06단원에서 자세히 봅니다. 지금은 “동사 쓰지 말자” 하나만 기억해도 충분합니다.

문제 13

```
const 나쁜주소 = new URL("/documents?page=2");
console.log(나쁜주소.pathname);
```

이 코드는 검사에서 걸립니다

왜:

URL 은 '완전한 주소' 를 요구합니다.

/documents 만으로는 어느 컴퓨터의 어느 포트인지 알 수가 없습니다.

http:// 로 시작하는 앞부분이 반드시 있어야 합니다.

어떻게 고치나:

두 번째 인자로 '기준 주소' 를 주면 됩니다.

```
new URL("/documents?page=2", "http://localhost:3000")
```

서버에서는 req.url 이 항상 /documents 같은 반쪽짜리로 옵니다.

그래서 개념03-04-05 에서 늘 이렇게 썼습니다.

```
new URL(req.url, `http://${req.headers.host}`)
```

host 헤더에는 브라우저가 접속한 주소가 들어 있습니다.

localhost:3000 이든 회사 서버 주소든 알아서 맞춰줍니다.

TypeError: Invalid URL

```
const 고친주소13 = new URL("/documents?page=2", "http://localhost:3000");
console.log(고친주소13.pathname);
```

출력 /documents

```
console.log(고친주소13.href);
```

출력 http://localhost:3000/documents?page=2

03단원 연습문제 서버 정답 HTTP와 서버

RUN node 연습문제_서버_정답.js

끄려면 Ctrl + C

먼저 스스로 만들어 본 다음에 보세요.

```
const http = require("http");

const PORT = process.env.PORT || 3000;
```

준비

```
let 설비들 = [
  { id: 1, name: "컨베이어", line: "A", status: "가동" },
  { id: 2, name: "프레스", line: "B", status: "정지" },
];

let 다음번호 = 3;

const 상태값들 = ["가동", "정지"];

function JSON응답(res, 상태코드, 데이터) {
  res.writeHead(상태코드, { "Content-Type": "application/json; charset=utf-8" });
  res.end(JSON.stringify(데이터));
}

function 본문읽기(req) {
  return new Promise((resolve, reject) => {
    let 쌓은글자 = "";
    req.setEncoding("utf-8");
    req.on("data", (조각) => (쌓은글자 += 조각));
    req.on("end", () => resolve(쌓은글자));
    req.on("error", reject);
  });
}
```

본문을 읽고 JSON 으로 바꾸는 것까지 한 번에 합니다. 세 번 넘게 반복될 코드라 미리 묶었습니다. 형식이 틀리면 null 을 돌려줍니다. 부르는 쪽에서 400 을 내면 됩니다.

```

async function JSON본문(req) {
  const 글자 = await 본문읽기(req);
  try {
    return JSON.parse(글자 || "{}");
  } catch {
    return null;
  }
}

const server = http.createServer(async (req, res) => {
  console.log(`${req.method} ${req.url}`);

  const 주소 = new URL(req.url, `http://${req.headers.host}`);
  const 경로 = 주소.pathname;

```

문제 1 /health

```

if (경로 === "/health") {
  JSON응답(res, 200, { status: "ok" });
  return;
}

```

GET /health

200 {"status":"ok"}

서버가 살아 있는지 확인하는 주소입니다. PART 4 에서 배포할 때, 이 주소가 200 을 주는지로 서버 상태를 감시합니다.

```

const 조각들 = 경로.split("/").filter(Boolean);
const 첫조각 = 조각들[0];
const 번호글자 = 조각들[1];

if (첫조각 !== "equipments") {
  JSON응답(res, 404, { error: "그런 주소가 없습니다" });
  return;
}

```

GET /없는주소

404 {"error":"그런 주소가 없습니다"}

문제 2 목록 + 라인으로 걸러 보기

```
if (번호글자 === undefined && req.method === "GET") {
  const 라인 = 주소.searchParams.get("line");
  const 결과 = 라인 ? 설비들.filter((설비) => 설비.line === 라인) : 설비들;

  JSON응답(res, 200, 결과);
  return;
}
```

GET /equipments

200 [{"id":1,"name":"컨베이어","line":"A","status":"가동"}, {"id":2,"name":"프레스","line":"B","status":"정지"}]

GET /equipments?line=A

200 [{"id":1,"name":"컨베이어","line":"A","status":"가동"}]

조건에 맞는 게 없으면 빈 배열 [] 과 200 입니다. 404 가 아닙니다. ?line=Z 로 시험해 보세요.

문제 3 하나만 보기

```
if (번호글자 !== undefined && req.method === "GET") {
  const 번호 = Number(번호글자);

  if (!Number.isInteger(번호)) {
    JSON응답(res, 400, { error: "번호는 숫자여야 합니다" });
    return;
  }
}
```

GET /equipments/abc

400 {"error":"번호는 숫자여야 합니다"}

```
const 설비 = 설비들.find((설비) => 설비.id === 번호);

if (!설비) {
  JSON응답(res, 404, { error: `${번호}번 설비가 없습니다` });
  return;
}
```

GET /equipments/99

404 {"error": "99번 설비가 없습니다"}

```
JSON응답(res, 200, 설비);
  return;
}
```

GET /equipments/1

200 {"id":1,"name":"컨베이어","line":"A","status":"가동"}

★ 숫자 검사를 먼저, 찾기를 나중에 하는 순서가 중요합니다. 순서를 바꾸면 /equipments/abc 에 404 가 나옵니다. "없는 설비" 와 "잘못 쓴 주소" 는 다른 문제입니다.

문제 4 새로 만들기

```
if (번호글자 === undefined && req.method === "POST") {
  const 데이터 = await JSON본문(req);

  if (데이터 === null) {
    JSON응답(res, 400, { error: "JSON 형식이 아닙니다" });
    return;
  }
}
```

필수 값이 여러 개라 빠진 것을 모아서 알려 줍니다. (연습문제.js 문제 11)

```
const 빠진것 = ["name", "line"].filter((키) => !데이터[키]);

if (빠진것.length > 0) {
  JSON응답(res, 400, { error: `${빠진것.join(", ")} 을(를) 넣어 주세요` });
  return;
}
```

POST /equipments

{"name": "용접로봇"}

400 {"error": "line 을(를) 넣어 주세요"}

```
const 새설비 = {
  id: 다음번호,
  name: 데이터.name,
  line: 데이터.line,
  status: "정지", // 새로 등록한 설비는 일단 정지 상태
}
```

```

};

다음번호 += 1;
설비들.push(새설비);

res.writeHead(201, {
  "Content-Type": "application/json; charset=utf-8",
  Location: `/equipments/${새설비.id}`,
});
res.end(JSON.stringify(새설비));
return;
}

```

POST /equipments

```
{ "name": "용접로봇", "line": "C" }
```

201 { "id": 3, "name": "용접로봇", "line": "C", "status": "정지" }

★ status 를 보낸 쪽이 정하게 두지 않았습니다. 서버가 규칙을 쥐고 있어야 합니다. 보낸 값을 그대로 믿으면 누군가 { "status": "폭발" } 을 보내는 순간 데이터가 망가집니다.

문제 5 상태만 바꾸기

```

if (번호글자 !== undefined && req.method === "PATCH") {
  const 번호 = Number(번호글자);
  const 설비 = 설비들.find((설비) => 설비.id === 번호);

  if (!설비) {
    JSON응답(res, 404, { error: `${번호}번 설비가 없습니다` });
    return;
  }

  const 데이터 = await JSON본문(req);

  if (데이터 === null) {
    JSON응답(res, 400, { error: "JSON 형식이 아닙니다" });
    return;
  }
}

```

정해진 값만 받습니다.

```
if (!상태값들.includes(데이터.status)) {
  JSON응답(res, 400, { error: "status 는 가동 또는 정지여야 합니다" });
  return;
}
```

```
PATCH /equipments/3
{"status":"이상한값"}
```

```
400 {"error":"status 는 가동 또는 정지여야 합니다"}
```

```
설비.status = 데이터.status;

JSON응답(res, 200, 설비);
return;
}
```

```
PATCH /equipments/3
{"status":"가동"}
```

```
200 {"id":3,"name":"용접로봇","line":"C","status":"가동"}
```

★ PUT 과 PATCH 의 차이 PUT 전체를 통째로 바꾼다 → 안 보낸 값은 사라지는 게 원칙 PATCH 일부만 바꾼다 → 보낸 것만 바뀐다 “상태만 바꾼다” 는 PATCH 가 어울립니다.

★ includes 로 값을 제한한 이유 status 에 아무 글자나 들어가면 나중에 “가동” 을 세는 코드가 다 틀립니다. 받을 수 있는 값이 정해져 있으면 반드시 목록으로 막으세요. (TS 를 배우면 이걸 타입으로도 막을 수 있습니다)

문제 6 지우기

```
if (번호글자 !== undefined && req.method === "DELETE") {
  const 번호 = Number(번호글자);
  const 있나 = 설비들.some((설비) => 설비.id === 번호);

  if (!있나) {
    JSON응답(res, 404, { error: `${번호}번 설비가 없습니다` });
    return;
  }

  설비들 = 설비들.filter((설비) => 설비.id !== 번호);

  res.writeHead(204);
}
```

```
res.end();
  return;
}
```

DELETE /equipments/3

204

DELETE /equipments/3

404 {"error": "3번 설비가 없습니다"}

★ 같은 요청을 두 번 보냈더니 답이 달라졌습니다. 이게 맞을까요? “지워라” 를 두 번 보내도 결과(그 설비가 없는 상태)는 똑같습니다. 그래서 두 번째에 204 를 주는 서버도 많습니다. 둘 다 틀리지 않습니다.

여기서는 “없는 걸 지우라고 했으니 알려 주자” 쪽을 골랐습니다. 중요한 건 어느 쪽이든 ‘정해 두고 일관되게’ 하는 것입니다.

문제 7 남은 경우

```
JSON응답(res, 405, { error: "이 주소에서는 쓸 수 없는 방법입니다" });
```

PUT /equipments

405 {"error": "이 주소에서는 쓸 수 없는 방법입니다"}

/equipments 는 있는 주소지만 PUT 은 안 만들었습니다. 404(그런 주소 없음)가 아니라 405(그 방법은 안 됨)가 정확합니다.

```
});
```

```
server.listen(PORT, () => {
  console.log(`설비 관리 API 가 켜졌습니다. http://localhost:${PORT}/equipments`);
});
```

지우고 나서 목록을 다시 보면

GET /equipments

200 [{"id":1,"name":"컨베이어","line":"A","status":"가동"}, {"id":2,"name":"프레스","line":"B","status":"정지"}]

이 코드를 보며 확인할 것

- ① 갈래를 나누는 조건이 벌써 일곱 개입니다. 설비 말고 작업지시·불량기록까지 만들면 스무 개가 넘습니다. 04단원에서 Express 로 다시 만들면 이 if 들이 전부 사라집니다.
- ② 반복이 보입니다.

```
Number(번호글자)
find 하고 없으면 404
JSON본문 하고 null 이면 400
```

문제 3·5·6 에 똑같은 코드가 세 번씩 들어 있습니다. 05단원 미들웨어를 배우면 이걸 한 곳에 모을 수 있습니다.

- ③ 검사하는 코드가 진짜 일하는 코드보다 깁니다. 이건 잘못된 게 아니라 정상입니다. 남이 보내는 것은 무엇이든 틀릴 수 있다고 생각해야 합니다.
- ④ 서버를 끄면 3번 설비가 사라집니다. PART 4 에서 데이터베이스를 붙이면 해결됩니다.

Postman 시나리오

```
1 GET    /health                200 {"status":"ok"}
2 GET    /equipments            2건
3 GET    /equipments?line=A    1건 (컨베이어)
4 GET    /equipments?line=Z    0건 - 200 과 [] 입니다
5 GET    /equipments/1        컨베이어 하나
6 GET    /equipments/99 404
7 GET    /equipments/abc      400
8 POST   /equipments            400 - line 을 안 넣었으니
        { "name": "용접로봇" }
```

9 POST /equipments 201, id 3, status 는 “정지”

```
{ "name": "용접로봇", "line": "C" }
```

10 PATCH /equipments/3 400

```
{ "status": "폭발" }
```

11 PATCH /equipments/3 200, status 가 “가동” 으로

```
        { "status": "가동" }
12 DELETE /equipments/3 204, 본문 없음
13 DELETE /equipments/3 404
14 PUT    /equipments            405
```

4번과 6번의 차이를 꼭 눈으로 확인하세요. 둘 다 “없다” 인데 하나는 200, 하나는 404 입니다. 이유를 설명할 수 있어야 합니다.

04단원 연습문제 정답 Express 시작하기

RUN node 연습문제_정답.js

04단원 연습문제 정답 (서버 없이 푸는 것)

문제 1

```
function 버전5로(경로) {
  return 경로
    .replace(/\\/(:[a-zA-Z]+)\?/g, "{/$1}") // /:year? → {/:year}
    .replace(/\*$/, "*splat"); // 끝의 * → *splat
}

console.log(버전5로("/reports/:year?"));
```

출력 /reports{/:year}

코드

▶ 출력

console.log(버전5로("/files/*"));

▶ /files/*splat

console.log(버전5로("/documents/:id"));

▶ /documents/:id

```
console.log(버전5로("/a/:b?/c/*"));
```

출력 /a{/:b}/c/*splat

세 번째는 바꿀 게 없어서 그대로 나옵니다. 이것도 확인해야 합니다. “바꿀 게 없는 것을 안 건드리는가” 도 함수가 맞는지 보는 방법입니다.

\$1 은 무엇인가 괄호로 묶은 부분을 그대로 가져다 쓰는 표시입니다. (: [a-zA-Z]+) 가 “:year” 를 잡았으면 \$1 자리에 “:year” 가 들어갑니다.

★ 이 함수는 연습용입니다. 실제로는 이런 걸 만들 일이 없습니다. 블로그 코드를 보고 “아 이건 4 문법이구나” 하고 알아보는 게 목적입니다.

문제 2

```
const 문서들 = [
  { id: 1, title: "작업표준서" },
  { id: 2, title: "검사성적서" },
];
```

Express 가 주는 req.params 는 이렇게 생겼습니다. 값이 전부 글자입니다.

```
const params = { id: "1" };

console.log(문서들.find((문서) => 문서.id === params.id));
```

출력 undefined

```
console.log(문서들.find((문서) => 문서.id === Number(params.id)));
```

출력 { id: 1, title: '작업표준서' }

★ 첫 줄이 undefined 인 이유 1 === "1" 은 false 입니다. 숫자와 글자는 다릅니다. 그래서 find 가 끝까지 훑고도 아무것도 못 찾습니다.

에러가 안 납니다. 그냥 undefined 가 나옵니다. 그래서 "왜 404 만 나오지?" 하고 한참 헤매게 됩니다.

★ 이걸 못 찾을 때 확인하는 법 console.log(typeof req.params.id) 를 찍어 보세요. string 이 나옵니다. Express 를 쓰는 내내 이 실수를 하게 됩니다. Number 를 습관으로.

== 를 쓰면 되지 않나요? 1 == "1" 은 true 라서 되긴 합니다. 하지만 쓰지 마세요. == 는 "" == 0 도 true 라서 나중에 엉뚱한 곳에서 사고가 납니다. === 로 비교하고, 대신 타입을 맞춰 주는 것이 정석입니다.

문제 3

```
const 라우트들 = ["/orders/:id", "/orders/new", "/orders", "/orders/:id/items"];

function 파라미터수(경로) {
  return (경로.match(/:/g) || []).length;
}

const 등록순서 = [...라우트들].sort((a, b) => 파라미터수(a) - 파라미터수(b));

console.log(등록순서);
```

출력 ['/orders/new', '/orders', '/orders/:id', '/orders/:id/items']

콜론이 적은 것(구체적인 것)이 먼저 오게 정렬했습니다.

★ match 는 못 찾으면 `null` 을 줍니다 `"/orders".match(/:/g)` 는 `null` 입니다. `null.length` 는 `TypeError` 입니다. 그래서 `|| []` 로 받쳐 두었습니다. 빈 배열의 `length` 는 0 입니다.

★ [...라우트들] 로 복사한 이유 sort 는 원본 배열을 진짜로 바꿔 버립니다. 원본을 지키려면 복사본을 정렬해야 합니다. (JS자료 06단원)

★ 이건 규칙을 이해하기 위한 연습입니다 실제로는 정렬 함수를 쓰지 않고, 사람이 코드를 그 순서로 적습니다. "구체적인 것 위, :id 아래" 를 손에 익히는 게 목적입니다.

문제 4

```
const 어디에담나 = {
  "3번 문서 하나 보기": "경로",
  "표준 종류만 걸러 보기": "쿼리",
  "몇 쪽을 볼지": "쿼리",
  "새 문서의 제목과 내용": "본문",
  "비밀번호": "본문",
  "로그인 증표": "헤더",
};

for (const 상황 of Object.keys(어디에담나)) {
  console.log(`${상황} → ${어디에담나[상황]}`);
}
```

출력	3번 문서 하나 보기 → 경로
	표준 종류만 걸러 보기 → 쿼리
	몇 쪽을 볼지 → 쿼리
	새 문서의 제목과 내용 → 본문
	비밀번호 → 본문
	로그인 증표 → 헤더

판단 기준 없으면 무슨 요청인지 알 수 없다 → 경로 없어도 요청은 성립한다 → 쿼리 길다 / 구조가 있다 /
남이 보면 안 된다 → 본문 내용이 아니라 요청에 대한 정보다 → 헤더

★ 비밀번호를 쿼리에 담으면 안 되는 이유 주소는 서버 접속 기록에 그대로 남습니다. 브라우저 방문 기록에도 남고, 링크를 복사해 보내면 딸려 갑니다.

문제 5

```
const 상황별코드 = {
  "res.json 으로 목록을 돌려줌": 200,
  "새 문서를 만들고 돌려줌": 201,
  "지웠고 돌려줄 내용이 없음": 204,
  "req.body 에 title 이 없음": 400,
  "그 번호의 문서를 못 찾음": 404,
  "라우트 안에서 TypeError 가 남": 500,
};

for (const 상황 of Object.keys(상황별코드)) {
  console.log(`${상황} → ${상황별코드[상황]}`);
}
```

출력	res.json 으로 목록을 돌려줌 → 200
	새 문서를 만들고 돌려줌 → 201
	지웠고 돌려줄 내용이 없음 → 204
	req.body 에 title 이 없음 → 400
	그 번호의 문서를 못 찾음 → 404
	라우트 안에서 TypeError 가 남 → 500

03단원과 같은 표입니다. Express 를 써도 상태코드는 그대로입니다. 도구가 바뀌어도 약속은 안 바뀝니다.

문제 6

```
const 적는순서 = [
  "app.use(express.json())",
  "app.use('/documents', documentsRouter)",
  "app.use((req, res) => 404 응답)",
  "app.use((err, req, res, next) => 에러 응답)",
];

적는순서.forEach((줄, 번호) => {
  console.log(`${번호 + 1}. ${줄}`);
});
```

출력	1. app.use(express.json())
	2. app.use('/documents', documentsRouter)
	3. app.use((req, res) => 404 응답)
	4. app.use((err, req, res, next) => 에러 응답)

이 순서를 어기면 이렇게 됩니다.

1을 2 아래에 두면 → req.body 가 undefined 3을 2 위에 두면 → 모든 요청이 404 4를 3 위에 두면 → 에러가 404 처리기에 먼저 걸림

forEach 의 두 번째 인자가 번호(0부터)입니다. 사람이 읽을 번호는 1부터라서 +1 을 했습니다.

문제 7

```
app.get("/orders/:id", (req, res) => { ... });
app.get("/orders/new", (req, res) => { ... });
```

무슨 일이 생기나: /orders/new 로 요청하면 위의 :id 라우트가 먼저 걸립니다. req.params.id 에 "new" 라는 글자가 들어갑니다. Number("new") 는 NaN 이라 아무 주문도 못 찾고 404 가 납니다. 아래 /orders/new 라우트는 영원히 실행되지 않습니다.

에러가 나냐: 안 납니다. 그래서 더 무섭습니다. "라우트를 분명히 만들었는데 안 걸린다" 로 보입니다.

어떻게 고치냐: 두 줄의 순서를 바꿉니다. /orders/new 를 위로 올립니다.

규칙: 구체적인 주소를 위에, :id 나 *splat 같은 것을 아래에.

확인하는 법: 라우트 맨 앞에 console.log("여기 걸림") 을 넣어 보세요. 안 찍히면 위쪽 어딘가에서 먼저 걸린 것입니다.

문제 8

req.body 가 undefined 인 원인 세 가지

① app.use(express.json()) 을 안 썼다

제일 흔합니다. 이 한 줄이 본문을 읽어 담는 일을 합니다.

② app.use(express.json()) 을 라우트 아래에 썼다

위에서부터 실행되니 라우트가 먼저 걸립니다. 담을 기회가 없습니다.

③ Content-Type 이 application/json 이 아니다

Postman 에서 Body → raw 만 고르고 JSON 으로 안 바꾼 경우입니다.
express.json() 은 "내가 읽을 형식이 아니네" 하고 그냥 지나갑니다.

그리고 하나 더 — Express 5 만의 사정 본문을 아예 안 보내면 req.body 는 {} 가 아니라 undefined 입니다. Express 4 는 {} 였습니다. 그래서 옛날 코드를 그대로 쓰면

```
const { title } = req.body 에서 TypeError 로 서버가 죽습니다.
```

어떻게 확인하나: 라우트 맨 앞에서 이 두 줄을 찍어 보세요.

```
console.log("타입:", typeof req.body);  
console.log("형식:", req.get("Content-Type"));
```

```
const 원인들 = [  
  "express.json() 을 안 씀",  
  "express.json() 을 라우트 아래에 씀",  
  "Content-Type 이 application/json 이 아님",  
];  
  
원인들.forEach((원인) => console.log(원인));
```

```
출력      express.json() 을 안 씀  
          express.json() 을 라우트 아래에 씀  
          Content-Type 이 application/json 이 아님
```

04단원 연습문제 서버 정답 Express 시작하기

RUN node 연습문제_서버_정답.js

또는 npx nodemon 연습문제_서버_정답.js

03단원 연습문제_서버_정답.js 와 나란히 놓고 보세요. 하는 일이 완전히 같습니다. 코드만 절반으로 줄었습니다.

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

준비

```
let 설비들 = [
  { id: 1, name: "컨베이어", line: "A", status: "가동" },
  { id: 2, name: "프레스", line: "B", status: "정지" },
];

let 다음번호 = 3;

const 상태값들 = ["가동", "정지"];
```

상태코드가 붙은 에러를 만드는 도구입니다. (개념04 섹션 4)

```
function HttpError(코드, 메시지) {
  const 에러 = new Error(메시지);
  에러.status = 코드;
  return 에러;
}
```

문제 0 공통 함수 만들기

번호로 설비를 찾고, 없으면 에러를 던집니다. 문제 3·5·6 에서 똑같이 쓰니까 함수로 묶었습니다.

```
function 설비찾기(번호글자) {
  const 번호 = Number(번호글자);
```

05단원 연습문제 정답 미들웨어

RUN node 연습문제_정답.js

05단원 연습문제 정답 (서버 없이 푸는 것)

문제 1

미들웨어를 파일로 빼 두면 이렇게 따로 불러다 시험할 수 있습니다.

```
const { 한줄검사 } = require("../middlewares/검증");

console.log(한줄검사({ 필수: true }, undefined));
```

출력 필수입니다

```
console.log(한줄검사({ 필수: false }, undefined));
```

출력 null

```
console.log(한줄검사({ 타입: "number" }, "3"));
```

출력 number 이어야 합니다 (지금은 string)

```
console.log(한줄검사({ 허용: ["A", "B"] }, "Z"));
```

출력 A, B 중 하나여야 합니다

```
console.log(한줄검사({ 최소길이: 2 }, "김"));
```

출력 2글자 이상이어야 합니다

```
console.log(한줄검사({ 최소: 1, 최대: 100 }, 200));
```

출력 100 이하여야 합니다

```
console.log(한줄검사({ 타입: "string", 최소길이: 2 }, "용접로봇"));
```

출력 null

★ 서버를 안 켜고 시험할 수 있는 게 큰 장점입니다. Postman 을 열고 요청을 보내면서 검증 규칙을 고치는 것보다 훨씬 빠릅니다. 함수를 파일로 빼는 이유 중 하나가 이것입니다.

★ `null` 이 '문제 없음' 입니다 "문제가 있으면 이유를, 없으면 `null`" 이라는 약속을 정한 것입니다.
`true/false` 로 만들면 "왜 틀렸는지" 를 알려 줄 수가 없습니다.

문제 2

```
const 작업지시규칙 = [
  { 키: "title", 필수: true, 타입: "string", 최소길이: 2, 최대길이: 30 },
  { 키: "line", 필수: true, 허용: ["A", "B", "C"] },
  { 키: "priority", 필수: false, 타입: "number", 최소: 1, 최대: 5 },
];

function 전체검증(규칙들, 데이터) {
  const 항목 = [];

  for (const 규칙 of 규칙들) {
    const 이유 = 한줄검사(규칙, 데이터[규칙.키]);

    if (이유) {
      항목.push({ 키: 규칙.키, 이유 });
    }
  }

  return 항목;
}

function 결과찍기(제목, 데이터) {
  const 항목 = 전체검증(작업지시규칙, 데이터);

  if (항목.length === 0) {
    console.log(`${제목}: 통과`);
    return;
  }

  항목.forEach((하나) => console.log(`${제목}: ${하나.키} - ${하나.이유}`));
}

결과찍기("빈 것", {});
```

출력 빈 것: title - 필수입니다
 빈 것: line - 필수입니다

```
결과찍기("라인 틀림", { title: "벨트 교체", line: "Z" });
```

출력 라인 틀림: line - A, B, C 중 하나여야 합니다

```
결과찍기("우선순위 범위 밖", { title: "벨트 교체", line: "A", priority: 9 });
```

출력 우선순위 범위 밖: priority - 5 이하여야 합니다

```
결과찍기("정상", { title: "벨트 교체", line: "A", priority: 3 });
```

출력 정상: 통과

```
결과찍기("우선순위 없음", { title: "벨트 교체", line: "A" });
```

출력 우선순위 없음: 통과

★ 마지막 두 개가 둘 다 통과인 것을 보세요. priority 는 필수가 아니라서, 없어도 되고 있으면 검사합니다.

★ 왜 객체를 그대로 `console.log` 하지 않았나

```
node 는 한글 속성 이름에 따옴표를 붙입니다.  
[ { '키': 'title', '이유': '필수입니다' } ]
```

보기 어려우니 글자로 만들어 찍었습니다. JSON 으로 나갈 때는 상관없습니다. 콘솔에서만 그렇습니다.

문제 3

```
function 역할판단(사용자, 허용역할들) {  
  if (!사용자) {  
    return 401; // 누구인지 모름  
  }  
  
  if (!허용역할들.includes(사용자.role)) {  
    return 403; // 누군지는 알지만 자격이 없음  
  }  
  
  return 200;  
}  
  
console.log(역할판단(null, ["admin"]));
```

출력 401

코드

```
console.log(역할판단({ role: "user" }, ["admin"]));
```

```
console.log(역할판단({ role: "admin" }, ["admin"]));
```

▶ 출력

▶ 403

▶ 200

```
console.log(역할판단({ role: "manager" }, ["admin", "manager"]));
```

출력 200

★ 401 과 403 을 가르는 것은 '사용자가 있느냐' 하나뿐입니다. 사용자가 없으면 자격을 따질 수조차 없습니다. 그래서 401 이 먼저입니다.

★ 허용역할들 을 배열로 받은 이유 역할이 하나만 될 거라고 가정하면, manager 가 생겼을 때 함수를 또 만들어야 합니다. 배열로 받으면 역할이 몇 개로 늘어도 그대로 씁니다.

문제 4

```
function 증표읽기(헤더값) {
  if (!헤더값) {
    return null; // 헤더 자체가 없음
  }

  const [방식, 키] = 헤더값.split(" ");

  if (방식 !== "Bearer" || !키) {
    return null; // 형식이 틀림
  }

  return 키;
}

console.log(증표읽기(undefined));
```

출력 null

코드	출력
console.log(증표읽기("key-user-1"));	null
console.log(증표읽기("Bearer"));	null
console.log(증표읽기("Bearer key-user-1"));	key-user-1

★ "Bearer" 만 왔을 때 split(" ") 결과가 ["Bearer"] 라서 키가 undefined 입니다. !키 로 걸러 내지 않으면 undefined 를 그대로 돌려주게 됩니다. 그러면 사용자 목록에서 undefined 로 찾게 되어, 결국 401 이 되긴 합니다. 되긴 하지만 "형식이 틀림" 과 "키가 틀림" 은 다른 문제입니다.

★ 이 함수는 401 인지 아닌지만 판단합니다 "키가 진짜 있는 키인가" 는 사용자 목록을 봐야 알 수 있습니다. 함수 하나가 한 가지만 하게 나눠 두면 시험하기 쉽습니다.

문제 5

```
const 적는순서 = [
  "app.use(번호붙이기)",
  "app.use(morgan(...))",
  "app.use(express.json())",
  'app.get("/me", 인증, ...)',
  "app.use((req, res) => 404 응답)",
  "app.use((err, req, res, next) => 에러 응답)",
];

적는순서.forEach((줄, 번호) => console.log(`${번호 + 1}. ${줄}`));
```

출력	1. app.use(번호붙이기)
	2. app.use(morgan(...))
	3. app.use(express.json())
	4. app.get("/me", 인증, ...)
	5. app.use((req, res) => 404 응답)
	6. app.use((err, req, res, next) => 에러 응답)

여기면 이렇게 됩니다. 1을 2 아래에 두면 → morgan 의 :reqid 가 - 로만 나옵니다 3을 4 아래에 두면 → req.body 가 undefined 5를 4 위에 두면 → 모든 요청이 404 6을 5 위에 두면 → 에러가 404 처리기에 먼저 걸립니다

★ 1과 2의 순서에 정답이 하나만 있는 건 아닙니다. 번호를 기록에 넣고 싶으면 번호붙이기가 먼저여야 합니다. 기록에 안 넣을 거라면 순서가 상관없습니다. “앞에서 만든 것을 뒤에서 쓴다” 는 관계만 지키면 됩니다.

문제 6

```
const 상황별코드 = {
  "Authorization 헤더가 아예 없음": 401,
  "Bearer 없이 키만 보냄": 401,
  "없는 키를 보냄": 401,
  "user 가 관리자 전용 주소에 들어옴": 403,
  "admin 이 관리자 전용 주소에 들어옴": 200,
  "로그인은 했는데 남의 글을 지우려 함": 403,
};

for (const 상황 of Object.keys(상황별코드)) {
  console.log(`${상황} → ${상황별코드[상황]}`);
}
```

출력 Authorization 헤더가 아예 없음 → 401
 Bearer 없이 키만 보냄 → 401
 없는 키를 보냄 → 401
 user 가 관리자 전용 주소에 들어옴 → 403
 admin 이 관리자 전용 주소에 들어옴 → 200
 로그인은 했는데 남의 글을 지우려 함 → 403

앞의 세 개는 전부 “누구인지 모르겠다” 입니다. 이유는 달라도 401 입니다. 뒤의 두 개는 “누군지는 알겠는데 안 된다” 입니다. 403 입니다.

★ 마지막 줄이 혼한 경우입니다 역할은 맞는데 ‘내 것’ 이 아니라서 안 되는 경우입니다. 이건 미들웨어가 아니라 라우트에서 확인해야 합니다. 그 글의 작성자가 누구인지는 데이터를 봐야 알 수 있으니까요.

문제 7

미들웨어에서 `next()` 를 안 부르고 응답도 안 보내면

무슨 일이 생기나: 요청이 그 자리에서 멈춥니다. 아래 미들웨어도 라우트도 실행되지 않습니다. 브라우저는 응답을 기다리며 계속 빙빙 돌립니다.

에러가 나나: 안 납니다. 터미널에도 아무것도 안 찍힙니다. 그래서 제일 찾기 어렵습니다.

어떻게 알아채나: ① 브라우저 탭이 계속 도는데 터미널은 조용하다 ② morgan 기록에 요청은 들어왔는데 응답 줄이 안 나온다 ③ 미들웨어마다 `console.log` 를 넣어 어디까지 갔는지 본다

어떻게 막나: 미들웨어의 모든 갈래가 셋 중 하나로 끝나게 합니다.

① <code>next()</code>	다음으로
② 응답을 보낸다	여기서 끝
③ <code>next(에러)</code>	에러 처리기로

특히 if 를 썼다면 else 쪽도 확인하세요. 한쪽에서만 `next()` 를 부르는 것이 가장 흔한 실수입니다.

문제 8

Express 5 에서 `req.query` 를 고칠 수 없는 이유

무슨 일이 일어나나: `req.query.q = "고친값"` 이라고 써도 에러가 안 납니다. 그런데 라우트에서 다시 읽으면 원래 값이 나옵니다. 고친 게 사라집니다.

왜 그런가: Express 5 에서 `req.query` 는 ‘읽을 때마다 주소를 다시 분석해 만드는 값’ 입니다. 내가 고친 것은 그 순간 만들어진 것이고, 다음에 읽으면 또 새로 만들어집니다.

왜 위험한가: 에러가 안 나기 때문입니다. “분명히 trim 했는데 왜 공백이 남아 있지?” 로 한참 헤매게 됩니다. Express 4 에서는 땀기 때문에, 옛날 코드를 옮길 때 조용히 깨집니다.

어떻게 하나:

```
새 이름에 답습니다. req.정리된쿼리 = { ... }
```

req 에 내가 만든 속성을 붙이는 것은 잘 됩니다.

규칙으로 정리하면: 받은 것(req.query, req.params)은 읽기만 하고, 바꾼 것은 새 이름으로 붙인다.

05단원 연습문제 서버 정답 미들웨어

RUN node 연습문제_서버_정답.js

```
const express = require("express");
const morgan = require("morgan");

const { 번호붙이기 } = require("./middlewares/기록");
const { 인증, 역할확인 } = require("./middlewares/인증");
const { 검증 } = require("./middlewares/검증");
const { HttpError } = require("./utils/HttpError");

const app = express();
const PORT = process.env.PORT || 3000;
```

05

문제 1 기본 미들웨어 엮기

```
morgan.token("user", (req) => (req.user ? req.user.name : "-"));
morgan.token("reqid", (req) => (req.요청번호 ? `#${req.요청번호}` : "-"));

app.use(번호붙이기);
app.use(morgan(":reqid :method :url :status :response-time ms - :user"));
app.use(express.json());
```

준비

```
let 작업지시들 = [
  { id: 1, title: "벨트 교체", line: "A", priority: 2, status: "대기", 등록자: "김민준" },
  { id: 2, title: "베어링 점검", line: "B", priority: 4, status: "완료", 등록자: "이서연" },
];

let 다음번호 = 3;

const 상태값들 = ["대기", "진행", "완료"];

const 작업지시규칙 = [
  { 키: "title", 필수: true, 타입: "string", 최소길이: 2, 최대길이: 30 },
```

```
{ 키: "line", 필수: true, 허용: ["A", "B", "C"] },
  { 키: "priority", 필수: false, 타입: "number", 최소: 1, 최대: 5 },
];
```

문제 2 :id 를 숫자로 바꾸는 미들웨어

```
app.param("id", (req, res, next, 값) => {
  const 번호 = Number(값);

  if (!Number.isInteger(번호)) {
    return next(HttpError(400, "번호는 숫자여야 합니다"));
  }

  req.번호 = 번호;
  next();
});
```

문제 3 작업지시를 찾아 req 에 붙이는 미들웨어

```
function 작업지시찾기(req, res, next) {
  const 작업 = 작업지시들.find((작업) => 작업.id === req.번호);

  if (!작업) {
    return next(HttpError(404, `${req.번호}번 작업지시가 없습니다`));
  }

  req.작업 = 작업;
  next();
}
```

★ 이게 미들웨어의 진짜 힘입니다. "번호를 숫자로 바꾸고 → 찾고 → 없으면 404" 가 미들웨어 두 개로 끝났습니다. 아래 라우트 세 개가 이 혜택을 함께 받습니다.

문제 4 공개 주소

```
app.get("/health", (req, res) => {
  res.json({ status: "ok" });
});
```


GET /health**200** {"status":"ok"}

문제 5

목록 (로그인만 하면 볼 수 있음)

```
app.get("/orders", 인증, (req, res) => {
  const { line, status } = req.query;

  let 결과 = 작업지시들;
  if (line) 결과 = 결과.filter((작업) => 작업.line === line);
  if (status) 결과 = 결과.filter((작업) => 작업.status === status);

  res.json(결과);
});
```

GET /orders**401** {"error":"로그인이 필요합니다"}**GET** /orders

Authorization: Bearer key-user-1

200 [{"id":1,"title":"벨트 교체","line":"A","priority":2,"status":"대기","등록자":"김민준"}, {"id":2,"title":"베어링 점검","line":"B","priority":4,"status":"완료","등록자":"이서연"}]**GET** /orders?line=A

Authorization: Bearer key-user-1

200 [{"id":1,"title":"벨트 교체","line":"A","priority":2,"status":"대기","등록자":"김민준"}]**GET** /orders?status=완료

Authorization: Bearer key-user-1

200 [{"id":2,"title":"베어링 점검","line":"B","priority":4,"status":"완료","등록자":"이서연"}]**GET** /orders?line=C

Authorization: Bearer key-user-1

200 []

★ 조건을 여러 개 받을 때는 결과를 계속 좁혀 나갑니다. `let` 으로 시작해서 조건마다 `filter` 를 겹칩니다. 조건이 없으면 그냥 통과하니 `if` 안에서만 좁히면 됩니다.

문제 6 하나만 보기

```
app.get("/orders/:id", 인증, 작업지시찾기, (req, res) => {
  res.json(req.작업);
});
```

GET /orders/1

Authorization: Bearer key-user-1

200 {"id":1,"title":"벨트 교체","line":"A","priority":2,"status":"대기","등록자":"김민준"}

GET /orders/99

Authorization: Bearer key-user-1

404 {"error":"99번 작업지시가 없습니다"}

GET /orders/abc

Authorization: Bearer key-user-1

400 {"error":"번호는 숫자여야 합니다"}

라우트가 한 줄입니다. 검사는 전부 앞에서 끝났습니다.

문제 7 새로 등록 (admin 만)

```
app.post("/orders", 인증, 역할확인("admin"), 검증(작업지시규칙), (req, res) => {
  const 새작업 = {
    id: 다음번호,
    title: req.body.title,
    line: req.body.line,
    priority: req.body.priority ?? 3, // 안 주면 보통 우선순위 status: "대기",
    등록자: req.user.name,
  };

  다음번호 += 1;
  작업지시들.push(새작업);

  res.status(201).location(`/orders/${새작업.id}`).json(새작업);
});
```

POST /orders`{"title":"모터 점검","line":"C"}`**401** `{"error":"로그인이 필요합니다"}`**POST** /orders

Authorization: Bearer key-user-1

`{"title":"모터 점검","line":"C"}`**403** `{"error":"admin 만 할 수 있습니다"}`**POST** /orders

Authorization: Bearer key-admin-1

`{"title":"모", "line":"Z"}`**400** `{"error":"입력값이 올바르지 않습니다","항목":[{"키":"title","이유":"2글자 이상이어야 합니다"}, {"키":"line","이유":"A, B, C 중 하나여야 합니다"}]}`**POST** /orders

Authorization: Bearer key-admin-1

`{"title":"모터 점검","line":"C"}`**201** `{"id":3,"title":"모터 점검","line":"C","priority":3,"status":"대기","등록자":"이서연"}`

★ 세 개의 400·401·403 이 각각 다른 미들웨어에서 나왔습니다. 401 은 인증에서, 403 은 역할확인에서, 400 은 검증에서. 라우트 함수는 한 번도 실행되지 않았습니다.

★ ?? 3 을 쓴 이유 || 3 으로 쓰면 priority: 0 을 보냈을 때 3 이 되어 버립니다. 지금 규칙은 1 이상이라 0 이 안 오지만, 습관을 들여 두세요.

★ 등록자를 본문에서 안 받았습니다 req.user.name 을 씁니다. 보낸 쪽이 정확하게 두면 남의 이름으로 등록할 수 있습니다. “누가 했는지” 는 언제나 서버가 정합니다.

문제 8 상태 바꾸기 (로그인만 하면 됨)

```
app.patch("/orders/:id", 인증, 작업지시찾기, (req, res, next) => {
  const { status } = req.body || {};

  if (!상태값들.includes(status)) {
    return next(HttpError(400, `status 는 ${상태값들.join(", ")} 중 하나여야 합니다`));
  }

  req.작업.status = status;

  res.json(req.작업);
});
```

PATCH /orders/3

Authorization: Bearer key-user-1

{ "status": "이상" }

400 { "error": "status 는 대기, 진행, 완료 중 하나여야 합니다" }

PATCH /orders/3

Authorization: Bearer key-user-1

{ "status": "진행" }

200 { "id": 3, "title": "모터 점검", "line": "C", "priority": 3, "status": "진행", "등록자": "이서연" }

★ 여기서는 검증 미들웨어를 안 쓰고 라우트 안에서 확인했습니다. 상태값들은 이 라우트에서만 쓰는 규칙이라, 굳이 규칙 표로 만들 필요가 없습니다. “두 곳 이상에서 쓰이면 빼낸다” 정도로 생각하면 됩니다.

문제 9 지우기 (admin 만)

```
app.delete("/orders/:id", 인증, 역할확인("admin"), 작업지시찾기, (req, res) => {
  작업지시들 = 작업지시들.filter((하나) => 하나.id !== req.작업.id);
  res.sendStatus(204);
});
```

DELETE /orders/3

Authorization: Bearer key-user-1

403 { "error": "admin 만 할 수 있습니다" }

DELETE /orders/3

Authorization: Bearer key-admin-1

204**DELETE** /orders/3

Authorization: Bearer key-admin-1

404 {"error": "3번 작업지시가 없습니다"}

★ 미들웨어 순서를 보세요. 인증 → 역할확인 → 작업지시찾기 자격이 없는 사람에게는 “그 작업이 있는지 없는지” 도 알려 주지 않습니다. 찾기를 먼저 하면, 403 을 받을 사람이 404 를 통해 “그 번호는 있구나” 를 알게 됩니다. 사소해 보이지만 이런 것이 정보를 흘리는 통로가 됩니다.

문제 10 405 · 404 · 에러 처리기

05

```
app.all("/orders", (req, res, next) => {
  next(HttpError(405, "이 주소에서는 쓸 수 없는 방법입니다"));
});
```

PUT /orders

Authorization: Bearer key-admin-1

405 {"error": "이 주소에서는 쓸 수 없는 방법입니다"}

```
app.use((req, res) => {
  res.status(404).json({ error: "그런 주소가 없습니다" });
});
```

GET /없는주소**404** {"error": "그런 주소가 없습니다"}

```
app.use((err, req, res, next) => {
  console.error(`  L*${req.요청번호} [에러] ${err.message}`);

  if (err.type === "entity.parse.failed") {
    return res.status(400).json({ error: "JSON 형식이 아닙니다" });
  }

  const 코드 = err.status || 500;

  if (코드 === 500) {
```

```

}

res.status(코드).json({ error: err.message });
});

app.listen(PORT, () => {
  console.log(`작업지시 API 가 켜졌습니다. http://localhost:${PORT}/health`);
});

```

지우고 나서 목록을 다시 보면

```

GET /orders
Authorization: Bearer key-user-1

200 [{"id":1,"title":"벨트 교체","line":"A","priority":2,"status":"대기","등록자":"김민준"}, {"id":2,"title":"베어링 점검","line":"B","priority":4,"status":"완료","등록자":"이서연"}]

```

이 파일에서 확인할 것

- ① 라우트 함수 안에 '검사 코드'가 거의 없습니다 로그인 확인, 권한 확인, 숫자 변환, 찾기, 404 — 전부 미들웨어입니다. 라우트는 "무엇을 돌려줄지"만 말합니다.
- ② 같은 미들웨어를 여러 라우트가 나눠 씁니다 작업지시찾기 하나를 GET·PATCH·DELETE 세 곳이 씁니다. 404 메시지를 바꾸고 싶으면 한 곳만 고치면 됩니다.
- ③ 미들웨어 순서가 곧 정책입니다 인증 → 역할확인 → 찾기 순서에는 이유가 있습니다. 순서를 바꾸면 응답이 달라지고, 때로는 정보가 새어 나갑니다.
- ④ 파일이 나뉘어 있습니다 이 파일 200줄 중 미들웨어 코드는 한 줄도 없습니다. 전부 require로 가져왔습니다. 다음 프로젝트에서도 middlewares 폴더를 통째로 복사해 쓸 수 있습니다.

Postman 시나리오

Headers에 Authorization: Bearer key-user-1 또는 key-admin-1

1 GET	/health		200	2 GET	/orders
(증표 없이)		401	3 GET	/orders	user
2건					
4 GET	/orders?line=A	user	1건		
5 GET	/orders?line=C	user	200	과 []	
6 GET	/orders/1	user	벨트 교체		
7 GET	/orders/99	user	404	8 GET	/orders/abc
user		400	9 POST	/orders	user
403					
	{ "title": "모터 점검", "line": "C" }				
10 POST	/orders	admin	201, id 3		
11 PATCH	/orders/3	user	{ "status": "진행" }	200	
12 DELETE	/orders/3	user	403		
13 DELETE	/orders/3	admin	204		
14 PUT	/orders	admin	405		

터미널을 함께 보세요. 요청 번호와 누가 했는지가 한 줄씩 남습니다.

#10 POST /orders 201 3.121 ms - 이서연 #12 DELETE /orders/3 403 1.004 ms - 김민준

이 기록만 봐도 “김민준이 지우려다 막혔다” 를 알 수 있습니다.

06단원 연습문제 정답 REST API 설계

RUN node 연습문제_정답.js

06단원 연습문제 정답 (서버 없이 푸는 것)

문제 1

```
function 받침있나(글자) {
  const 코드 = 글자.charCodeAt(0);
```

완성형 한글은 가(44032)부터 힉(55203)까지입니다.

```
if (코드 < 44032 || 코드 > 55203) {
  return null; // 한글이 아님
}
```

한글은 (초성 × 21 + 중성) × 28 + 종성 으로 만들어집니다. 그래서 28 로 나눈 나머지가 종성(받침) 번호입니다. 0이면 받침이 없습니다.

```
return (코드 - 44032) % 28 !== 0;
}

function 조사붙이기(단어, 받침있을때, 받침없을때) {
  const 마지막 = 단어[단어.length - 1];
  const 있나 = 받침있나(마지막);

  if (있나 === null) {
    return `${단어}(${받침있을때}/${받침없을때})`; // 한글이 아니면 둘 다 보여 줍니다
  }

  return 단어 + (있나 ? 받침있을때 : 받침없을때);
}

console.log(조사붙이기("설비", "을", "를"));
```

출력 설비를

코드	출력
<code>console.log(조사불이기("점검기록", "을", "를"));</code>	▶ 점검기록을
<code>console.log(조사불이기("작업지시", "이", "가"));</code>	▶ 작업지시가
<code>console.log(조사불이기("사용자", "은", "는"));</code>	▶ 사용자는
<code>console.log(조사불이기("로봇", "은", "는"));</code>	▶ 로봇은
<code>console.log(조사불이기("API", "을", "를"));</code>	▶ API(을/를)

★ 28 로 나누는 이유 한글 한 글자의 번호는 이렇게 정해져 있습니다.

$$44032 + (\text{초성번호} \times 21 \times 28) + (\text{중성번호} \times 28) + \text{종성번호}$$

앞의 두 덩어리는 28 의 배수라 나머지가 0 입니다. 그래서 나머지가 곧 종성(받침) 번호입니다. 0 이면 받침이 없습니다.

‘비’ 는 나머지가 0 → 받침 없음 → 설비‘를’ ‘록’ 은 나머지가 1 → 받침 있음 → 점검기록‘을’

★ 영어·숫자가 오면 ‘API’ 의 마지막 ‘I’ 는 한글 범위 밖이라 `null` 입니다. 무엇을 붙일지 정할 수 없으니 ‘(을/를)’ 로 둡니다. 실무에서도 이렇게 처리하는 곳이 많습니다.

★ 왜 이런 걸 배우나 에러 메시지가 “설비을(를) 찾을 수 없습니다” 로 나가면 어색합니다. 사용자에게 보이는 글은 이런 것 하나로 인상이 달라집니다.

문제 2

```
function 쪽정보(query, 기본 = 10, 최대 = 100) {
  let page = Number(query.page);
  let limit = Number(query.limit);

  if (!Number.isInteger(page) || page < 1) page = 1;
  if (!Number.isInteger(limit) || limit < 1) limit = 기본;
  if (limit > 최대) limit = 최대;

  return { page, limit };
}

function 쪽메타(total, page, limit) {
  return { page, limit, total, totalPages: Math.ceil(total / limit) };
}

console.log(쪽정보({}));
```

출력 { page: 1, limit: 10 }

```
console.log(쪽정보({ page: "3", limit: "5" }));
```

```
출력      { page: 3, limit: 5 }
```

```
console.log(쪽정보({ page: "abc" }));
```

```
출력      { page: 1, limit: 10 }
```

```
console.log(쪽정보({ limit: "999" }));
```

```
출력      { page: 1, limit: 100 }
```

```
console.log(쪽정보({ page: "0", limit: "-5" }));
```

```
출력      { page: 1, limit: 10 }
```

```
console.log(쪽메타(12, 1, 5));
```

```
출력      { page: 1, limit: 5, total: 12, totalPages: 3 }
```

```
console.log(쪽메타(0, 1, 10));
```

```
출력      { page: 1, limit: 10, total: 0, totalPages: 0 }
```

★ 마지막 줄을 보세요. 0건이면 totalPages 도 0 입니다. `Math.ceil(0 / 10)` 이 0 이기 때문입니다. 화면에 "1 / 0 쪽" 이라고 나오면 어색하니, 프론트에서 따로 처리하거나 여기서 `Math.max(1, ...)` 로 최소 1을 보장하기도 합니다. 정하기 나름입니다.

★ `Number("3")` 은 3 이지만 `Number("abc")` 는 `NaN` 입니다. 그런데 `Number(undefined)` 도 `NaN` 이고, `Number(null)` 은 0 입니다. `Number.isInteger` 하나로 `NaN` 과 소수를 함께 걸러 냅니다. 0 은 정수라서 통과하니, `page < 1` 이 반드시 필요합니다.

문제 3

```
function 정렬조건(sort) {  
  if (!sort) return [];  
  
  return sort  
    .split(",")  
    .map((조각) => 조각.trim())  
    .filter(Boolean)  
    .map((조각) =>
```

```

    조각.startsWith("-")
      ? { 키: 조각.slice(1), 방향: -1 }
      : { 키: 조각, 방향: 1 }
  );
}

function 조건찍기(sort) {
  const 조건들 = 정렬조건(sort);

  if (조건들.length === 0) {
    console.log(`${sort} === undefined ? "(없음)" : `${sort}` → 정렬 안 함`);
    return;
  }

  const 글 = 조건들.map((c) => `${c.키} ${c.방향} === 1 ? "오름" : "내림"`).join(",");
  console.log(`${sort} → ${글}`);
}

조건찍기(undefined);

```

출력 (없음) → 정렬 안 함

```
조건찍기("name");
```

출력 "name" → name 오름

```
조건찍기("-id");
```

출력 "-id" → id 내림

```
조건찍기("line,-id");
```

출력 "line,-id" → line 오름, id 내림

```
조건찍기("line, -id ");
```

출력 "line, -id " → line 오름, id 내림

```
조건찍기("");
```

출력 "" → 정렬 안 함

★ 다섯 번째를 보세요. 공백이 섞여도 됩니다. `trim()` 을 안 하면 " -id" 가 되어 `startsWith("-")` 가 `false` 가 됩니다. 그러면 키가 " -id" 라는 이상한 이름이 되어 정렬이 조용히 안 됩니다.

★ filter(Boolean) 이 필요한 이유 "line,,id" 처럼 쉼표가 겹치면 빈 조각이 생깁니다. 그대로 두면 키가 "" 인 조건이 만들어집니다.

문제 4

```
function 필드고르기(항목, fields) {
  if (!fields) return 항목;

  const 결과 = {};

  for (const 키 of fields.split(",").map((조각) => 조각.trim())) {
    if (키 in 항목) {
      결과[키] = 항목[키];
    }
  }

  return 결과;
}
```

```
const 설비 = { id: 1, name: "컨베이어 1호", line: "A", status: "가동" };
```

```
console.log(필드고르기(설비, "id,name"));
```

```
출력      { id: 1, name: '컨베이어 1호' }
```

```
console.log(필드고르기(설비, undefined));
```

```
출력      { id: 1, name: '컨베이어 1호', line: 'A', status: '가동' }
```

```
console.log(필드고르기(설비, "id, 없는것"));
```

```
출력      { id: 1 }
```

```
console.log(필드고르기(설비, ""));
```

```
출력      { id: 1, name: '컨베이어 1호', line: 'A', status: '가동' }
```

★ 세 번째 — 없는 이름은 그냥 무시했습니다. 400 을 내는 것도 방법이지만, “달라고 한 것 중 있는 것만” 이 다루기 쉽습니다.

★ 네 번째 — ?fields= 로 빈 값이 오면 전부 줍니다. !fields 가 빈 글자도 걸러 내기 때문입니다. 빈 객체를 돌려주는 것보다 이쪽이 자연스럽습니다.

★ 키 in 항목 그 속성이 있는지 확인합니다. 항목[키] !== undefined 로 해도 되지만, 값이 진짜 undefined 인 경우와 구별이 안 됩니다. in 이 정확합니다.

문제 5

```
const 고친주소 = [
  "GET /api/v1/equipments",
  "GET /api/v1/equipments/3",
  "POST /api/v1/equipments",
  "PATCH /api/v1/equipments/3",
  "DELETE /api/v1/equipments/3",
  "GET /api/v1/equipments?line=A&status=가동",
  "GET /api/v1/equipments/3/logs",
  "POST /api/v1/equipments/3/logs",
];
```

```
고친주소.forEach((줄) => console.log(줄));
```

```
출력      GET /api/v1/equipments
          GET /api/v1/equipments/3
          POST /api/v1/equipments
          PATCH /api/v1/equipments/3
          DELETE /api/v1/equipments/3
          GET /api/v1/equipments?line=A&status=가동
          GET /api/v1/equipments/3/logs
          POST /api/v1/equipments/3/logs
```

원래 주소가 무엇이 틀렸는지

/getEquipmentList	동사. GET 이 이미 '가져온다' 입니다
/equipment/3	단수형. 컬렉션에서 하나를 고르는 것이니 복수형
/addEquipment	동사 + 어떤 메서드인지 안 보임
/modifyEquipment?id=3	동사 + 하나를 가리키는 건 경로로
/removeEquipment?id=3	★ GET 으로 지우면 안 됩니다
/searchEquipment?...	조건마다 주소를 만들면 끝이 없습니다
/getLogsOfEquipment?..	포함 관계는 중첩 주소로
/addLog	어느 설비의 기록인지 주소에 안 보입니다

문제 6

```
const 변경들 = {
  "설비 응답에 마지막점검일을 추가": "안 올려도 됨",
  "line 을 글자에서 객체로 바꿈": "올려야 함",
  "?sort= 를 새로 지원": "안 올려도 됨",
  "설비 등록에 담당자를 필수로 추가": "올려야 함",
  "404 메시지 글자를 다듬음": "안 올려도 됨",
}
```

06단원 연습문제 서버 정답 REST API 설계

RUN node 연습문제_서버_정답.js

감사무시: 대조 — 이 정답은 번호 블록이 아니라 ‘완성된 서버 한 벌’ 입니다.

문제 파일의 11개를 다 반영했지만 코드가 한 흐름으로 이어져
번호로 자를 수 없습니다.

이 파일은 같은 폴더의 README.md 를 그대로 구현한 것입니다. 문서와 코드가 한 글자도 어긋나면 안
됩니다.

```
const express = require("express");
const { 설비목록, 점검기록목록 } = require("./data/설비데이터");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

준비 —

데이터는 서버가 켜질 때 한 번만 가져옵니다. 매번 설비목록() 을 부르면 추가·삭제한 것이 사라집니다.

```
let 설비들 = 설비목록();
const 점검기록들 = 점검기록목록();

let 다음번호 = 13;

const 라인값들 = ["A", "B", "C"];
const 상태값들 = ["가동", "정지", "점검중"];

const 키별사용자 = {
  "key-user-1": { name: "김민준", role: "user" },
  "key-admin-1": { name: "이서연", role: "admin" },
};
```

연습문제.js 에서 만든 함수들

```

function 받침있나(글자) {
  const 코드 = 글자.charCodeAt(0);
  if (코드 < 44032 || 코드 > 55203) return null;
  return (코드 - 44032) % 28 !== 0;
}

function 조사붙이기(단어, 받침있을때, 받침없을때) {
  const 있나 = 받침있나(단어[단어.length - 1]);
  if (있나 === null) return `${단어}(${받침있을때}/${받침없을때})`;
  return 단어 + (있나 ? 받침있을때 : 받침없을때);
}

function 쪽정보(query, 기본 = 10, 최대 = 100) {
  let page = Number(query.page);
  let limit = Number(query.limit);

  if (!Number.isInteger(page) || page < 1) page = 1;
  if (!Number.isInteger(limit) || limit < 1) limit = 기본;
  if (limit > 최대) limit = 최대;

  return { page, limit };
}

function 정렬조건(sort) {
  if (!sort) return [];

  return sort
    .split(",")
    .map((조각) => 조각.trim())
    .filter(Boolean)
    .map((조각) =>
      조각.startsWith("-") ? { 키: 조각.slice(1), 방향: -1 } : { 키: 조각, 방향: 1 }
    );
}

function 비교하기(a, b, 조건들) {

```

```

for (const { 키, 방향 } of 조건들) {
  const 왼쪽 = a[키];
  const 오른쪽 = b[키];

  if (typeof 왼쪽 === "string" && typeof 오른쪽 === "string") {
    const 결과 = 왼쪽.localeCompare(오른쪽, "ko", { numeric: true });
    if (결과 !== 0) return 결과 * 방향;
    continue;
  }

  if (왼쪽 < 오른쪽) return -1 * 방향;
  if (왼쪽 > 오른쪽) return 1 * 방향;
}

return 0;
}

function 필드고르기(항목, fields) {
  if (!fields) return 항목;

  const 결과 = {};

  for (const 키 of fields.split(",").map((조각) => 조각.trim())) {
    if (키 in 항목) 결과[키] = 항목[키];
  }

  return 결과;
}

```

에러

```

class AppError extends Error {
  constructor(status, code, message, details) {
    super(message);
    this.name = "AppError";
    this.status = status;
    this.code = code;
    this.details = details;
  }
}

const 에러 = {
  없음: (무엇) => new AppError(404, "NOT_FOUND", `${조사붙이기(무엇, "을", "를")}`
  찾을 수 없습니다`),

```



```
검증실패: (항목들) => new AppError(400, "VALIDATION_FAILED", "입력값이 올바르지 않습니다", 항목들),
```

```
로그인필요: () => new AppError(401, "UNAUTHENTICATED", "로그인이 필요합니다"),
```

```
권한없음: (무엇) => new AppError(403, "FORBIDDEN", `${무엇} 권한이 없습니다`),
```

```
주소없음: () => new AppError(404, "ROUTE_NOT_FOUND", "그런 주소가 없습니다"),
};
```

응답 도우미

```
function 보내기(res, data, meta) {
  const 몸통 = { data };
  if (meta) 몸통.meta = meta;
  res.json(몸통);
}
```

미들웨어

```
function 인증(req, res, next) {
  const 헤더값 = req.get("Authorization");
  if (!헤더값) return next(에러.로그인필요());

  const [방식, 키] = 헤더값.split(" ");
  if (방식 !== "Bearer" || !키) return next(에러.로그인필요());

  const 사용자 = 키별사용자[키];
  if (!사용자) return next(에러.로그인필요());

  req.user = 사용자;
  next();
}

function 역할확인(...허용역할들) {
  return (req, res, next) => {
    if (!req.user) return next(에러.로그인필요());
    if (!허용역할들.includes(req.user.role)) return next(에러.권한없음("이 작업을 할")));
    next();
  };
}
```

:id 를 숫자로. 문서에 “숫자여야 합니다” 라고 적었으니 400 입니다.

```

app.param("id", (req, res, next, 값) => {
  const 번호 = Number(값);

  if (!Number.isInteger(번호)) {
    return next(에러.검증실패([키: "id", 이유: "숫자여야 합니다" ]));
  }

  req.번호 = 번호;
  next();
});

function 설비찾기(req, res, next) {
  const 설비 = 설비들.find((설비) => 설비.id === req.번호);
  if (!설비) return next(에러.없음("설비"));

  req.설비 = 설비;
  next();
}

```

GET /api/v1/equipments

```

app.get("/api/v1/equipments", (req, res) => {
  const q = (req.query.q ?? "").trim().toLowerCase();
  const { line, status, sort, fields } = req.query;

  let 결과 = 설비들;

```

① 검색

```

if (q) {
  결과 = 결과.filter((설비) =>
    ["name", "line", "status"].some((키) => String(설비
[키]).toLowerCase().includes(q)
  )
);
}

```

② 필터

```

if (line) {
  const 라인들 = line.split(",").map((조각) => 조각.trim());
  결과 = 결과.filter((설비) => 라인들.includes(설비.line));
}
if (status) {
  결과 = 결과.filter((설비) => 설비.status === status);
}

```

③ 정렬

```

const 조건들 = 정렬조건(sort);
if (조건들.length > 0) {
  결과 = [...결과].sort((a, b) => 비교하기(a, b, 조건들));
}

```

④ 개수는 자르기 전에

```
const total = 결과.length;
```

⑤ 자르기

```

const { page, limit } = 쪽정보(req.query);
const 시작 = (page - 1) * limit;
const data = 결과.slice(시작, 시작 + limit).map((설비) => 필드고르기(설비,
fields));

보내기(res, data, { page, limit, total, totalPages: Math.ceil(total / limit) });
});

```

```
GET /api/v1/equipments?limit=2&fields=id
```

```
200 {"data":[{"id":1},{"id":2}],"meta":{"page":1,"limit":2,"total":12,"totalPages":6}}
```

```
GET /api/v1/equipments?line=A&limit=2&sort=-id&fields=id,name
```

```
200 {"data":[{"id":11,"name":"적재로봇 1호"},{"id":7,"name":"검사기 1호"}],"meta":{"page":1,"limit":2,"total":4,"totalPages":2}}
```

★ README 의 예시 응답과 한 글자도 다르지 않아야 합니다.

```
GET /api/v1/equipments?q=로봇&fields=id
```

```
200 {"data":[{"id":5},{"id":6},{"id":11},{"id":12}],"meta":{"page":1,"limit":10,"total":4,"totalPages":1}}
```

GET /api/v1/equipments?line=A,B&status=정지&fields=id

200 {"data":[{"id":2},{"id":8}], "meta":{"page":1, "limit":10, "total":2, "totalPages":1}}

GET /api/v1/equipments?line=Z&fields=id

200 {"data":[], "meta":{"page":1, "limit":10, "total":0, "totalPages":0}}

GET /api/v1/equipments?page=99&limit=5&fields=id

200 {"data":[], "meta":{"page":99, "limit":5, "total":12, "totalPages":3}}

★ 조건에 맞는 게 없어도, 쪽 번호를 넘겨도 200 입니다. 404 가 아닙니다.

GET /api/v1/equipments/:id

```
app.get("/api/v1/equipments/:id", 설비찾기, (req, res) => {  
  보내기(res, req.설비);  
});
```

GET /api/v1/equipments/3

200 {"data":{"id":3, "name":"프레스 1호", "line":"B", "status":"가동", "설치일":"2019-07-02"}}

GET /api/v1/equipments/abc

400 {"error":{"code":"VALIDATION_FAILED", "message":"입력값이 올바르지 않습니다", "details":[{"키":"id", "이유":"숫자여야 합니다"}]}}

GET /api/v1/equipments/99

404 {"error":{"code":"NOT_FOUND", "message":"설비를 찾을 수 없습니다"}}

★ "설비를" 입니다. "설비을(를)" 이 아닙니다. 연습문제 1의 조사붙이기 가 여기서 쓰입니다.

GET /api/v1/equipments/:id/logs

```
app.get("/api/v1/equipments/:id/logs", 설비찾기, (req, res) => {
  const 결과 = 점검기록들.filter((기록) => 기록.equipmentId === req.번호);
  const { page, limit } = 쪽정보(req.query);
  const 시작 = (page - 1) * limit;

  보내기(res, 결과.slice(시작, 시작 + limit), {
    page,
    limit,
    total: 결과.length,
    totalPages: Math.ceil(결과.length / limit),
  });
});
```

GET /api/v1/equipments/1/logs

```
200 {"data":[{"id":1,"equipmentId":1,"result":"정상","점검일":"2026-08-01","담당자":"김민준"}, {"id":2,"equipmentId":1,"result":"이상","점검일":"2026-08-08","담당자":"이서연"}, {"id":5,"equipmentId":1,"result":"정상","점검일":"2026-08-12","담당자":"김민준"}], "meta":{"page":1,"limit":10,"total":3,"totalPages":1}}
```

GET /api/v1/equipments/7/logs

```
200 {"data":[], "meta":{"page":1,"limit":10,"total":0,"totalPages":0}}
```

GET /api/v1/equipments/99/logs

```
404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}
```

★★ 두 번째와 세 번째를 꼭 비교하세요. 7번 설비는 있는데 기록이 0건 → 200 과 빈 배열 99번 설비는 아예 없음 → 404

설비찾기 미들웨어를 안 붙이면 둘 다 200 과 빈 배열이 됩니다. “없는 설비의 기록을 달라” 는 요청에 “0건입니다” 라고 답하는 셈입니다. 중첩 주소에서는 위쪽 자원이 있는지를 반드시 확인해야 합니다.

POST /api/v1/equipments

```
app.post("/api/v1/equipments", 인증, 역할확인("admin"), (req, res, next) => {
  const { name, line } = req.body || {};
  const 항목 = [];

  if (name === undefined || name === "") 항목.push({ 키: "name", 이유: "필수입니다" });
  else if (typeof name !== "string") 항목.push({ 키: "name", 이유: "글자여야 합니다" });
});
```

```
});
  else if (typeof name !== "string") 항목.push({ 키: "name", 이유: "글자여야 합니다"
});
  else if (name.length < 2) 항목.push({ 키: "name", 이유: "2글자 이상이어야 합니다"
});
  else if (name.length > 20) 항목.push({ 키: "name", 이유: "20글자 이하여야 합니다"
});

  if (line === undefined || line === "") 항목.push({ 키: "line", 이유: "필수입니다"
});
  else if (!라인값들.includes(line))
    항목.push({ 키: "line", 이유: "A, B, C 중 하나여야 합니다" });

  if (항목.length > 0) return next(에러.검증실패(항목));
```

★ status 는 본문에서 안 받습니다. 서버가 정합니다.

```
const 새설비 = { id: 다음번호, name, line, status: "정지" };

다음번호 += 1;
설비들.push(새설비);

res.status(201).location(`/api/v1/equipments/${새설비.id}`).json({ data: 새설비 });
});
```

POST /api/v1/equipments

```
{ "name": "용접로봇 3호", "line": "C" }
```

401 {"error":{"code":"UNAUTHENTICATED","message":"로그인이 필요합니다"}}

POST /api/v1/equipments

Authorization: Bearer key-user-1

```
{ "name": "용접로봇 3호", "line": "C" }
```

403 {"error":{"code":"FORBIDDEN","message":"이 작업을 할 권한이 없습니다"}}

POST /api/v1/equipments

Authorization: Bearer key-admin-1

```
{ "name": "용", "line": "Z" }
```

400 {"error":{"code":"VALIDATION_FAILED","message":"입력값이 올바르지 않습니다","details":[{"키":"name","이유":"2글자 이상이어야 합니다"}, {"키":"line","이유":"A, B, C 중 하나여야 합니다"}]}}

POST /api/v1/equipments

Authorization: Bearer key-admin-1

```
{ "name": "용접로봇 3호", "line": "C", "status": "가동" }
```

```
201 { "data": { "id": 13, "name": "용접로봇 3호", "line": "C", "status": "정지" } }
```

★ 마지막을 보세요. status: "가동" 을 보냈는데 "정지" 로 만들어졌습니다. 문서에 "받지 않습니다" 라고 적었고, 코드도 그렇게 되어 있습니다. 보낸 값을 그대로 믿으면 규칙이 무너집니다.

PATCH /api/v1/equipments/:id

```
app.patch("/api/v1/equipments/:id", 인증, 설비찾기, (req, res, next) => {
  const { status } = req.body || {};

  if (!상태값들.includes(status)) {
    return next(
      에러.검증실패([ { 키: "status", 이유: `${상태값들.join(", ")}` 중 하나여야 합니다
    ` } ] )
    );
  }

  req.설비.status = status;

  보내기(res, req.설비);
});
```

PATCH /api/v1/equipments/13

Authorization: Bearer key-user-1

```
{ "status": "이상" }
```

```
400 { "error": { "code": "VALIDATION_FAILED", "message": "입력값이 올바르지 않습니다", "details": [ { "키": "status", "이유": "가동, 정지, 점검중 중 하나여야 합니다" } ] } }
```

PATCH /api/v1/equipments/13

Authorization: Bearer key-user-1

```
{ "status": "점검중" }
```

```
200 { "data": { "id": 13, "name": "용접로봇 3호", "line": "C", "status": "점검중" } }
```

PATCH /api/v1/equipments/99

Authorization: Bearer key-user-1

```
{ "status": "가동" }
```

```
404 { "error": { "code": "NOT_FOUND", "message": "설비를 찾을 수 없습니다" } }
```

DELETE /api/v1/equipments/:id

```
app.delete("/api/v1/equipments/:id", 인증, 역할확인("admin"), 설비찾기, (req, res) => {
  설비들 = 설비들.filter((하나) => 하나.id !== req.번호);
  res.sendStatus(204);
});
```

DELETE /api/v1/equipments/13

Authorization: Bearer key-user-1

403 {"error":{"code":"FORBIDDEN","message":"이 작업을 할 권한이 없습니다"}}

DELETE /api/v1/equipments/13

Authorization: Bearer key-admin-1

204

DELETE /api/v1/equipments/13

Authorization: Bearer key-admin-1

404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}

★ 미들웨어 순서: 인증 → 역할확인 → 설비찾기 자격이 없는 사람에게는 있는지 없는지도 알려 주지 않습니다. README 에도 그렇게 적어 두었습니다.

404 와 에러 처리기

```
app.use((req, res, next) => {
  next(에러.주소없음());
});
```

GET /api/v1/없는주소

404 {"error":{"code":"ROUTE_NOT_FOUND","message":"그런 주소가 없습니다"}}

```
app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.name}: ${err.message}`);

  if (err.type === "entity.parse.failed") {
    return res.status(400).json({
      error: { code: "INVALID_JSON", message: "JSON 형식이 아닙니다" },
    });
  }
});
```


부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.